

Capitolo 1

Introduzione alla programmazione

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume
informatica per progetti – Programmare in Python, di M. Fiore, A. Zanga, ISBN 9788808899477

Rappresentazione dell'informazione

Cosa significa il termine INFORMATICA?

- unione delle parole *informazione* e *automatica*;
- scienza dell'informazione;
- computer science;
- information technology.

Non abbiamo un significato unico del termine *informatica*, ma siamo certi che abbia a che fare con:

- informazione;
- scienza;
- computer.

Cosa significa «informazione»?

Osserviamo l'immagine e su un foglio riportiamo **cosa vediamo**.



Dati

Ciò che osserviamo sono dati, cioè **osservazioni e rilevazioni oggettive** della realtà

Ora spieghiamo il motivo per cui abbiamo fatto queste osservazioni scrivendo una tabella:

DATI	SPIEGAZIONE

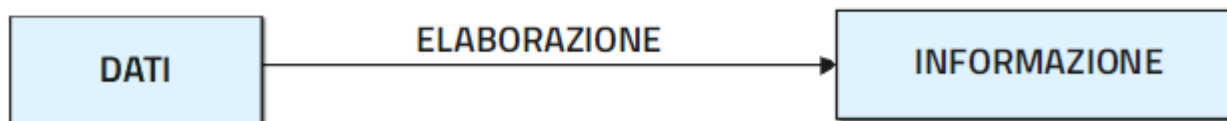
Dati e Informazioni

La spiegazione delle osservazioni fatte è dare un significato alle nostre osservazioni, riflettendo su quanto fatto.

Abbiamo quindi fatto una elaborazione delle nostre osservazioni: le informazioni sono esattamente questo, dati elaborati.

I **dati** sono delle conoscenze elementari che caratterizzano una situazione reale.

Le **informazioni** sono dati che sono stati elaborati così da poter dare loro un significato.



Come rappresentare l'informazione

Con la rappresentazione delle informazioni consentiamo al computer di utilizzare le informazioni (che sono astratte) e questo tipo di rappresentazione deve necessariamente essere il più possibile precisa.

Vediamo un esempio:

INFORMAZIONE	RAPPRESENTAZIONE
una persona abita a Milano	residenza = Milano
io sono alta	statura = 180 (cm)
la verifica di informatica è andata bene	voto = 10
le scarpe che vorrei costano tanto	prezzo = 200 (€)
ho vinto la battaglia a Clash of Clans	risultato = True

Dal problema al processo risolutivo

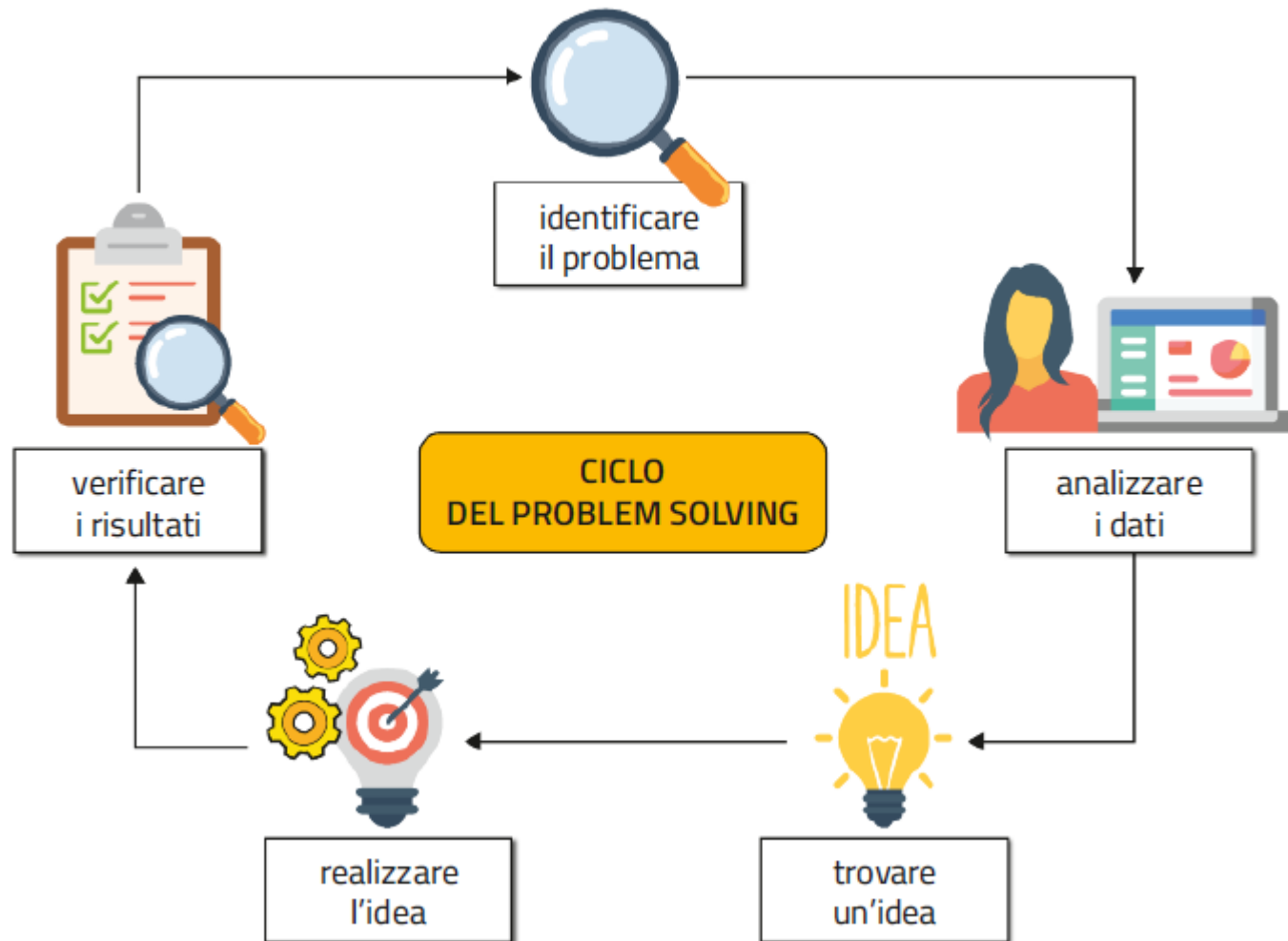
Per quanto possa sembrare intelligente un computer, in realtà non lo è affatto poiché si limita ad eseguire programmi software scritti dagli umani.

Se gli umani scrivono programmi software intelligenti e validi, i risultati delle elaborazioni del computer saranno intelligenti e validi.

Quotidianamente se dobbiamo risolvere un problema, di qualsiasi natura esso sia, cerchiamo di individuare **l'insieme di azioni** che devono essere svolte in una certa sequenza per arrivare a una soluzione.

Applichiamo il **problem solving** che ci aiuta a individuare quali azioni e quali risorse siano utili per trovare la soluzione del nostro problema.

Ciclo del problem solving



Soluzione del problema

La soluzione del nostro problema è composta da **dati** e da **azioni** che possono essere effettuate sui dati per ottenere dei **risultati** che concorrono alla soluzione del problema.

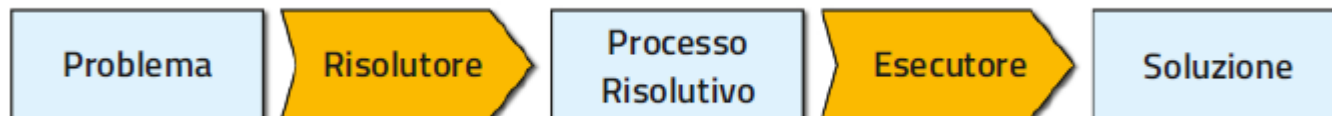


Risolutore ed esecutore

per scrivere programmi software che siano validi e intelligenti dobbiamo:

- con il problem solving trovare le azioni necessarie alla soluzione del problema - descrivere il processo risolutivo;
- eseguire il processo risolutivo.

- Il **risolutore** è colui che si occupa di risolvere il problema che gli è stato affidato e che decide in che modo effettuare il processo risolutivo, a seconda delle sue abilità tecniche e scegliendo, tra più soluzioni alternative, quella più adatta in base alle caratteristiche del problema specifico.
- L'**esecutore** è invece colui che riceve la soluzione del problema e si limita a eseguire un'istruzione dopo l'altra, fino alla fine.



Processo di esecuzione

La risoluzione di un problema spesso è **generica** rispetto ai dati in ingresso:

- fare una lista di acquisti...
- individuare diversi negozi...

ma quando dobbiamo eseguire queste azioni è **necessario indicare**:

- quali sono gli acquisti da fare: latte, pane, yogurt...
- quali sono i negozi dove recarci: supermercato, panettiere, ...

altrimenti non siamo in grado di svolgere il compito assegnato!

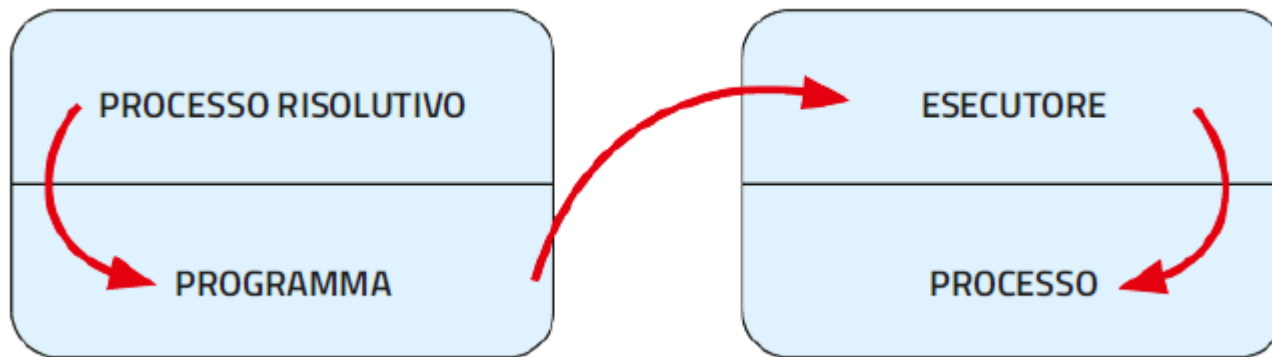
Il processo risolutivo deve essere generale per consentire l'utilizzo in casi diversi:

- la lista della spesa può essere diversa: calze, maglione, ...
- i negozi possono essere vari: macellaio, negozio di abbigliamento...

Il processo di esecuzione invece deve specificare **a cosa ci riferiamo** e si deve adattare all'esigenza specifica in **modo dinamico** e in base alla necessità.

Programma e processo

Il processo risolutivo porta alla costruzione di un *programma*, l'esecuzione del programma stesso viene detto processo di esecuzione, o più semplicemente, *processo*.



Algoritmo

Algoritmo: una sequenza di istruzioni che partendo dai dati iniziali produce una soluzione al problema a esso associato.

Un algoritmo deve essere:

1. **non ambiguo**, ossia, le istruzioni che lo compongono devono corrispondere ad una ed una sola azione da parte dell'esecutore;
2. **finito**, sia per numero di istruzioni che per tempo di esecuzione;
3. **ordinato**, ogni istruzione viene eseguita esattamente nell'ordine in cui è riportata nella sequenza che compone l'algoritmo;
4. **deterministico**, a parità di dati iniziali, l'algoritmo deve portare sempre alla stessa soluzione, in modo da garantire la riproducibilità del processo di esecuzione;
5. **completo**, cioè deve avere una soluzione da seguire per tutti i casi che si possono verificare durante l'esecuzione.

Quali sono algoritmi?

- una ricetta
- le indicazioni stradali
- il calcolo dell'area di un rettangolo
- i processi risolutivi che hai scritto in precedenza...

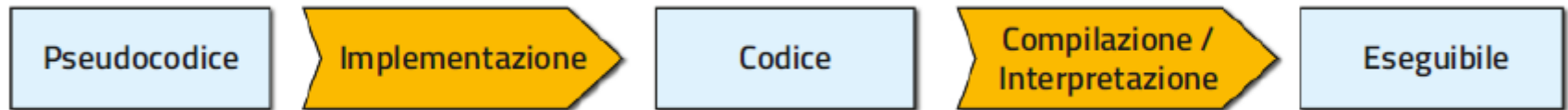
Progettare un algoritmo

La realizzazione di un algoritmo passa attraverso la sua formalizzazione, il processo tramite cui descriviamo la sequenza di istruzioni che compone l'algoritmo in un linguaggio formale, chiamato *pseudocodice*.



Programmazione di un algoritmo

Dopo aver risolto il problema e scritto la sequenza di istruzioni che compongono l'algoritmo in pseudocodice, dobbiamo tradurre lo pseudocodice in un programma che l'esecutore possa effettivamente mettere in azione.

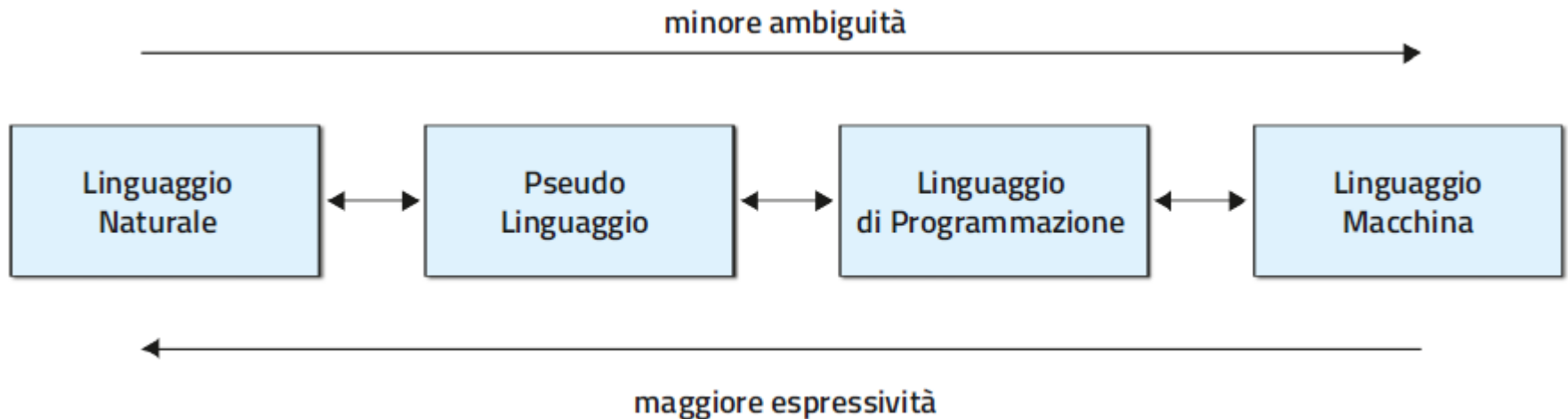


Implementazione: l'algoritmo formalizzato in pseudocodice viene convertito in codice.

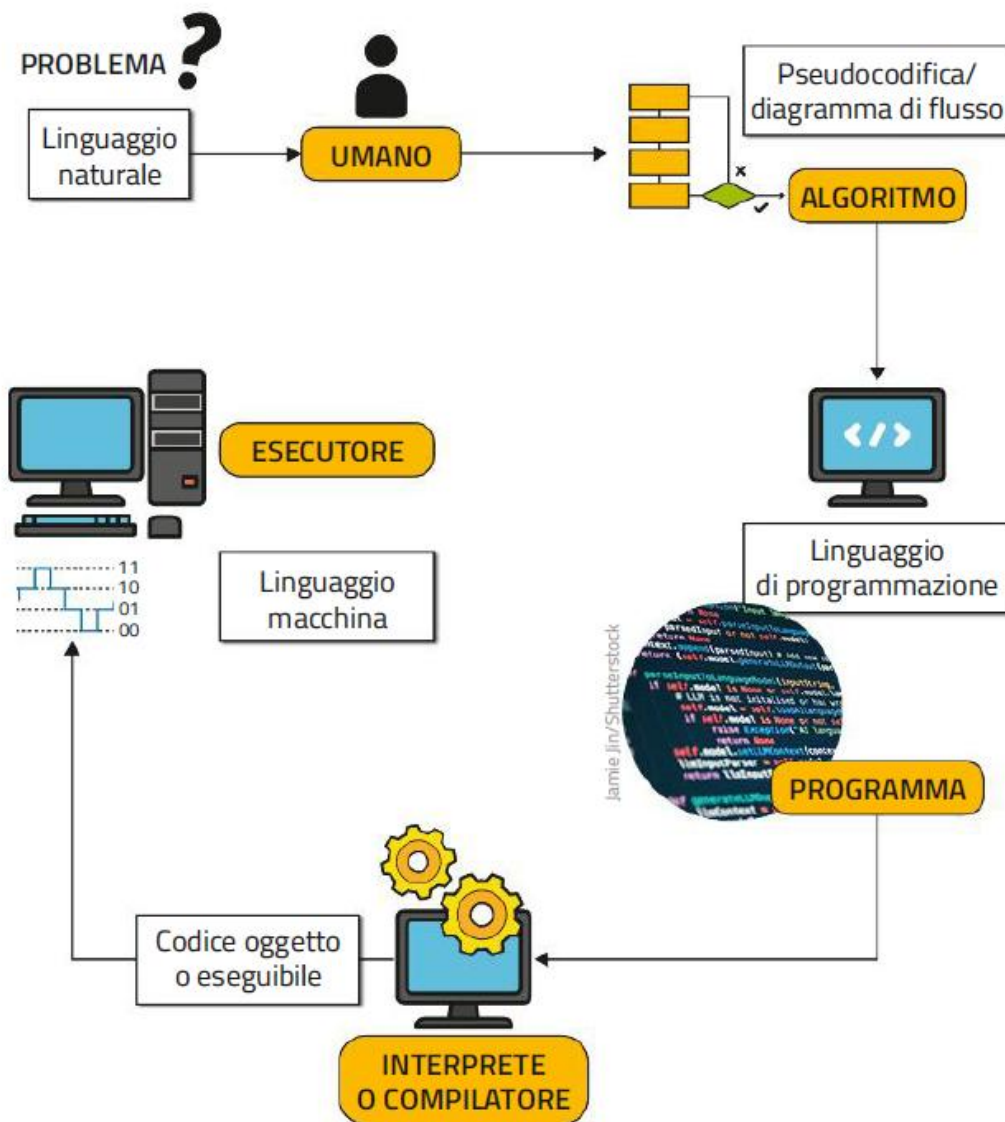
Compilazione/Interpretazione: il codice viene utilizzato per costruire un programma che verrà eseguito.

Compilazione/Interpretazione

Lo pseudocodice viene tradotto in codice utilizzando un linguaggio di programmazione e il codice, attraverso la compilazione o l'interpretazione, viene tradotto in linguaggio macchina, un linguaggio binario costituito solo dai simboli 0 e 1.



Dal problema all'esecuzione



Capitolo 2

Programmazione strutturata

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume
informatica per progetti – Programmare in Python, di M. Fiore, A. Zanga, ISBN 9788808899477

Pseudocodice

Lo pseudocodice è un linguaggio intermedio tra linguaggio naturale e linguaggio di programmazione.

Ad esempio ecco lo pseudocodice che permette di calcolare il costo del totale di un carrello che contiene tre prodotti.

```
1  ALGORITMO
2      costo_totale_carrello
3  DATI
4      costo_1      (numero reale)
5      costo_2      (numero reale)
6      costo_3      (numero reale)
7      costo_totale  (numero reale)
8  ISTRUZIONI
9      INIZIO
10         INPUT costo_1, costo_2, costo_3
11         costo_totale = costo_1 + costo_2 + costo_3
12         OUTPUT costo_totale
13     FINE
```


Caratteristiche dello pseudocodice

Parole riservate:

- **ALGORITMO**, viene utilizzata per identificare l'inizio dell'algoritmo ed immediatamente seguita dal nome dell'algoritmo;
- **DATI**, riporta i dati utilizzati dall'algoritmo, come ad esempio il nome di una variabile e il tipo della variabile;
- **ISTRUZIONI**, dopo aver specificato il nome dell'algoritmo e i dati dell'algoritmo, rimangono da scrivere le istruzioni dell'algoritmo vero e proprio;
- **INIZIO**, indica l'inizio delle istruzioni dell'algoritmo;
- **FINE**, indica la fine delle istruzioni dell'algoritmo;
- **INPUT**, indica l'inserimento di dati da tastiera;
- **OUTPUT**, indica la visualizzazione di risultati dell'algoritmo;
- ...

Variabili

Un algoritmo manipola i dati a sua disposizione tramite l'utilizzo delle variabili.

Una **variabile** rappresenta un valore presente in memoria e viene utilizzata come riferimento a tale valore durante l'esecuzione dell'algoritmo.

L'assegnamento di un valore ad una variabile avviene tramite l'**operatore di assegnamento**, rappresentato dal simbolo "=" :

```
nome_variabile = valore_variabile
```

Per esempio:

```
età_cliente = 26
```

Usare nomi di variabili significativi!

Operatori

OPERATORE	OPERAZIONE	ESEMPIO
=	assegnamento	$A = 0$
+	somma	$A + B$
-	sottrazione	$A - 1$
*	moltiplicazione	$A * B$
/	divisione	A / B
//	divisione intera (quoziente)	$A // B$
%	divisione intera (resto)	$A \% B$
==	confronto	$A == B$

Operatori logici e tabelle di verità

Gli operatori logici di congiunzione “E”, disgiunzione “O” e negazione “NON” permettono di valutare la veridicità di alcune condizioni logiche, ad esempio “piove E NON ho l’ombrello”.

Gli operatori logici si usano secondo le loro tabelle di verità:

Operatore NON:

C'È UNA PROMOZIONE	NON C'È UNA PROMOZIONE
<i>Falso</i>	<i>Vero</i>
<i>Vero</i>	<i>Falso</i>

Tabelle di verità

Operatore **E**:

C'È UNA PROMOZIONE	IL PRODOTTO È IN PROMOZIONE	C'È UNA PROMOZIONE E IL PRODOTTO È IN PROMOZIONE
<i>Falso</i>	<i>Falso</i>	<i>Falso</i>
<i>Falso</i>	<i>Vero</i>	<i>Falso</i>
<i>Vero</i>	<i>Falso</i>	<i>Falso</i>
<i>Vero</i>	<i>Vero</i>	<i>Vero</i>

Operatore **O**:

C'È UNA PROMOZIONE	IL PRODOTTO È IN PROMOZIONE	C'È UNA PROMOZIONE O IL PRODOTTO È IN PROMOZIONE
<i>Falso</i>	<i>Falso</i>	<i>Falso</i>
<i>Falso</i>	<i>Vero</i>	<i>Vero</i>
<i>Vero</i>	<i>Falso</i>	<i>Vero</i>
<i>Vero</i>	<i>Vero</i>	<i>Vero</i>

Strutture di controllo

L'ordine con cui una o più istruzioni vengono eseguite viene detto “flusso di esecuzione” e dipende da alcune strutture chiamate strutture di controllo del flusso, o più semplicemente, “strutture di controllo”.

Esistono tre categorie di strutture di controllo:

- sequenza;
- selezione;
- iterazione.

Sequenza

La sequenza è la più semplice ed è formata da una lista ordinata di istruzioni, eseguite nello stesso ordine con cui sono scritte:

LINGUAGGIO NATURALE	ISTRUZIONI	
Per aggiungere un nuovo prodotto al nostro catalogo online dobbiamo inserire il nome del prodotto, inserire la descrizione, inserire il prezzo, caricare una foto e salvare le informazioni nel catalogo.	1	Inserire il nome,
	2	Inserire la descrizione,
	3	Inserire il prezzo,
	4	Inserire la foto,
	5	Salvare nel catalogo.

Selezione

La selezione permette di scegliere tra flussi di esecuzioni alternativi in base ad una condizione. Per questo motivo, la struttura di selezione viene espressa con il condizionale “SE ... ALLORA: ...”

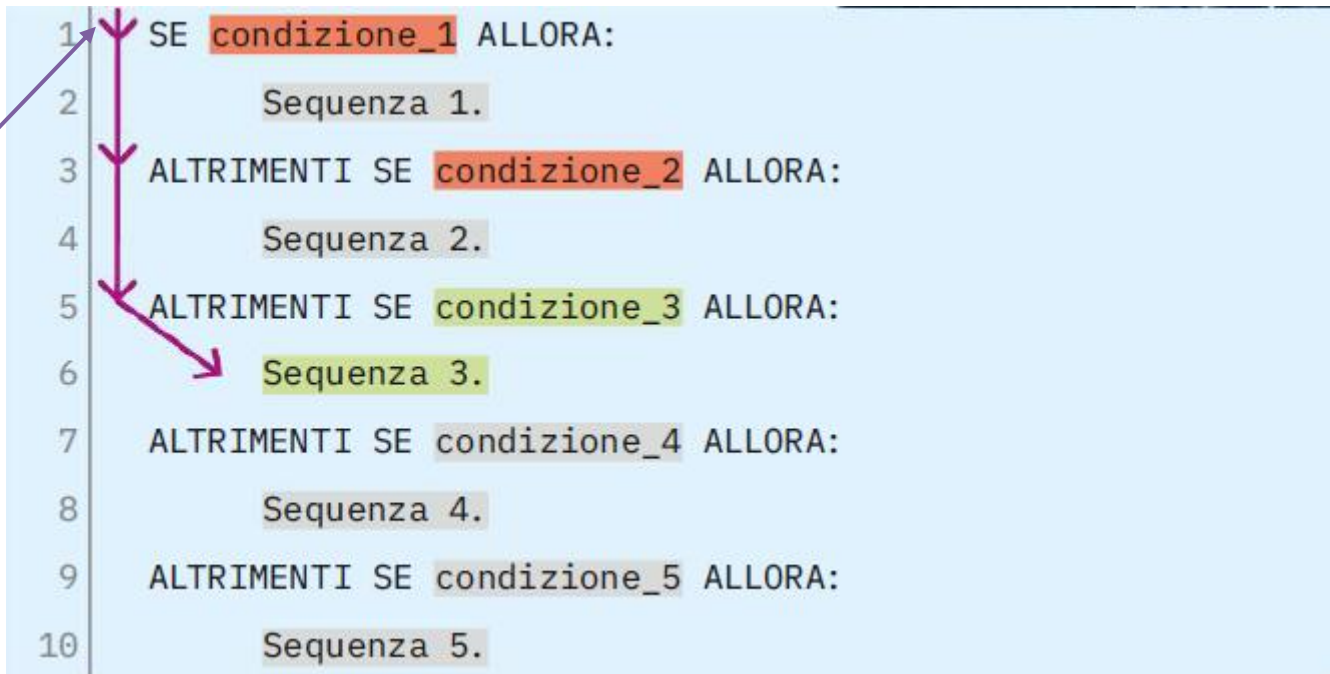
LINGUAGGIO NATURALE	ISTRUZIONI
Per aggiungere un nuovo prodotto al nostro catalogo online dobbiamo inserire il nome del prodotto, inserire la descrizione e inserire il prezzo. Se è disponibile una foto del prodotto, allora caricare la foto. Infine, salvare le informazioni nel catalogo.	1 Inserire il nome, 2 Inserire la descrizione, 3 Inserire il prezzo, 4 SE abbiamo una foto ALLORA: 5 Inserire la foto, 6 Salvare nel catalogo.

L’istruzione “inserire la foto” viene eseguita se e solo se la condizione “abbiamo una foto” è vera. Se la condizione è falsa la riga 5 non viene eseguita e si salta alla riga 6.

Struttura Selezione

SELEZIONE...	...SENZA SEQUENZA FINALE	...CON SEQUENZA FINALE
...semplice	1 SE condizione ALLORA: 2 Sequenza.	1 SE condizione ALLORA: 2 Sequenza. 3 ALTRIMENTI: 4 Sequenza finale.
...composta	1 SE condizione 1 ALLORA: 2 Sequenza 1. 3 ALTRIMENTI SE cond. 2 ALLORA: 4 Sequenza 2. 5 ... 6 ALTRIMENTI SE cond. N ALLORA: 7 Sequenza N.	1 SE condizione 1 ALLORA: 2 Sequenza 1. 3 ALTRIMENTI SE cond. 2 ALLORA: 4 Sequenza 2. 5 ... 6 ALTRIMENTI SE cond. N ALLORA: 7 Sequenza N. 8 ALTRIMENTI: 9 Sequenza finale.

Flusso di esecuzione Selezione composta



Struttura Iterazione definita

L'iterazione è la struttura di controllo che permette di ripetere una sequenza di istruzioni in base ad una condizione.

Esistono due tipologie di iterazione: **definita** e **indefinita**.

L'iterazione **definita** ripete la sequenza un numero prefissato di volte. Utilizza una *variabile contatore* che tiene conto del numero di ripetizione corrente.

Viene espressa tramite l'espressione **“RIPETI PER contatore DA 1 A N :”**.

Il contatore viene incrementato implicitamente al termine dell'esecuzione della sequenza interna all'iterazione.

LINGUAGGIO NATURALE	ISTRUZIONI
Per aggiungere un nuovo prodotto al nostro catalogo online dobbiamo inserire il nome del prodotto, inserire la descrizione e inserire il prezzo. Aggiungere quindi tre foto del prodotto. Infine, salvare le informazioni nel catalogo.	1 Inserire il nome, 2 Inserire la descrizione, 3 Inserire il prezzo, 4 RIPETI PER i DA 1 A 3: 5 Inserire i-esima foto, 6 Salvare nel catalogo.

Struttura Iterazione indefinita

L'iterazione **indefinita** ripete la sequenza se e solo se una data condizione è **vera**.

In questo caso, lo pseudocodice corrispondente è “**RIPETI MENTRE condizione :**”.

Nel caso in cui la condizione sia falsa fin dall'inizio, allora la sequenza interna non verrà eseguita nemmeno una volta, in modo analogo alla struttura di selezione con condizione falsa.

LINGUAGGIO NATURALE	ISTRUZIONI
Per aggiungere un nuovo prodotto al nostro catalogo online dobbiamo inserire il nome del prodotto, inserire la descrizione e inserire il prezzo. Aggiungere quindi foto del prodotto fino a esaurimento. Infine, salvare le informazioni nel catalogo.	1 Inserire il nome, 2 Inserire la descrizione, 3 Inserire il prezzo, 4 RIPETI MENTRE ci sono ancora foto: 5 Inserire la foto, 6 Salvare nel catalogo.

I diagrammi di flusso

Per scrivere algoritmi possiamo utilizzare anche un formalismo diverso, più legato a una rappresentazione con simboli grafici: i diagrammi di flusso.

I **diagrammi di flusso** rappresentano in modo visuale gli algoritmi, facendo ricorso a simboli grafici che corrispondono alle strutture di controllo viste.

SIMBOLI DEL DIAGRAMMA DI FLUSSO	
SIMBOLO	SIGNIFICATO
	INIZIO ALGORITMO FINE ALGORITMO
	INPUT DATI OUTPUT DATI
	ELABORAZIONE DATI
	STRUTTURA DI SELEZIONE
	DIREZIONE DEL FLUSSO DELLE OPERAZIONI

Diagramma di flusso Sequenza

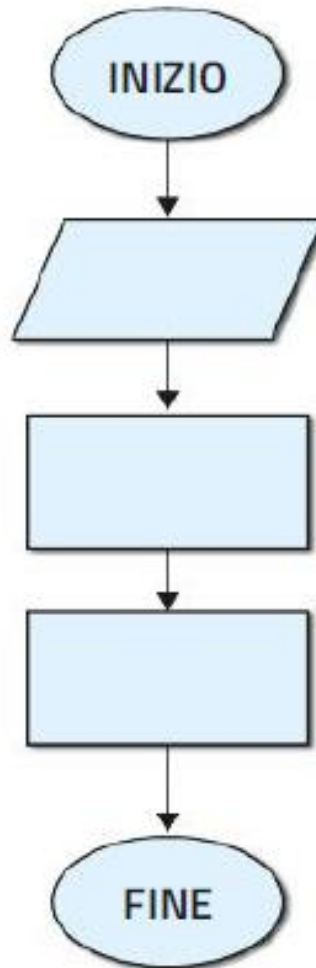


Diagramma di flusso Selezione

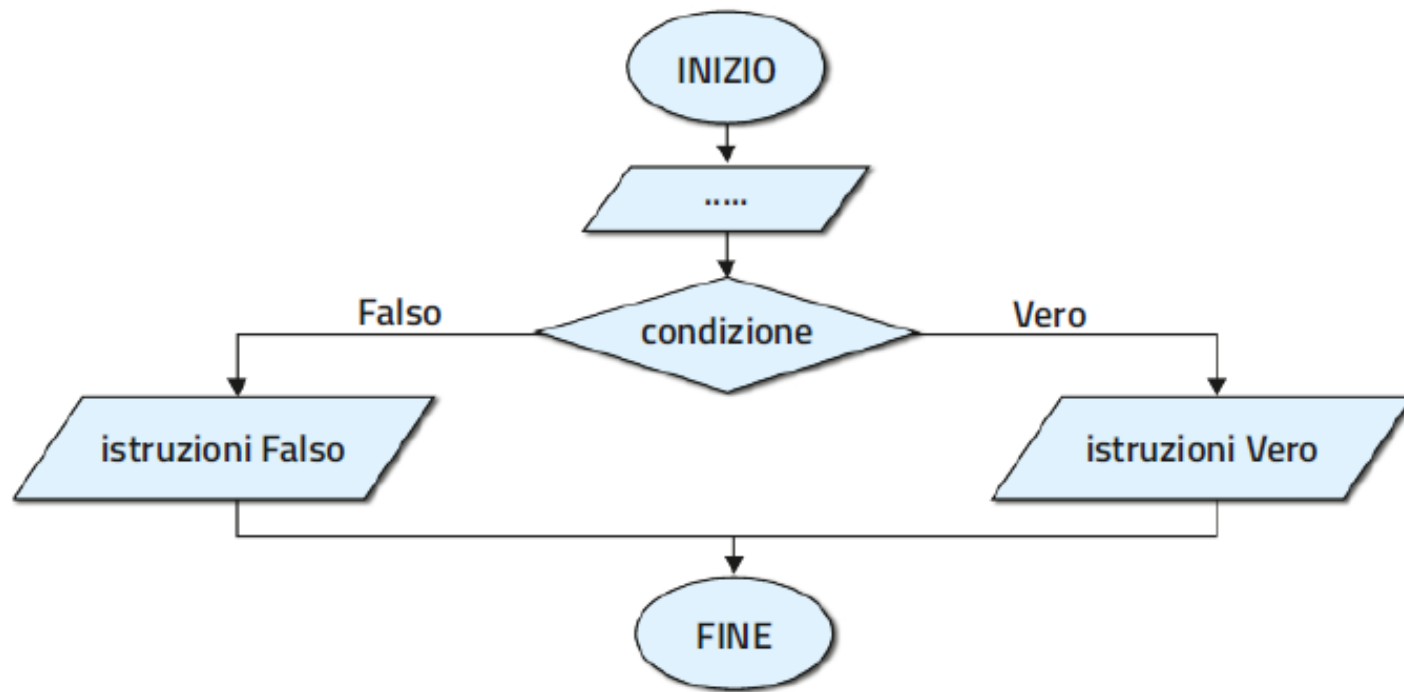


Diagramma di flusso Iterazione Indefinita

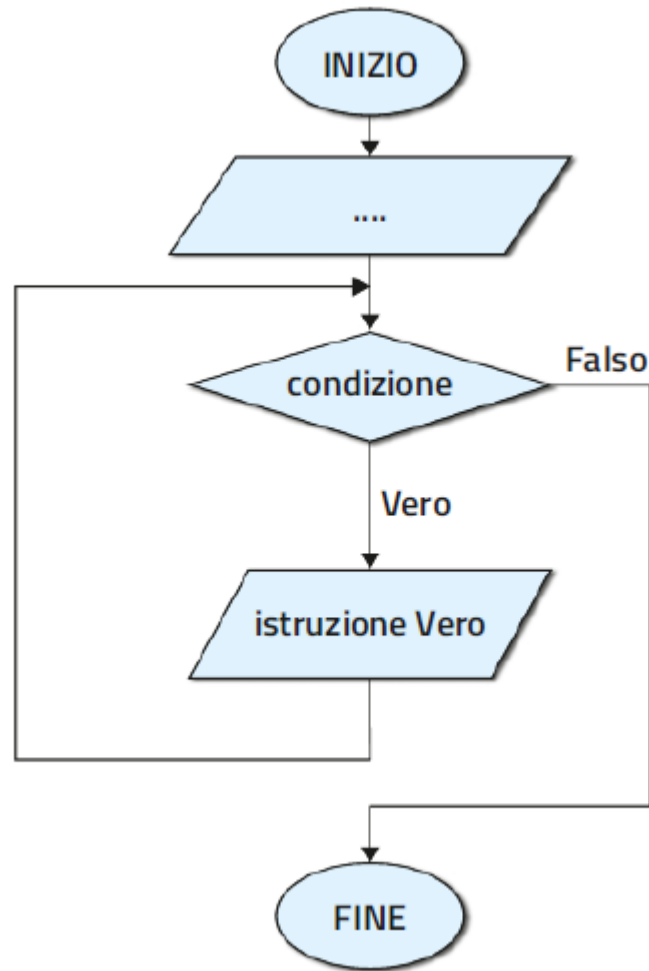


Diagramma di flusso Iterazione Definita

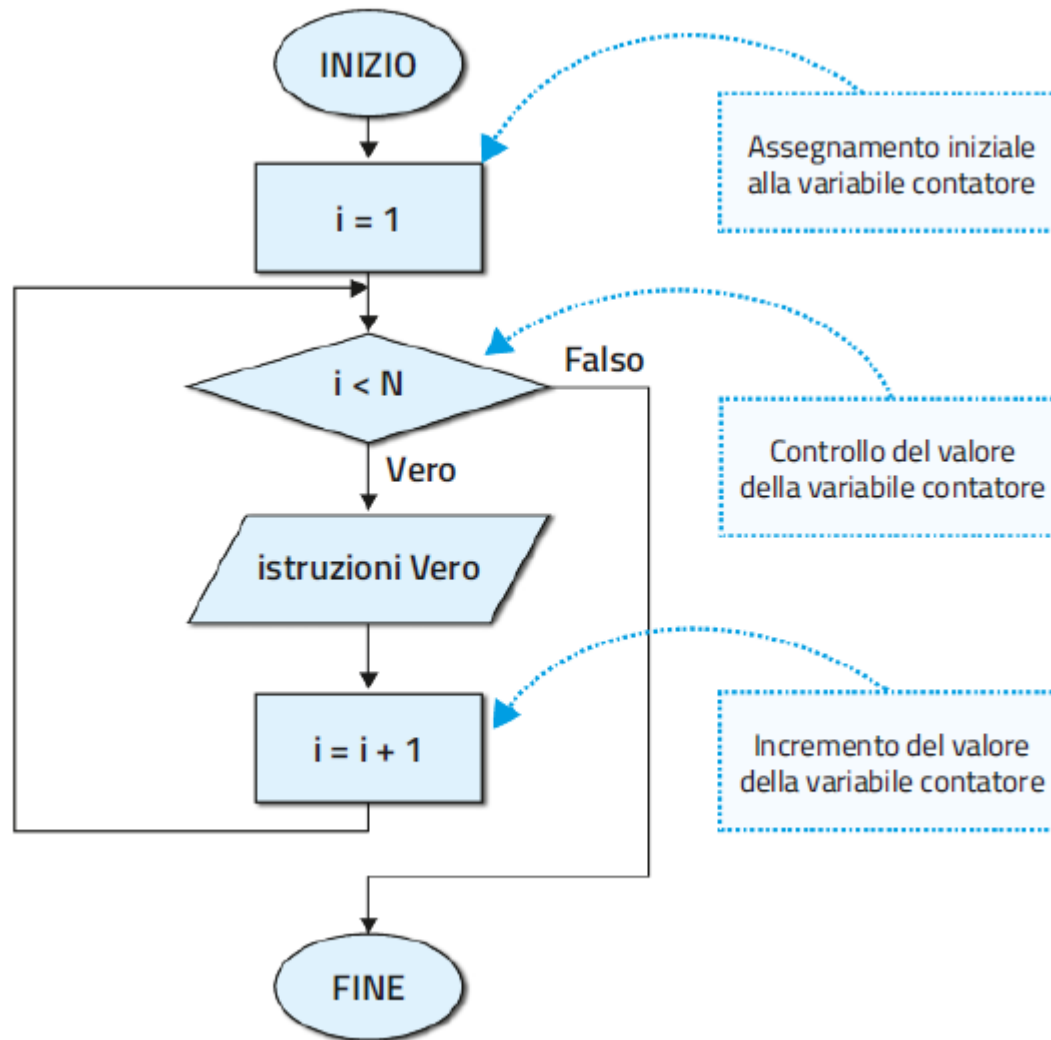


Tabella di esecuzione o di tracking

Utilizzando la pseudocodifica non si ha a disposizione un computer che possa eseguire il codice che abbiamo scritto, e allora come possiamo verificare che il nostro algoritmo sia corretto se non lo possiamo eseguire?

La **tabella di esecuzione** riproduce passo dopo passo, i differenti stati di un programma durante il flusso di esecuzione e il valore che le variabili assumono.

Per costruire la tabella di esecuzione sono necessarie:

1. una colonna che indica il numero della riga
2. una colonna per indicare le istruzioni
3. tante colonne quante sono le variabili previste nell'algoritmo
4. una colonna per l'output.

Esempio

```
1  ALGORITMO
2      perimetro_triangolo
3  DATI
4      cateto_1  (numero intero)
5      cateto_2  (numero intero)
6      ipotenusa (numero reale)
7      perimetro (numero reale)
8  ISTRUZIONI
9      INIZIO
10         INPUT cateto_1, cateto_2
11         ipotenusa = radice(cateto_1 * cateto_1 + cateto_2 * cateto_2)
12         perimetro = ipotenusa + cateto_1 + cateto_2
13         OUTPUT perimetro
14     FINE
```

Tabella di tracking

Costruiamo la tabella di esecuzione, predisponendo le colonne necessarie.

riga	istruzioni	cateto_1	cateto_2	ipotenusa	perimetro	output
------	------------	----------	----------	-----------	-----------	--------

La tabella di esecuzione dell'algoritmo perimetro_triangolo:

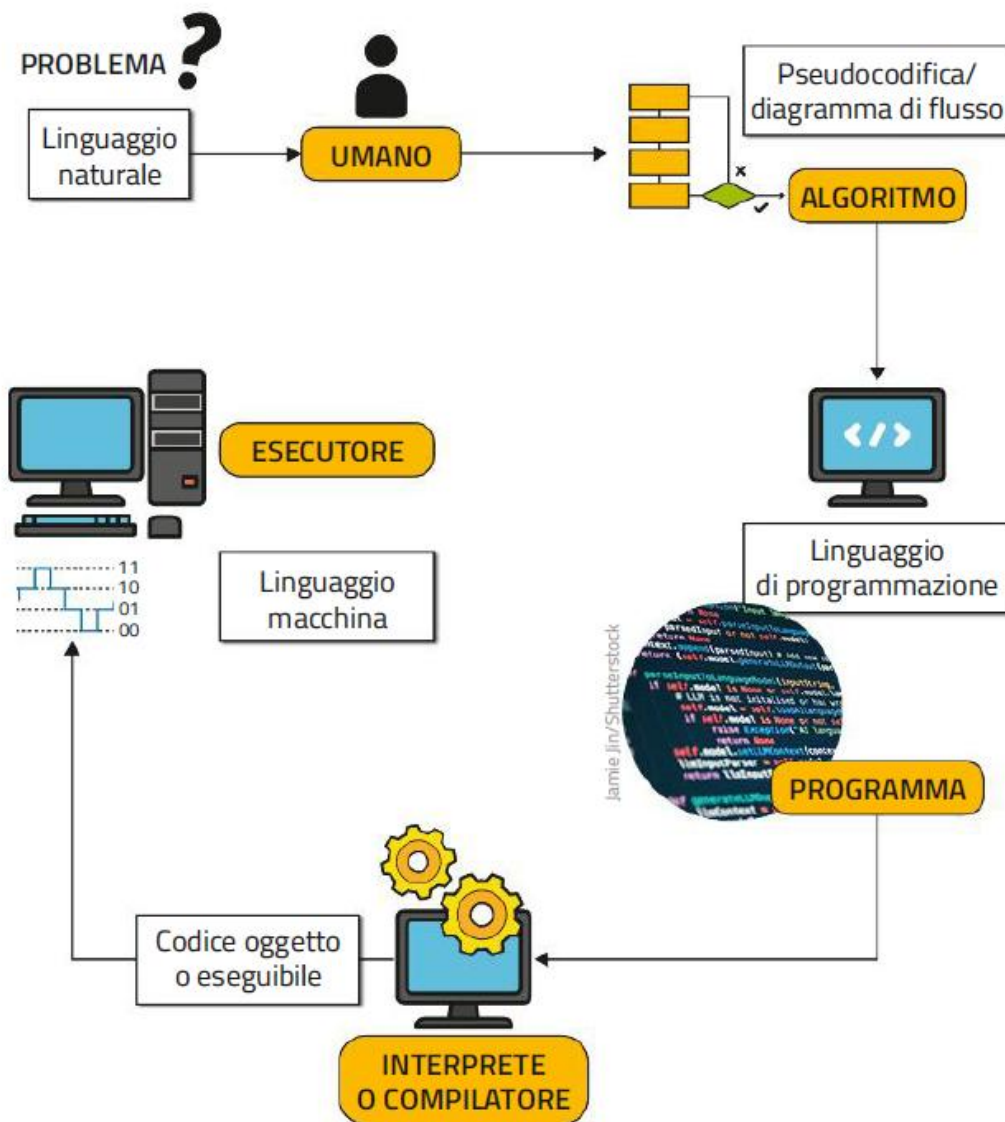
riga	istruzioni	cateto_1	cateto_2	ipotenusa	perimetro	output
9	INIZIO					
10	INPUT cateto1, cateto2	4	6			
11	ipotenusa = radice (cateto1 * cateto1 + cateto2 * cateto2)			7,20		
12	perimetro = ipotenusa + cateto1 + cateto2				17,20	
13	OUTPUT perimetro					17,20
14	FINE					

Capitolo 3

Cominciamo a programmare

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume
informatica per progetti – Programmare in Python, di M. Fiore, A. Zanga, ISBN 9788808899477

Dal problema all'esecuzione



Linguaggi di programmazione

Nel corso degli anni i linguaggi di programmazione si sono molto evoluti, ma tutti sono costituiti da un **insieme di comandi e strutture funzionali** che permettono di tradurre in programmi eseguibili dal computer le istruzioni in pseudocodice degli algoritmi.

I linguaggi di programmazione «**di basso livello**», sono quelli il cui funzionamento è molto vicino all'hardware del computer (e molto lontano dalla logica del programmatore!): ad esempio Assembly.

I linguaggi di programmazione «**di alto livello**», sono quelli che consentono di scrivere codice indipendente dall'hardware del computer e che hanno caratteristiche che lo rendono sempre più semplice da utilizzare dal punto di vista del programmatore: ad esempio Pascal, C, Fortran e i più recenti C++, Java, Python.

Compilatori e Interpreti

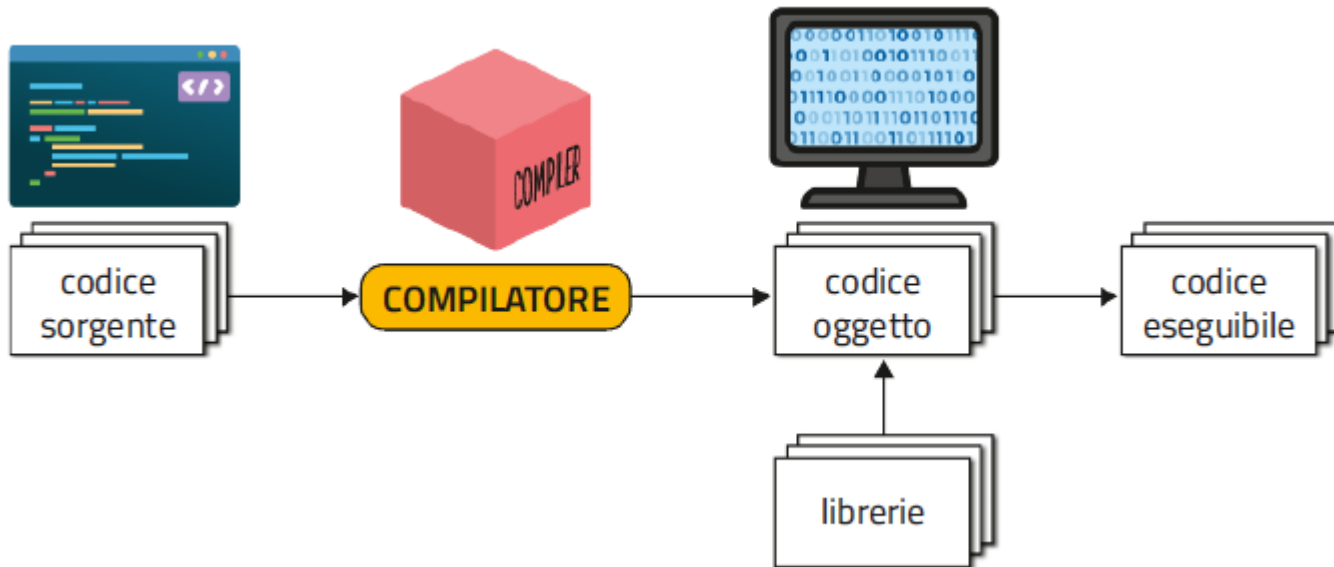
Il programma scritto in un linguaggio di programmazione, per poter essere eseguito dal computer (l'esecutore), ha necessità di essere tradotto in linguaggio macchina, cioè quel linguaggio che può essere compreso dal computer.

Esistono diversi programmi software che consentono questa traduzione da programma scritto in linguaggio di alto livello a programma scritto in linguaggio di basso livello, e si identificano principalmente in due categorie: **compilatori e interpreti**.

Un **compilatore** è un programma software che traduce una serie di istruzioni scritte in un linguaggio di alto livello (detto **codice sorgente**) in istruzioni di un altro linguaggio (detto **codice oggetto**).

Con una operazione successiva del compilatore, il codice oggetto viene tradotto in linguaggio macchina (**codice eseguibile**) ed è pronto per l'esecuzione.

Fasi della compilazione



Interprete

Un **interprete** è un programma software che è in grado di eseguire un programma scritto in un linguaggio di programmazione di alto livello, elaborando le istruzioni una alla volta e traducendole direttamente in linguaggio macchina così che siano eseguibili dal computer.

Un interprete, a differenza di un compilatore, non prevede le fasi di creazione del codice oggetto e poi del codice eseguibile, ma traduce direttamente le singole istruzioni del codice sorgente in istruzioni eseguibili dal computer, una dopo l'altra.

Python, usa un interprete ed è attualmente tra i più utilizzati al mondo proprio per le sue caratteristiche di versatilità e flessibilità.

Il linguaggio Python

Python è un linguaggio di **alto livello** creato nel 1991 da Guido Van Rossum. Il suo nome nasce come omaggio a una serie televisiva britannica: *Monty Python's Flying Circus*.

Python è stato sviluppato a partire dal linguaggio di programmazione ABC, con l'intenzione di semplificarne notevolmente le strutture e ottenere un linguaggio che fosse molto vicino allo pseudocodice e avesse tutte le caratteristiche necessarie per sviluppare applicazioni di ogni genere.

Python supporta infatti **la programmazione strutturata**, quella a **oggetti**, la **programmazione web** e la **programmazione numerica**, ed è adatto per lavorare su **data science**, applicazioni di **intelligenza artificiale** e molte altre ancora.

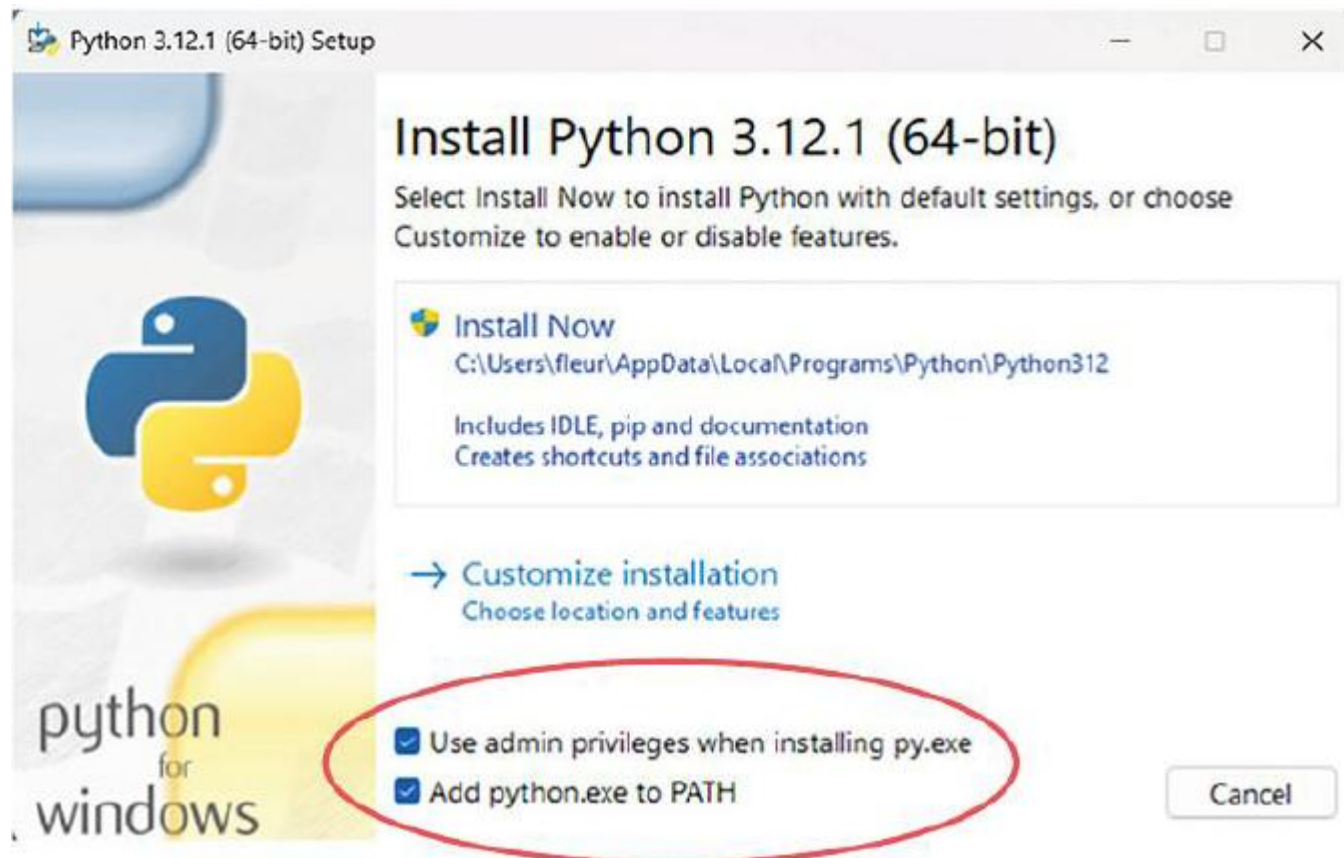
Attualmente è tra i linguaggi più utilizzati, è open source e può contare su una community di sviluppatori tra le più estese al mondo.

Il sito di riferimento per il linguaggio Python si trova alla URL <https://www.python.org>



Installazione di Python

Dal sito ufficiale scaricare la versione più recente di Python e idonea al sistema operativo del proprio personal computer.

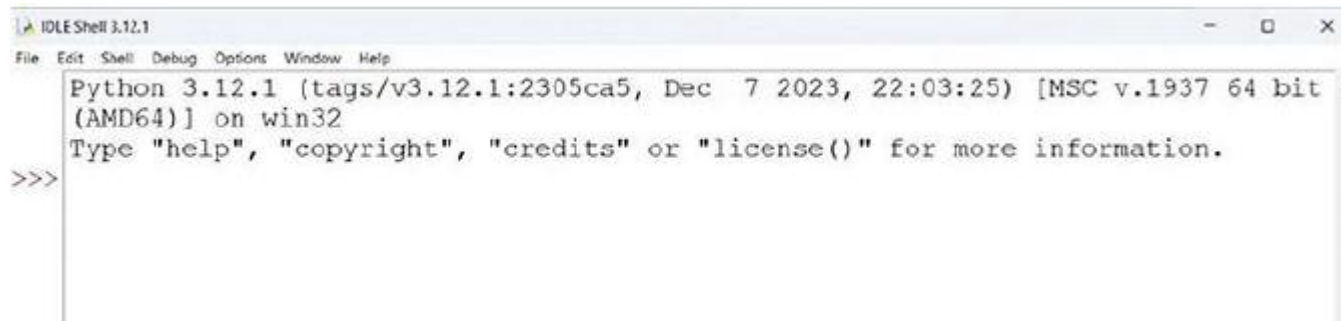


Usare l'interprete Python

L'interprete Python è utilizzabile in due modalità:

1. linea di comandi (modalità interattiva) IDLE Shell;
2. script.

Per avviare Python in modalità «linea di comandi», bisogna avviare l'applicazione denominata IDLE (*Integrated Development and Learning Environment*).



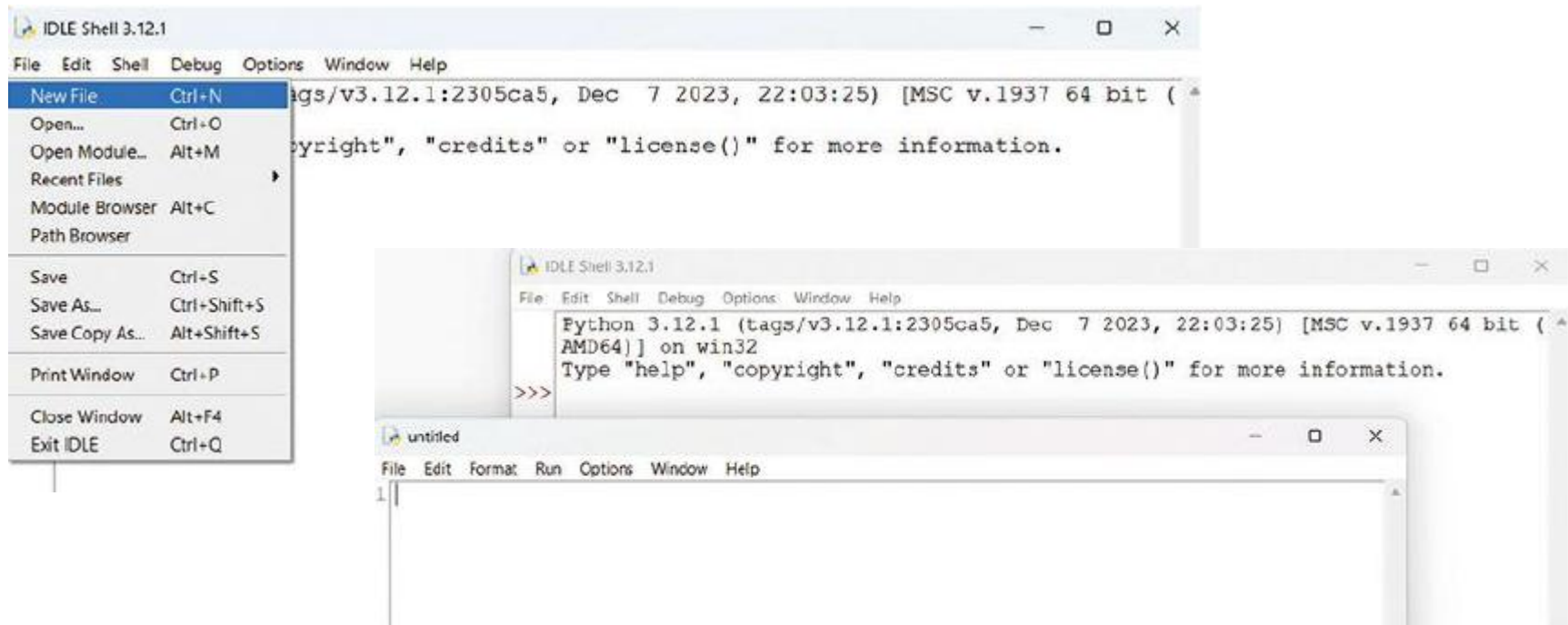
```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
```

Il prompt dei comandi è rappresentato dal simbolo >>> che compare a sinistra e che mostra la riga disponibile per inserire i comandi del linguaggio Python. Ogni comando scritto in una riga verrà eseguito da Python immediatamente.

Script

La modalità «linea di comandi» non è molto comoda quando vogliamo scrivere più righe di codice e memorizzarle per poterle utilizzare anche in futuro.

Python consente quindi la scrittura di **script** che contengono più istruzioni e che vengono salvati in file con estensione **.py**



Commenti

Quando si inizia a scrivere uno script Python è buona norma utilizzare i commenti per documentare il codice. I commenti sono molto utili e possono per esempio servire a:

1. illustrare lo scopo dello script che si sta sviluppando;
2. spiegare il significato di un blocco di istruzioni;
3. documentare il funzionamento di specifiche parti di codice.

Esistono due diverse modalità di implementazione per i commenti:

1. commento su una singola riga;
2. commento su più righe.

Per inserire un commento su una singola riga si usa il simbolo # (hashtag) all'inizio del commento.

Per inserire un commento su più righe vanno inseriti i caratteri ''' (tre apici singoli o doppi) all'inizio del commento e ''' (tre apici singoli o doppi) alla fine del commento.

Tipi di dati semplici

Per **tipo di dato** intendiamo l'insieme dei valori che il dato può assumere. Python è un linguaggio di programmazione **debolmente tipizzato**, ovvero un linguaggio per il quale non è necessario indicare esplicitamente il tipo di dato.

È l'interprete a decidere il tipo di dato, a seconda del valore che gli assegneremo.

I seguenti sono i quattro tipi di dato semplice che è possibile rappresentare in Python:

Tipo	Codice	Valore	Codice di esempio
Valore booleano	bool	{False, True}	<code>>>> soleggiato = True</code>
Numero intero	int	Insieme Z	<code>>>> prodotti = 42</code>
Numero in virgola mobile	float	Insieme R	<code>>>> sconto = 0.31</code>
Numero complesso	complex	Insieme C	<code>>>> fase = 1.23 + 3.50j</code>

Operatori aritmetici e logici

Per i tipi numerici:

Operatore	Operazione
<pre>>>> 12 + 13 25</pre>	Somma
<pre>>>> 12 - 13 -1</pre>	Sottrazione
<pre>>>> 12 * 13 156</pre>	Moltiplicazione
<pre>>>> 12 / 13 0.9230769230769231</pre>	Divisione
<pre>>>> 12 // 13 0</pre>	Divisione intera
<pre>>>> 12 % 13 12</pre>	Modulo, resto della divisione intera
<pre>>>> 12 ** 13 106993205379072</pre>	Elevamento a potenza

Per i tipi booleani:

- not
- and
- or

```
>>> piove = True
>>> soleggiato = False
>>> not piove and soleggiato
False
```

Conversione di tipo o casting

Il risultato delle operazioni aritmetiche può essere un tipo di dato diverso da quelli di partenza: ad esempio la divisione di due int può restituire un float.

Il passaggio da un tipo di dato a un altro viene detto **conversione** o **casting**. La conversione può essere implicita o esplicita.

- **implicita**: quando sommiamo due numeri di tipo diverso stiamo convertendo il tipo intero a virgola mobile prima di effettuare l'operazione
- **esplicita**: siamo noi a decidere il tipo del dato che vogliamo utilizzare. Per decidere il tipo di numero che vogliamo usare è sufficiente scrivere il valore del dato tra parentesi, preceduto dal tipo di dato.

```
>>> float(78)
78.0
```

Per controllare il tipo di un dato o una variabile possiamo utilizzare **type**.

```
>>> type(42)
<class 'int'>
```

Input/Output

In Python i dati in input sono trattati come testo e di solito vengono convertiti subito dopo a un altro tipo, per l'output, invece, è necessario convertire gli altri tipi in testo.

Flusso delle operazioni:

1. Inserimento dei dati in input.
2. Conversione da stringa a un altro tipo di dato.
3. Esecuzione del programma.
4. Conversione da un altro tipo di dato a stringa.
5. Output per visualizzare il risultato.

```
>>> sconto = input("Inserire lo sconto da applicare: ") #Assegnazione dell'input
Inserire lo sconto da applicare al totale: 0.35
>>> type(sconto) #Controllo del tipo in input
<class 'str'>
>>> sconto = float(sconto) #Conversione a float
>>> type(sconto) #Controllo del tipo convertito
<class 'float'>
```

Formattazione output

Per utilizzare l'istruzione `print` con stringhe combinate con le variabili in modo da ottenere un output più complesso, dobbiamo effettuare la formattazione delle stringhe.

Possiamo formattare le stringhe in due modi:

1. usando l'istruzione `format`;
2. tramite l'interpolazione delle stringhe, dette anche `f-strings`.

Nel primo caso, l'istruzione `format` permette di inserire una variabile all'interno di una stringa tramite un segnaposto indicato da una coppia di parentesi graffe: `{}`.

```
>>> imc = 65 / 1.70 ** 2
>>> print("L'IMC è: {}".format(imc))
L'IMC è: 22.49134948096886
```

Possiamo anche utilizzare più di un segnaposto alla volta, i segnaposto saranno sostituiti nello stesso ordine delle variabili in `format`.

```
>>> massa = 65
>>> altezza = 1.70
>>> imc = massa / altezza ** 2
>>> print("L'IMC per massa {} kg e altezza {} m è: {}".format(massa, altezza, imc))
L'IMC per massa 65 kg e altezza 1.7 m è: 22.49134948096886
```

f-strings

L'istruzione `format` viene sostituita con una lettera `f` anteposta agli apici della stringa in cui sostituiamo i nomi dei segnaposto direttamente con le variabili da visualizzare.

```
>>> massa = 65
>>> altezza = 1.70
>>> imc = massa / altezza ** 2
>>> print(f"L'IMC per altezza {altezza} m e massa {massa} kg è: {imc}")
L'IMC per altezza 1.7 m e massa 65 kg è: 22.49134948096886
```

Sia con l'istruzione `format` sia con le *f-strings* possiamo ottenere un output ordinato, ma per renderlo più leggibile abbiamo bisogno di un modo per poter controllare come visualizzare il valore di ogni variabile: lo **specificatore di formato**.

Specificatore di formato	Tipo della variabile
:d	Rappresentazione di int.
:f	Rappresentazione di float con sei cifre dopo la virgola.
:a.bf	Rappresentazione di float con: a - Lunghezza minima da visualizzare; b - Numero di cifre da visualizzare dopo la virgola. Possiamo anche omettere a e utilizzare solo b.

Selezione: if else elif

Selezione ...	Pseudocodice	Codice in Python
... semplice senza sequenza finale	<pre> 1 SE condizione ALLORA: 2 Sequenza.</pre>	<pre> 1 if condizione: 2 Sequenza.</pre>
... semplice con sequenza finale	<pre> 1 SE condizione ALLORA: 2 Sequenza. 3 ALTRIMENTI: 4 Sequenza finale.</pre>	<pre> 1 if condizione: 2 Sequenza. 3 else: 4 Sequenza finale.</pre>
... composta senza sequenza finale	<pre> 1 SE condizione 1 ALLORA: 2 Sequenza 1. 3 ALTRIMENTI SE cond. 2 ALLORA: 4 Sequenza 2. 5 ... 6 ALTRIMENTI SE cond. N ALLORA: 7 Sequenza N.</pre>	<pre> 1 if condizione 1: 2 Sequenza 1. 3 elif condizione 2: 4 Sequenza 2. 5 ... 6 elif condizione N: 7 Sequenza N.</pre>
... composta con sequenza finale	<pre> 1 SE condizione 1 ALLORA: 2 Sequenza 1. 3 ALTRIMENTI SE cond. 2 ALLORA: 4 Sequenza 2. 5 ... 6 ALTRIMENTI SE cond. N ALLORA: 7 Sequenza N. 8 ALTRIMENTI: 9 Sequenza finale.</pre>	<pre> 1 if condizione 1: 2 Sequenza 1. 3 elif condizione 2: 4 Sequenza 2. 5 ... 6 elif condizione N: 7 Sequenza N. 8 else: 9 Sequenza finale.</pre>

Selezione: match-case

Selezione composta con if-else	Selezione composta con match-case
<pre>1 giorno = 3 2 if giorno == 1: 3 giorno = "Lunedì" 4 elif giorno == 2: 5 giorno = "Martedì" 6 elif giorno == 3: 7 giorno = "Mercoledì" 8 elif giorno == 4: 9 giorno = "Giovedì" 10 elif giorno == 5: 11 giorno = "Venerdì" 12 elif giorno == 6: 13 giorno = "Sabato" 14 elif giorno == 7: 15 giorno = "Domenica" 16 else: 17 giorno = "ERRORE" 18 print(f"Oggi è {giorno}!")</pre> OUTPUT: Oggi è Mercoledì!	<pre>1 giorno = 3 2 match giorno: 3 case 1: 4 giorno = "Lunedì" 5 case 2: 6 giorno = "Martedì" 7 case 3: 8 giorno = "Mercoledì" 9 case 4: 10 giorno = "Giovedì" 11 case 5: 12 giorno = "Venerdì" 13 case 6: 14 giorno = "Sabato" 15 case 7: 16 giorno = "Domenica" 17 case _: 18 giorno = "ERRORE" 19 print(f"Oggi è {giorno}!")</pre> OUTPUT: Oggi è Mercoledì!

Iterazione definita: for

Pseudocodice	Codice in Python
<pre>1 somma = 0 2 RIPETI PER i DA 1 A 3: 3 somma = somma + i 4 OUTPUT somma OUTPUT: 6</pre>	<pre>1 somma = 0 2 for i in range(1, 4): 3 somma += i 4 print(somma) OUTPUT: 6</pre>

L'intervallo può essere implementato con l'istruzione `range` in diversi modi:

- `range(n + 1)` per valori da 0 a n ;
- `range(m, n + 1)` per valori da m a n ;
- `range(m, n + 1, s)` per valori da m a n , dove il valore successivo è ottenuto sommando s al valore precedente.

L'intervallo che costruiamo con l'istruzione `range` include sempre il valore di partenza ed esclude sempre il valore di arrivo.

Uso di range

Per i da ...	Pseudocodice	Codice in Python
... 0 a n	<pre> 1 RIPETI PER i DA 0 A 3: 2 OUTPUT i OUTPUT: 0 1 2 3 </pre>	<pre> 1 for i in range(4): 2 print(i) OUTPUT: 0 1 2 3 </pre>
... m a n	<pre> 1 RIPETI PER i DA 1 A 3: 2 OUTPUT i OUTPUT: 1 2 3 </pre>	<pre> 1 for i in range(1, 4): 2 print(i) OUTPUT: 1 2 3 </pre>
... m a n, con incremento 2	<pre> 1 RIPETI PER i DA 1 A 3: 2 OUTPUT i 3 i = i + 1 OUTPUT: 1 3 </pre>	<pre> 1 for i in range(1, 4, 2): 2 print(i) OUTPUT: 1 3 </pre>
... m a n, con decremento 1	<pre> 1 RIPETI PER i DA 3 a 0: 2 OUTPUT i OUTPUT: 3 2 1 </pre>	<pre> 1 for i in range(3, 0, -1): 2 print(i) OUTPUT: 3 2 1 </pre>

Iterazione indefinita: while

Pseudocodice	Codice in Python
<pre>1 somma = 0 2 i = 1 3 RIPETI MENTRE i < 4: 4 somma = somma + i 5 i = i + 1 6 OUTPUT somma OUTPUT: 6</pre>	<pre>1 somma = 0 2 i = 1 3 while i < 4: 4 somma += i 5 i += 1 6 print(somma) OUTPUT: 6</pre>

L'iterazione indefinita con il **while** funziona esattamente come una selezione con l'if, con l'unica differenza che quando la condizione del while è vera la sequenza all'interno della struttura viene eseguita più di una volta, finché la condizione di iterazione non diventa falsa.

Capitolo 4

Organizziamo il codice

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume
informatica per progetti – Programmare in Python, di M. Fiore, A. Zanga, ISBN 9788808769978

Funzioni

Quando la sequenza di istruzioni che compone il programma diventa troppo lunga, c'è il rischio che il codice diventi troppo complesso e difficile da interpretare.

Per questo è utile suddividere una sequenza di istruzioni troppo lunga in sequenze più corte.

Una **funzione** è una sequenza di istruzioni che riceve dei dati in ingresso e restituisce un risultato in uscita.

Una funzione in Python rappresenta l'equivalente di un algoritmo in pseudocodice e, in genere, risolve un semplice problema.

Dichiarazione di Funzione

La definizione di una funzione in Python è composta:

1. dalla parola riservata **def**;
2. dal nome della funzione;
3. da dati in ingresso, inseriti tra parentesi tonde seguite dai due punti;
4. da una sequenza di istruzioni, indentate a destra rispetto alla prima riga;
5. dalla parola riservata **return** seguita dal risultato.

```
1 def nome_funzione(dato_1, dato_2, dato_3):  
2     risultato = dato_1 * dato_2 + dato_3  
3     return risultato
```

Caratteristiche di una Funzione

- I dati in ingresso sono anche detti parametri della funzione.
- La prima riga della definizione viene chiamata anche *firma* della funzione.
- L'esecuzione di una funzione avviene tramite la sua *chiamata* o *invocazione*, che consiste nello scrivere il nome della funzione seguito dai valori dei parametri tra parentesi tonde.
- Dati in ingresso e return sono opzionali, quindi non sono sempre presenti.

Parametri formali e parametri attuali

- I **parametri formali** sono i parametri indicati tra parentesi tonde nella dichiarazione di una funzione e rappresentano il modo in cui la funzione utilizza le variabili corrispondenti.
- I **parametri attuali** sono i parametri che, nella chiamata della funzione da parte del programma principale, sono indicati tra parentesi tonde. Tali parametri rappresentano la modalità con cui il programma principale passa alla funzione i valori sui quali fare le elaborazioni.

Osservazioni:

Il nome dei parametri formali indicati tra parentesi dopo la dichiarazione di funzione è lo stesso delle variabili usate *nella funzione*.

Questi parametri rappresentano delle variabili locali alla funzione.

Esempio di utilizzo

Parametri formali

```
1 def calcola_media_prezzi(totale, numero):
2     media = totale / numero
3     print(f"La media dei prezzi dei prodotti del giorno è: {media:7.2f}")
4
5 #main()
6 somma = 0
7 prodotti = int(input("Inserisci il numero totale dei prodotti: "))
8 for i in range(prodotti):
9     prezzo = float(input("Inserisci il prezzo del prodotto: "))
10    somma = somma + prezzo
11 calcola_media_prezzi(somma, prodotti)
```

Parametri attuali

- Nel programma principale avviene l'acquisizione dell'input per quanto riguarda il numero di prodotti venduti nella giornata e il loro prezzo. Questi valori sono contenuti nelle due variabili prodotti e somma (righe da 6 a 10).
- La chiamata della funzione che calcola la media dei prezzi dei prodotti venduti nella giornata è effettuata alla riga 11 e avviene con il passaggio dei valori delle variabili somma e prodotti, che vengono scritti tra parentesi dopo il nome della funzione.

Variabili locali e variabili globali

- Le **variabili locali** sono variabili che sono utilizzabili solo dalla funzione che le ha definite, poiché sono visibili solo a quest'ultima.
- Le **variabili globali** sono variabili che vengono definite nel programma principale e che sono visibili ovunque in quest'ultimo, ma non sono modificabili dalle funzioni.

Tipo di variabile	Dove viene definita	Visibilità
variabile locale	nella funzione	solo alla funzione che la dichiara
variabile globale	nel programma principale	• solo in lettura per le funzioni • ovunque nel programma

Le variabili globali sono visibili anche alle funzioni contenute nel programma che le ha dichiarate, ma solo in lettura, cioè le funzioni che sono state inserite nel programma avranno la possibilità di vedere il valore delle variabili globali, ma non potranno modificarlo.

Parametri opzionali

Oltre a chiamare funzioni passando i valori dei parametri in modo *posizionale*, ossia rispettando l'ordine con cui sono stati definiti i parametri all'interno della definizione, Python supporta anche il passaggio dei *valori per nome*.

```
nome_funzione(..., nome_parametro = valore_parametro)
```

```
...
5 #main()
6 somma = 0
7 prodotti = int(input("Inserisci il numero totale dei prodotti: "))
8 for i in range(prodotti):
9     prezzo = float(input("Inserisci il prezzo del prodotto: "))
10     somma = somma + prezzo
11 calcola_media_prezzi(somma, numero = prodotti)
```

Parametri di default

Un'altra alternativa consiste nell'assegnare dei valori di default direttamente all'interno della dichiarazione della funzione.

```
def nome_funzione(..., nome_parametro = valore_default):  
    ...
```

```
1 def calcola_media_prezzi(totale, numero = 3):  
2     media = totale / numero  
3     print(f"La media dei prezzi dei prodotti del giorno è: {media:7.2f}")  
4  
5 #main()  
6 somma = float(input("Inserisci il prezzo del primo prodotto: "))  
7 somma += float(input("Inserisci il prezzo del secondo prodotto: "))  
8 somma += float(input("Inserisci il prezzo del terzo prodotto: "))  
9 calcola_media_prezzi(somma)
```

Librerie

Una **libreria** o modulo è una raccolta di porzioni di codice correlate, rappresentate con le funzioni, che è utilizzabile per l'inclusione negli script realizzati dai programmatori così da non dover scrivere il codice da zero.

Una libreria è quindi una raccolta di funzioni nello stesso script con estensione `.py` che fanno riferimento a un medesimo argomento.

Python ha librerie built-in e librerie standard.

Librerie built-in

Python ha una libreria built-in, cioè una libreria presente al suo interno che contiene una serie di funzioni di carattere generale: questa libreria viene installata insieme all'interprete ed è utilizzata di default da Python, quindi non è necessario includerla negli script che scriviamo.

A abs() aiter() all() anext() any() ascii() B bin() bool() breakpoint() bytearray() bytes() C callable() chr() classmethod() compile() complex() D delattr() dict() dir() divmod()	E enumerate() eval() exec() F filter() float() format() frozenset() G getattr() globals() H hasattr() hash() help() hex() I id() input() int() isinstance() issubclass() iter()	L len() list() locals() M map() max() memoryview() min() N next() O object() oct() open() ord() P pow() print() property()	R range() repr() reversed() round() S set() setattr() slice() sorted() staticmethod() str() sum() super() T tuple() type() V vars() Z zip() - __import__()
---	---	--	---

Librerie standard

Python ha inoltre un elevato numero di librerie standard, che sono comode da utilizzare poiché contengono funzioni molto utili. Per usare queste librerie è sufficiente importarle all'interno degli script con un semplice comando:

```
import nome_libreria
```

Se si desidera importare da una libreria solo alcune funzioni o costanti, è possibile effettuare una importazione ridotta, ad esempio:

```
from math import sqrt, pi
```

E' possibile anche utilizzare un alias per il nome della libreria grazie al comando `as`:

```
import math as ma
```

e fare riferimento alle funzioni della libreria usando la formula `ma.nome_funzione`.

Librerie esterne

Esistono più di 500.000 librerie esterne di Python, orientate a diversi utilizzi e per varie applicazioni (inclusa la programmazione web), soprattutto quelle di Data Science e Machine Learning.

Esempi di librerie esterne:

- *Matplotlib*, che serve per realizzare grafici a due o più dimensioni a partire da set di dati forniti in input
- *Numpy* e *Pandas*, particolarmente utili nella Data Science

Per poter utilizzare le librerie esterne, queste devono essere installate.

Il programma da usare per installare pacchetti software nell'ambiente di Python si chiama *pip*.

Ogni libreria ha il proprio comando *pip* per l'installazione, ma in genere il comando è:

pip install nome_pacchetto

Questo comando viene lanciato dalla command line.

Docstrings

La documentazione del codice è importante perchè:

- il codice deve essere riutilizzabile e dunque è necessario che venga ben spiegato che cosa il codice fa e come lo fa;
- ci permette di riprenderlo successivamente, per esempio per un nuovo utilizzo o un aggiornamento, chiarendo le modalità che avevamo usato e le scelte che avevamo fatto nel corso del suo sviluppo.

Le docstrings servono per associare della documentazione a funzioni, moduli e altre porzioni di codice Python, con lo scopo di descrivere a che cosa servono e che cosa fanno.

Le docstrings vengono di solito usate in modo diverso rispetto ai commenti: le docstrings si usano per descrivere che cosa una determinata porzione di codice fa, mentre i commenti per descrivere come lo fa.

Per dichiarare una docstring si usano le triple virgolette (""") oppure i tripli apici singoli (''), che vengono inseriti subito dopo la dichiarazione di funzione o di modulo per spiegare il suo scopo.

Gestione delle eccezioni

Durante l'esecuzione può capitare che si verifichino delle situazioni inaspettate.

Per esempio possiamo incontrare una divisione per zero:

```
>>> 0 / 0
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    0 / 0
ZeroDivisionError: division by zero
>>>
```

In questo caso, l'interprete Python restituisce l'errore `ZeroDivisionError` e termina l'esecuzione del programma.

In Python gli errori vengono gestiti tramite le eccezioni.

Un'**eccezione** è l'occorrenza di una situazione anomala che altera il normale flusso di esecuzione del programma.

Try ... except

La gestione dell'eccezione avviene attraverso l'utilizzo del blocco try-except:

```
1 def calcola_media_prezzi(totale, numero):  
2     try:  
3         media = totale / numero  
4         print(f"La media dei prezzi dei prodotti del giorno è: {media:7.2f}")  
5     except ZeroDivisionError:  
6         print("Divisione per zero!")
```

L'istruzione try indica l'inizio del blocco di codice che potrebbe generare un'eccezione, mentre l'istruzione except associa a ogni eccezione il blocco di codice da eseguire per gestire quell'eccezione.

Try ... except

Il blocco try-except può essere scritto in diversi modi:

Try-except con...	Codice
...una sola eccezione	<pre>1 try: 2 ... 3 except Exception: 4 ...</pre>
...più eccezioni con una sola gestione	<pre>1 try: 2 ... 3 except (Exception1, Exception2, ...): 4 ...</pre>
...più eccezioni con diverse gestioni	<pre>1 try: 2 ... 3 except Exception1: 4 ... 5 except Exception2: 6 ...</pre>

Esempio: validazione input utente

Il blocco try-except ci è utile per controllare l'inserimento dei dati da parte dell'utente:

```
1 def inserisci_costo():
2     costo_corretto = False
3     while not costo_corretto:
4         try:
5             costo = float(input("Inserire costo: "))
6             costo_corretto = True
7             print("Inserimento corretto! Valore inserito: ", costo)
8         except ValueError:
9             print("Errore! Inserire un numero, ex: 0.10")
10
11 #main()
12 inserisci_costo()
```

Ciclo di vita di un progetto software

Per realizzare un progetto informatico è necessario seguire un processo che guida il suo sviluppo e la sua gestione, dal momento in cui viene concepito fino alla sua conclusione o dismissione.

Questo processo prende il nome di **ciclo di vita di un progetto software** e si suddivide in diverse fasi, ciascuna con specifici obiettivi e attività.

Le principali fasi del ciclo di vita di un progetto informatico sono:

1. raccolta e analisi dei requisiti;
2. pianificazione del progetto;
3. progettazione o design;
4. sviluppo;
5. test e verifica;
6. rilascio;
7. manutenzione e supporto;
8. conclusione o eventuale dismissione.

Modelli di sviluppo dei progetti informatici

- Il **modello a cascata** (waterfall) segue una sequenza lineare, in cui ogni fase viene completata prima di passare alla successiva. È ideale per progetti con requisiti stabili e ben definiti.
- Il **modello incrementale** suddivide il progetto in piccoli incrementi funzionali che vengono rilasciati e testati progressivamente.
- Il **modello agile** adotta un approccio iterativo e incrementale, con brevi cicli di sviluppo (*sprint*) che permettono di adattarsi ai cambiamenti nei requisiti in corso d'opera. È particolarmente indicato per progetti complessi e dinamici.
- Il **modello DevOps** integra lo sviluppo (*Dev*) e le operazioni (*Ops*) per garantire una collaborazione continua tra i team di sviluppo e quelli di gestione operativa, migliorando la velocità e l'efficienza del rilascio e della manutenzione del sistema.

Capitolo 5

Le strutture dati in Python

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume
informatica per progetti – Programmare in Python, di M. Fiore, A. Zanga, ISBN 9788808899477

Tipi di dati semplici

Tipo	Codice	Valore	Esempio
Valore booleano	bool	{False, True}	>>> soleggiato = True
Numero intero	int	Insieme Z	>>> prodotti = 42
Numero in virgola mobile	float	Insieme R	>>> sconto = 0.31
Numero complesso	complex	Insieme C	>>> fase = 1.23 + 3.50j

Tipi di dati complessi

Esistono tipi di dati che vengono definiti **complessi** perché raggruppano una struttura di dati in un'unica variabile: questi dati possono a loro volta essere semplici o complessi, dando origine a strutture anche molto articolate.

Un **tipo di dato complesso** è una struttura dati che può contenere più elementi, alcuni anche di tipo diverso.

In Python i tipi di dati complessi sono molto importanti e molto utilizzati e includono **liste, tuple, insiemi e dizionari**. Anche il tipo di dato **stringa** è un tipo complesso, ma è un po' particolare.

Stringhe

In Python non esiste il tipo di dato semplice “carattere”, ma esiste un tipo di dato complesso che si chiama **stringa** e che è molto utilizzato.

Una **stringa** è una sequenza di caratteri **immutabile**, che può contenere lettere, numeri, simboli e spazi.

Si possono definire utilizzando singoli apici (') o doppi apici (").

```
>>> parola = "computer"
```

```
>>> frase = "questa è una stringa"
```

Operatore []

Una stringa è una sequenza di caratteri ed è possibile accedere al singolo carattere attraverso l'operatore parentesi quadre [].

```
>>> lettera_singola = parola[1]
>>> lettera_singola
'o'
```

Le posizioni dei caratteri nelle stringhe partono da zero e non da uno. Se rappresentiamo la stringa *parola* indicando le posizioni dei suoi caratteri, abbiamo:

parola	c	o	m	p	u	t	e	r
posizione	0	1	2	3	4	5	6	7

L'operatore [] è molto utile perché ci permette di accedere al singolo carattere della stringa conoscendo la sua posizione.

Indice

La posizione del carattere all'interno della stringa è indicata dall'**indice**, che è un numero intero che parte da zero e arriva fino alla *lunghezza della stringa - 1*.

L'**indice** è un numero intero che indica la posizione di un carattere all'interno della sequenza.

L'indice può essere anche *una variabile o un'espressione*:

```
>>> i = 2
>>> parola[i]
'm'
>>> parola[i + 1]
'p'
```

L'indice può essere anche un *numero negativo* e in questo caso indica la corrispondente posizione partendo dalla fine della stringa:

```
>>> parola[-1]
'r'
>>> parola[-2]
'e'
```

Lunghezza di una stringa

La funzione `len()` è una funzione predefinita che ci restituisce la lunghezza di una stringa, cioè il numero di caratteri che la stringa contiene:

```
>>> frase = "questa è una stringa"
```

```
>>> len(frase)
20
```

Utilizzando in modo combinato l'indice e la funzione `len` siamo in grado di estrarre tutti i caratteri che compongono la stringa con un semplice ciclo di `for`.

```
1 parola = "computer"
2 for i in range(len(parola)):
3     print(parola[i])
```

Per scorrere tutti i caratteri della stringa possiamo usare anche un'altra forma del ciclo `for`, che usa l'operatore `in`:

```
1 for lettera in parola:
2     print(lettera, end = " ")
```

Slicing

Uno **slice** è una porzione di una stringa, o sottostringa, contenuta tra due indici. È possibile selezionare porzioni di stringhe con un'operazione definita slicing.

```
>>> stringa = "Linguaggio Python"
>>> stringa[0:6]
'Lingua'
```

L'istruzione `stringa[0:6]` ci permette di estrarre i caratteri che vanno dall'indice 0 (compreso) all'indice 6 (escluso)
Se vogliamo estrarre la sottostringa finale:

```
>>> stringa[11:17]
'Python'
```

Casi d'uso slicing

stringa	L	i	n	g	u	a	g	g	i	o		P	y	t	h	o	n
indice	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Slicing	Sottostringa	Spiegazione
<code>>>> stringa[0:6]</code>	'Lingua '	porzione di stringa dall'indice 0 incluso all'indice 6 escluso
<code>>>> stringa[:6]</code>	'Lingua '	porzione di stringa dall'inizio all'indice 6 escluso
<code>>>> stringa[11:17]</code>	'Python '	porzione di stringa dall'indice 11 incluso all'indice 17 escluso
<code>>>> stringa[11:]</code>	'Python '	porzione di stringa dall'indice 11 incluso al termine della stringa
<code>>>> stringa[4:10]</code>	'aggio'	porzione di stringa dall'indice 4 incluso all'indice 10 escluso
<code>>>> stringa[4:4]</code>	''	stringa vuota, quando il primo indice è maggiore o uguale al secondo
<code>>>> stringa[6:3]</code>	''	stringa vuota, quando il primo indice è maggiore o uguale al secondo
<code>>>> stringa[:]</code>	'Linguaggio Python '	tutta la stringa

Concatenazione di stringhe

Una stringa è **immutabile**: ciò vuol dire che non è possibile cambiare una stringa già dichiarata e usata.

Se vogliamo modificare una stringa la cosa migliore è crearne una nuova, per esempio come **concatenazione** di due stringhe.

La concatenazione è l'operazione di unione di stringhe, si esprime tramite il simbolo + e fornisce come risultato una stringa.

```
>>> nome = 'Mario'
>>> cognome = 'Rossi'
>>> user = nome + cognome
>>> user
'MarioRossi'
```


Metodi e funzioni per le stringhe - 1

Operatore/ Funzione/Metodo	Utilizzo	Esempio
<code>s1 + s2</code>	concatenazione	<pre>>>> s1 = 'Oggi' >>> s2 = 'piove' >>> s1 + s2 'Oggipiove'</pre>
<code>s1 * n</code>	ripetizione della stringa <code>s1</code> per <code>n</code> volte	<pre>>>> s1 = 'Oggi' >>> s1 * 2 'OggiOggi'</pre>
<code>s1 == s2</code>	confronta <code>s1</code> con <code>s2</code> : se sono uguali restituisce <code>True</code> , altrimenti restituisce <code>False</code>	<pre>>>> s1 = 'Oggi' >>> s2 = 'piove' >>> s1 == s2 False</pre>
<code>s1 < s2</code> <code>s1 > s2</code>	confronta <code>s1</code> con <code>s2</code> : se <code>s1</code> viene prima (o dopo) di <code>s2</code> in ordine alfabetico restituisce <code>True</code> , altrimenti <code>False</code> .	<pre>>>> s1 = 'Oggi' >>> s2 = 'piove' >>> s1 < s2 True</pre>

Metodi e funzioni per le stringhe - 2

<code>s in s1</code>	controlla se <code>s</code> è presente nella stringa <code>s1</code> : in caso positivo restituisce <code>True</code> , altrimenti restituisce <code>False</code>	<pre>>>> s1 = 'Oggi' >>> 'g' in s1 True >>> 'M' in s1 False</pre>
<code>s1.upper()</code>	mette <code>s1</code> in maiuscolo	<pre>>>> s1 = 'Oggi' >>> s1.upper() 'OGGI'</pre>
<code>s1.lower()</code>	mette <code>s1</code> in minuscolo	<pre>>>> s1 = 'Oggi' >>> s1.lower() 'oggi'</pre>
<code>s2.capitalize()</code>	mette l'iniziale di <code>s2</code> in maiuscolo	<pre>>>> s2 = 'piove' >>> s2.capitalize() 'Piove'</pre>
<code>s.strip()</code> <code>s.strip(sottostr)</code>	elimina i caratteri di spaziatura inseriti all'inizio e alla fine della stringa (o della sottostringa)	<pre>>>> nome = ' Mario ' >>> nome.strip() 'Mario'</pre>
<code>s.count(sottostr)</code>	conta quante volte la sottostringa si ripete nella stringa	<pre>>>> frase = 'Amo la mamma' >>> frase.count('ma') 2</pre>

Vettori o array

In Python **non** esiste un tipo di dato *vettore*.

Un **vettore**, o **array**, è una struttura dati di dimensione fissa, che permette di memorizzare dati tutti dello stesso tipo semplice, identificati da un indice.

Le *caratteristiche* del vettore sono:

1. lunghezza fissa;
2. elementi tutti dello stesso tipo semplice;
3. indice per identificare la posizione dell'elemento nel vettore.

```
>>> array_interi = [4, 18, 23, 2, 145]
>>> array_interi
[4, 18, 23, 2, 145]
```

La *dichiarazione* di un vettore è piuttosto semplice:

1. il vettore è racchiuso tra parentesi quadre [];
2. gli elementi sono separati da virgole “,”;
3. gli elementi sono tutti dello stesso tipo.

Indice del vettore

Ogni elemento del vettore ha una posizione che è indicata dall'**indice**, che come per le stringhe parte dal valore 0.

array_interi	4	18	23	2	145
indice	0	1	2	3	4

E' possibile usare la funzione `len()` per trovare la lunghezza del vettore, che è fissata nel momento della sua dichiarazione.

```
>>> len(array_interi)
5
>>> array_interi[1]
18
```

Utilizzando in modo combinato l'indice e la funzione `len()` è possibile estrarre tutti gli elementi del vettore con un ciclo `for`.

```
1 array_interi = [4, 18, 23, 2, 145]
2 for i in range(len(array_interi)):
3     print(array_interi[i], end = " ")
4 print()
5 for intero in array_interi:
6     print(intero, end = " ")
```

Creare un vettore da valori in input

```
1  #Carica un array di 5 elementi interi
2  #dichiaro il vettore di 5 elementi tutti 0
3  array = [0, 0, 0, 0, 0]
4  for i in range(len(array)):
5      #controllo che l'input sia intero
6      err_intero = False
7      while not err_intero:
8          try:
9              array[i] = int(input("Inserisci un numero intero: "))
10             err_intero = True
11         except ValueError:
12             print("Inserire un numero intero!")
13 print("Array: ", array)
```

Creare un vettore di dimensione variabile

```
1  #Definisci un array di dimensione chiesta in input e caricalo con interi
2  #chiedo la dimensione e controllo sia un numero intero
3  err_int = False
4  while not err_int:
5      try:
6          dim = int(input("Inserisci la dimensione del vettore: "))
7          err_int = True
8      except ValueError:
9          print("Inserire un numero intero!")
10 #creo il vettore contenente tutti 0
11 array = [0] * dim
12 #carico i valori nel vettore
13 for i in range(len(array)):
14     #controllo che l'input sia intero
15     err_intero = False
16     while not err_intero:
17         try:
18             array[i] = int(input("Inserisci un numero intero: "))
19             err_intero = True
20         except ValueError:
21             print("Inserire un numero intero!")
22 print("Array: ", array)
```

Ricerca di un valore in un vettore

Per cercare un elemento in un vettore dobbiamo utilizzare un algoritmo che determina se l'elemento appartiene al vettore e restituisce il suo indice per poter identificare la sua posizione.

L'algoritmo più semplice che svolge questa funzione è l'algoritmo di *ricerca lineare*, che scorre gli elementi del vettore nel loro ordine e si ferma quando incontra l'elemento cercato.

Nel caso peggiore, cioè quello in cui l'elemento cercato si trova in fondo al vettore, l'algoritmo deve scorrere tutto il vettore.

L'algoritmo di ricerca lineare si chiama così perché è *lineare* il tempo di esecuzione per svolgere la sua ricerca.

```
1  #ricerca lineare in un vettore
2  def ricerca_array(array, elemento):
3      trovato = False
4      indice = -1
5      for i in range(len(array)):
6          if array[i] == elemento:
7              trovato = True
8              indice = i
9              break
10     return indice, trovato
11
```

Ordinamento di un vettore

La ricerca lineare di un elemento in un vettore scorre tutto il vettore per cercare l'elemento che cerchiamo: se il vettore fosse ordinato, la ricerca potrebbe fermarsi non appena abbiamo superato nell'ordine l'elemento da cercare.

Il tempo di esecuzione del nostro algoritmo di ricerca potrebbe diminuire (*potrebbe* diminuire, perché se l'elemento che cerchiamo si trova in fondo, l'algoritmo deve comunque scorrere tutto il vettore).

Un **algoritmo di ordinamento di un vettore** viene utilizzato per posizionare gli elementi di un insieme secondo una sequenza stabilita, in modo che ogni elemento sia minore o maggiore di quello che lo segue.

Esistono diversi algoritmi di ordinamento dei vettori. I due più semplici sono:

1. Selection Sort;
2. Bubble Sort.

Selection Sort

- L'algoritmo **Selection Sort** confronta il primo elemento del vettore con il secondo, e se è minore (o maggiore, a seconda di come vogliamo ordinare il vettore) lo scambia di posto.
- L'algoritmo continua in questo modo fino alla fine del vettore, poi passa a considerare il secondo elemento, che verrà confrontato con il terzo, e così via.
- Al termine di queste operazioni di confronto e di eventuale scambio, il vettore sarà ordinato.
- Si dice che l'algoritmo effettua gli scambi *in place*, poiché gli scambi degli elementi vengono fatti direttamente sul vettore, per cui al termine delle operazioni il vettore iniziale non ci sarà più, ma avremo il vettore con gli elementi ordinati.
- A quel punto potremo salvare il vettore ordinato e restituirlo dalla funzione in una nuova variabile, come *vettore_ordinato*.

```
1  #Ordinamento array con Selection Sort
2  def selection_sort(array):
3      for i in range(len(array)):
4          for j in range(i+1, len(array)):
5              if array[j] < array[i]:
6                  temp = array[i]
7                  array[i] = array[j]
8                  array[j] = temp
9      return array
```

Bubble Sort

- L'algoritmo Bubble Sort confronta gli elementi a coppie: il primo elemento del vettore viene confrontato con il secondo e, se è minore (o maggiore, a seconda di come si vuole ordinare il vettore), lo scambia di posto.
- L'algoritmo continua in questo modo, confrontando il secondo con il terzo e scambiandoli se necessario, e così via fino alla fine del vettore.
- Al termine di queste operazioni di confronto e degli eventuali scambi, il vettore sarà ordinato.
- Anche l'algoritmo Bubble Sort effettua gli scambi *in place* per cui al termine delle operazioni il vettore iniziale non ci sarà più ma avremo il vettore con gli elementi ordinati.
- A quel punto potremo salvare il vettore ordinato e restituito dalla funzione in una nuova variabile, come *vettore_ordinato*.

Bubble Sort

```
1  #Ordinamento array con Bubble Sort
2  def bubble_sort(array):
3      #flag indica quando fare gli scambi
4      flag = 1
5      #stop indica la penultima posizione dell'array
6      stop = len(array) - 1
7      while flag == 1:
8          #flag = 0 consente di uscire dal ciclo while
9          flag = 0
10         for i in range(stop):
11             if array[i] > array[i+1]:
12                 temp = array[i]
13                 array[i] = array[i+1]
14                 array[i+1] = temp
15                 #continuo a fare scambi
16                 flag = 1
17             #a ogni passaggio escludo l'ultimo elemento dell'array
18             stop = stop - 1
19     return array
```

Liste

In Python il tipo di dato vettore non esiste e il modo in cui è implementato nel linguaggio è attraverso la struttura dati complessa della **lista**.

Una **lista** in Python è una struttura di dati complessa, costituita da una sequenza di valori (anche di tipo diverso e a loro volta complessi), che non ha una lunghezza fissa e i cui elementi sono caratterizzati dall'indice che ne indica la posizione all'interno della sequenza.

Una lista può essere dichiarata in Python in diversi modi, usando sempre l'operatore parentesi quadre `[]` e separando gli elementi con la virgola.

```
>>> lista_interi = [4, 18, 23, 2, 145]
>>> lista_interi
[4, 18, 23, 2, 145]
```

Può anche essere dichiarata una lista vuota

```
>>> lista_vuota = []
>>> lista_vuota
[]
```

Caratteristiche delle liste - 1

Per scorrere gli elementi di una lista possiamo utilizzare l'indice che indica la posizione dell'elemento nella lista; il primo elemento ha indice zero, e l'ultimo ha indice pari a `len(lista)`

In Python, la lista è *l'implementazione della struttura vettore*.

Ma la lista, in Python, ha anche altre caratteristiche che i vettori non hanno:

1. una dimensione che può variare dinamicamente;
2. la possibilità di contenere al suo interno elementi di tipo diverso, anche di tipo complesso.

```
>>> lista_prodotti = ["latte", 1.30, "pane", 3.2, "gelato", 1.5]
>>> type(lista_prodotti)
<class 'list'>
```

Caratteristiche delle liste - 2

Le liste, al contrario delle stringhe, sono mutabili: utilizzando l'operatore `[ ]` possiamo accedere all'elemento della lista e fare un'assegnazione.

```
>>> lista_interi = [4, 18, 23, 2, 145]
>>> lista_interi[2] = 12
>>> lista_interi
[4, 18, 12, 2, 145]
```

Le operazioni di concatenazione, iterazione e di slicing valgono anche per le liste, così come i metodi per scorrere una lista (for con l'uso dell'operatore in).

Dimensione dinamica – aggiungere elementi

La dimensione di una lista può variare dinamicamente: è possibile creare una lista partendo dalla lista vuota e «appendendo» uno alla volta i nuovi elementi, che verranno inseriti nella lista dopo l'ultimo elemento presente.

```
>>> lista_vuota = []
>>> lista_vuota.append(18)
>>> lista_vuota
[18]
>>> lista_vuota.append(5)
>>> lista_vuota
[18, 5]
>>> lista_vuota.append(14)
>>> lista_vuota
[18, 5, 14]
```

Il metodo `append()` permette di aggiungere un elemento alla fine della lista.

Dimensione dinamica – eliminare elementi

Primo metodo: usare `pop(indice)`. In questo caso abbiamo bisogno di conoscere l'indice dell'elemento che vogliamo eliminare, a meno che non vogliamo eliminare l'ultimo elemento e in questo caso è sufficiente scrivere `pop()`. Il metodo `pop()` restituisce l'elemento rimosso dalla lista.

```
>>> lista_interi = [4, 18, 23, 2, 145]
>>> lista_interi.pop(1)
18
>>> lista_interi
[4, 23, 2, 145]
```

Secondo metodo: usare `remove(elemento)`. Questo metodo si può usare quando conosciamo qual è l'elemento da eliminare, ma per esempio non il suo indice. Il metodo `remove()` restituisce `None`.

```
>>> lista_interi = [4, 18, 23, 2, 145]
>>> lista_interi.remove(18)
>>> lista_interi
[4, 23, 2, 145]
```

Terzo metodo: usare l'operatore `del`. In questo caso bisogna indicare con precisione quale elemento eliminare, usando l'operatore `[ ]`. L'operatore `del` non restituisce l'elemento rimosso.

```
>>> lista_interi = [4, 18, 23, 2, 145]
>>> del lista_interi[1]
>>> lista_interi
[4, 23, 2, 145]
```


Liste parallele

Quando abbiamo più informazioni che si riferiscono allo stesso oggetto, possiamo usare le liste parallele.

Ad esempio se abbiamo più prodotti con la loro descrizione e il loro prezzo possiamo usare 2 liste parallele:

`lista_prodotti`

elemento	scarpe	calze	maglia
indice	0	1	2

`lista_prezzi`

elemento	25.90	2.5	15
indice	0	1	2

Allo stesso indice nelle due liste corrisponde logicamente lo stesso dato. Il prezzo del prodotto “scarpe” è 25.90 poiché entrambi gli elementi si trovano allo stesso indice 0 delle due diverse liste parallele.

Metodi e funzioni per le liste

Operatore/Funzione/Metodo	Utilizzo
<code>l1 + l2</code>	concatenazione di liste
<code>l1 * n</code>	ripetizione della lista l1 per <i>n</i> volte
<code>l1 == l2</code>	confronta la lista l1 con la lista l2: se sono uguali restituisce True, altrimenti restituisce False
<code>l1 = l2</code>	l1 e l2 sono la stessa lista, quindi le operazioni effettuate su l1 sono effettuate anche su l2
<code>sum(l1)</code>	restituisce la somma degli elementi della lista l1
<code>max(l1) ; min(l1)</code>	restituiscono rispettivamente il valore massimo e il valore minimo della lista l1
<code>l1.sort()</code>	ordina la lista l1 <i>in place</i>
<code>l2 = sorted(l1)</code>	restituisce la nuova lista l2 che è la lista l1 ordinata; l1 non si modifica
<code>l1.append(elem)</code>	inserisce elem alla fine della lista l1
<code>l1.pop()</code>	elimina l'ultimo elemento della lista l1
<code>l1.pop(ind)</code>	elimina l'elemento di indice ind della lista l1
<code>l1.index(elem)</code>	restituisce l'indice dell'elemento elem della lista l1
<code>l1.insert(indice, elem)</code>	inserisce elem in posizione indice
<code>l1.insert(0,elem)</code>	inserisce elem all'inizio della lista l1

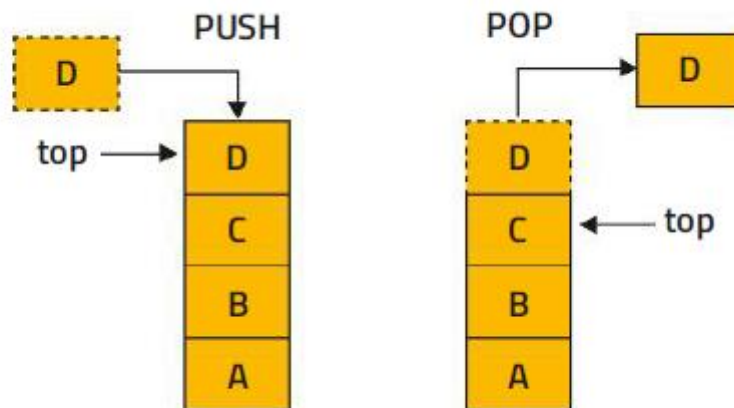
Stack

Uno **stack** (o **pila**) è un tipo di dato astratto di genere LIFO (*Last In First Out*), che permette l'inserimento e l'estrazione di valori solo dalla cima; per questo viene chiamata pila (come una pila di piatti).

Consente solo due tipi di operazioni:

1. push: per inserire un elemento in cima alla pila;
2. pop: per estrarre un elemento dalla cima della pila.

STACK



Stack e lista

Lo stack viene implementato in Python usando una lista.

E' possibile definire le due operazioni `push` e `pop` facendo inserimenti ed estrazioni solo dal fondo della lista:

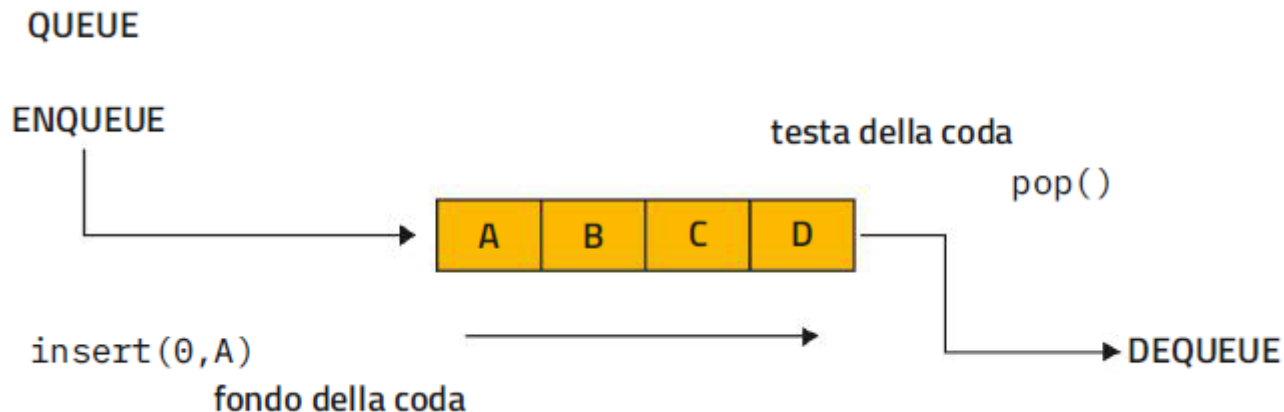
- usando il metodo `append()` per inserire un elemento alla fine
- usando il metodo `pop()` che permette di estrarre l'ultimo elemento della lista

Queue

Una **queue (o coda)** è un tipo di dato astratto di genere FIFO (*First In First Out*), che permette la gestione di una sequenza di valori aggiungendo entità a un estremo e rimuovendole dall'altro estremo, come in una coda di persone in attesa di fare il biglietto per assistere a un concerto.

Le operazioni possibili per una coda sono solo due:

1. enqueue, per inserire un elemento in fondo alla coda;
2. dequeue, per estrarre l'elemento in testa alla coda.



Queue e lista

Anche la queue viene implementata in Python utilizzando le liste.

Le due uniche operazioni possibili, `enqueue` e `dequeue`, vengono realizzate attraverso:

- il metodo `insert(0, elemento)`, che consente l'inserimento in posizione di indice 0 (quindi all'inizio della lista),
- il metodo `pop()`, che consente di estrarre l'ultimo elemento della lista

Tuple

Una **tupla** in Python è una sequenza di valori, anche di tipo diverso, che non ha una lunghezza fissa e i cui elementi sono caratterizzati dall'indice che ne indica la posizione nella sequenza.

La tupla rappresenta una sequenza di valori spesso racchiusa tra parentesi tonde () e separati dalla virgola.

La tupla è **immutabile** (come la stringa) quindi, una volta che una tupla è stata già dichiarata, non è possibile fare un'assegnazione a uno dei suoi elementi.

Per questa sua caratteristica, in genere si utilizza la tupla per rappresentare dati che non cambiano, come ad esempio i nomi dei giorni della settimana.

```
>>> giorni_feriali = ('lunedì', 'martedì', 'mercoledì', 'giovedì', 'venerdì', 'sabato')
>>> type(giorni_feriali)
<class 'tuple'>
```

Uso delle tuple

Per creare una tupla con un solo elemento è necessario aggiungere la virgola dopo l'elemento, altrimenti la tupla non viene riconosciuta come tale, ma come tipo dell'elemento inserito.

Se vogliamo scambiare tra loro i valori di due variabili `var1` e `var2`, possiamo usare le tuple. A sinistra dell'uguale abbiamo una tupla di variabili e a destra una tupla di espressioni. Per effettuare correttamente questa assegnazione è necessario che il numero delle variabili a sinistra sia uguale al numero delle espressioni a destra dell'assegnazione.

```
>>> tupla_stringa = ('sole')
>>> type(tupla_stringa)
<class 'str'>
>>> tupla_stringa = ('sole',)
>>> type(tupla_stringa)
<class 'tuple'>
>>> tupla_int = (8)
>>> type(tupla_int)
<class 'int'>
>>> tupla_int = (8,)
>>> type(tupla_int)
<class 'tuple'>
```

```
>>> var1, var2 = var2, var1
```


Insiemi

Vettori, liste e tuple sono collezioni *sequenziali*, in cui ogni elemento ha una precisa posizione all'interno della collezione, al contrario di quanto succede per gli insiemi.

Un **insieme** è una struttura di dati non sequenziale che contiene elementi non ripetuti.

Gli elementi di un insieme non hanno un ordine e la struttura è mutabile, permettendo di aggiungere e togliere elementi in base alla necessità.

In Python possiamo costruire un insieme con la funzione `set`.

```
>>> A = ["banane", "mele", "arance", "banane"] #Lista
>>> A = set(A) #Costruzione dell'insieme da una lista
>>> A
{"arance", "banane", "mele"}
```

Costruzione di insiemi

Possiamo costruire un insieme usando una coppia di parentesi graffe { } nello stesso modo con cui usiamo le parentesi quadre [] per la costruzione di una lista.

Una volta costruito l'insieme possiamo conoscerne la lunghezza con la funzione `len`.

```
>>> A = {"banane", "mele", "arance", "banane"} #Costruzione di un insieme
>>> len(A)                                     #Numero di elementi
3
>>> len(A) > 0                                #L'insieme contiene elementi?
True
>>> bool(A)                                    #Equivalente al precedente
True
>>> if A:                                       #Funziona anche con l'if
>>>     print("L'insieme non è vuoto")
>>> else:
>>>     print("L'insieme è vuoto")
L'insieme non è vuoto
```

Accedere agli elementi di un insieme

Dato che un insieme non è una collezione sequenziale, non possiamo accedere agli elementi tramite un indice, ma possiamo controllare la loro esistenza con l'operatore `in`:

```
>>> A = {"banane", "mele", "arance", "banane"} #Costruzione di un insieme
>>> "mele" in A
True
```

Oppure scorrere la collezione con un ciclo `for`:

```
>>> A = {"banane", "mele", "arance", "banane"}
>>> for a in A:
>>>     print(a)
mele
arance
banane
```

Metodi per gli insiemi

- Aggiungere un elemento:
- Aggiungere più elementi:
- Rimuovere un elemento (l'elemento deve essere contenuto, altrimenti viene sollevata una eccezione che deve essere gestita con try...except...):
- Rimuovere un elemento (può anche non essere contenuto):

```
>>> A = {1, 2, 3}
>>> A.add(4)
>>> A
{1, 2, 3, 4}
```

```
>>> A = {1, 2, 3}
>>> A.update([4, 5, 6])
>>> A
{1, 2, 3, 4, 5, 6}
```

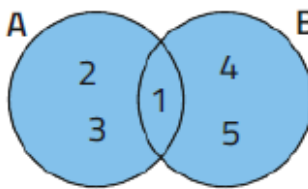
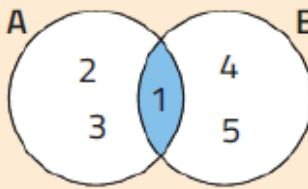
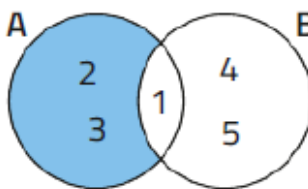
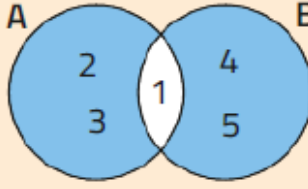
```
>>> A = {1, 2, 3}
>>> A.remove(2)
>>> A
{1, 3}
```

```
>>> A = {1, 2, 3}
>>> A.discard(2)
>>> A
{1, 3}

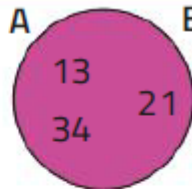
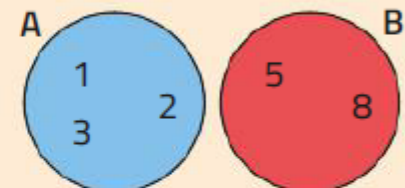
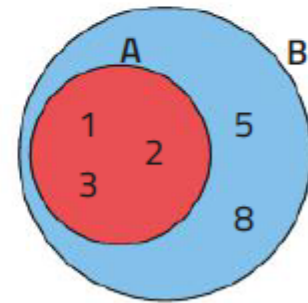
>>> A.discard(5)
>>> A
{1, 3}
```

Operazioni tra insiemi

Dati due insiemi $A = \{1, 2, 3\}$ e $B = \{1, 4, 5\}$ possiamo eseguire diverse operazioni di comparazione degli elementi:

Operazione	Codice	Diagramma di Eulero-Venn
Unione di A e B	<pre>>>> A.union(B) #Metodo {1, 2, 3, 4, 5} >>> A B #Operatore {1, 2, 3, 4, 5}</pre>	
Intersezione di A e B	<pre>>>> A.intersection(B) {1} >>> A & B {1}</pre>	
Differenza di A e B	<pre>>>> A.difference(B) {2, 3} >>> A - B {2, 3}</pre>	
Differenza simmetrica di A e B	<pre>>>> A.symmetric_difference(B) {2, 3, 4, 5} >>> A ^ B {2, 3, 4, 5}</pre>	

Relazioni tra insiemi

Operazione	Codice	Diagramma di Eulero-Venn
Coincidenza	<pre>>>> A = {13, 21, 34} >>> B = {13, 21, 34} >>> A == B True</pre>	
Disgiunzione	<pre>>>> A = {1, 2, 3} >>> B = {5, 8} >>> A.isdisjoint(B) True</pre>	
Sottoinsieme e sovrainsieme	<pre>>>> A = {1, 2, 3} >>> B = {1, 2, 3, 5, 8} >>> A.issubset(B) #Metodo True >>> A <= B #Operatore True >>> A.issuperset(B) #Metodo False >>> A >= B #Operatore False</pre>	

Dizionari

Un **dizionario** è una struttura dati formata da coppie chiave-valore in cui ogni chiave identifica univocamente il valore associato.

Possiamo costruire un dizionario con la funzione `dict`.

```
>>> sconto = dict([                                #Sconto per ogni categoria
>>>     ("magliette", 0.20),                        #Coppia chiave - valore,
>>>     ("scarpe", 0.10)                           #come una tupla di due elementi
>>> ])
>>> sconto
{'magliette': 0.2, 'scarpe': 0.1}
```

La rappresentazione più efficace di un dizionario è quella di una tabella a due colonne:

l'interprete Python cerca nella prima colonna la chiave e restituisce il valore della riga corrispondente.

Chiave	Valore
magliette	0.20
scarpe	0.10

Costruzione di un dizionario

Per la costruzione di un dizionario utilizziamo le parentesi graffe, come per gli insiemi, ma separiamo chiave e valore con i due punti.

```
>>> sconto = {                                     #Sconto per ogni categoria.
>>>     "magliette": 0.20,                         #Coppia chiave - valore,
>>>     "scarpe": 0.10                             #separate dai due punti.
>>> }
>>> sconto
{'magliette': 0.2, 'scarpe': 0.1}
```

Bisogna fare attenzione al caso del **dizionario vuoto**, in cui la costruzione di un dizionario può sembrare un insieme vuoto (che si crea invece con **set**):

```
>>> { }                                             #Costruzione di un insieme
>>> { }                                             #Insieme vuoto?
>>> type({ })                                     #Controlliamo con type ...
dict                                              #No! Dizionario vuoto!
```


Accedere agli elementi del dizionario

Per accedere a un elemento è sufficiente utilizzare la sua chiave (se la chiave non è presente all'interno del dizionario, allora viene sollevata un'eccezione):

```
>>> sconto = {"magliette": 0.20, "scarpe": 0.10}
>>> sconto["magliette"]
0.20
```

L'operatore `in` ci permette di controllare se la chiave è presente nel dizionario prima di tentare l'accesso:

```
>>> sconto = {"magliette": 0.20, "scarpe": 0.10}
>>> chiave = "cappelli"
>>> if chiave in sconto:
>>>     print(sconto[chiave])
>>> else:
>>>     print(f"Errore! la chiave '{chiave}' non esiste!")
Errore! La chiave 'cappelli' non esiste!
```

Aggiungere/rimuovere elementi al dizionario

Se una chiave non esiste, è possibile aggiungerla al dizionario e assegnarle un valore:

```
>>> sconto = {"magliette": 0.20, "scarpe": 0.10}
>>> sconto["cappelli"] = 0.15
>>> sconto
{'magliette': 0.2, 'scarpe': 0.1, 'cappelli': 0.15}
```

è anche possibile aggiornare il valore di una chiave già esistente:

```
>>> sconto = {"magliette": 0.20, "scarpe": 0.10}
>>> sconto["magliette"] = 0.15
>>> sconto
{'magliette': 0.15, 'scarpe': 0.1}
```

la rimozione di un elemento da un dizionario utilizza la keyword `del`, come per le liste:

```
>>> sconto = {"magliette": 0.20, "scarpe": 0.10}
>>> del sconto["magliette"]
>>> sconto
{'scarpe': 0.1}
```

Iterazioni sui dizionari

Iterazione per...	Codice
... chiave-valore	<pre>>>> for (chiave, valore) in sconto.items() >>> print(f"{chiave} -> {valore}") magliette -> 0.2 scarpe -> 0.1</pre>
... chiavi	<pre>>>> for chiave in sconto.keys() #Il metodo è opzionale >>> print(chiave) magliette scarpe</pre>
... valori	<pre>>>> for valore in sconto.values() >>> print(valore) 0.2 0.1</pre>

Metodi per i dizionari

- Accedere ai valori di un dizionario: metodo `get`

```
>>> sconto = {"magliette": 0.20, "scarpe": 0.10}
>>> sconto.get("cappelli")           #Se la chiave non esiste, non restituisce nulla
```

- Aggiornare un dizionario con un altro, sovrascrivendo i valori delle chiavi esistenti o aggiungendo quelle mancanti: metodo `update`

```
>>> sconto = {"magliette": 0.20, "scarpe": 0.10}
>>> sconto.update({"collane": 0.15, "magliette": 0.60})
>>> sconto
{'magliette': 0.6, 'scarpe': 0.1, 'collane': 0.15}
```

- Rimuovere elementi dal dizionario:
 - Metodo `pop`: occorre specificare le chiavi dell'elemento da rimuovere
 - Metodo `clear`: rimuove tutti gli elementi da un dizionario

Archivi o file

Un **file**, o **archivio**, è una collezione di informazioni salvata in modo persistente in un computer.

Un file è identificato univocamente da una stringa, detta *percorso*, che è composta da tre parti:

1. la cartella che contiene il file;
2. il nome del file;
3. l'estensione del file.

C:\Users\nome_utente\nome_file.txt

Estensione	Descrizione
.txt	File che contiene testo libero e stringhe, senza una struttura ben definita.
.csv	<i>Comma-Separated Values</i> , file di testo che contiene informazioni strutturate, simili a tabelle, in cui le celle sono separate da virgole.

Operare con i file

Dato che un file è salvato su un tipo di memoria diversa da quella su cui eseguiamo il programma, **è necessario trasferire i dati dal file al programma e viceversa.**

Per fare questo ci serviremo di alcuni punti di riferimento che ci indicano lo stato del trasferimento.

Riferimento	Descrizione
Puntatore	Rappresenta la posizione corrente nel file.
Fine della riga - End Of Line (EOL)	Segnala la fine della riga corrente, rappresentata dalla stringa <code>\n</code> , dove «n» sta per <i>newline</i> , «nuova riga».
Fine del file - End Of File (EOF)	Segnala la fine del file, rappresentata dalla stringa vuota <code>""</code> .

I file hanno un inizio e una fine, e quando apriamo un file per leggere dei dati e caricarli nel programma, Python restituisce un *puntatore* all'inizio del file, che può essere utilizzato per scorrerlo fino alla fine. Se il file contiene del testo, allora ogni riga di testo è segnata da un *fine riga*, che può essere particolarmente utile se vogliamo caricare le informazioni riga per riga.

Operazioni sui file

Oltre a **leggere** un file, possiamo anche **scrivere** dei dati su un file. Python supporta due modalità di scrittura:

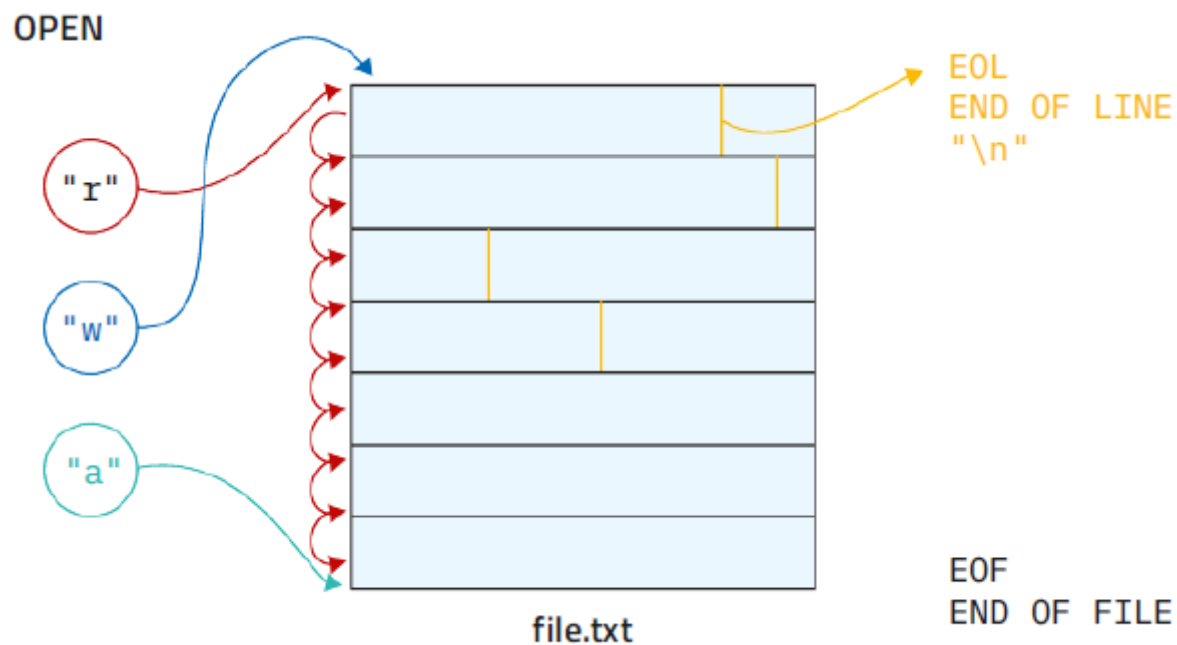
- *aggiunta*, con cui possiamo inserire i dati alla fine del file;
- *scrittura*, con cui sovrascriviamo un file esistente con dei nuovi dati.

Modalità di apertura	Descrizione
"r" (read)	Apre in sola lettura. Se il file non esiste, solleva un'eccezione.
"a" (append)	Apre in «aggiunta». Se il file non esiste, lo crea.
"w" (write)	Apre in scrittura. Se il file non esiste, lo crea. Se il file esiste, lo sovrascrive.

Python fornisce la funzione **open** che, dati il percorso del file e la modalità di accesso, restituisce il puntatore al file con cui effettuare le operazioni di lettura e/o scrittura. Una volta terminate le operazioni, possiamo chiudere il file invocando il metodo **close** sul puntatore.

```
1 | p = open("file.txt", "r")      #Apertura di un file in modalità lettura
2 | ...                            #Operazioni sui file
3 | p.close()                      #Chiusura del file
```

Modalità di accesso a un file



Lettura di un file una riga alla volta

Per leggere un file possiamo caricare una riga alla volta con il ciclo `while` e il metodo `readline`:

```
1 p = open("file.txt", "r")      # Apertura del file in lettura
2 riga = p.readline()            # Lettura della prima riga
3 while riga != "":              # Riga vuota? Fine del file?
4     print(riga)                 # Visualizzazione della riga
5     riga = p.readline()         # Lettura riga successiva
6 p.close()                      # Chiusura del file
```

Mentre il ciclo `while` ci dà il controllo sulla lettura del file, il ciclo `for` automatizza il processo di lettura e controllo della fine della riga.

```
1 p = open("file.txt", "r")      #Apertura del file in lettura
2 for riga in p:                 #Iterazione sulle righe
3     print(riga)                 #Visualizzazione della riga
4 p.close()                      #Chiusura del file
```

Lettura di un file più righe insieme

È possibile leggere un file e salvarne il contenuto in una lista d'appoggio con il metodo `readlines`:

```
1 p = open("file.txt", "r")      #Apertura del file in lettura
2 testo = p.readlines()          #Lettura dell'intero file
3 print(testo)                   #Visualizzazione del testo
4 p.close()                      #Chiusura del file
```

Scrittura di un file

Possiamo anche scrivere il contenuto di una lista in un file

```
1  #Testo da scrivere, con EOL per ogni riga
2  testo = [
3      "Nel mezzo del cammin di nostra vita\n",
4      "mi ritrovai per una selva oscura,\n",
5      "ché la diritta via era smarrita.\n"
6  ]
7  p = open("inferno.txt", "w")      #Apertura del file in scrittura
8  for riga in testo:                #Iterazione sulle righe del testo
9      p.write(riga)                  #Scrittura delle righe
10 p.close()                         #Chiusura del file
```

Aprire un file in scrittura ha due effetti collaterali che dobbiamo sempre tenere presenti:

- Se il file non esiste, allora verrà creato un nuovo file vuoto; lo stesso succede se apriamo il file in aggiunta.
- Se il file esiste e contiene dei dati, allora verranno sovrascritti con i nuovi dati.

File strutturati - record

I file CSV (*Comma-Separated Values*) sono file di testo strutturati in modo che ogni riga sia composta da un record.

Un record rappresenta una sequenza di dati (per esempio nome, descrizione, prezzo) separati da una virgola.

Per **leggere** i record contenuti in un file possiamo procedere:

```
1 p = open("magazzino.csv", "r") #Apertura del file in lettura
2 for riga in p:                 #Iterazione sui record
3     riga = riga.strip("\n")    #Rimozione EOL
4     riga = riga.split(",")     #Separazione dei campi
5     print(riga)               #Visualizzazione campi separati
6 p.close()                     #Chiusura del file

['nome', 'descrizione', 'prezzo']
['maglietta', 't-shirt di colore blu', '8.50€']
['cappello', 'cappellino di lana', '7.00€']
['jeans', 'jeans slim fit', '11.95€']
```

File strutturati - record

Per **scrivere** i record in un file possiamo procedere:

```
1      #Rappresentazione dei prodotti da aggiungere
2      prodotti = [
3          ["scarpe", "mocassini marroni", "68.50€"],
4          ["cintura", "cintura di pelle", "37.60€"]
5      ]
6      p = open("magazzino.csv", "a")      #Apertura magazzino in aggiunta
7      for prodotto in prodotti:          #Iterazione sui prodotti
8          prodotto = ",".join(prodotto)   #Unione delle stringhe con la virgola
9          prodotto = prodotto + "\n"      #Aggiunta EOL
10         p.write(prodotto)               #Scrittura su file
11     p.close()                           #Chiusura del file
```



Capitolo 6

Programmazione a oggetti

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume
informatica per progetti – Programmare in Python, di M. Fiore, A. Zanga, ISBN 9788808899477

Programmazione a oggetti - OOP

La programmazione orientata agli oggetti OOP è un paradigma di programmazione che si basa sul concetto di oggetto.

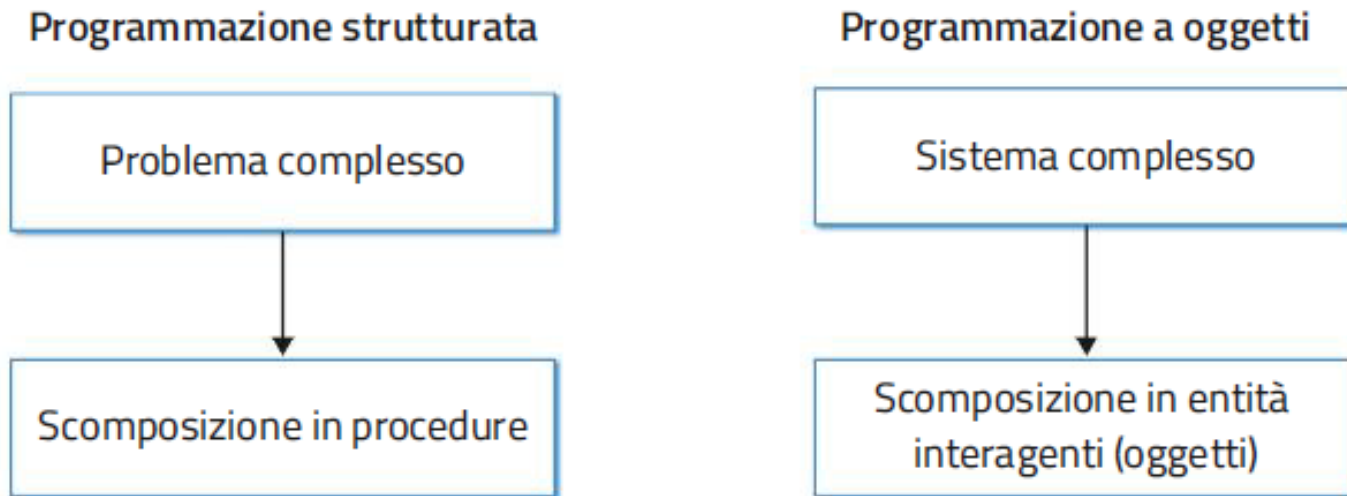
Un **oggetto** rappresenta un'entità della realtà che vogliamo scrivere in codice.

Gli oggetti sono un'astrazione di entità e concetti e contengono tutte le informazioni sullo stato di quell'entità che sono utili per risolvere il problema.

È possibile anche associare a un oggetto le operazioni che è in grado di compiere.

Programmazione a oggetti vs programmazione strutturata

La programmazione a oggetti non è un paradigma di programmazione alternativo alla programmazione strutturata, ma è complementare a essa:



Vantaggi della OOP

La programmazione orientata agli oggetti offre alcuni vantaggi che la programmazione strutturata non possiede:

- **Facilità di lettura e comprensione:** rappresentare entità del mondo reale come oggetti ci permette di seguire con più facilità la semantica del codice.
- **Rapidità della manutenzione:** modificare un oggetto è più facile rispetto a dover cercare variabili e funzioni nel codice.
- **Robustezza:** raggruppare variabili e funzioni comuni nella stessa regione di codice riduce la probabilità di commettere errori di implementazione.
- **Riusabilità:** definire il comportamento di un oggetto ci permette di scrivere meno codice e riutilizzare quello già esistente.

Oggetti, classi, istanze

Un **oggetto** è definito attraverso i suoi attributi e metodi:

- Un attributo è una variabile che caratterizza lo stato di un oggetto.
- Un metodo è una funzione che definisce il comportamento di un oggetto.

Una **classe** è una descrizione astratta che definisce attributi e metodi di un oggetto.

In pratica, una classe è l'implementazione in codice di un oggetto, che a sua volta è un'astrazione, una rappresentazione semplificata di un'entità del problema che vogliamo risolvere.

Quando assegniamo dei valori agli attributi di una classe otteniamo un'istanza.

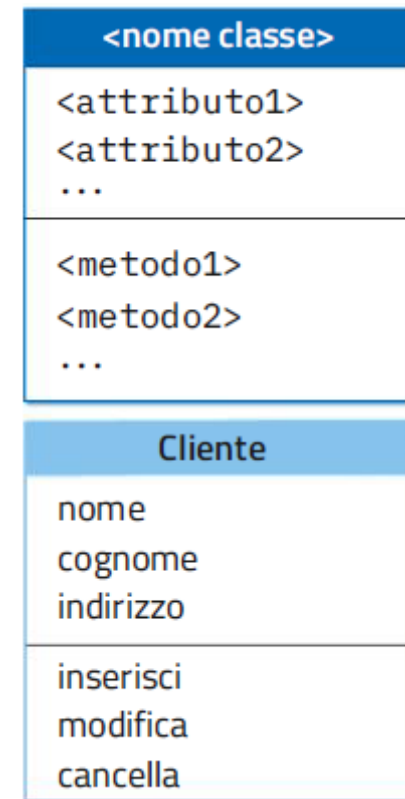
L'**istanza** di un oggetto è un esemplare della classe con specifici valori degli attributi.

Rappresentazione di una classe con UML

Il diagramma delle classi è una rappresentazione grafica del codice che dovremo implementare. I simboli utilizzati nel diagramma seguono il Linguaggio di Modellazione Unificato, in inglese *Unified Modeling Language* (UML).

Per convenzione, una classe è rappresentata come una tabella composta da tre parti:

- il nome della classe,
- la lista degli attributi
- la lista dei metodi.



Dichiarazione di una classe

La dichiarazione di una classe in Python è composta da:

1. la parola chiave `class`;
2. il nome della classe;
3. la classe base, inserita tra parentesi tonde seguite dai due punti;
4. gli attributi (opzionali), indentati a destra rispetto alla prima riga;
5. i metodi (opzionali), indentati a destra rispetto alla prima riga.

In Python esiste una classe base chiamata `object` da cui derivano tutte le altre classi.

```
1 class NomeClasse(object): #Dichiarazione della classe
2     pass                  #Corpo della classe (vuoto per ora)
```

Per costruire un'istanza della classe è sufficiente scrivere il suo nome seguito da una coppia di parentesi tonde.

```
1 class NomeClasse(object): #Dichiarazione della classe
2     pass                  #Corpo della classe (vuoto per ora)
3
4 #main
5 istanza = NomeClasse()   #Creazione istanza
6 print(istanza)
```

Metodi fondamentali

Fase del ciclo di vita	Metodo	Descrizione
Costruzione	<code>__new__</code>	• Crea una nuova istanza non inizializzata • Viene chiamata in fase di creazione • Se non è definito, viene chiamato quello della classe base <code>object</code>
Inizializzazione	<code>__init__</code>	• Inizializza l'istanza appena creata • Viene chiamata dopo <code>__new__</code> • Il suo scopo è quello di assegnare un valore agli attributi dell'istanza • Se non è definito, viene chiamato quello della classe base <code>object</code>
Distruzione	<code>__del__</code>	• Distrugge l'istanza • Viene chiamato implicitamente quando l'istanza non è più visibile, oppure esplicitamente con <code>del</code> • Se non è definito, viene chiamato quello della classe base <code>object</code>

In generale, non definiremo né il costruttore né il distruttore, ma solo l'inizializzatore, in modo da delegare a `object` la gestione di queste due fasi del ciclo di vita della classe.

Esempio di classe

```
1 class Cliente(object):
2
3     #Attributi
4     nome = None
5     cognome = None
6     indirizzo = None
7
8     #Inizializzatore
9     def __init__(self, nome, cognome, indirizzo):
10         self.nome = nome
11         self.cognome = cognome
12         self.indirizzo = indirizzo
13
14 #main
15 cliente = Cliente("Mario", "Rossi", "Via Monte Napoleone")
16 print(f"Nome: {cliente.nome}, Cognome: {cliente.cognome}")
17 print(f"Indirizzo: {cliente.indirizzo}")
18
19 Nome: Mario, Cognome: Rossi
20 Indirizzo: Via Monte Napoleone
```

Definizione di attributi

Riferimento all'istanza

Notazione punto

Creazione dell'istanza

Visualizzazione di una istanza

La visualizzazione è un'operazione così importante che Python prevede un metodo speciale `__str__` che consente di restituire una stringa formattata da passare alla funzione `print`.

```
...
14     #Visualizzazione
15     def __str__(self):
16         s = f"Nome: {self.nome}, Cognome: {self.cognome}\n"
17         s += f"Indirizzo: {self.indirizzo}"
18         return s
19
20 #main
21 cliente = Cliente("Mario", "Rossi", "Via Monte Napoleone")
22 print(cliente)

Nome: Mario, Cognome: Rossi
Indirizzo: Via Monte Napoleone
```

Metodi di classe e di istanza

I **metodi di classe** sono funzioni definite all'interno di una classe e sono applicabili alla classe stessa. Utilizzano `cls` per indicare il riferimento alla classe.

Per definire un metodo di classe dobbiamo utilizzare la funzione `classmethod`.

```
1 class Classe(object):
2     def metodo_classe(cls):                #Dichiarazione metodo
3         print("Metodo di classe")         #Corpo metodo
4     #Sovrascrittura con metodo di classe
5     metodo_classe = classmethod(metodo_classe)
```

I metodi di classe sono invocabili sia dalla classe sia su una sua istanza. Per questo motivo sono particolarmente utili quando il metodo non dipende dallo stato dell'istanza, cioè quando non facciamo riferimento agli attributi dell'istanza.

I **metodi di istanza** sono funzioni definite all'interno di una classe e sono invocabili solo sull'istanza della classe. Utilizzano `self` per indicare il riferimento all'istanza.

Attributi di classe e di istanza

Un **attributo di classe** è un attributo il cui valore è noto durante la dichiarazione della classe.

Un attributo di classe è condiviso da tutte le istanze della classe e viene automaticamente aggiunto come attributo di istanza con il valore che ha al momento della creazione.

```
>>> class Classe(object):           #Dichiarazione classe
>>>     attributo_classe = 0        #Dichiarazione attributo di classe
>>>
>>> print(Classe.attributo_classe)  #Visualizzazione attributo di classe
0

>>> istanza_1 = Classe()           #Creazione nuova istanza
>>> print(istanza_1.attributo_classe) #Visualizzazione attributo di classe
0

>>> Classe.attributo_classe = 10    #Modifica dell'attributo di classe
>>> istanza_2 = Classe()           #Creazione nuova istanza
>>> print(istanza_2.attributo_classe) #Visualizzazione attributo di classe
10
```

Un **attributo di istanza** è un attributo il cui valore è noto durante l'inizializzazione di un'istanza.

A differenza di un attributo di classe, un attributo di istanza definisce lo stato dell'istanza, ossia l'insieme dei valori delle variabili che ci permettono di differenziare le istanze tra loro.

Visibilità

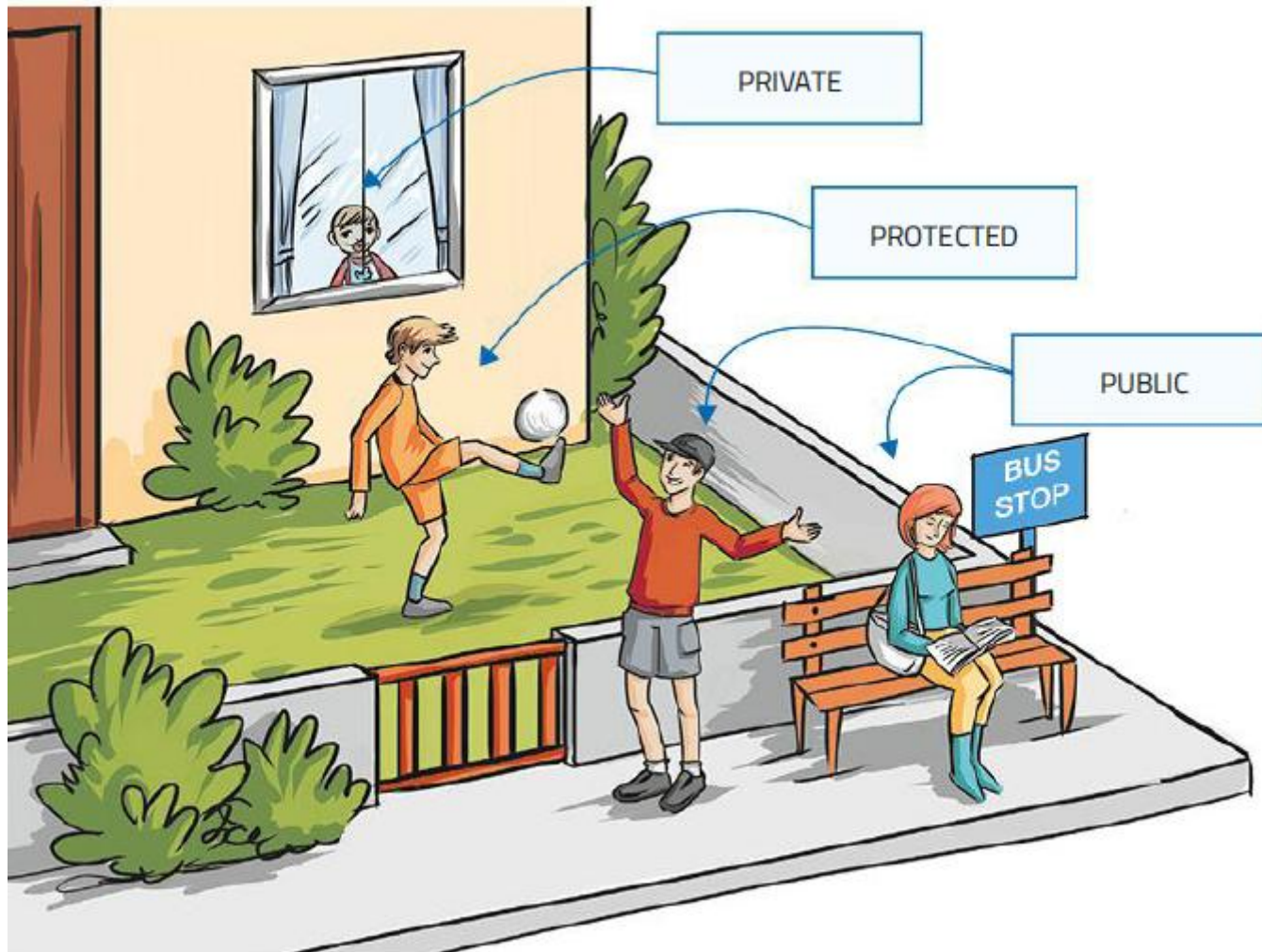
Nella programmazione a oggetti il concetto di visibilità dell'informazione è fondamentale.

Si distinguono tre livelli di accesso:

- **public** contiene i membri a cui si può accedere dall'esterno della classe;
- **private** contiene i membri ai quali è possibile accedere solo dall'interno della classe;
- **protected** contiene membri ai quali si può accedere anche all'interno di classi derivate della stessa.

I livelli di visibilità danno la possibilità allo sviluppatore di controllare l'informazione che vuole esporre all'esterno della classe.

Livelli di visibilità



Visibilità in Python

Mentre in altri linguaggi di programmazione i membri di una classe sono considerati privati, in Python non c'è una distinzione nella sintassi del linguaggio e di base **i membri sono considerati pubblici**.

Esiste una convenzione per cui gli attributi da considerare privati sono preceduti da un doppio *underscore*.

Ma perché distinguere tra membri pubblici e privati?

I motivi sono due.

1. Per imporre vincoli concettuali: in caso di codice scritto da più sviluppatori è utile poter comunicare l'intenzione di esporre o meno alcuni dettagli di implementazione.
2. Per non fornire accesso ad alcune parti della classe: nel caso di sviluppo di applicazioni commerciali potrebbe essere necessario nascondere i dettagli dell'implementazione specifica di alcune parti del codice.

Incapsulamento

Se i membri privati non sono visibili all'esterno della classe, al contrario i membri pubblici lo sono e definiscono quella che viene chiamata interfaccia.

L'**interfaccia di una classe** indica l'insieme dei membri pubblici, cioè l'insieme delle informazioni e funzionalità utilizzabili dalle istanze della classe.

Rendere espliciti, cioè pubblici, i membri di una classe significa esporre in modo visibile le possibili interazioni con le istanze, in modo che tutti gli sviluppatori sappiano quali azioni possono e non possono fare con quelle istanze.

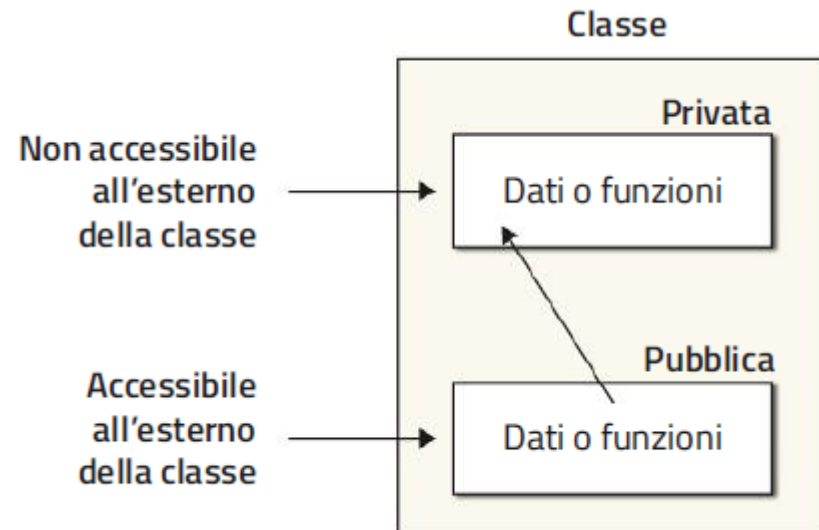
Il termine **incapsulamento** indica la qualità degli oggetti di poter incorporare al loro interno sia gli attributi sia i metodi, cioè le caratteristiche e i comportamenti dell'oggetto

Information hiding

Se combiniamo visibilità e incapsulamento possiamo attuare quello che viene chiamato *information hiding* che indica una particolare modalità di implementazione di un oggetto che rende disponibili all'esterno solo alcune funzionalità.

I dettagli sulle caratteristiche e la struttura dell'oggetto sono nascosti all'interno.

Distinguere tra membri pubblici e privati ci permette di utilizzare queste qualità che sono alla base della programmazione a oggetti, garantendo una maggiore capacità di astrazione e flessibilità di implementazione.



Ereditarietà

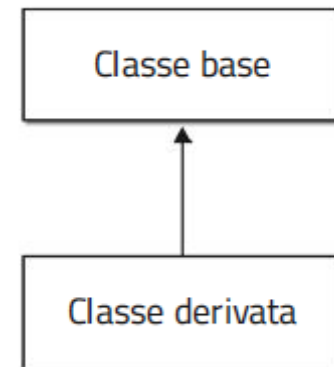
L'**ereditarietà** rappresenta la capacità di creare nuove classi a partire da classi già esistenti: le nuove classi create rappresentano una versione modificata della classe di partenza che viene definita superclasse o classe base.

La nuova classe eredita tutti gli attributi e i metodi della classe base e può essere arricchita con nuovi attributi e nuovi metodi.

Si definisce **sottoclasse** o classe derivata la classe che viene tratta da un'altra classe con il meccanismo di ereditarietà.

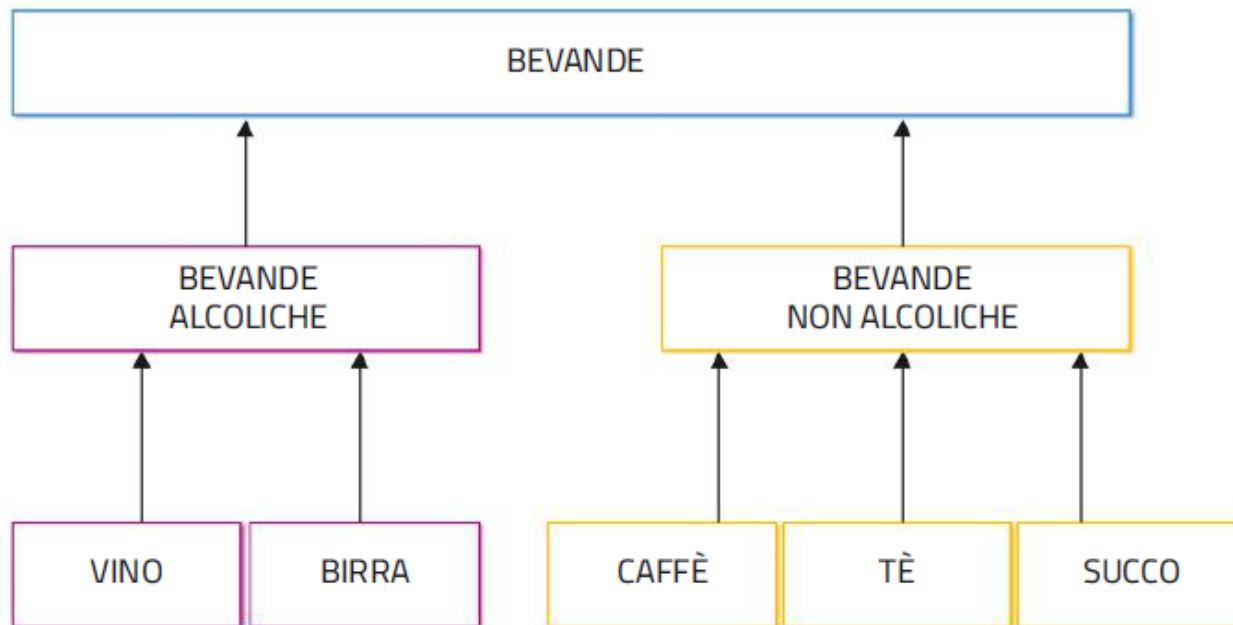
Si definisce **superclasse** o classe base la classe generatrice di una sottoclasse.

Nel diagramma UML delle classi, la classe derivata si indica con una freccia che la collega alla classe base.



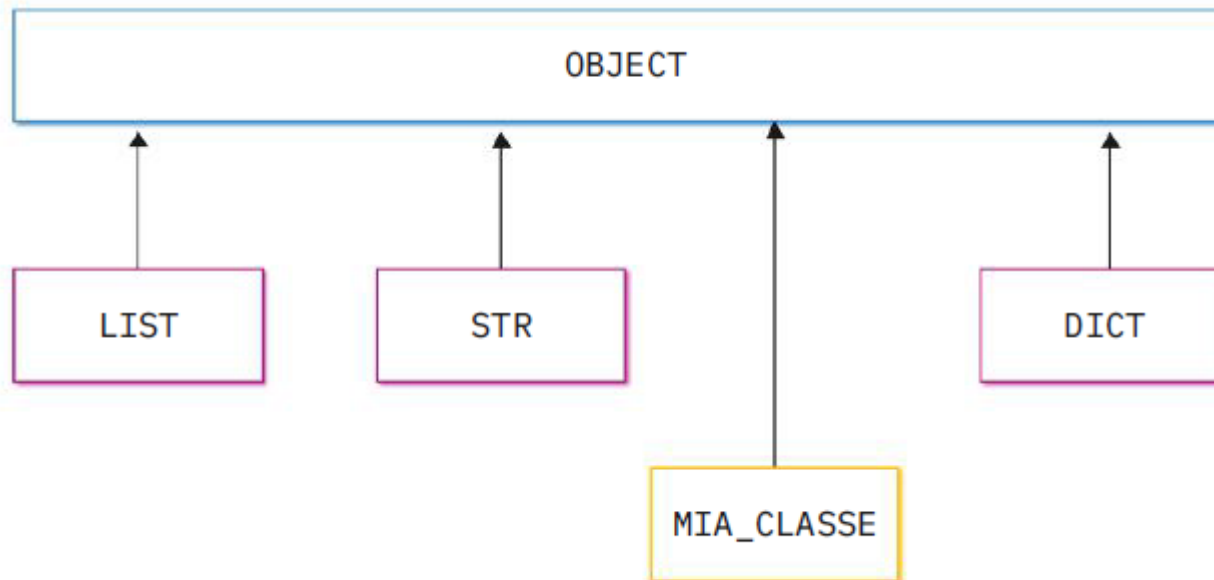
Gerarchia di classi

Il meccanismo di ereditarietà consente di creare *classi figlie* a partire da *classi genitori* permettendo il riutilizzo del codice e rendendo gli script più modulari e organizzati; consente anche la creazione di più classi a livelli differenti, permettendo così la costruzione di una gerarchia di classi, in cui le classi derivate possono estendere o modificare i comportamenti delle classi base.



Classe object

La classe object è la classe base di Python da cui derivano tutti i tipi di dati built-in: dalla classe object derivano anche tutte le classi definite in Python, utilizzando il meccanismo di ereditarietà.



Dichiarazione di classe derivata

L'ereditarietà permette di creare nuove classi in due modalità:

- per **estensione**;
- per **ridefinizione**.

Una classe **derivata per estensione** indica una classe che aggiunge ai metodi e agli attributi della classe base nuovi attributi e metodi che vengono definiti nella classe derivata.

Una classe **derivata per ridefinizione** indica una classe che modifica o ridefinisce i metodi ereditati dalla classe base.

```
class BevandeAlcoliche(Bevande):  
    #attributi  
    ...  
    #metodi  
    ...
```

Le classi derivate ereditano attributi e metodi dalla superclasse e per poterli utilizzare è necessario fare riferimento a essi usando la funzione `super()`, che consente di fare riferimento ad attributi e metodi (anche il costruttore e l'inizializzatore) della classe da cui deriva.

Polimorfismo

Con l'ereditarietà è possibile creare nuove classi derivandole da altre già esistenti, per estensione della classe base oppure per ridefinizione; il polimorfismo rappresenta per un oggetto la possibilità di assumere forme diverse in base alla sua implementazione specifica.

Il **polimorfismo degli oggetti** rappresenta la capacità degli oggetti di assumere comportamenti diversi in base a una diversa implementazione dei suoi metodi.

Il polimorfismo **facilita il riutilizzo del software**; un esempio è la possibilità di usare lo stesso metodo di una classe, ma con tipi diversi di dati.

Facilita inoltre **la scrittura di codice in modo estensibile**, poiché l'aggiunta, a una gerarchia di classi già predisposta, di una nuova classe che implementa i metodi in modo differente non influenza il codice precedentemente scritto.

Overriding

La derivazione di una classe che effettua la ridefinizione dei metodi della classe base si chiama **overriding** (o sovrascrittura del metodo).

L'overriding non cambia il nome del metodo né il numero dei parametri (la sua firma), ma modifica le sue istruzioni.

L'**overriding** è un polimorfismo che prevede di sovrascrivere nella classe derivata un metodo della classe base, cambiando le istruzioni e dunque cambiando il suo comportamento.

```
1 class Libro(object):
2     #attributi
3     titolo = ""
4     autore = ""
5     anno_pubblicazione = 0
6     #metodi
7     def __init__(self, titolo, autore, anno_pubblicazione):
8         self.titolo = titolo
9         self.autore = autore
10        self.anno_pubblicazione = anno_pubblicazione
11
12    def descrivi(self):
13        '''usato per descrivere l'istanza del libro'''
14        return f"{self.titolo} scritto da {self.autore}, pubblicato
15        nel {self.anno_pubblicazione}"
```

```
16 class Fantasy(Libro):
17     #attributi aggiuntivi
18     genere = ""
19     #metodi
20     def __init__(self, titolo, autore, anno_pubblicazione, genere):
21         super().__init__(titolo, autore, anno_pubblicazione)
22         self.genere = genere
23
24     def descrivi(self):
25         return f"{super().descrivi()}, genere: {self.genere}"
26
```

overriding nel metodo `descrivi` della classe `Fantasy`: anche la classe `Libro` ha un metodo che si chiama `descrivi`, ma il nuovo metodo `descrivi` della classe `Fantasy` lo sovrascrive aggiungendo al suo comportamento la possibilità di stampare anche il nuovo attributo `genere` che abbiamo aggiunto alla classe derivata.

Overloading

Un secondo tipo di polimorfismo è l'**overloading**, cioè la possibilità di definire più di una versione di una funzione o di un operatore con lo stesso nome ma con diversi tipi di parametri o un numero diverso di parametri.

In altre parole, le funzioni o gli operatori possono avere lo stesso nome ma comportarsi in modo diverso in base al tipo o al numero di parametri passati.

L'**overloading** è un polimorfismo che permette, in una classe derivata, di aggiungere parametri al metodo ereditato dalla classe base e di poter modificare le istruzioni.

In fase di chiamata del metodo, il compilatore o l'interprete del linguaggio identifica quale versione del metodo deve essere eseguita in base al numero dei parametri passati.

Overloading degli operatori e Python

Python non supporta l'overloading delle funzioni, mentre supporta solo quello degli operatori.

Infatti in Python se ci sono più definizioni di funzioni con lo stesso nome ma con parametri diversi, la dichiarazione dell'ultima funzione sovrascrive le precedenti e il tentativo di usarne una che non abbia il numero di parametri previsto da quella che è stata dichiarata per ultima provoca un errore.

Un esempio di overloading degli operatori è quello della funzione print: in genere viene fatto un overloading con il metodo `__str__` che permette di restituire la rappresentazione dell'oggetto in forma di stringa, cioè in forma leggibile.

Librerie di classi

Attraverso l'ereditarietà viene reso possibile il riutilizzo del software creato senza dover partire da zero nella scrittura di un nuovo programma: è sufficiente utilizzare un insieme di classi già definite che hanno dei comportamenti simili a quelli che vogliamo realizzare nel nuovo programma ed estenderle per aggiungere i comportamenti desiderati alla nuova applicazione.

Nel nuovo programma le classi già esistenti non verranno riscritte, portando un vantaggio in termini di tempo e di affidabilità:

- in termini di tempo, poiché non è necessario ricodificare le classi esistenti;
- in termini di affidabilità, poiché le classi già scritte sono state collaudate da altri programmi che le hanno usate in precedenza e quindi sono quasi sicuramente prive di errori.

Capitolo 7

Il gaming in Python

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume *informatica per progetti – Programmare in Python*, di M. Fiore, A. Zanga, ISBN 9788808899477

Interfacce grafiche

Le **interfacce grafiche** (o GUI, *Graphical User Interfaces*) sono il modo principale con cui gli utenti interagiscono con i sistemi informatici attraverso elementi visivi e grafici.

Una GUI utilizza componenti grafici che rendono i sistemi più accessibili e intuitivi per l'utente, come:

- **finestre**, ovvero porzioni dello schermo dedicate a specifiche applicazioni o contenuti;
- **icone**, che sono rappresentazioni grafiche di programmi, funzioni o file;
- **bottoni**, cioè componenti cliccabili per eseguire azioni specifiche;
- **menu**, ovvero elenchi di comandi o di opzioni disponibili;
- **barre degli strumenti**, che contengono pulsanti per funzioni di uso comune, per esempio il salvataggio di dati;
- **campi di testo**, per l'inserimento di dati testuali.

Programmazione guidata da eventi

La **programmazione guidata da eventi** (*event-driven programming*) è un paradigma di programmazione utilizzato per lo sviluppo di software interattivi dotati di interfacce utente grafiche (GUI) e capaci di rispondere in tempo reale alle azioni dell'utente, fornendo un'esperienza di uso fluida e dinamica.

Il **flusso del programma** non segue una sequenza lineare predefinita, ma è controllato da eventi esterni che possono verificarsi in modo asincrono e imprevedibile.

Gli **eventi** possono essere generati da varie fonti, come le azioni dell'utente (per esempio il click del mouse, la pressione dei tasti, il trascinamento di oggetti) o cambiamenti di stato nel sistema (per esempio l'aggiornamento di un file o l'arrivo di dati da una rete).

Gaming in Python: Pygame

La libreria open source *Pygame* offre un insieme di moduli che semplificano la gestione di grafica, suoni e input utente ed è ideale per chiunque voglia avvicinarsi allo sviluppo di giochi 2D.

La libreria fornisce moduli specifici per le operazioni di rendering, gestione di immagini, suoni, eventi e finestre grafiche.

L'installazione di *Pygame* è semplice e può essere eseguita attraverso il gestore di pacchetti *pip*.

Il comando per l'installazione è il seguente.

```
pip install pygame
```

Moduli di Pygame

I moduli principali di *Pygame* includono:

- `pygame.display`, per la gestione delle finestre o delle schermate su cui viene disegnata la grafica;
- `pygame.draw`, per disegnare primitive grafiche come linee, cerchi, rettangoli e poligoni;
- `pygame.image`, per il caricamento e la manipolazione di immagini in vari formati;
- `pygame.sprite`, per gestire oggetti grafici (sprite) in modo strutturato, permettendo animazioni e collisioni;
- `pygame.mixer`, per la gestione dell'audio, incluse la riproduzione di suoni ed effetti musicali;
- `pygame.event`, per la gestione degli eventi, come input da tastiera, mouse e joystick;
- `pygame.font`, per il rendering di testo su schermo.

Game Loop

La struttura di base di un programma scritto con *Pygame* segue un flusso piuttosto semplice e ripetitivo, chiamato **game loop**.

Questo ciclo viene utilizzato per aggiornare continuamente la schermata, gestire gli eventi, aggiornare lo stato del gioco e disegnare i vari elementi grafici.

Game Loop

```
1 import pygame
2 import sys
3
4 #Inizializzazione di Pygame
5 pygame.init()
6
7 #Creazione di una finestra
8 screen = pygame.display.set_mode((800, 600))
9 pygame.display.set_caption("Il mio primo gioco con Pygame")
10
11 #Impostazione del colore dello sfondo
12 background_color = (0, 0, 0) #Nero
13
14 #Game loop
15 running = True
16 while running:
17     #Gestione degli eventi
18     #pygame.QUIT significa che l'utente fa clic su X per chiudere la finestra
19     for event in pygame.event.get():
20         if event.type == pygame.QUIT:
21             running = False
22
23     #Riempi lo schermo con il colore di sfondo
24     screen.fill(background_color)
25
26     #Blocco istruzioni per il gioco
27
28     #flip() permette l'aggiornamento della finestra
29     pygame.display.flip()
30
31 #Chiusura di Pygame
32 pygame.quit()
33 sys.exit()
```



insta_photos/Shutterstock

Gestione delle immagini - 1

Il modulo `pygame.display` permette di creare una finestra o uno schermo, mentre il modulo `pygame.draw` fornisce strumenti per disegnare forme geometriche di base. Per creare giochi più complessi, è possibile caricare e manipolare immagini attraverso il modulo `pygame.image`.

La funzione `blit()` si usa per copiare il contenuto dell'immagine su una superficie, come ad esempio, quella della finestra principale.

Gestione delle immagini - 2

```
14 #Caricamento di un'immagine
15 image = pygame.image.load("owl.png")
16
17 #Game loop
18 running = True
19 while running:
20     #Gestione degli eventi
21     #pygame.QUIT significa che l'utente fa click su X per chiudere la finestra
22     for event in pygame.event.get():
23         if event.type == pygame.QUIT:
24             running = False
25
26     #Riempie lo schermo con il colore di sfondo
27     screen.fill(background_color)
28
29     #Disegna l'immagine nella posizione specificata
30     screen.blit(image, (100, 150))
31
32     #Blocco istruzioni per il gioco
33
34     #flip() permette l'aggiornamento della finestra
35     pygame.display.flip()
```


Gestione delle immagini - 3

Usando il modulo `pygame.draw` è possibile anche tracciare linee e disegnare rettangoli, cerchi e altre forme.

Per farlo si usano le funzioni:

- `line()` per tracciare linee da un punto di partenza a un punto di arrivo, dotate di uno specifico spessore e di un colore;
- `rect()` per disegnare forme rettangolari fornendo le coordinate dei quattro punti che costituiscono i vertici del rettangolo, il colore e lo spessore del bordo;
- `circle()` per disegnare cerchi e porzioni di cerchio, indicando il colore, le coordinate del centro del cerchio, la lunghezza in pixel del raggio e lo spessore del bordo;
- `polygon()` per disegnare un poligono con un numero qualsiasi di vertici, fornendo le coordinate dei punti che li costituiscono, il colore e lo spessore del bordo.

Usando queste funzioni per disegnare le forme non serve la funzione `blit()` per visualizzarle sullo schermo.

Oggetto Surface

Un oggetto **Surface** rappresenta l'oggetto grafico che viene mostrato attualmente ed è restituito dalla funzione `display.set_mode()`.

All'inizio dello script di base, creiamo una finestra in cui faremo accadere le azioni del nostro gioco; ogni azione fa riferimento all'oggetto Surface restituito da questa funzione.

Qualsiasi disegno venga fatto su questa Surface sarà visibile sul monitor; anche le immagini sono degli oggetti Surface.

La funzione `pygame.image.load()` restituisce un oggetto Surface che ci permette di gestire l'immagine che abbiamo caricato.

Oggetto Rect

Un oggetto **Rect** rappresenta un'area rettangolare utile per le animazioni.

Un oggetto Rect fornisce indicazioni relative alle dimensioni e alle posizioni degli oggetti da animare, oltre a altre informazioni specifiche che consentono di muovere l'oggetto nello spazio di gioco con molta precisione.

Gestione degli sfondi

In una finestra possono essere messi più oggetti, che possono essere immagini caricate ma anche disegni realizzati da noi.

In genere in un gioco abbiamo molti oggetti che si muovono e a volte interagiscono tra loro (per esempio, se si toccano accade qualcosa).

Possiamo anche caricare uno sfondo che non sia un semplice colore, ma un'immagine (per esempio un paesaggio).

```
13  #Caricamento immagine dello sfondo
14  background_image = pygame.image.load("background.png")
```

```
31  #Disegna lo sfondo
32  screen.blit(background_image, (0, 0))
```

Eventi

Per consentire agli utenti di interagire con un gioco, è necessario utilizzare il sistema degli eventi di *Pygame*.

Utilizzando `pygame.event.get()`, è possibile catturare gli input dell'utente, come i movimenti del mouse, i clic e la pressione dei tasti.

Ogni evento può essere gestito con un ciclo che verifica il tipo di evento e reagisce di conseguenza.

```
34     #Rileva lo stato dei tasti premuti
35     keys = pygame.key.get_pressed()
36
37     #Movimento del cerchio con i tasti freccia
38
39     if keys[pygame.K_LEFT]:
40         circle_x -= circle_speed #Muove a sinistra
41     if keys[pygame.K_RIGHT]:
42         circle_x += circle_speed #Muove a destra
43     if keys[pygame.K_UP]:
44         circle_y -= circle_speed #Muove in su
45     if keys[pygame.K_DOWN]:
46         circle_y += circle_speed #Muove in giù
```

Tipi di eventi rilevati - 1

Evento	Descrizione	Attributi
QUIT	Viene generato quando l'utente cerca di chiudere la finestra (per esempio, cliccando sulla X di chiusura).	
KEYDOWN	Viene generato quando viene premuto un tasto sulla tastiera.	• key: codice del tasto premuto (per esempio <code>pygame.K_LEFT</code> per la freccia sinistra) • mod: modificatori attivi (Shift, Ctrl, Alt)
KEYUP	Viene generato quando un tasto viene rilasciato.	• key: codice del tasto rilasciato • mod: modificatori attivi (Shift, Ctrl, Alt)
MOUSEBUTTONDOWN	Viene generato quando un pulsante del mouse viene premuto.	• button: pulsante del mouse premuto (1 per il pulsante sinistro, 2 per il centrale, 3 per il destro) • pos: posizione del mouse nel momento della pressione
MOUSEBUTTONUP	Viene generato quando un pulsante del mouse viene rilasciato.	• button: pulsante del mouse rilasciato • pos: posizione del mouse nel momento del rilascio
MOUSEMOTION	Viene generato quando il mouse si muove.	• pos: nuova posizione del mouse • rel: spostamento relativo del mouse rispetto alla posizione precedente • buttons: stato dei pulsanti del mouse (una lista di tre valori booleani per sinistro, centrale, destro)

Tipi di eventi rilevati - 2

VIDEORESIZE	Viene generato quando la finestra cambia dimensione.	• size: nuova dimensione della finestra • w: nuova larghezza • h: nuova altezza
VIDEOEXPOSE	Viene generato quando la finestra deve essere ridisegnata, per esempio quando viene coperta e poi scoperta.	
JOYAXISMOTION, JOYBUTTONDOWN, JOYBUTTONUP, JOYHATMOTION	Eventi generati da pulsanti di un joystick premuti o da assi del joystick che si muovono.	
USEREVENT	Evento personalizzabile che è possibile generare in autonomia, utile per timer o altri scopi specifici del gioco.	
FINGERDOWN, FINGERUP, FINGERMOTION	Eventi generati quando si interagisce con dispositivi touchscreen. • FINGERDOWN: Quando un dito tocca lo schermo. • FINGERUP: Quando un dito lascia lo schermo. • FINGERMOTION: Quando un dito si muove sullo schermo.	



```
1 import pygame
2 import sys
3
4 pygame.init()
5
6 #Crea una finestra di gioco
7 screen = pygame.display.set_mode((800, 600))
8 pygame.display.set_caption("Rilevamento eventi")
9
10 #Game loop
11 running = True
12 while running:
13     for event in pygame.event.get():
14         if event.type == pygame.QUIT:
15             running = False
16         if event.type == pygame.KEYDOWN:
17             print(f"Tasto premuto: {pygame.key.name(event.key)}")
18         if event.type == pygame.MOUSEBUTTONDOWN:
19             print(f"Pulsante mouse premuto: {event.button} in posizione
20                 {event.pos}")
21         if event.type == pygame.MOUSEMOTION:
22             print(f"Mouse mosso: {event.pos}")
23
24     pygame.display.flip()
25
26 pygame.quit()
27 sys.exit()
```


Inserire testo nella finestra di gioco

Per inserire e visualizzare del testo nella finestra di gioco, per esempio per gestire il punteggio, bisogna seguire alcuni passaggi specifici:

1. **Creare un oggetto font**: ciò permette di definire il carattere, la dimensione e lo stile del testo.
2. **Renderizzare il testo**: utilizzando l'oggetto font creato, si genera un oggetto Surface che contiene il testo, che può essere successivamente disegnato sulla finestra.
3. **Visualizzare il testo**: l'oggetto Surface con il testo viene disegnato sulla finestra utilizzando il metodo `blit()`.

Audio e suoni

Pygame supporta anche la gestione dell'audio.

Tramite il modulo `pygame.mixer`, è possibile caricare e riprodurre file audio, che possono essere effetti sonori oppure musica di sottofondo.

Pygame supporta vari formati audio, tra cui .wav, .mp3 e .ogg.

Pygame permette anche di riprodurre musica in loop o in sequenza, cosa che risulta particolarmente utile per la colonna sonora del gioco.

Gli Sprite

Uno **sprite** è un'immagine o un oggetto grafico 2D che rappresenta un'entità del gioco, come un personaggio o un oggetto interattivo, e può avere attributi come posizione, velocità e animazione.

Viene gestito dal modulo **pygame.sprite**

Il modulo **pygame.sprite** semplifica l'organizzazione degli sprite in gruppi, facilitando la gestione di più oggetti contemporaneamente, specialmente per operazioni come il rilevamento delle collisioni.

Il modulo utilizza la programmazione a oggetti (OOP) per gestire i comportamenti degli sprite.

Le caratteristiche principali di questo modulo sono:

- La classe **Sprite** è la classe di base da cui tutti gli sprite ereditano attributi e metodi.
- Gli sprite possono essere raggruppati in oggetti **Group** per una gestione collettiva (per esempio per aggiornamento, disegno e collisione).
- È possibile rilevare collisioni tra sprite o gruppi di sprite.

Gli Sprite

```
15 #Classe Player per il nostro sprite (un semplice rettangolo)
16 class Player(pygame.sprite.Sprite):
17     def __init__(self):
18         super().__init__() #Inizializza la classe Sprite
19         self.image = pygame.Surface((50, 50)) #Crea una superficie 50x50
20         # per rappresentare lo sprite
21         self.image.fill((0, 128, 255)) #Colora lo sprite di blu
22         self.rect = self.image.get_rect() #Ottiene il rettangolo (rect) che
23         # circonda lo sprite
24         self.rect.center = (width // 2, height // 2) #Posiziona lo sprite
25         # al centro della finestra
26
27     def update(self):
28         keys = pygame.key.get_pressed() #Controlla i tasti premuti frecce
29         if keys[pygame.K_LEFT]:
30             self.rect.x -= 5
31         if keys[pygame.K_RIGHT]:
32             self.rect.x += 5
33         if keys[pygame.K_UP]:
34             self.rect.y -= 5
35         if keys[pygame.K_DOWN]:
36             self.rect.y += 5
37
38 #Crea un gruppo per gestire tutti gli sprite
39 all_sprites = pygame.sprite.Group()
40
41 #Crea il giocatore (Player) e lo aggiunge al gruppo
42 player = Player()
43 all_sprites.add(player)
```

Collisioni

Uno dei vantaggi di utilizzare gli sprite in *Pygame* è la gestione delle collisioni, cioè le situazioni in cui due o più sprite si sovrappongono o entrano in contatto.

Pygame include funzioni per rilevare le collisioni tra sprite o tra uno sprite e un gruppo di sprite.

A questo scopo abbiamo a disposizione funzioni specifiche come `pygame.sprite.spritecollide()` e `pygame.sprite.groupcollide()`.

```
# Controlla se un singolo sprite collide con un gruppo di sprite
if pygame.sprite.spritecollide(player, enemy_group, False):
    print("Collisione rilevata!")
```

Il parametro `False` indica se lo sprite con cui collide deve poi essere eliminato dal gruppo.

Se fosse `True`, il nemico con cui si scontra verrebbe rimosso.

Progettare un gioco

1. Definizione del concetto di base e degli obiettivi: questa fase include la scelta del tipo di gioco, il tema, la storia e il pubblico di destinazione.
2. La progettazione (game design), in cui vengono delineati gli aspetti più dettagliati del gioco: si progettano le regole, le dinamiche di gioco, i personaggi, l'ambientazione e i livelli.
3. Creare un prototipo: vengono implementati gli elementi di base del gioco, come i controlli, la grafica e le interazioni.
4. Implementazione della logica centrale del gioco: programmare come il gioco si comporta in risposta agli input dell'utente e agli eventi che accadono nel mondo di gioco.
5. Aggiungere elementi di grafica e suoni per migliorare l'esperienza del gioco.
6. Testing: è necessario testare attentamente il gioco per assicurarsi che funzioni correttamente.
7. Funzionalità extra: come punteggi, Vite e Salute, Livelli, Effetti speciali.
8. Pubblicazione del gioco.

Capitolo 8

Data Science

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume *informatica per progetti – Programmare in Python*, di M. Fiore, A. Zanga, ISBN 9788808899477

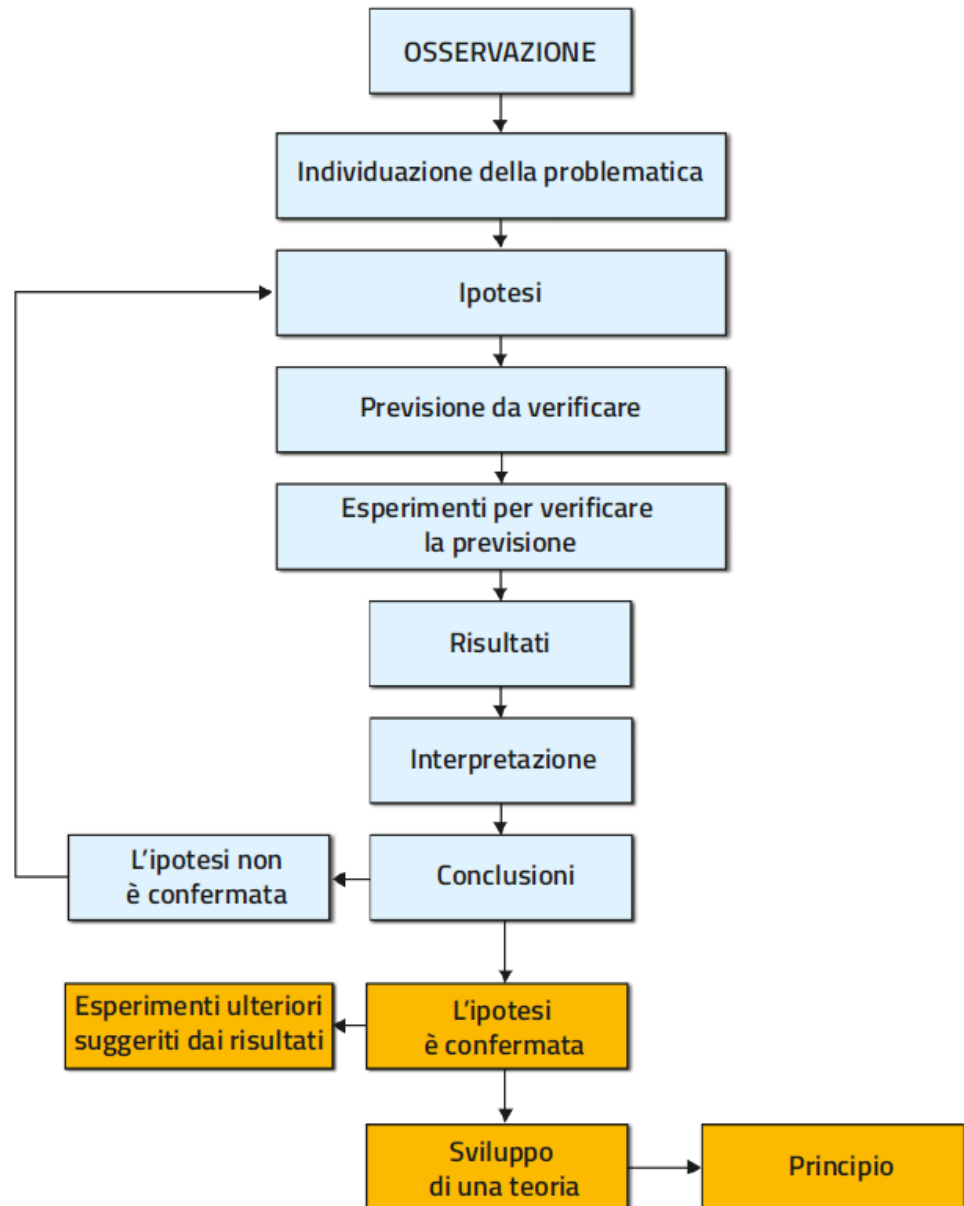
Data Science

La scienza dei dati, più comunemente chiamata **Data Science**, è la disciplina che si occupa dello studio dei dati.

L'obiettivo primario di questa scienza è l'interpretazione e l'estrazione di informazioni complesse a partire dai dati raccolti. Per esempio, dallo storico degli ordini di un negozio è possibile estrarre un elenco degli articoli più venduti e sapere quelli che hanno un maggior margine di profitto, oppure, al contrario, quelli che sono rimasti invenduti e occupano spazio prezioso nel magazzino.

A differenza dell'analisi dei dati, nella quale ci si limita a estrarre informazione dai dati, nella scienza dei dati estraiamo informazione con lo scopo ultimo di **confermare o meno un'ipotesi iniziale**.

Metodo induttivo



Dati grezzi

Per **dati grezzi** intendiamo quei dati che sono stati raccolti ma non sono ancora stati elaborati, analizzati o trasformati in alcun modo.

I dati grezzi rappresentano la forma più elementare delle informazioni raccolte e non sono stati sottoposti a nessun processo di pulizia, normalizzazione, aggregazione o interpretazione.

Data Science Pipeline (o workflow)

1. **Acquisizione dei dati:** *raccolta* dati, è la fase che nel metodo induttivo corrisponde all'osservazione, in cui i dati grezzi vengono registrati su un supporto esattamente come vengono prodotti dalla sorgente dei dati.
2. **Elaborazione dei dati:** *pre-processing* dei dati per distinguerlo dalla fase di analisi, consiste in una prima fase di raffinamento dei dati grezzi in dati più organizzati «puliti», in cui vengono rimosse le osservazioni che contengono anomalie, per esempio errori di raccolta, valori mancanti e altre discrepanze.
3. **Analisi dei dati:** il cuore della fase sperimentale, in cui vengono applicate metodologie di analisi anche molto diverse tra loro, partendo dalla più semplice statistica descrittiva fino ad arrivare ai metodi di Intelligenza Artificiale.
4. **Validazione:** i risultati dell'analisi dei dati devono essere validati, ossia controllati, per essere sicuri che l'analisi sia stata eseguita correttamente.
5. **Conclusione:** se la validazione ha dato esito positivo, allora è effettivamente possibile utilizzare i risultati sperimentali per confermare o smentire l'ipotesi iniziale.

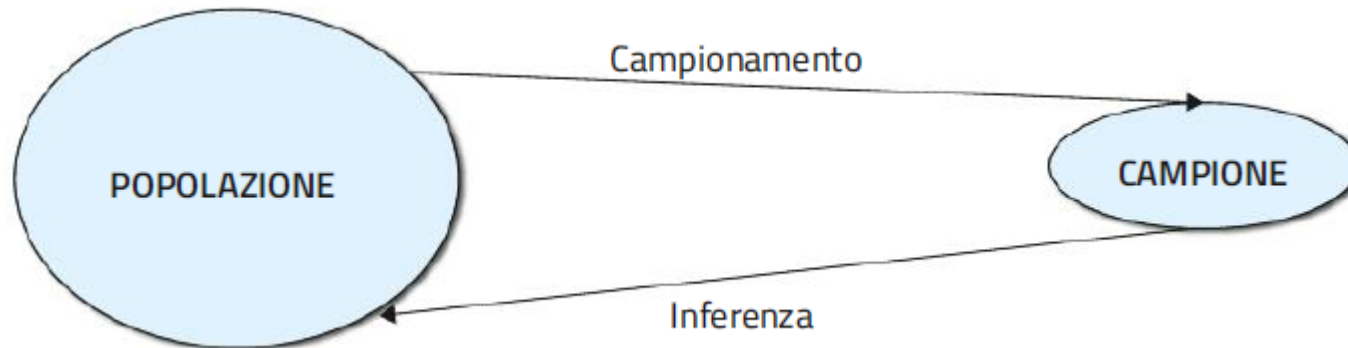
Campionamento e inferenza

Per **popolazione** intendiamo l'insieme delle istanze, anche dette unità, che sono oggetto di studio.

Un **campione** è un sottoinsieme delle unità contenute nella popolazione.

Per **inferenza** intendiamo la generalizzazione dei risultati ottenuti dall'analisi di un campione all'intera popolazione da cui è stato estratto il campione stesso.

Perché l'inferenza sia generalizzabile alla popolazione, il campione deve essere rappresentativo della popolazione, ossia, deve avere caratteristiche simili alla popolazione da cui è stato estratto.



Variabili quantitative e variabili qualitative

- **variabili quantitative**: sono rappresentate da valori numerici, per esempio il peso o l'età di una persona
- **variabili qualitative**: sono rappresentate da valori non numerici, per esempio il genere di una persona o il suo colore preferito.

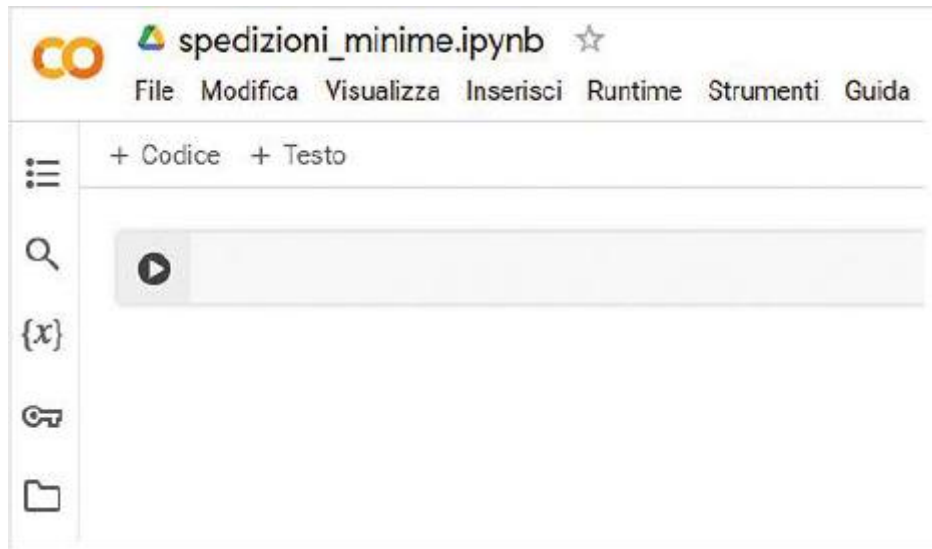
Conoscere il tipo di una variabile ci permette di sapere quali sono le informazioni che possiamo estrarre da un campione.

Colaboratory (Colab)

Colaboratory è realizzato da Google e è pensato proprio per l'analisi dei dati: questo ambiente di sviluppo fornisce un'interfaccia web ed esegue il codice Python in remoto sui server di Google gratuitamente.

E' disponibile all'indirizzo <https://colab.research.google.com>.

L'interfaccia di Colab si basa sul concetto di notebook, ovvero blocco note, in cui si alternano gruppi, chiamati celle, di righe di codice e di testo.

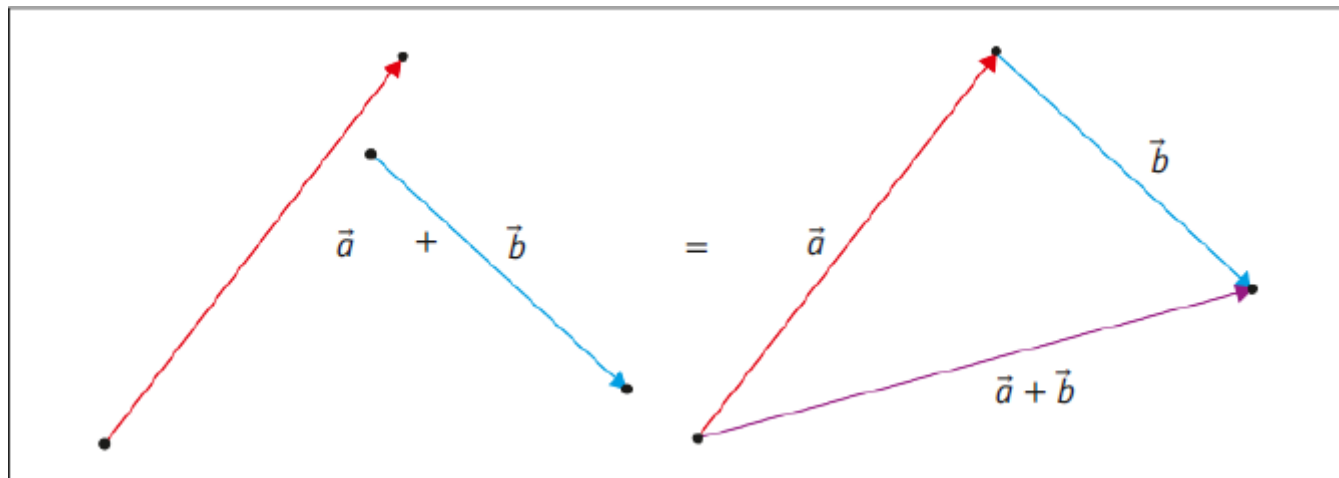


Dati vettoriali

In Informatica un vettore o array monodimensionale è una sequenza di valori omogenei, ossia, dello stesso tipo.

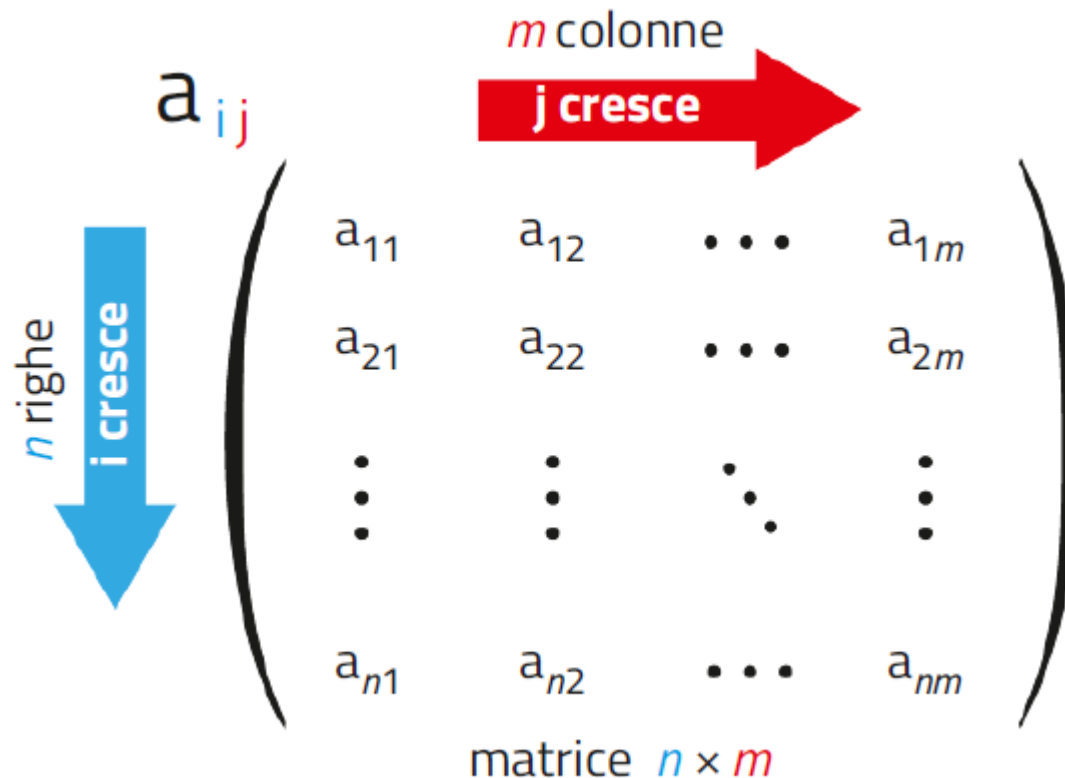
In Matematica, un **vettore** è un elemento di uno spazio vettoriale, ossia un insieme di elementi che supportano operazioni binarie, come addizione e moltiplicazione.

I valori che compongono un vettore sono detti componenti del vettore. È possibile rappresentare i vettori a due componenti su un piano cartesiano



Matrici

In Matematica, una **matrice** è una tabella, o array bidimensionale, di numeri ordinati in righe e colonne.



Altre tipologie di dato sono rappresentabili con array di dimensioni superiori.

Numpy

Per rappresentare vettore, matrici e array n -dimensionali in Python si utilizza la libreria *Numpy*.

Questa libreria è già installata nell'ambiente Colab, quindi è possibile importarla direttamente nella prima cella di codice del notebook.

```
✓ [1] import numpy as np
```

La prima operazione fondamentale è la creazione di un vettore. Dato che *Numpy* non fa differenza tra vettori, matrici e array n -dimensionali, la funzione base per la creazione di un oggetto *Numpy* si chiama *array*.

```
✓ [2] data = np.array([1, 2, 3])  
  
✓ [3] data  
→ array([1, 2, 3])
```

La funzione accetta in input una lista di numeri e restituisce in output un array 1-dimensionale, un array 1D, che supporta tutte le operazioni vettoriali matematiche più utilizzate.

Array 1D

	data		ones				
<code>data = np.array([1,2])</code>	<table><tr><td>1</td></tr><tr><td>2</td></tr></table>	1	2	<code>ones = np.ones(2)</code>	<table><tr><td>1</td></tr><tr><td>1</td></tr></table>	1	1
1							
2							
1							
1							

Come per le liste in Python, gli array D di *Numpy* supportano l'accesso tramite indice, [indexing](#), e la selezione di un sottosequenza dell'array, ovvero lo [slicing](#).

✓ 0s	[4] data[0]
	⇒ 1
✓ 0s	[5] data[1]
	⇒ 2
✓ 0s	[6] data[0:2]
	⇒ array([1, 2])
✓ 0s	[7] data[1:]
	⇒ array([2, 3])
✓ 0s	[8] data[-2:]
	⇒ array([2, 3])

Indexing e slicing di array 1D

Sia indexing che slicing supportano per gli indici valori negativi, che consentono di accedere agli elementi dell'array a partire dalla fine: -1 è l'indice dell'ultimo elemento dell'array, -2 del penultimo e così via, come per le liste e le stringhe.

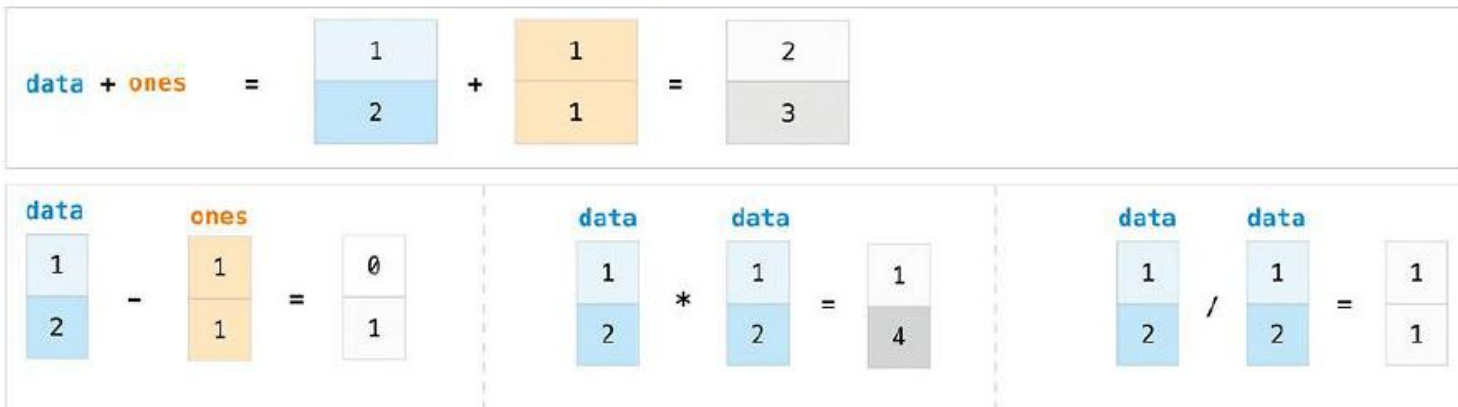
	data	data[0]	data[1]	data[0:2]	data[1:]	data[-2:]	
0	1	1		1			0
1	2		2	2	2	2	1
2	3				3	3	2
							3

Operazioni con gli array 1D

La somma di due array si traduce immediatamente nella somma delle componenti che costituiscono gli array stessi.

Il vantaggio di operare con è evidente: le operazioni elementari sono già implementate e non necessitano di essere re-implementate tramite cicli.

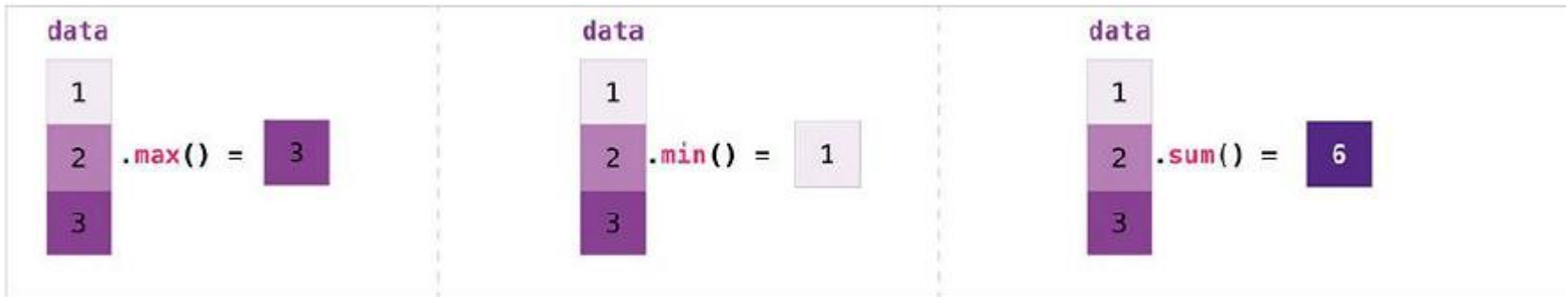
```
✓ [9] data, ones = np.array([1, 2]), np.array([1, 1])  
✓ [10] data + ones  
    ↪ array([2, 3])  
✓ # Differenza, prodotto e quoziente.  
  (data - ones, data * data, data / data)  
    ↪ (array([0, 1]), array([1, 4]), array([1., 1.]))
```



Array come oggetti

Un array è un oggetto *Numpy* e quindi, come un qualsiasi altro oggetto in Python, possiede attributi e metodi.

```
✓ [12] data = np.array([1, 2, 3])  
0s  
  
✓ [13] # Massimo, minimo e somma.  
0s      (data.max(), data.min(), data.sum())  
  
➡ (3, 1, 6)
```



Matrici come array 2D

Se un vettore viene creato usando una lista, possiamo creare una matrice usando una lista di liste.

```
✓ [14] data = np.array([  
0s     [1, 2],  
     [3, 4],  
     [5, 6]  
])
```

```
✓ [15] data  
0s  
⇌ array([[1, 2],  
        [3, 4],  
        [5, 6]])
```

```
np.array([[1,2],[3,4],[5,6]])
```



1	2
3	4
5	6

Indexing e slicing di array 2D

Indexing e slicing di una matrice funzionano in modo analogo a quelli di un vettore; l'unica differenza è data dal fatto che, visto che l'array è bidimensionale, sono presenti due indici invece di uno solo: il primo indice seleziona le righe, mentre il secondo seleziona le colonne.

Se il secondo indice non viene esplicitato, vengono selezionate automaticamente le righe:

```
✓ [16] data[0, 1]
```

```
⇒ 2
```

```
✓ [17] data[1:3]
```

```
⇒ array([[3, 4],  
        [5, 6]])
```

```
✓ [18] data[0:2, 0]
```

```
⇒ array([1, 3])
```

data		data[0,1]		data[1:3]		data[0:2,0]	
0	1	0	1	0	1	0	1
0	1	2		1	2	1	
1	3	4	3	4	3	3	4
2	5	6	5	6	5	5	6

Operazioni con gli array 2D

Le operazione vettoriali vengono eseguite allineando le righe e le colonne ed eseguendo le operazioni elemento per elemento.

```
✓ [19] data, ones = np.array([[1, 2], [3, 4]]), np.array([[1, 1], [1, 1]])
```

✓ [20] (data, ones)

```
⇒ (array([[1, 2],
          [3, 4]]),
    array([[1, 1],
          [1, 1]]))
```

✓ [21] data + ones

```
array([[2, 3],
       [4, 5]])
```

`data + ones` =

data	
1	2
3	4

ones	
1	1
1	1

=

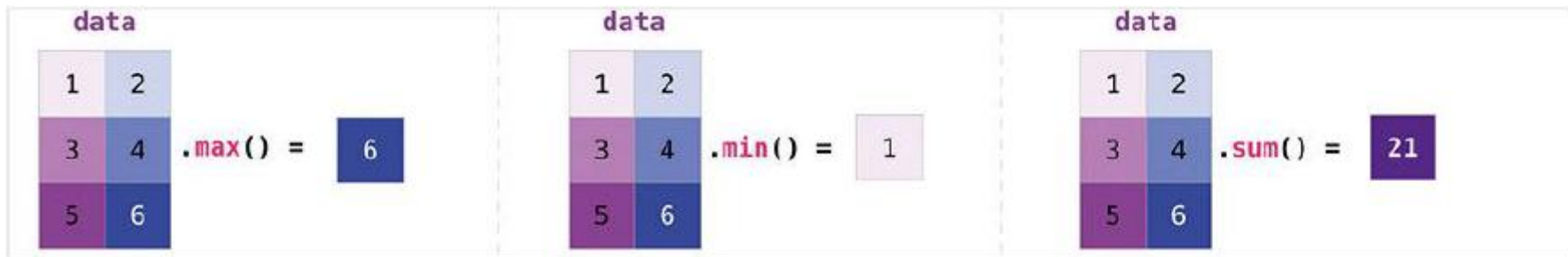
2	3
4	5

Operazioni massimo e minimo

```
✓ [22] data = np.array([  
    0s      [1, 2],  
           [3, 4],  
           [5, 6]  
    ])

✓ [23] (data.max(), data.min(), data.sum())  
      ↪ (6, 1, 21)
```

In questo caso, è necessario fare particolare attenzione al verso, o asse, lungo cui applichiamo l'operazione. Se non specifichiamo l'asse, allora l'operazione verrà eseguita su tutti gli elementi che compongono la matrice.



Operazioni lungo gli assi

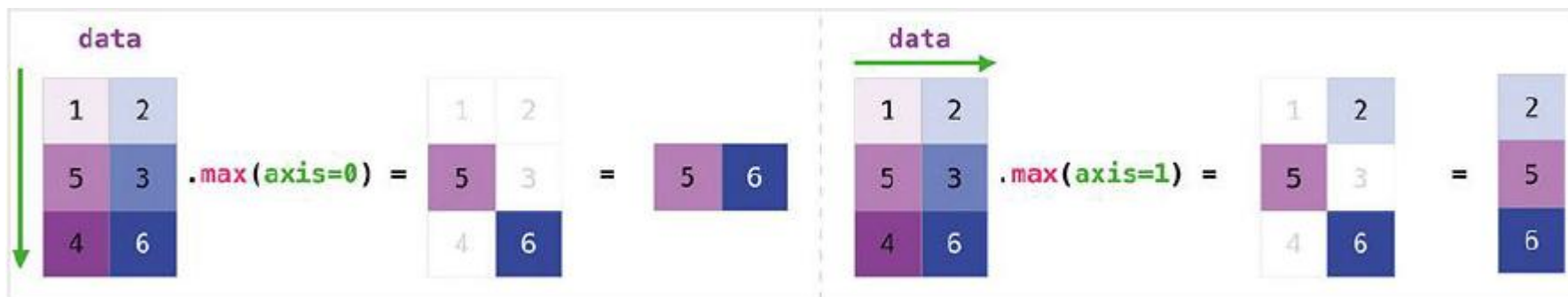
Se specifichiamo l'asse lungo cui calcolare il massimo, allora è possibile ottenere risultati per righe o per colonne configurando il valore del parametro opzionale `axis`.

Se `axis` è zero allora si fa riferimento alle righe, mentre se `axis` è 1 allora il riferimento è alle colonne.

```
✓ [24] data = np.array([  
0s      [1, 2],  
      [5, 3],  
      [4, 6]  
])
```

```
✓ [25] (data.max(axis = 0), data.min(axis = 0), data.sum(axis = 0))  
0s  
⇒ (array([5, 6]), array([1, 2]), array([10, 11]))
```

```
✓ [26] (data.max(axis = 1), data.min(axis = 1), data.sum(axis = 1))  
0s  
⇒ (array([2, 5, 6]), array([1, 3, 4]), array([ 3,  8, 10]))
```



Matrice trasposta

È possibile scambiare gli assi di una matrice righe e colonne grazie all'operatore `.T`, ottenendo così la matrice trasposta.

```
✓ [27] data = np.array([  
0s      [1, 2],  
      [3, 4],  
      [5, 6]  
      ])  
  
✓ [28] (data, data.T)  
⇒ (array([[1, 2],  
          [3, 4],  
          [5, 6]]),  
   array([[1, 3, 5],  
          [2, 4, 6]]))
```

data	
1	2
3	4
5	6

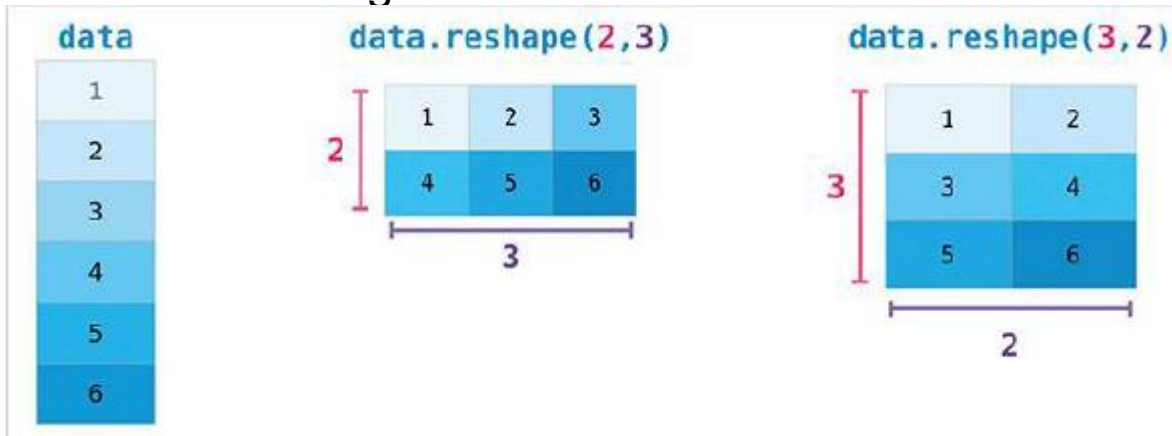
data.T		
1	3	5
2	4	6

Reshape

Se invece che scambiare gli assi vogliamo modificarne le lunghezze, possiamo utilizzare il metodo `reshape` per cambiare la forma dell'array

```
✓ [29] data = np.array([1, 2, 3, 4, 5, 6])  
  
✓ [30] (data, data.reshape(2, 3), data.reshape(3, 2))  
  
➔ (array([1, 2, 3, 4, 5, 6]),  
    array([[1, 2, 3],  
          [4, 5, 6]]),  
    array([[1, 2],  
          [3, 4],  
          [5, 6]]))
```

Ciò ci permette anche di passare da vettore a matrice e viceversa, a patto che il numero degli elementi della nuova forma sia lo stesso.



Generatore di numeri pseudo-casuali - 1

La generazione di numeri casuali non mai è completamente casuale, e per questo motivo viene detta pseudo-casuale. Questo perché ogni numero nella sequenza è generato a partire dal precedente, quindi, anche se la sequenza appare casuale, in realtà esiste una funzione che collega un numero al suo successivo.

Questa funzione viene detta **generatore di numeri pseudo-casuali**.

Uno dei più semplici generatori di numeri pseudo-casuali è il Generatore Lineare Congruenziale, LCG.

Un LCG si basa su una funzione ricorrente, che genera il numero x_{i+1} in funzione del numero x_i .

$$x_{i+1} = (ax_i + b) \bmod c$$

I parametri a , b e c sono detti rispettivamente *moltiplicatore*, *incremento* e *modulo*, e vengono scelti durante l'inizializzazione del generatore in modo da cambiare la sequenza.

Una volta fissati i parametri, è sufficiente scegliere un numero iniziale e applicare la funzione per poter generare il numero successivo.

Generatore di numeri pseudo-casuali - 2

Il generatore ha un periodo, ovvero una lunghezza massima della sequenza oltre la quale la sequenza si ripete.

Per un LCG il periodo massimo è dato dal parametro c .

A parità di parametri, il numero iniziale determina il punto di inizio della sequenza. Questo numero viene chiamato *seed*.

Per ottenere di nuovo la stessa sequenza basta scegliere lo stesso *seed*.

Come generare i numeri pseudo-casuali usando *Numpy*:

```
✓ [2] rng = np.random.default_rng()
```

Per generare un numero dobbiamo chiamare il metodo *random*

```
✓ [3] print(rng.random())
```

```
➞ 0.6042619999005413
```

Possiamo specificare il *seed* del generatore durante l'inizializzazione.

```
✓ [4] seed = 42                                     # Inizializzazione del seed.  
      rng = np.random.default_rng(seed)             # Assegnazione del seed.  
      print(rng.random())
```

```
➞ 0.7739560485559633
```

Generatore di numeri pseudo-casuali - 3

Il metodo `random` accetta anche un parametro opzionale in ingresso che viene utilizzato per determinare la forma di un array, permettendo in questo modo di generare array n -dimensionali di numeri pseudo-casuali.

Di base, i numeri generati sono in virgola mobile, ma possiamo utilizzare il metodo `integers` e specificare l'intervallo dei numeri interi da generare.

```
✓ [5] A = rng.random((2, 3))           # Matrice 2 x 3 numeri reali.  
    B = rng.integers(0, 10, (2, 3))    # Matrice 2 x 3 numeri interi da 0 a 9.  
    print("A: \n", A)  
    print("B: \n", B)
```

```
⇒ A:  
  [[0.43887844 0.85859792 0.69736803]  
   [0.09417735 0.97562235 0.7611397 ]]  
 B:  
  [[7 7 5]  
   [1 8 4]]
```

Generatore di stringhe

La libreria *Faker*, <https://faker.readthedocs.io>, utilizza un generatore di numeri casuali per simulare dei dati realistici come stringhe.

Faker non è una libreria presente di default nell'ambiente Colab, quindi è necessario installarla utilizzando il comando *pip*.

Il punto esclamativo all'inizio della cella di codice permette di eseguire comandi direttamente nel terminale Colab invece che nell'interprete Python.

```
✓ [1] !pip install faker
```

Per utilizzarla la dovremo importare e proseguire nell'inizializzazione del *seed*. Inoltre, *Faker* supporta la generazione di dati in diverse lingue anche più lingue contemporaneamente.

Costruiamo il generatore:

```
✓ [3] seed = 42                # Inizializzazione del seed.  
      Faker.seed(seed)        # Assegnazione del seed.  
      rng = Faker("it_IT")    # Costruzione del generatore.
```

Per generare un valore casuale è sufficiente invocare uno dei tanti metodi disponibili per questo oggetto.

Generatore di un cliente casuale

Nella libreria *Faker* esistono metodi specifici a seconda del tipo di valore che vogliamo generare.

Per esempio, il metodo `first_name` genera un nome di persona, mentre il metodo `city` genera un nome di città.

Per semplicità possiamo definire una funzione che accetta il generatore come parametro e restituisce un dizionario che rappresenta un cliente casuale.

```
✓ [4] def rng_cliente(rng):  
0s     return {  
        "nome": rng.first_name(),  
        "cognome": rng.last_name(),  
        "città": rng.city(),  
        "cap": rng.postcode(),  
        "via": rng.street_address(),  
        "telefono": rng.phone_number()  
    }
```

```
✓ [5] rng_cliente(rng)  
0s  
⇒ {'nome': 'Eugenia',  
   'cognome': 'Caracciolo',  
   'città': 'Bianco',  
   'cap': '46100',  
   'via': 'Via Loredana, 18',  
   'telefono': '+39 0322338903'}
```

Pandas e i dati tabulari

La libreria *Pandas*, è già installata nell'ambiente Colab, quindi è possibile importarla direttamente nella prima cella di codice del notebook.

Si basa su un oggetto chiamato *DataFrame*, che rappresenta di fatto una tabella di dati.

Per costruire l'oggetto basta chiamare il suo costruttore e passargli la lista di dizionari.

In questo oggetto, le chiavi dei dizionari corrispondono alle colonne della tabella, mentre le righe sono dati dai valori dei dizionari.

Ogni riga viene numerata in modo incrementale partendo da zero fino al numero massimo di elementi meno uno. Questa numerazione prende il nome di indice, che verrà utilizzato nella fase di accesso e manipolazione dei dati.

```
✓ 6s ▶ clienti = pd.DataFrame(clienti)

clienti
```

	nome	cognome	città	cap	via	telefono
0	Ricciotti	Lettiere	Seriate	00043	Contrada Cremonesi, 5 Piano 1	3534594071
1	Amanda	Magnani	Termini	19121	Canale Gianluigi, 1	0536752551
2	Concetta	Donini	Corciano	00077	Contrada Volta, 5 Appartamento 55	+39 04329537622
3	Gioffre	Zamorani	Viazzano	93017	Borgo Cicilia, 1 Appartamento 22	0709784805
4	Costanzo	Magrassi	Osteria Del Gatto	00031	Contrada Temistocle, 98 Appartamento 32	+39 0965570153
...	...	...	...	...	...	...
4995	Elisa	Cignaroli	Castelfidardo	28823	Vicolo Eliana, 840	371349677
4996	Massimo	Pulci	Rovito	02016	Stretto Ennio, 8 Appartamento 5	034394742
4997	Iolanda	Barsanti	Sovere	20831	Piazza Calogero, 77	+39 04314315795
4998	Loretta	Bolto	San Felice A Cancelli	26029	Incrocio Tatiana, 86 Appartamento 3	+39 0304376326
4999	Luisa	Garibaldi	San Giuseppe Jato	33036	Canale Paolo, 17 Appartamento 5	+39 043800767

5000 rows × 6 columns

Input e output di dati tabulari

È possibile salvare su un file in formato CSV i dati che abbiamo appena generato con il metodo `to_csv`. Di solito, l'indice viene escluso dal salvataggio su file.

```
✓ [8] clienti.to_csv("clienti.csv", index = False)
```

Se apriamo il file con l'I/O di Python vediamo che la prima riga contiene l'intestazione delle colonne e i dati partono dalla seconda riga del file.

```
✓ [9] f = open("clienti.csv", "r")  
      print(f.readline())  
      print(f.readline())  
      f.close()
```

```
➡ nome,cognome,città,cap,via,telefono
```

```
Ricciotti,Lettiere,Seriate,00043,"Contrada Cremonesi, 5 Piano 1",3534594071
```

Per leggere il file CSV possiamo usare la funzione `pd.read_csv`.

Accesso al DataFrame - 1

I metodi `head` e `tail`, visualizzano rispettivamente le prime cinque righe e le ultime cinque righe del DataFrame.

```
[11] clienti.head()
```

	nome	cognome	città	cap	via	telefono
0	Ricciotti	Lettiere	Seriate	00043	Contrada Cremonesi, 5 Piano 1	3534594071
1	Amanda	Magnani	Termini	19121	Corso Garibaldi, 1	0536388551
2	Concetta	Donini	Corciano	00077		
3	Gioffre	Zamorani	Viazzano	93017		
4	Costanzo	Magrassi	Osteria Del Gatto	00031	C	

```
[12] clienti.tail()
```

	nome	cognome	città	cap	via	telefono
4995	Elisa	Cignaroli	Castelfidardo	28823	Vicolo Eliana, 840	371349677
4996	Massimo	Pulci	Rovito	02016	Stretto Ennio, 8 Appartamento 5	034394742
4997	Iolanda	Barsanti	Sovere	20831	Piazza Calogero, 77	+39 04314315795
4998	Loretta	Boito	San Felice A Cancellio	26029	Incrocio Tatiana, 86 Appartamento 3	+39 0304376326
4999	Luisa	Garibaldi	San Giuseppe Jato	33036	Canale Paolo, 17 Appartamento 5	+39 043800767

Se vogliamo accedere arbitrariamente a righe e colonne dobbiamo basarci sui valori dell'indice e sui nomi delle colonne.

```
[13] clienti.index
```

```
RangeIndex(start=0, stop=5000, step=1)
```

```
[14] clienti.columns
```

```
Index(['nome', 'cognome', 'città', 'cap', 'via', 'telefono'], dtype='object')
```

Accesso al DataFrame - 2

Se vogliamo accedere alla colonna nome è sufficiente passare questa stringa al DataFrame tra parentesi quadre.

Mentre se volessimo un sottoinsieme delle righe ci basterebbe passare un intervallo di valori.

```
✓ [15] clienti["nome"]
0s
➡ 0 Ricciotti
1 Amanda
2 Concetta
3 Gioffre
4 Costanzo
...
4995 Elisa
4996 Massimo
4997 Iolanda
4998 Loretta
4999 Luisa
Name: nome, Length: 5000, dtype: object
```

```
✓ [16] clienti[5:12]
0s
➡
```

	nome	cognome	città	cap	via	telefono
5	Gioacchino	Zola	Ruvo Del Monte	84075	Stretto Marisa, 92 Piano 8	37763834671
6	Gioffre	Gasperi	Caporosso	13853	Vicolo Giancarlo, 1	+39 3511051837
7	Adriana	Cherubini	Paina	26833	Borgo Caccioppoli, 786 Appartamento 1	34351333822
8	Gustavo	Scandone	Altofonte	80100	Piazza Fernanda, 1 Piano 7	04326026044
9	Piersanti	Vivaldi	Tremignon	02043	Borgo Biagi, 5	086282081
10	Carolina	Boldù	Pomaretto	74018	Vicolo Carolina, 99	37114353427
11	Luchino	Bonolis	Obermais	97019	Stretto Giampaolo, 22	0541278491

Accesso al DataFrame - loc

Il metodo `loc` combina i due approcci e permette di passare in ordine prima l'intervallo per le righe e poi il nome delle colonne.

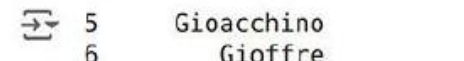
Il metodo `loc` è molto flessibile e accetta anche una lista di colonne per estrarre un sottoinsieme del DataFrame.

```
✓ [18] clienti.loc[5:12, ["nome", "cognome", "telefono"]]
```



	nome	cognome	telefono
5	Gioacchino	Zola	37763834671
6	Gioffre	Gasperi	+39 3511051837
7	Adriana	Cherubini	34351333822
8	Gustavo	Scandone	04326026044
9	Piersanti	Vivaldi	086282081
10	Carolina	Boldù	37114353427
11	Luchino	Bonolis	0541278491
12	Tina	Chindamo	05001640022

```
✓ [17] clienti.loc[5:12, "nome"]
```



```
5    Gioacchino
6      Gioffre
7       Adriana
8       Gustavo
9     Piersanti
10    Carolina
11     Luchino
12        Tina
Name: nome, dtype: object
```


Selezione di dati - 1

Le **maschere binarie** di selezione sono sequenze di valori vero o falso che vengono utilizzate per selezionare le righe, utilizzando una condizione.

Una volta definita la maschera binaria, questa può essere utilizzata per accedere alle righe. In questo modo, verranno selezionate solo le righe che soddisfano

Questa condizione, e cioè che presentano il valore True nella maschera.

```
✓ [19] clienti["nome"] == "Maria"
0s
```

0	False
1	False
2	False
3	False
4	False
...	
4995	False
4996	False
4997	False
4998	False
4999	False

Name: nome, Length: 5000, dtype: bool

```
✓ [20] clienti[clienti["nome"] == "Maria"].head()
0s
```

	nome	cognome	città	cap	via	telefono
181	Maria	Squarcione	Castellanza	81014	Borgo Bataglia, 788 Appartamento 20	37774989269
192	Maria	Morosini	Balbiano	91017	Strada Guglielmi, 6	0736588041
285	Maria	Mazzocchi	Capodarco	52047	Viale Mariana, 4	08516479434
353	Maria	Contarini	Alessandria Della Rocca	26024	Viale Gianpaolo, 247	3246377802
548	Maria	Foscari	San Rocchetto	56028	Incrocio Federico, 891 Appartamento 7	+39 0706615544

Selezione di dati - 2

Quando è necessario confrontare più valori è possibile utilizzare il metodo `isin`, che restituisce vero se il valore della riga è contenuto all'interno di una lista di selezione.

```
✓ [21] clienti[clienti["nome"].isin(["Maria", "Luca"])].head()
```



	nome	cognome	città	cap	via	telefono
181	Maria	Squarcione	Castellanza	81014	Borgo Bataglia, 788 Appartamento 20	37774989269
192	Maria	Morosini	Balbiano	91017	Strada Guglielmi, 6	0736588041
285	Maria	Mazzocchi	Capodarco	52047	Viale Mariana, 4	08516479434
313	Luca	Santorio	Colpalombo	20094	Borgo Fiorino, 299 Appartamento 20	+39 03977697461
353	Maria	Contarini	Alessandria Della Rocca	26024	Viale Gianpaolo, 247	3246377802

Manipolazione di dati

È possibile aggiungere nuovi dati e manipolare quelli esistenti, per esempio, è possibile aggiungere una nuova colonna nello stesso modo in cui aggiungiamo un valore a un dizionario.

```
[22] data_di_nascita = []
      for _ in range(len(clienti)):
          data_di_nascita.append(
              rng.date_of_birth(
                  minimum_age = 18,
                  maximum_age = 95
              )
          )
      clienti["data_di_nascita"] = data_di_nascita

      clienti.head()
```

	nome	cognome	città	cap	via	telefono	data_di_nascita
0	Ricciotti	Lettiere	Seriate	00043	Contrada Cremonesi, 5 Piano 1	3534594071	1990-06-10
1	Amanda	Magnani	Termini	19121	Canale Gianluigi, 1	0536752551	1929-10-23
2	Concetta	Donini	Corciano	00077	Contrada Volta, 5 Appartamento 55	+39 04329537622	1964-09-29
3	Gioffre	Zamorani	Viazzano	93017	Borgo Cicilia, 1 Appartamento 22	0709784805	1939-07-27
4	Costanzo	Magrassi	Osteria Del Gatto	00031	Contrada Temistocle, 98 Appartamento 32	+39 0965570153	1942-01-23

Gestire le date

Python supporta nativamente il tipo di valore `data`, che è composto da una tupla di tre numeri rispettivamente anno, mese e giorno

```
✓ [23] clienti.loc[0, "data_di_nascita"]  
↳ datetime.date(1990, 6, 10)
```

Pandas fornisce il metodo `apply` che accetta in input una funzione e restituisce una nuova sequenza di dati generata dall'applicazione della funzione a una colonna.

```
✓ [24] eta = clienti["data_di_nascita"].apply(lambda x: 2025 - x.year)  
eta  
↳  
0      35  
1      96  
2      61  
3      86  
4      83  
..  
4995   56  
4996   62  
4997   22  
4998   65  
4999   86  
Name: data_di_nascita, Length: 5000, dtype: int64
```

Dati mancanti

I dati mancanti assumono il valore `pd.NA`, dove NA sta per *Not available*, ossia non disponibile, mancante.

È possibile controllare la presenza di dati mancanti con la funzione `isna`.

```
✓ [29] clienti.isna()
```

	nome	cognome	città	cap	via	telefono	data_di_nascita	età
0	False	False	False	False	False	False	False	False
1	False	False	False	False	False	True	False	False
2	False	False	False	False	False	True	False	False
3	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...
4995	False	False	False	False	False	False	False	False
4996	False	False	False	False	False	True	False	False
4997	False	False	False	False	False	True	False	False
4998	False	False	False	False	False	True	False	False
4999	False	False	False	False	False	False	False	False

5000 rows x 8 columns

Visualizzare dati - Matplotlib

Per visualizzare i dati in un progetto Python utilizziamo la libreria *Matplotlib*, che è già inclusa in Colab.

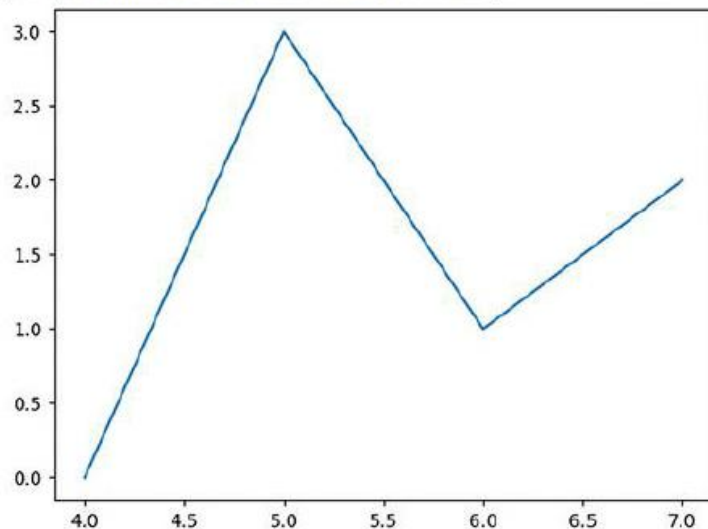
```
[1] import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Grazie a questa libreria è possibile visualizzare dei dati su un piano cartesiano semplicemente fornendo le coppie di coordinate.

La funzione `plt.plot` collega automaticamente le coordinate che vengono inserite, formando una linea spezzata che mostra l'andamento dei valori.

```
x = np.array([4, 5, 6, 7])
y = np.array([0, 3, 1, 2])
plt.plot(x, y)
```

[<matplotlib.lines.Line2D at 0x7d912392e650>]



Anatomia di una figura

La funzione `plt.plot` è utile per visualizzare dati velocemente senza dover utilizzare combinazioni complesse di oggetti e metodi.

In una figura ogni elemento assolve a una funzione specifica:

- `plt.figure` alloca memoria per la figura;
- `fig.subplots` inizializza il piano cartesiano;
- `ax.plot` permette di visualizzare dati sul piano cartesiano;
- `ax.scatter` visualizza i punti sul piano;
- `ax.set_title` imposta i titola della figura;
- `ax.set_xlabel` e `ax.set_ylabel` impostano le etichette degli assi;
- `ax.legend` imposta la legenda della figura.

Capitolo 9

Intelligenza Artificiale

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume *informatica per progetti – Programmare in Python*, di M. Fiore, A. Zanga, ISBN 9788808899477

Intelligenza Artificiale IA

Con l'espressione **Intelligenza Artificiale** (IA) intendiamo un sistema artificiale, costruito dall'uomo, in grado di simulare l'intelligenza umana.

Possiamo considerare come un'applicazione di Intelligenza Artificiale anche un programma che gestisce il riconoscimento e la rimozione automatica delle mele difettose immesse nella linea di produzione di un impianto industriale.

Il sistema è **artificiale**: è composto da un sensore che cattura le immagini delle mele che si muovono su un nastro trasportatore, ma la definizione di «mela difettosa» è vaga, non è precisa, può essere data da diversi fattori, quindi è complessa.

Il sistema non è detto sia **intelligente**: saper riconoscere una mela buona da una non buona richiede un certo livello di «intelligenza» e bisogna essere in grado di mettere insieme diverse informazioni, per poi decidere che, effettivamente, quella mela è da rimuovere.

Intelligenza Artificiale e modelli

Un modello di Intelligenza Artificiale è un **modello matematico**.

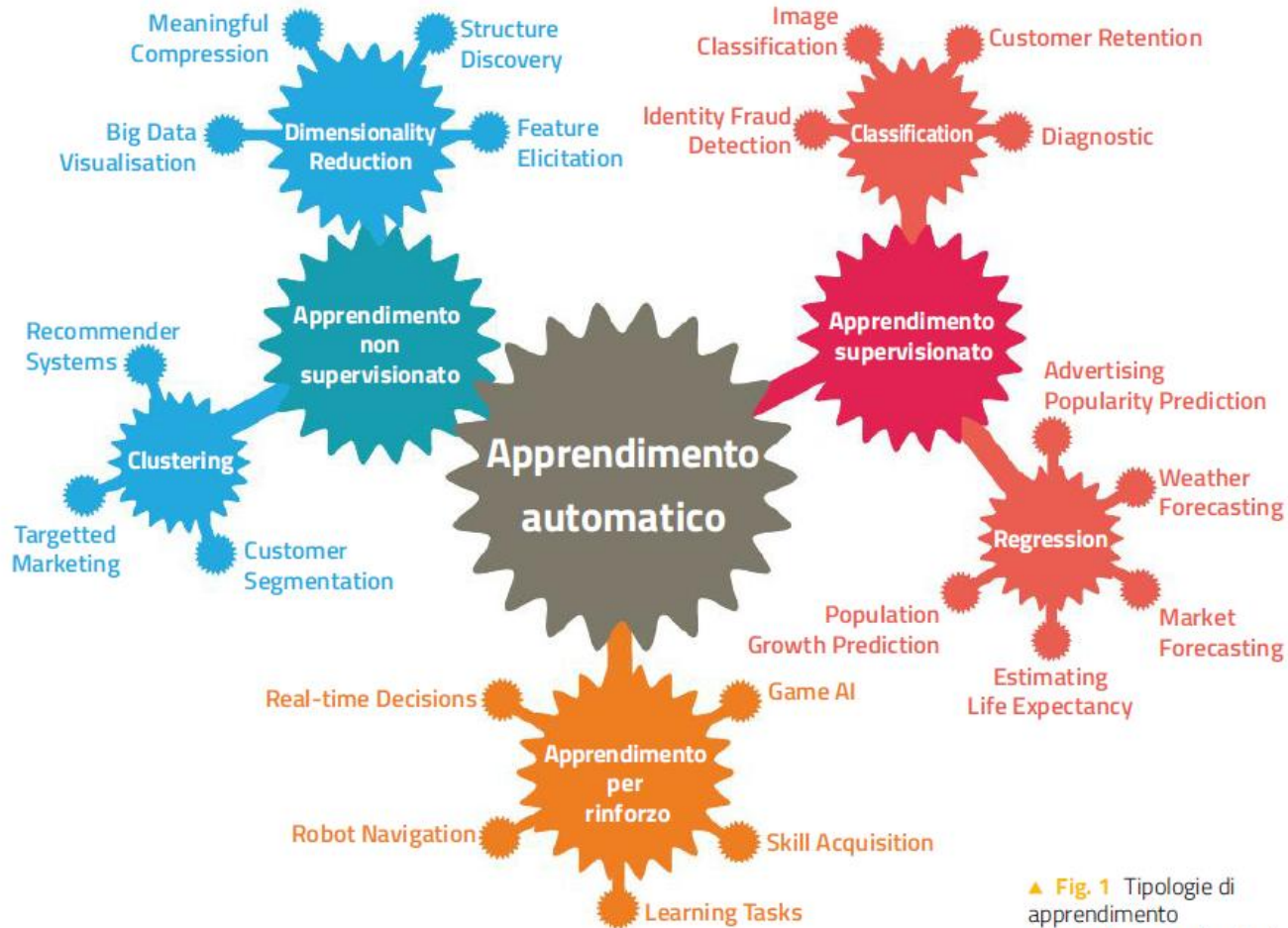
Possiamo dire che è una funzione matematica che rappresenta in modo formale e quantitativo, attraverso i numeri, un fenomeno naturale complesso.

Un modello di Intelligenza Artificiale ha dei **parametri** che determinano il valore dell'output al variare dell'input.

I parametri di un modello di Intelligenza Artificiale vengono appresi dai dati.

Per questo motivo si utilizza l'espressione **Machine learning**, in cui esiste un algoritmo che apprende, impara e stima i parametri dai dati che vengono forniti in ingresso.

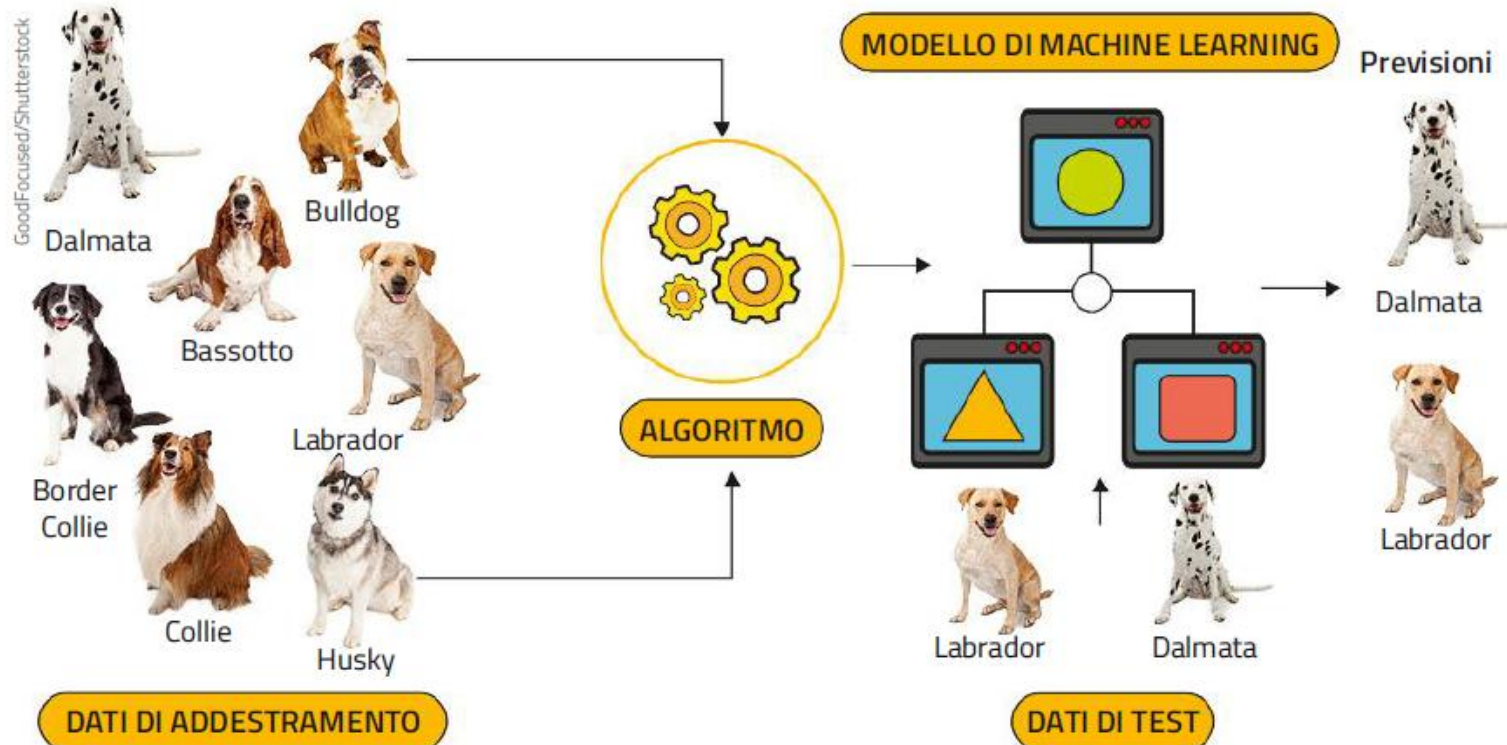
Apprendimento automatico



▲ Fig. 1 Tipologie di apprendimento automatico

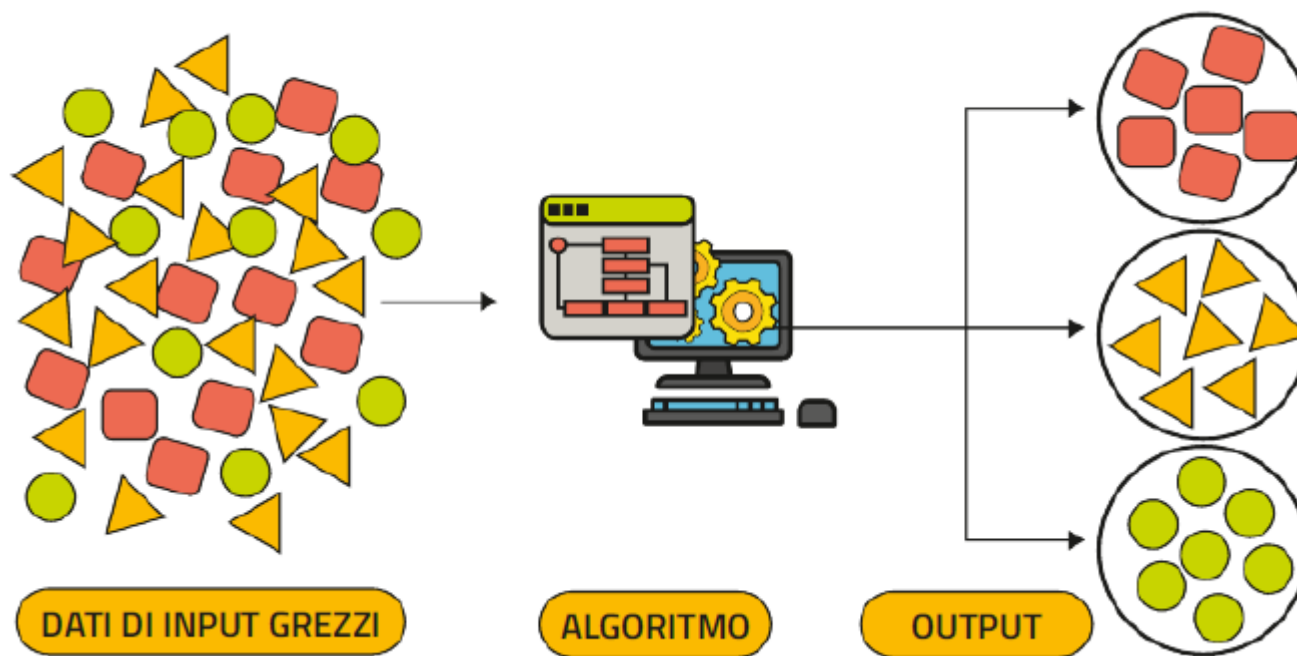
Apprendimento supervisionato

I dati in ingresso sono associati a dei valori in uscita, spesso chiamati *categorie*, *classi* o *etichette*. Il processo di apprendimento ha come obiettivo quello di stimare i parametri del modello in modo che sia in grado di predire la classe di appartenenza partendo dai dati in ingresso.



Apprendimento non supervisionato

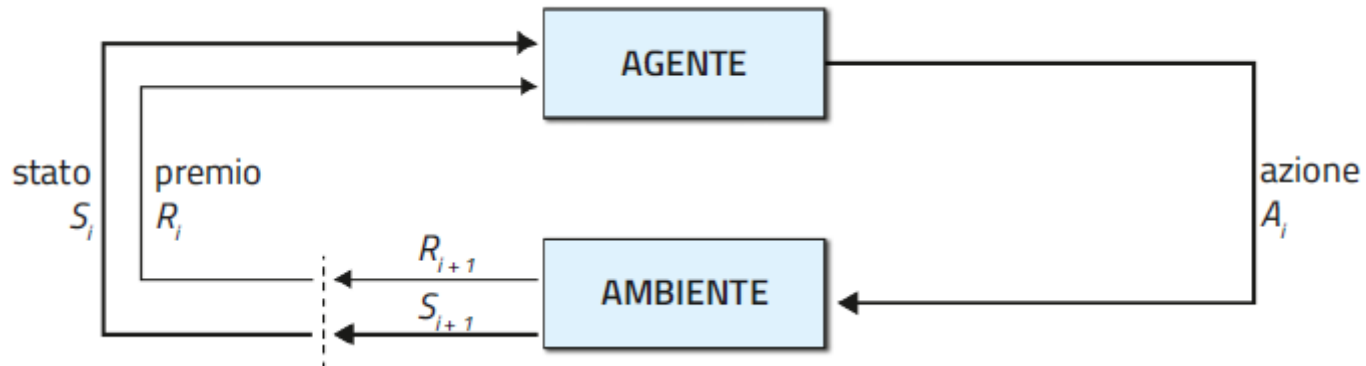
Al contrario dell'apprendimento supervisionato, i dati non sono etichettati. Lo scopo di questo tipo di apprendimento è quello di scoprire similarità e dissimilarità tra i dati in ingresso, in modo da evidenziare una possibile struttura nascosta nei dati.



Apprendimento per rinforzo

Il modello è in grado di interagire con l'ambiente circostante eseguendo azioni e osservando l'effetto di tali azioni sull'ambiente.

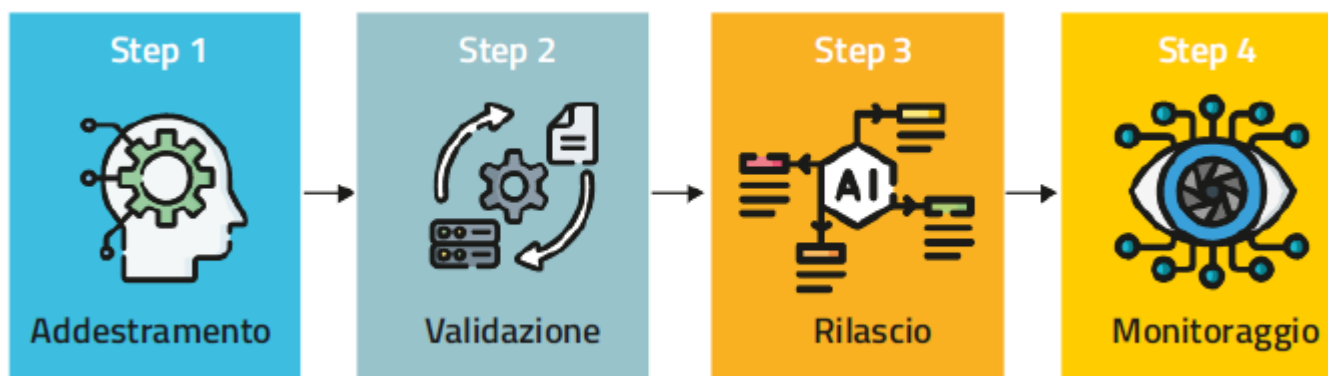
A ogni azione viene associato un valore che rappresenta l'utilità, il premio, che il modello riceve nel compiere tale azione. Per questo motivo spesso si parla di *agente* più che di modello, proprio per sottolineare la capacità di influenzare l'ambiente in cui si trova.



Ciclo di sviluppo di un modello

L'intero processo di sviluppo di un modello di Intelligenza Artificiale è costituito da quattro fasi:

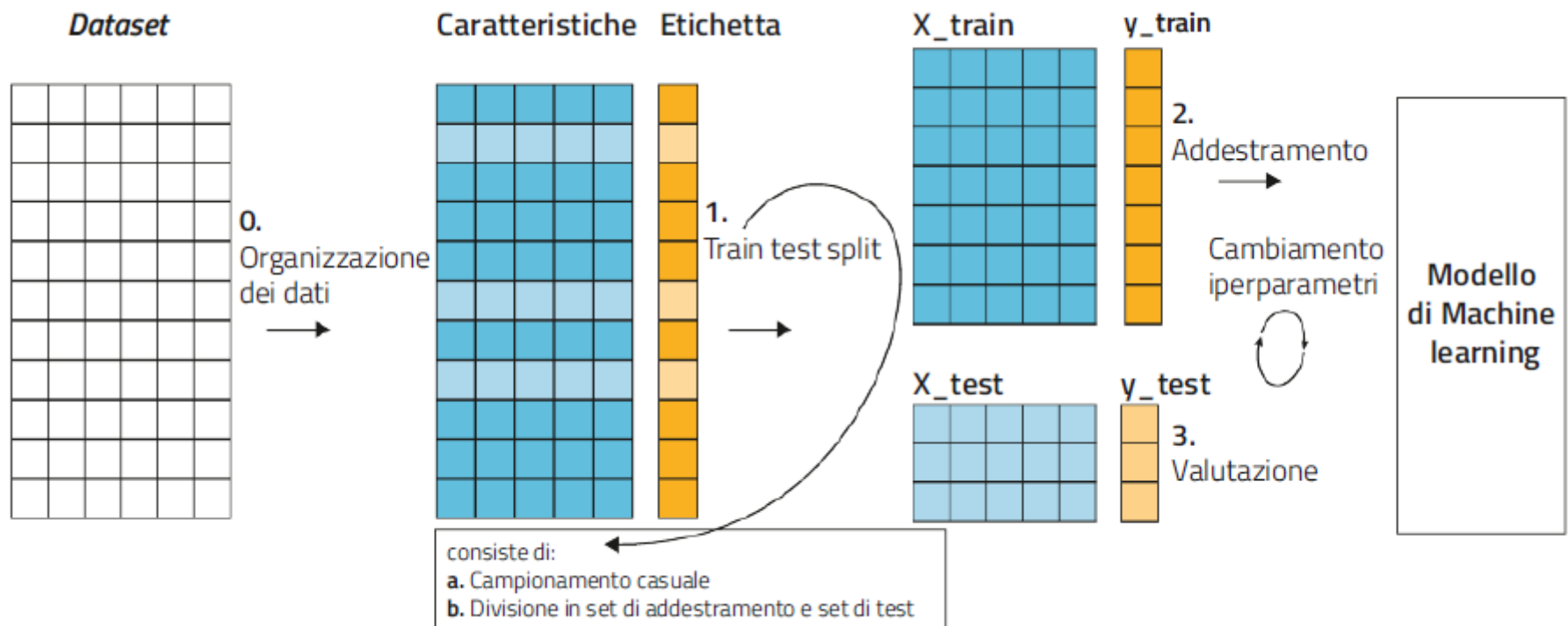
1. **Addestramento**, che ha come scopo principale la stima dei parametri di un modello.
2. **Validazione**, che consiste in una fase di valutazione che permette di acquisire informazioni sulle prestazioni del modello nello specifico compito per cui è stato addestrato.
3. **Rilascio**, in cui, una volta addestrato e validato, il modello viene utilizzato.
4. **Monitoraggio**, in cui le prestazioni del modello vengono costantemente monitorate, così da valutarne l'adeguatezza e l'efficienza. Se le prestazioni dovessero essere inferiori a quanto stimato, allora il ciclo riprende dall'inizio per individuare un nuovo modello più adatto.



Train-test split

Il train-test split, consiste nel suddividere i dati in due parti, una per la fase di addestramento e l'altra per la fase di validazione.

I parametri del modello verranno stimati nella fase di addestramento su un sottoinsieme dei dati disgiunto da quello che verrà usato nella fase di validazione, ottenendo una valutazione delle prestazioni del modello che non si basa sui dati già utilizzati in precedenza. Una volta validato il modello, la fase di sviluppo termina.



Scikit-Learn

La libreria Python **Scikit-Learn** permette di utilizzare algoritmi di apprendimento che sono considerati fondanti nel campo dell'apprendimento automatico.

Ogni modello consiste in una classe Python che implementa un'interfaccia comune, di cui ci basta conoscere alcuni metodi base:

Metodo	Descrizione
<code>fit</code>	Esegue la fase di stima dei parametri del modello.
<code>predict</code>	Stima i valori di output partendo dai valori in input e i parametri appresi.
<code>score</code>	Fornisce una metrica di valutazione del modello che varia a seconda del contesto specifico in cui viene calcolata.

Metodi fondamentali

Fase del ciclo di vita	Metodo	Descrizione
Costruzione	<code>__new__</code>	• Crea una nuova istanza non inizializzata • Viene chiamata in fase di creazione • Se non è definito, viene chiamato quello della classe base <code>object</code>
Inizializzazione	<code>__init__</code>	• Inizializza l'istanza appena creata • Viene chiamata dopo <code>__new__</code> • Il suo scopo è quello di assegnare un valore agli attributi dell'istanza • Se non è definito, viene chiamato quello della classe base <code>object</code>
Distruzione	<code>__del__</code>	• Distrugge l'istanza • Viene chiamato implicitamente quando l'istanza non è più visibile, oppure esplicitamente con <code>del</code> • Se non è definito, viene chiamato quello della classe base <code>object</code>

In generale, non definiremo né il costruttore né il distruttore, ma solo l'inizializzatore, in modo da delegare a `object` la gestione di queste due fasi del ciclo di vita della classe.

Associazione o causalità?

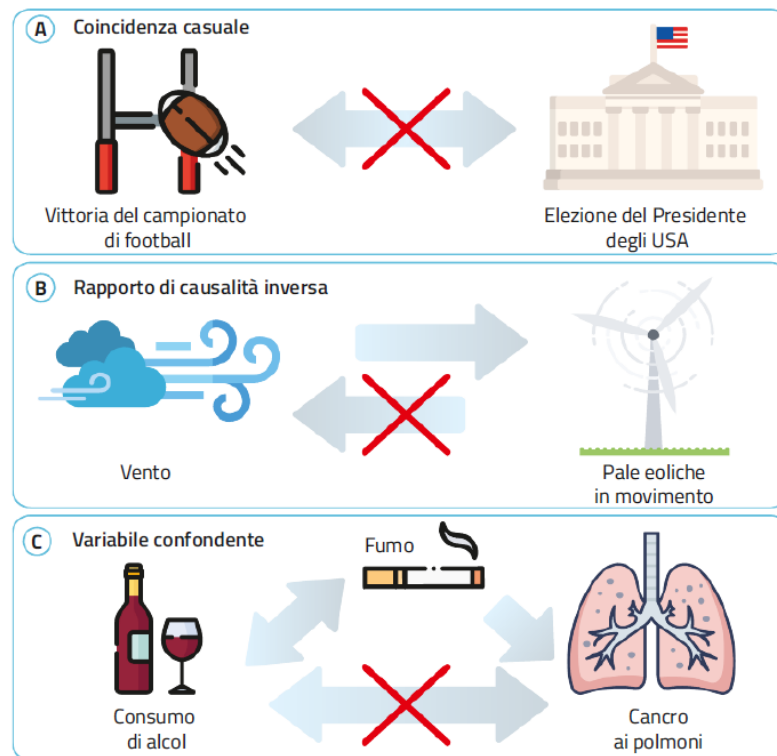
L'associazione non è causalità.

Un modello di Intelligenza Artificiale viene addestrato **per riconoscere schemi ricorrenti nei dati forniti**, ma un modello matematico non è senziente, non ha una nozione di causa ed effetto e non è in grado di comprendere il meccanismo che genera i dati.

Coincidenza casuale: due fenomeni avvengono in contemporanea anche se nessuno dei due influenza l'altro.

Rapporto di causalità inversa: la causa viene scambiata per l'effetto, e viceversa.

Variabile confondente: una terza variabile è causa di due variabili che sembrano essere tra di loro associate.

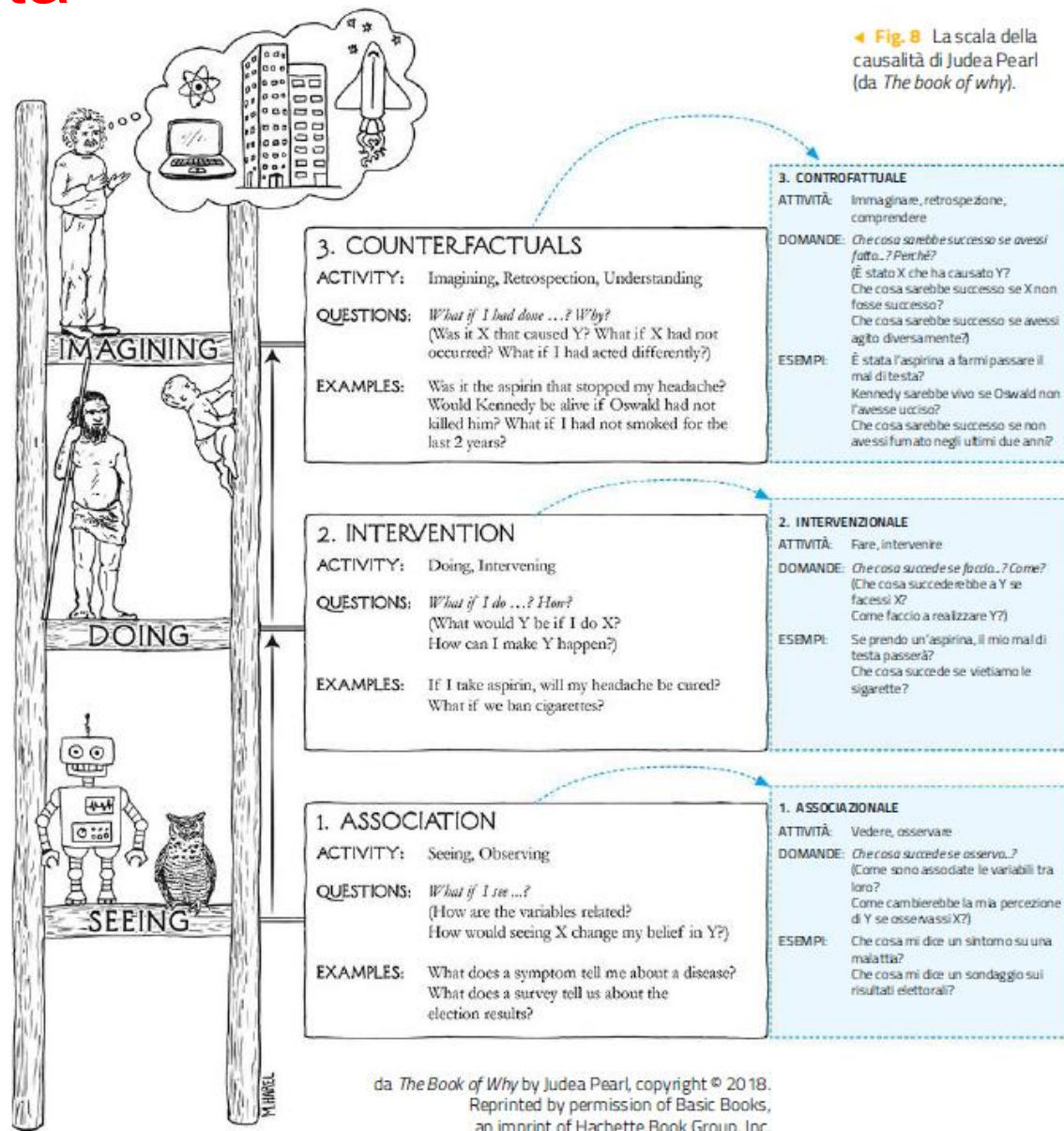


Scala della causalità

I dati da soli non bastano a spiegare il fenomeno naturale da cui li abbiamo raccolti.

Serve poter dare una interpretazione causa-effetto per essere in grado di comprendere ciò che avviene nella realtà dei fatti.

La **scala della causalità** ideata dall'informatico Judea Pearl, assegna ai vari modelli di Intelligenza Artificiale un livello di complessità e identifica le domande a cui possono rispondere.



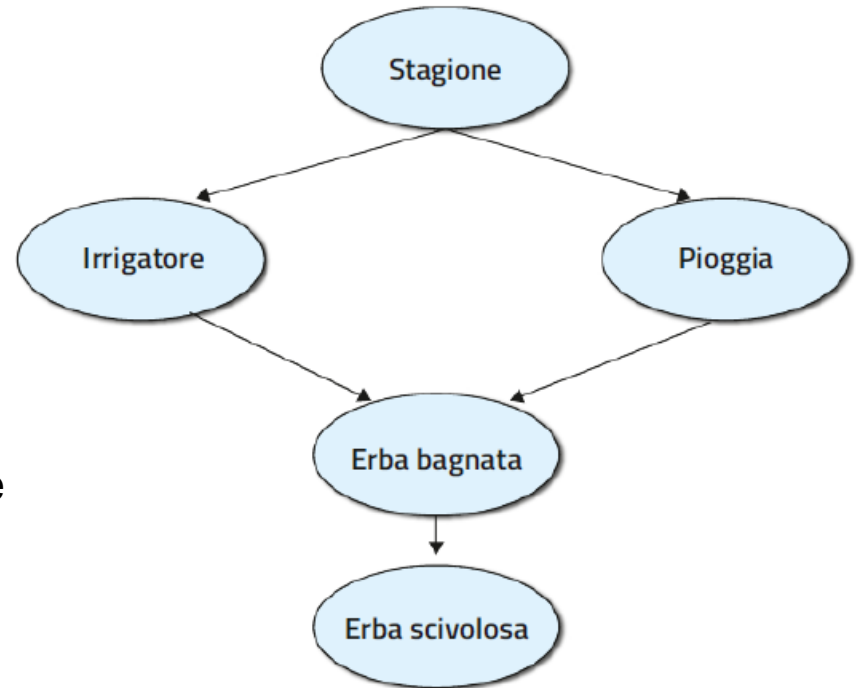
Grafo causale

Un modello di Intelligenza Artificiale in grado di rappresentare queste relazioni è il grafo causale.

Un **grafo causale** è un grafo, cioè un modello matematico composto da nodi e archi, che rappresenta le cause e i loro effetti con una struttura gerarchica.

Ogni nodo rappresenta una variabile di cui raccogliamo i dati, ovvero una colonna nel nostro *dataset*, e ogni arco orientato da un nodo a un altro significa che il primo è causa del secondo e, viceversa, il secondo è effetto del primo.

Questo grafo è molto efficace per spiegare la catena di cause ed effetti anche in caso di un numero elevato di variabili: basta seguire il verso degli archi.



Apprendimento supervisionato

L'apprendimento supervisionato può essere utilizzato per diversi scopi, uno dei quali è quello della classificazione.

Il problema della **classificazione** consiste nel classificare, riconoscere e predire la classe di appartenenza di una determinata istanza.

In questo caso abbiamo una collezione di oggetti con delle caratteristiche a cui è stata associata una classe, detta anche *categoria*.

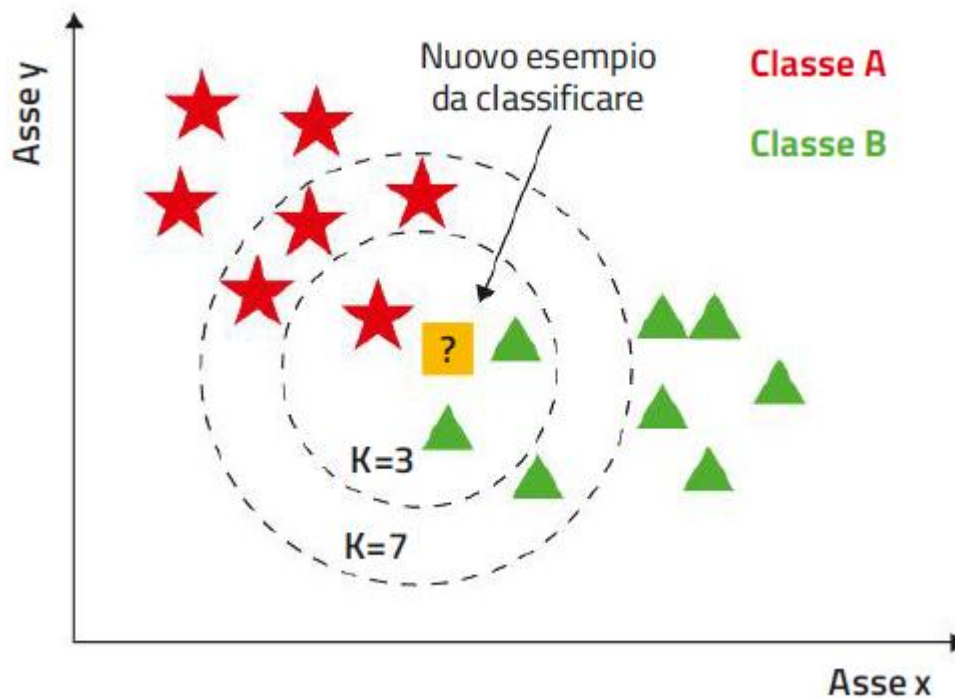
Risolvere un problema di classificazione significa addestrare un modello in grado di imparare la relazione tra le caratteristiche di un oggetto e la sua categoria di appartenenza.

I principali modelli noti sono:

- *k-Nearest Neighbors* (k-NN), in italiano «i primi *k*-vicini»;
- *Support Vector Classifier* (SVC), tradotto come «classificatore a vettori di supporto»;
- *Decision Tree Classifier* (DTC), un albero decisionale per problemi di classificazione.

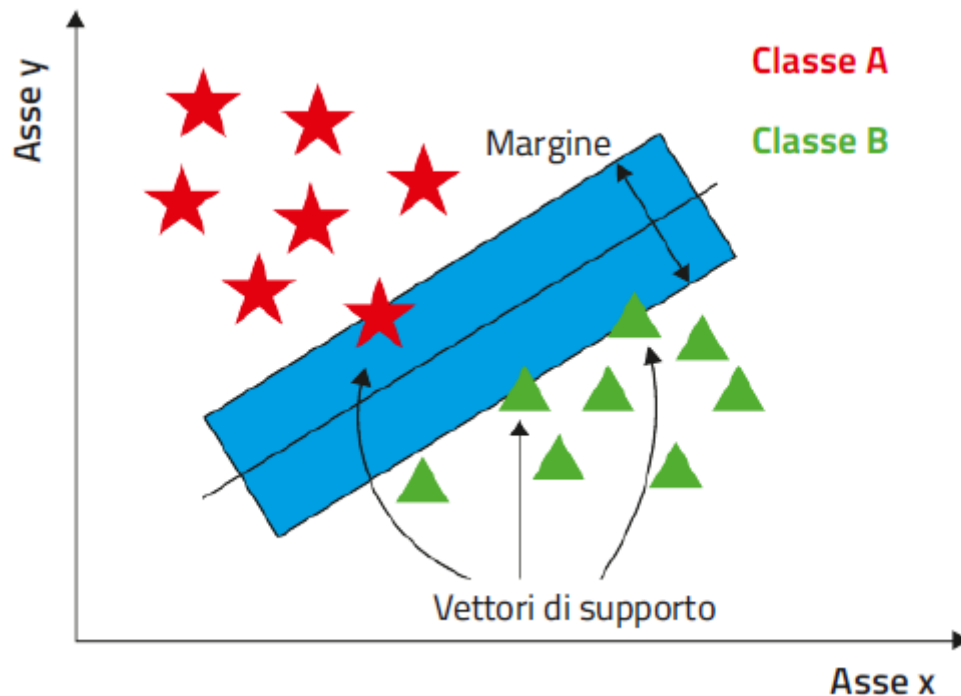
k-Nearest Neighbors (k-NN)

k-NN memorizza i campioni in fase di addestramento e classifica i nuovi campioni in fase di valutazione in base ai primi k elementi più vicini.



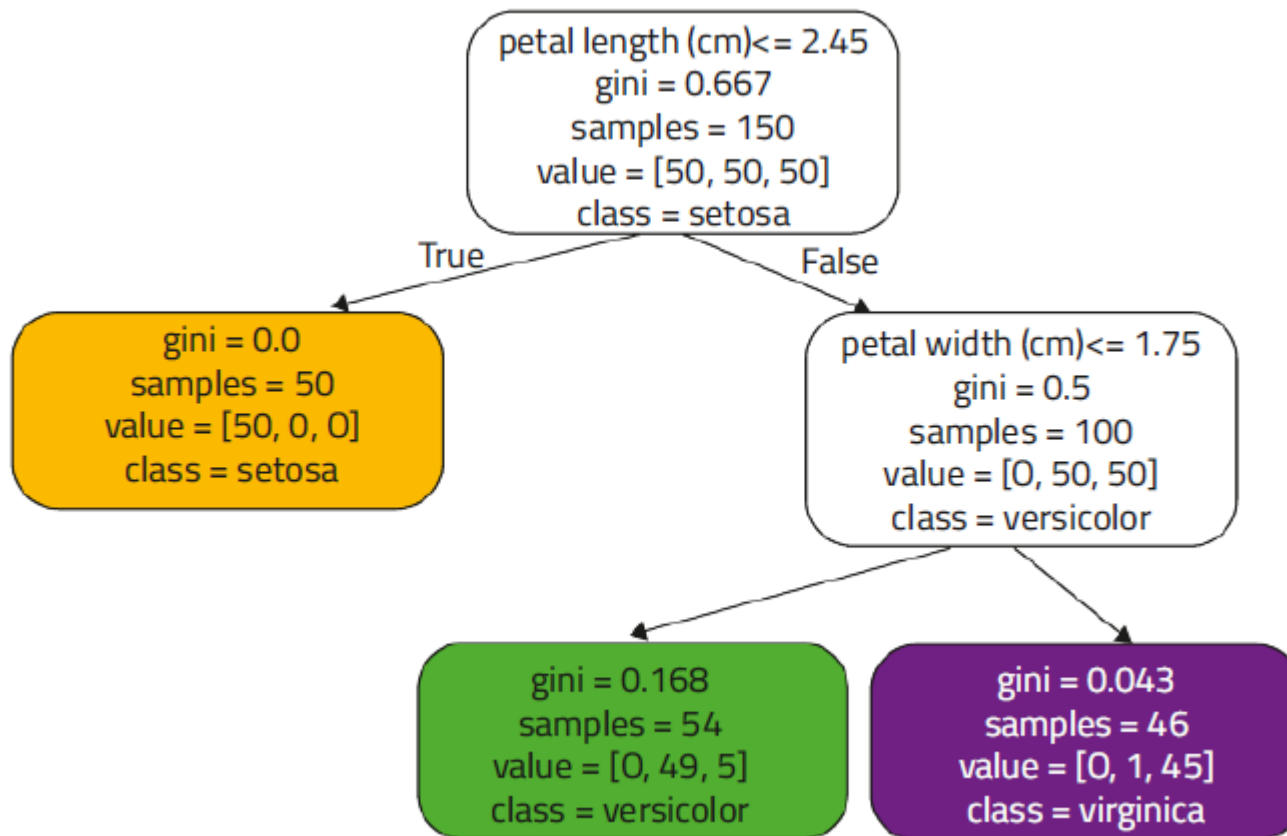
Support Vector Classifier (SVC)

SVC si basa sul concetto di *vettori di supporto*, ossia quegli elementi che si trovano sul confine di classificazione e che quindi fanno da supporto al piano che separa le varie categorie.



Decision Tree Classifier (DTC)

DTC: un albero decisionale si basa sul concetto di *struttura ad albero*, con nodi e archi, dove ogni nodo rappresenta una regola di decisione e ogni arco rappresenta un percorso decisionale.



Apprendimento non supervisionato

Una delle tante applicazioni dell'apprendimento non supervisionato è quella del raggruppamento (*clustering*).

Il problema del **raggruppamento** consiste nell'assegnare le istanze di un insieme a un gruppo in modo che tutte le istanze assegnate allo stesso gruppo siano simili tra loro.

Essendo un problema di apprendimento non supervisionato, non conosciamo a priori né l'assegnamento dei gruppi per le varie istanze né il numero di gruppi effettivamente presenti.

Spesso il raggruppamento viene utilizzato come metodo per scoprire similarità e dissimilarità ignote all'interno dei dati che stiamo analizzando.

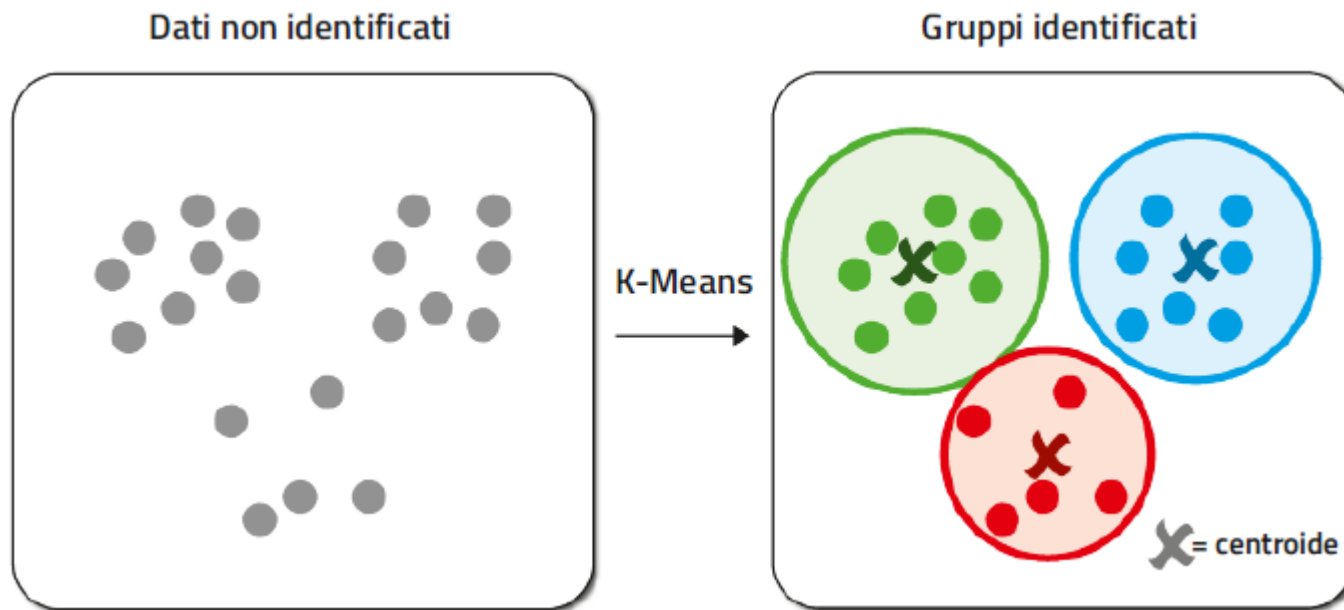
Tra le tecniche più utilizzate:

- K-Means, un algoritmo basato sul calcolo dei baricentri dei vari gruppi;
- raggruppamento agglomerativo e divisivo (*Agglomerative Clustering*), un processo di raggruppamento che, partendo dalle singole istanze, raggruppa elementi vicini e simili, fino a formare dei gruppi più grandi;
- DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*), un metodo che raggruppa gli elementi che si trovano entro un certo raggio di distanza.

K-Means

Questo algoritmo di raggruppamento si basa sul concetto di *centroide*, un punto nello spazio delle caratteristiche delle istanze che rappresenta il centro di un dato gruppo.

Il numero di gruppi e quindi di centroidi è un *iperparametro* di questo metodo.



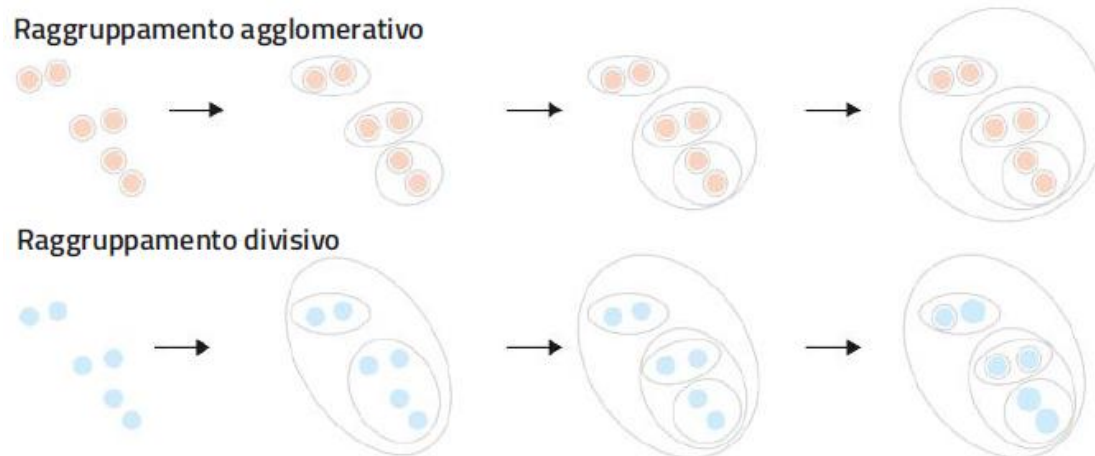
Agglomerative Clustering

In questo algoritmo la strategia di allocazione dei gruppi si basa sulla definizione di una gerarchia, cioè un ordinamento dei gruppi.

Esistono due tipi di raggruppamento gerarchico: *agglomerativo* e *divisivo*.

Nel raggruppamento *agglomerativo* ogni istanza appartiene a un gruppo separato e, iterativamente, i gruppi vengono uniti tra loro, formando mano a mano gruppi più grandi. La gerarchia è implicita ed è data dal fatto che i gruppi formati nelle iterazioni precedenti sono più simili rispetto a quelli formati nelle iterazioni successive.

Il raggruppamento *divisivo* procede esattamente nel modo opposto: si parte da un unico gruppo che contiene tutti i campioni e si suddivide il gruppo in gruppi più piccoli minimizzando le diversità tra gli elementi dei nuovi gruppi. In entrambi i casi, ci si ferma quando si raggiunge il numero di gruppi prefissato.



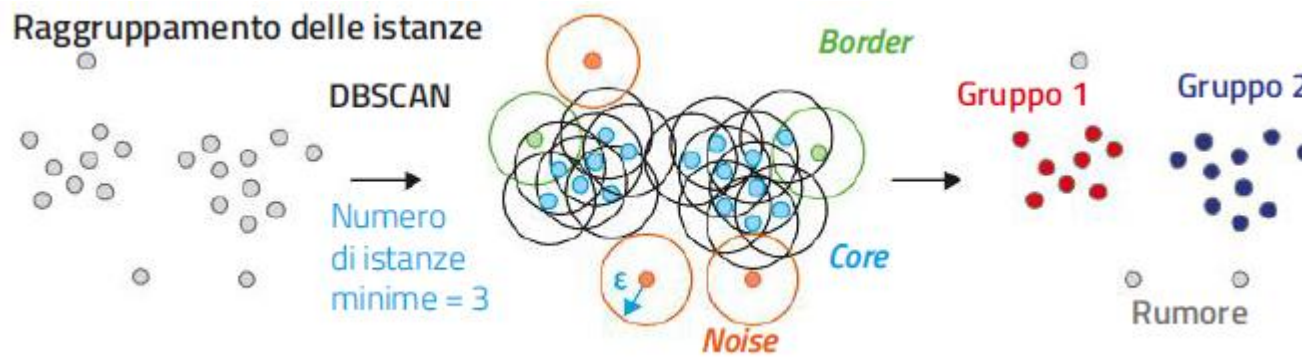
DBSCAN

Il raggruppamento effettuato tramite questo metodo parte dalla definizione di un raggio massimo di distanza, detto *epsilon*, che serve per individuare i vicini delle altre istanze.

In pratica, le istanze vengono divise in tre tipologie:

- *Core*, un'istanza che ha almeno k altre istanze nel suo raggio d'azione;
- *Border*, un'istanza che ha almeno un *Core* nel suo raggio d'azione;
- *Noise*, un'istanza che non è né un *Core* né un *Border*.

In questo modo, i campioni che sono troppo dissimili dagli altri vengono etichettati come *noise*, «rumore», e non sono inclusi all'interno di un gruppo.



Capitolo 10

Il web programming

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume *informatica per progetti – Programmare in Python*, di M. Fiore, A. Zanga, ISBN 9788808899477

HTML

HTML (*Hyper Text Markup Language*) è il linguaggio standard utilizzato per creare pagine web, poiché è supportato in maniera nativa dai browser web.

A esso si affiancano **CSS**, che è utilizzato per gli aspetti relativi alla presentazione grafica, e **JavaScript**, linguaggio di programmazione utile per sviluppare gli aspetti di interattività.

Per definire la struttura e il contenuto delle pagine web, HTML utilizza degli elementi, chiamati **tag**.

Ogni elemento HTML è racchiuso tra un tag di apertura e uno di chiusura.

```
<tag>contenuto</tag>
```

I tag possono avere degli attributi, cioè proprietà o parametri che forniscono informazioni aggiuntive sull'elemento HTML

```
<nome-elemento attributo1="valore1" attributo2="valore2">  
Contenuto</nome-elemento>
```

Attributi dei tag

- L'attributo `id` serve ad assegnare un identificatore univoco a un elemento
- L'attributo `class` serve ad assegnare una o più classi a un elemento

```
<div id="intestazione">Questo è un header</div>
```

```
<p class="testo-grande">Questo è un paragrafo con testo  
grande</p>
```

- L'attributo `href` viene usato con il tag `<a>` per specificare l'URL di destinazione di un link

```
<a href="https://www.example.com">Vai a Example</a>
```

- L'attributo `src` viene usato con i tag `<img>`, `<script>` e `<iframe>` per specificare la fonte di un'immagine, uno script o un contenuto

```

```

Attributi dei tag

- L'attributo `alt` viene usato con il tag `<img>` per fornire un testo alternativo che viene visualizzato se l'immagine non può essere caricata

```

```

- L'attributo `style` è usato per applicare stili in linea direttamente a un elemento

```
<p style="color: red; font-size: 20px;">Testo in rosso e di  
grandi dimensioni</p>
```

- L'attributo `title` viene usato per dare informazioni aggiuntive visualizzate come *tooltip*

```
<abbr title="HyperText Markup Language">HTML</abbr>
```

Tag semantici e non semantici

I **tag semantici** sono elementi HTML che definiscono la struttura del contenuto di una pagina web indicando anche il significato del tag.

I **tag non semantici** indicano semplicemente la struttura della pagina.

I tag semantici descrivono il ruolo del contenuto stesso nel contesto della pagina e migliorano l'accessibilità per le tecnologie assistive.

I principali tag semantici sono:

- `<header>` , definisce l'intestazione di una sezione o di un documento;
- `<nav>`, indica una sezione di navigazione;
- `<article>`, rappresenta un contenuto autonomo e indipendente;
- `<section>`, usato per raggruppare contenuti correlati e definisce la sezione di un documento;
- `<footer>`, rappresenta il piè di pagina di un documento o di una sezione.

Principali tag HTML

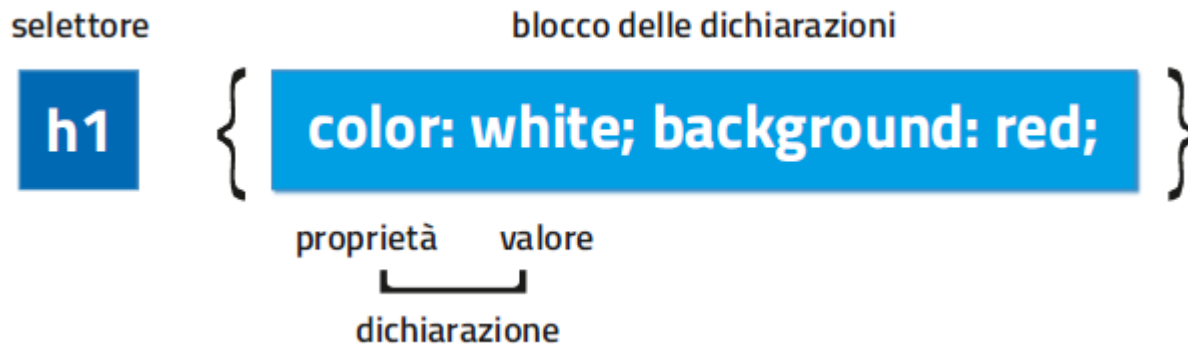
Tag	Significato
<html>	Contenitore principale di una pagina HTML
<head>	Contiene metadati come il titolo della pagina (<title>), unico e obbligatorio, i link a fogli di stile (<link>) e gli script (<script>)
<body>	Racchiude il contenuto visibile della pagina: testo, immagini, link ed elementi multimediali
<h1> - <h6>	Tag di intestazione, che definiscono titoli di diversa importanza
<p>	Tag per i paragrafi di testo
 	Tag per inserire un'interruzione di riga. Non ha un tag di chiusura.
<a>	Tag per i link ipertestuali, di solito ha l'attributo href e la struttura
	Tag per inserire immagini, di solito ha gli attributi src e alt e la struttura
, ,	Tag per elenchi non ordinati (<i>bullet points</i>), elenchi ordinati (numerati) e per gli elementi della lista
<div>, 	Tag per la divisione e la formattazione del contenuto, senza un significato semantico specifico
, <i>, <u>, 	Diversi tag per la formattazione del testo: grassetto (), corsivo (<i>), sottolineato (<u>), grassetto semantico ()
<header>, <footer>, <article>, <section>, <nav>	Tag semantici che descrivono il significato del contenuto, utili a migliorare l'accessibilità
<audio>, <video>	Tag per incorporare contenuti audio e video

CSS

Il linguaggio **CSS** (*Cascading Style Sheets*) viene usato per descrivere l'aspetto e la formattazione di un documento HTML.

La struttura di un CSS si basa su regole che definiscono come gli elementi HTML devono essere visualizzati.

Una regola CSS è composta da un selettore e da un blocco di dichiarazioni.



Il selettore definisce la parte del documento HTML a cui viene applicata la regola espressa nel blocco delle dichiarazioni.

```
p {font: 12px Verdana, Arial;}
```

Dichiarazioni

Il blocco delle dichiarazioni è racchiuso tra parentesi graffe { } e può contenere più dichiarazioni separate da punto e virgola ; .

Ogni dichiarazione è composta da due parti: la **proprietà** e il **valore**; per ogni dichiarazione è possibile indicare solo una proprietà, mentre la proprietà può avere anche più di un valore, separato da virgole.

Principali proprietà:

Proprietà	Significato
color	colore del testo
background-color	colore di sfondo
font-size	dimensione del carattere
margin	spazio esterno sui quattro lati di un elemento (superiore, destro, inferiore, sinistro)
padding	spazio interno di un elemento
border	bordo di un elemento
display	modalità di visualizzazione di un elemento (block, inline, none)
position	posizione di un elemento (static, absolute, fixed, sticky)

CSS esterni o interni

I fogli di stile CSS possono essere esterni, interni alla pagina o interni a un tag.

Se il CSS è esterno, può essere collegato a un documento HTML tramite l'elemento `<link>` inserito nella sezione `<head>`.

```
<html>
  <head>
    <link href="stile.css" rel="stylesheet" type="text/css">
  </head>
  <body>
    [...]
  </body>
</html>
```

Se il foglio di stile CSS è interno a una pagina, viene incorporato nel documento HTML tramite l'elemento `<style>` inserito sempre nella sezione `<head>`.

È preferibile avere fogli di stile **CSS esterni**.

```
<html>
  <head>
    <style type="text/css">
      body {background: white;}
      p {color: black;}
      [...]
    </style>
  </head>
  <body>
    [...]
  </body>
</html>
```

Accessibilità siti web

L'**accessibilità dei siti web** è il risultato della pratica di progettare e sviluppare i siti web in modo che siano utilizzabili da tutte le persone.

L'obiettivo è garantire che il contenuto, le funzionalità e le interazioni di un sito web siano fruibili da chiunque, comprese le persone con disabilità visive, uditive, motorie, cognitive, o neurologiche.

L'accessibilità web si basa su alcuni principi fondamentali, descritti nelle Linee Guida per l'Accessibilità dei Contenuti Web (WCAG), sviluppate dal World Wide Web Consortium (W3C) attraverso la *Web Accessibility Initiative* (WAI).

WCAG principi chiave

Le WCAG si suddividono in quattro principi chiave, ciascuno composto da diversi elementi.

1. Le informazioni e i componenti dell'interfaccia utente devono essere **presentabili** agli utenti in modo che li possano percepire.
2. I componenti dell'interfaccia utente e la navigazione devono essere **utilizzabili**.
3. Le informazioni e il funzionamento dell'interfaccia utente devono essere chiaramente **comprensibili**.
4. Il contenuto deve essere sufficientemente **robusto** da poter essere interpretato in modo affidabile da un'ampia gamma di strumenti dell'utente, incluse le tecnologie **assistive**.

Progettazione web

Un **sito web** è un insieme di pagine correlate tra di loro tramite link ipertestuali.

È generalmente ospitato su un server ed è raggiungibile mediante l'uso di un browser a un indirizzo detto URL (*Uniform Resource Locator*).

I siti web hanno un architettura applicativa **client/server**, dove il client è il browser e il server è rappresentato dal server web che ospita il sito.

In base alla loro struttura e alle loro caratteristiche, i siti web si dividono in:

- **siti statici**, in cui le pagine sono principalmente costituite da documenti HTML, che ne racchiudono il contenuto, e CSS, correlati tra loro per mezzo di link ipertestuali e sono di sola lettura;
- **siti dinamici**, in cui le pagine vengono costruite dinamicamente con contenuti in genere disponibili in un database e che possono variare in base a diversi fattori, per esempio le ricerche effettuate, l'utente collegato ecc.

Realizzazione di un sito web

La realizzazione di un sito web prevede diverse fasi che prendono in considerazione aspetti di comunicazione, aspetti di design e aspetti di programmazione.

È possibile identificare le seguenti fasi di sviluppo di un sito web.

1. **Definizione degli obiettivi di comunicazione** del sito web. In questa fase bisogna decidere che tipo di sito web si vuole sviluppare (statico o dinamico), quali sono gli utenti ai quali ci si rivolge e quali sono le risorse a disposizione per lo sviluppo.
2. **Web design**. In questa fase vengono definiti la struttura e gli aspetti grafici del sito, tenendo in considerazione gli aspetti di comunicazione, ma anche quelli di usabilità e di accessibilità.
3. **Sviluppo**. In questa fase vengono definite le tecnologie da utilizzare per la realizzazione del sito web e le modalità di sviluppo, e viene messa in pratica la realizzazione vera e propria.
4. **Rilascio**. Dopo le fasi precedenti il sito web viene messo online e vengono messe in opera attività di aggiornamento, manutenzione, analisi del traffico e marketing.

Protocollo HTTP

I browser comunicano con i web server utilizzando il protocollo HTTP (*Hyper Text Transfer Protocol*).

Quando l'utente fa clic su un collegamento all'interno di una pagina web, oppure invia un modulo con dei dati o esegue una ricerca, il browser invia una richiesta HTTP al web server.

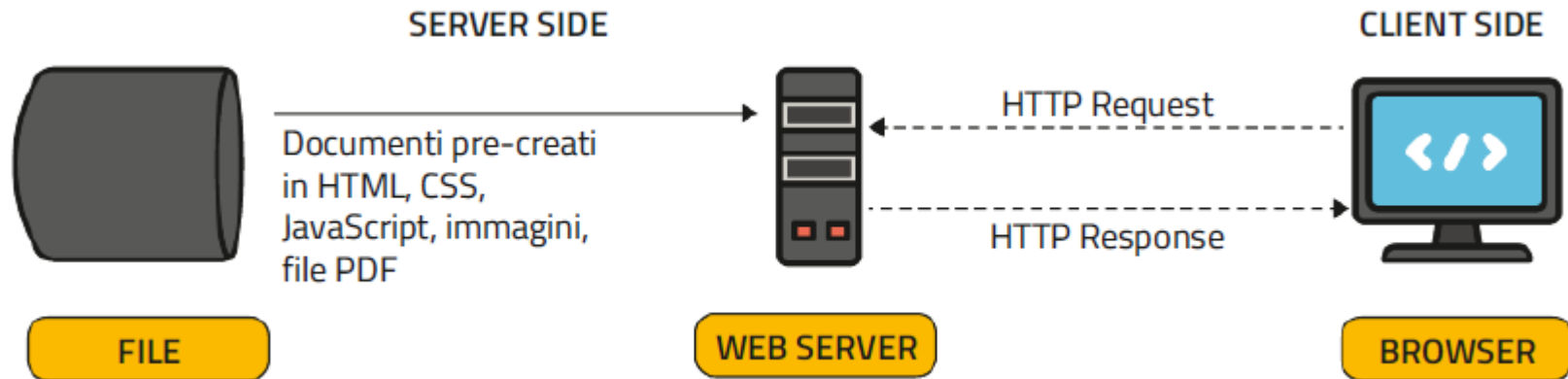
Una richiesta HTTP include:

- un **indirizzo URL** che identifica il server di destinazione e la risorsa al suo interno (per esempio un file HTML);
- un **metodo** che definisce l'azione richiesta (per esempio ottenere un file oppure salvare o aggiornare alcuni dati).

Principali metodi HTTP

Metodo	Azione
GET	Ottenere una risorsa specifica: una pagina HTML, un elenco di prodotti ecc.
POST	Creare una nuova risorsa: aggiungere un nuovo contatto a un database ecc.
HEAD	Ottenere le informazioni su una specifica risorsa: GET ottiene tutta la risorsa, mentre HEAD solo le informazioni come l'ultimo aggiornamento per quella risorsa
PUT	Aggiorna una risorsa esistente (oppure, se non esiste, ne crea una nuova)

Architettura applicativa sito statico

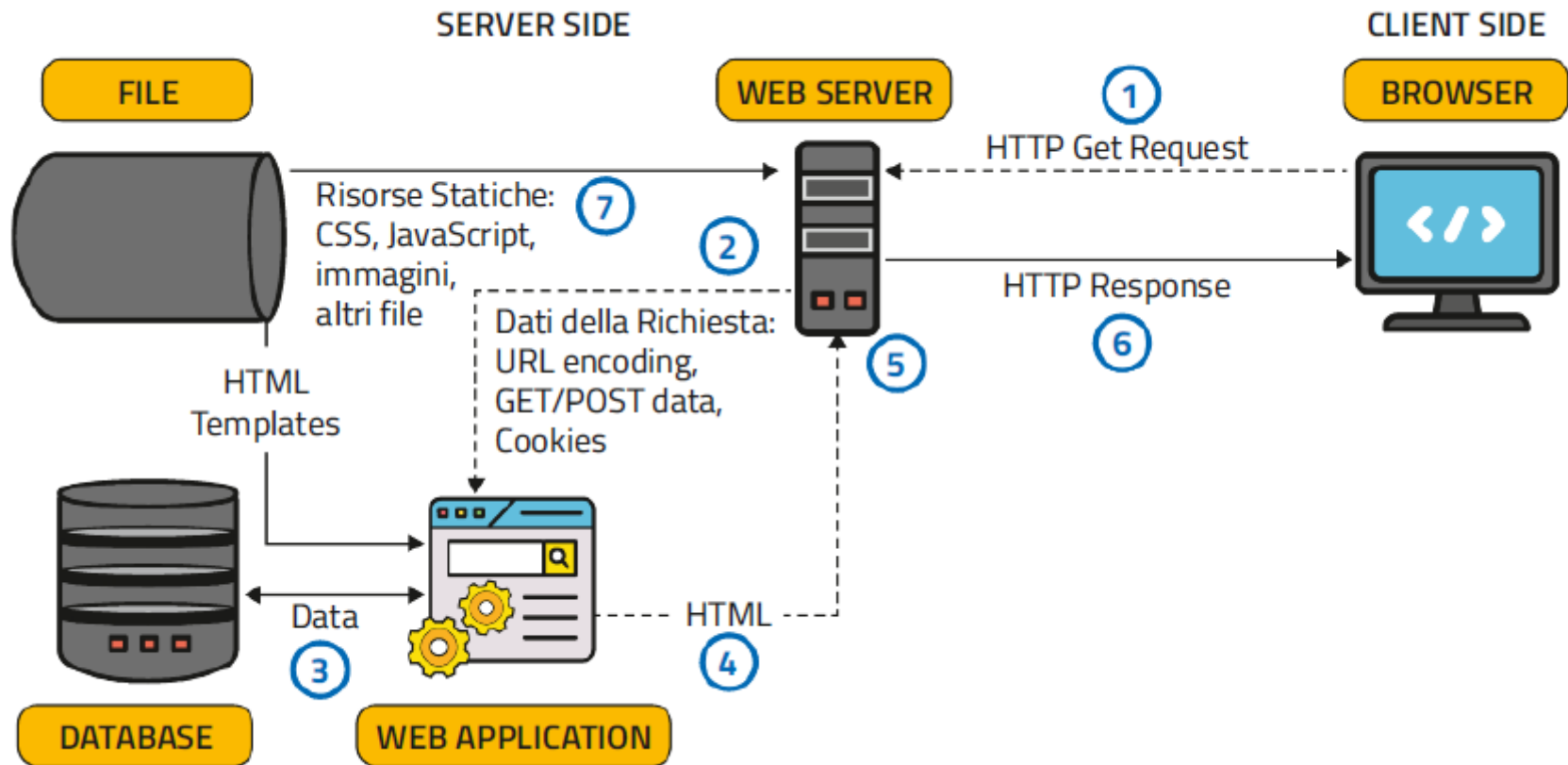


Quando un utente desidera accedere a un documento, il browser invia una richiesta HTTP di tipo GET al web server, specificando il suo URL.

Il web server recupera il documento richiesto dal suo file system e restituisce una risposta HTTP contenente il documento e uno stato di successo relativo all'operazione.

Se, per qualche motivo, il documento non può essere recuperato, viene restituito uno stato di errore

Architettura applicativa sito dinamico



Architettura applicativa sito dinamico

- ① Il browser invia una richiesta HTTP al server (GET Request).
- ② Il server elabora e interpreta la richiesta di una pagina dinamica.
- ③ Il web server passa la richiesta alla web application, che procede a recuperare i dati necessari dal database.
- ④ I dati provenienti dal database vengono inseriti nei segnaposto dei template HTML.
- ⑤ La pagina così composta viene restituita al web server.
- ⑥ Il web server restituisce al browser (client) il documento HTML così generato (Response).
- ⑦ In caso di richiesta di risorse statiche, queste vengono gestite allo stesso modo dei siti statici.

Web Application

Una web application ha il compito, tra gli altri, di gestire tutte le fasi di preparazione delle pagine dinamiche, recuperando i dati che sono contenuti nel database, utilizzando i template HTML in cui inserire i dati nei *placeholder* e combinando eventuali contenuti statici associati alle richieste effettuate dal browser.

Per poter essere utilizzato nello sviluppo di una web application, Python ha bisogno di alcune componenti che consentono una corretta comunicazione con il front-end (client-side) che utilizza le tecnologie HTML, CSS e JavaScript. Queste componenti sono contenute in diversi framework, come Django e Flask.

Un **framework** è un'architettura logica di supporto (cioè un insieme di librerie, funzioni e altri elementi già pronti) su cui un'applicazione web può essere progettata e sviluppata, semplificando molto il lavoro del programmatore.

ORM

I framework hanno anche un livello di gestione database in cui, con un'astrazione, è semplificata l'esecuzione delle operazioni del linguaggio DML (*Data Manipulation Language*) come INSERT, UPDATE e DELETE e QL (*Query Language*), come SELECT.

Questo livello di astrazione del database relazionale è detto **ORM**, sigla che sta per *Object Relational Mapper*.

Vantaggi di ORM

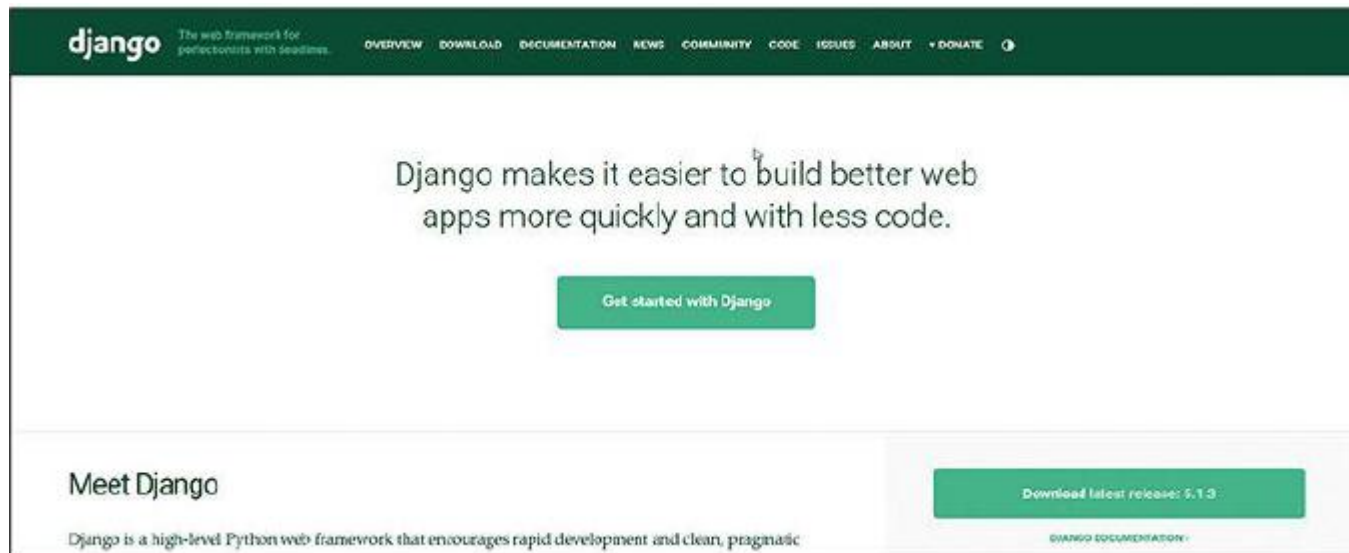
L'uso di un ORM ha diversi vantaggi:

- la logica di definizione dello schema del database viene fatta nell'applicazione web ed è indipendente dal tipo di DBMS che viene utilizzato;
- è dunque possibile, se necessario, sostituire il DBMS senza avere conseguenze o dover modificare l'applicazione che lo utilizza;
- agli sviluppatori è consentito ottimizzare il codice in base all'uso specifico che ne fanno e non in base alle caratteristiche del DBMS;
- la validazione dei dati viene fatta a livello dell'applicazione, semplificando e rendendo più sicuro il controllo relativo alla corretta archiviazione dei dati (per esempio il formato del dato o la sua dimensione);
- lo schema logico a oggetti permette di utilizzare la programmazione a oggetti OOP, e attraverso attributi e metodi è possibile agire sui dati senza usare le istruzioni SQL, e questo, in genere, è molto apprezzato dai programmatori.

Il framework Django

Django è un framework web di alto livello, scritto in Python, per lo sviluppo rapido ed efficace di applicazioni web.

Django (<https://www.djangoproject.com>) è gratuito e open source.



Caratteristiche di Django

- La sua architettura, di tipo MTV (Model-Template-View), separa la logica dell'applicazione (Model), la logica di presentazione (Template) e le richieste del client (View);
- ha un'interfaccia di amministrazione generata automaticamente e basata sui Model;
- ORM (Object Relational Mapper) permette di interagire con il database utilizzando le classi Python invece del linguaggio SQL e consente l'utilizzo di diversi DBMS, come SQLite, PostgreSQL, MySQL, e Oracle;
- ha un sistema di autenticazione integrato che consente di gestire l'autenticazione e la registrazione di utenti, permessi e gruppi;
- Con il sistema di routing le richieste HTTP del client vengono indirizzate a View in grado di soddisfare le richieste stesse;
- Il sistema di template permette di generare pagine HTML dinamiche;
- ha strumenti specifici per la gestione delle form di input di dati dell'utente, con la possibilità anche di effettuare la validazione automatica dei dati inseriti.

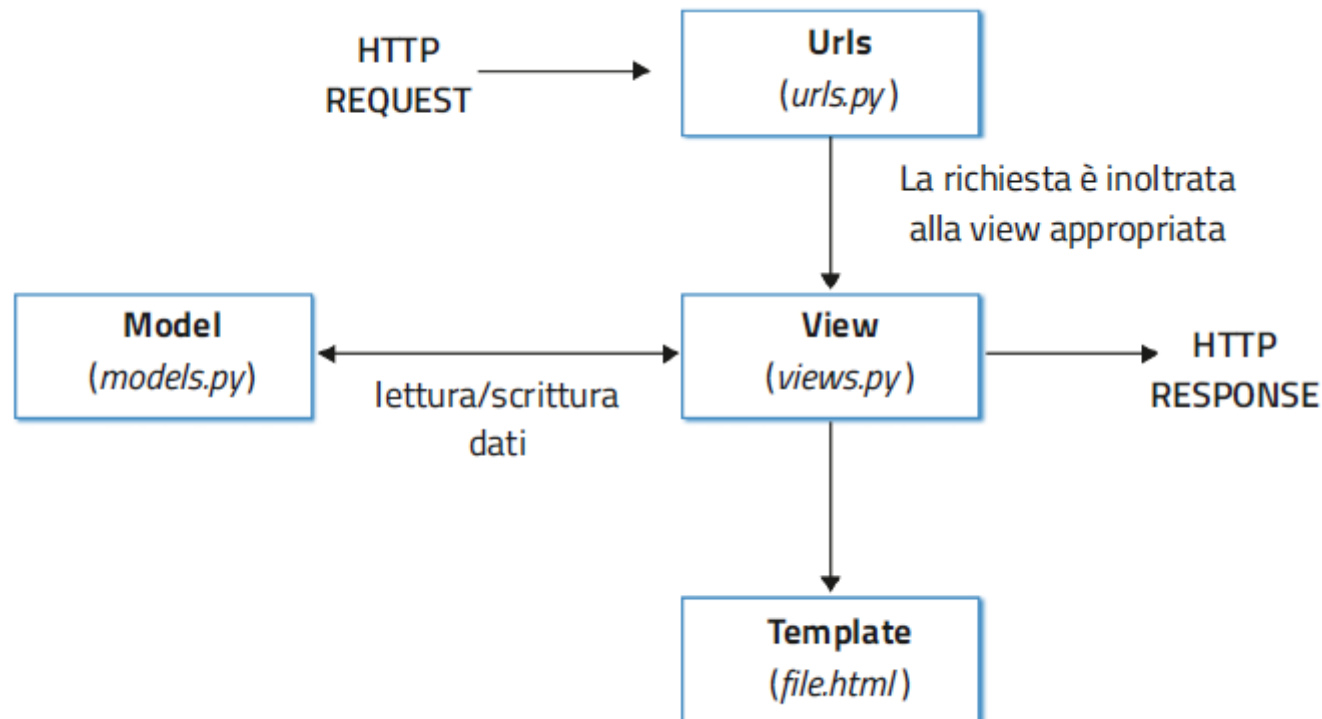
Caratteristiche di Django

- In termini di internazionalizzazione e localizzazione, permette la traduzione dell'applicazione web in diverse lingue e la gestione delle specificità locali, come il formato di data e ora;
- è dotato di misure di sicurezza per proteggere le applicazioni web da vulnerabilità comuni, per esempio le *SQL Injection* (cioè l'inserimento nel database di codice malevolo attraverso le interrogazioni SQL);
- per garantire la qualità del codice il framework ha strumenti di test integrati;
- il sistema di migrazione dati consente di gestire le modifiche al database attraverso le migrazioni, rendendo semplice applicare e mantenere le modifiche nel tempo.

Architettura MTV

Django ha un'architettura MTV (Model-Template-View), detta anche a tre livelli o *three tier*:

1. *Data Access layer* (Model);
2. *Presentation layer* (Template);
3. *Business Logic layer* (View).



Architettura MTV

Ogni modulo è rappresentato in Django da un insieme di *script.py* chiamati:

- il *Data Access layer*, **Model**, definisce la struttura delle classi che diverranno tabelle nel database;
- il *Presentation layer*, **Template**, descrive come vengono mostrati i dati nelle pagine HTML;
- il *Business Logic layer*, **View**, gestisce le richieste e le risposte HTTP, conosce la logica con la quale accedere ai dati necessari (attraverso Model), e usa Template per la loro formattazione e presentazione;
- L'ultimo modulo, **Urls**, rappresenta un elenco delle risorse disponibili e gestibili dall'applicazione web.

Come funziona Django

1. Il modulo **Urls** funziona come un mappatore di risorse e svolge la funzione di reindirizzare le richieste HTTP alla view appropriata in base alla URL della richiesta.
2. Il modulo **View** riceve la richiesta e la esegue effettuando anche i controlli, per esempio quelli relativi all'autorizzazione dell'utente a svolgere una determinata operazione. View accede ai dati necessari per soddisfare le richieste (tramite Model) e delega la formattazione della risposta a Template; al termine del lavoro di Template, View manda la risposta al browser.
3. Il modulo **Model** contiene le classi Python che definiscono la struttura dei dati dell'applicazione e fornisce i meccanismi per gestire (aggiungere, modificare, eliminare) e interrogare i record nel database.
4. Il modulo **Template** contiene file di testo che definiscono la struttura o il layout di un file (come una pagina HTML), con i segnaposto utilizzati per rappresentare il contenuto effettivo. Una view può creare dinamicamente una pagina HTML utilizzando un template HTML e popolandolo con i dati provenienti da un Model. Un template può essere utilizzato per definire la struttura di qualsiasi tipo di file e non deve essere necessariamente HTML.

Creare un ambiente virtuale Python

La creazione di un ambiente virtuale Python permette di mantenere separati i diversi ambienti di programmazione, ad esempio le diverse versioni di Django.

Per creare l'ambiente virtuale Python occorre utilizzare il prompt dei comandi:

1. creiamo una directory in cui vogliamo creare l'ambiente virtuale e posizioniamoci al suo interno;
2. creiamo l'ambiente virtuale con il seguente comando:

```
python -m venv nome_ambiente_virtuale
```

Una volta creato l'ambiente virtuale, è possibile attivarlo con il seguente comando

```
nome_ambiente_virtuale\Scripts\activate
```

Per disattivarlo

```
deactivate
```

Esempio di creazione di un ambiente virtuale

```
C:\Users\fleur\Documents\Codice Python>mkdir Web_Project
```

```
C:\Users\fleur\Documents\Codice Python>cd Web_Project
```

```
C:\Users\fleur\Documents\Codice Python\Web_Project>|
```

```
C:\Users\fleur\Documents\Codice Python\Web_Project>python -m venv Venv_Django
```

```
C:\Users\fleur\Documents\Codice Python\Web_Project>dir
```

```
Il volume nell'unità C è Windows  
Numero di serie del volume: 849E-EB83
```

```
Directory di C:\Users\fleur\Documents\Codice Python\Web_Project
```

```
11/06/2024  17:43    <DIR>          .  
11/06/2024  17:41    <DIR>          ..  
11/06/2024  17:43    <DIR>          Venv_Django  
               0 File              0 byte  
               3 Directory  430.470.033.408 byte disponibili
```

```
C:\Users\fleur\Documents\Codice Python\Web_Project>|
```

```
C:\Users\fleur\Documents\Codice Python\Web_Project>Venv_Django\Scripts\activate
```

```
(Venv_Django) C:\Users\fleur\Documents\Codice Python\Web_Project>|
```

```
(Venv_Django) C:\Users\fleur\Documents\Codice Python\Web_Project>deactivate
```

```
C:\Users\fleur\Documents\Codice Python\Web_Project>|
```


Installazione di Django

Per installare correttamente Django dobbiamo attivare nuovamente l'ambiente virtuale e al suo interno usare il comando

```
python -m pip install django
```

È possibile verificare che l'installazione sia avvenuta correttamente chiedendo qual è la versione di Django installata

```
django-admin --version
```

```
(Venv_Django) C:\Users\fleur\Documents\Codice Python\Web_Project>django-admin --version  
5.0.6
```

```
(Venv_Django) C:\Users\fleur\Documents\Codice Python\Web_Project>|
```

Sviluppo di un sito web dinamico

Dopo l'installazione del framework Django, procediamo a sviluppare un semplice sito web dinamico per la visualizzazione di un catalogo di prodotti, che mostri in home page i prodotti disponibili e permetta di selezionarne uno per visualizzare i suoi dettagli.

Il sito web potrà essere successivamente esteso inserendo la funzionalità che consente a un utente registrato e autenticato la possibilità di selezionare un prodotto e di inserirlo nella propria lista di preferiti.

Il sito consiste di due parti:

- una parte pubblica, che consente di visualizzare i prodotti del catalogo e selezionare un prodotto per vedere il suo dettaglio;
- un'area di amministrazione, che permette di inserire nuovi prodotti, modificarli o eliminarli.

Creare un progetto

La configurazione iniziale di Django prevede la creazione di un **progetto**, cioè un pacchetto Python, ovvero una directory del codice, che contiene tutte le impostazioni relative a un'istanza Django.

Il pacchetto include la configurazione del database, le opzioni specifiche di Django e le impostazioni relative all'applicazione.

Django usa in modo integrato il DBMS SQLite, un semplice DBMS adatto a sviluppi di semplici siti web.

Chiamiamo il progetto *catalogo*, che sarà anche il nome di tutto il nostro sito web e procediamo con i seguenti passi:

1. dal prompt dei comandi attiviamo l'ambiente virtuale;
2. entriamo nella cartella *Web_Project*, dove memorizzeremo tutto il codice;
3. creiamo il progetto col seguente comando

```
django-admin startproject catalogo
```

4. controlliamo che cosa Django ha creato in questa cartella

Cartella del progetto

- La directory principale *catalogo/* è il contenitore del nostro progetto.
- Il file *manage.py* è lo script con cui potremo interagire con il progetto.
- La cartella interna *catalogo/* è il vero pacchetto Python del progetto; dovremo usare questo nome per importare qualsiasi cosa dal suo interno e per fare i riferimenti che ci serviranno.
- Il file *catalogo/__init__.py* è un file vuoto che indica a Python che questa directory deve essere considerata un pacchetto Python.
- Il file *catalogo/settings.py* contiene le configurazioni del progetto Django.
- Il file *catalogo/urls.py* contiene le dichiarazioni delle URL per questo progetto Django, quindi una specie di mappa del sito.
- Il file *catalogo/asgi.py* contiene le configurazioni per distribuire il sito su server web compatibili con ASGI (lo standard di Python per i web server e le applicazioni asincrone). Per il nostro progetto non lo useremo.
- Il file *catalogo/wsgi.py* contiene le configurazioni per distribuire il sito su web server compatibili con WSGI (standard Python per web server e applicazioni). Per il nostro progetto non lo useremo, poiché non pubblicheremo il sito online, mantenendolo in locale (*localhost*).

```
catalogo/  
    manage.py  
    catalogo/  
        __init__.py  
        settings.py  
        urls.py  
        asgi.py  
        wsgi.py
```

Avviare Django

Per avviare Django usiamo

```
python manage.py runserver
```

```
(Venv_Django) C:\Users\fleur\Documents\Codice Python\Web_Project\catalogo>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin,
auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
June 12, 2024 - 15:21:34
Django version 5.0.6, using settings 'catalogo.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Apriamo quindi con il browser il link indicato (che corrisponde a *localhost*), dove possiamo vedere Django in azione

Per disattivare il server di sviluppo, va inserito nel prompt il comando CTRL + C



Creare una app

Un progetto è una collezione di configurazioni (i file che abbiamo visto prima) e di app; un progetto può infatti contenere più app.

In Django una **app** è un'applicazione web che fa qualcosa, per esempio un blog o un sondaggio.

Per la logica del riuso, una app può fare parte di più progetti.

Per creare la nostra app di nome *prodotti* procediamo come segue:

1. ci posizioniamo nella cartella dove è presente il file *manage.py*;
2. inseriamo il comando `startapp`

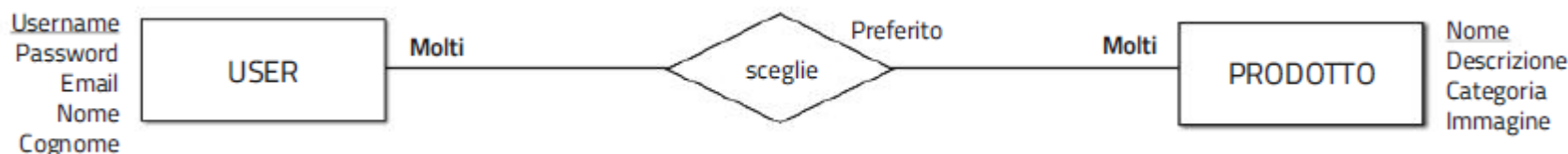
```
python manage.py startapp prodotti
```

3. Controlliamo cosa Django ha creato nella cartella

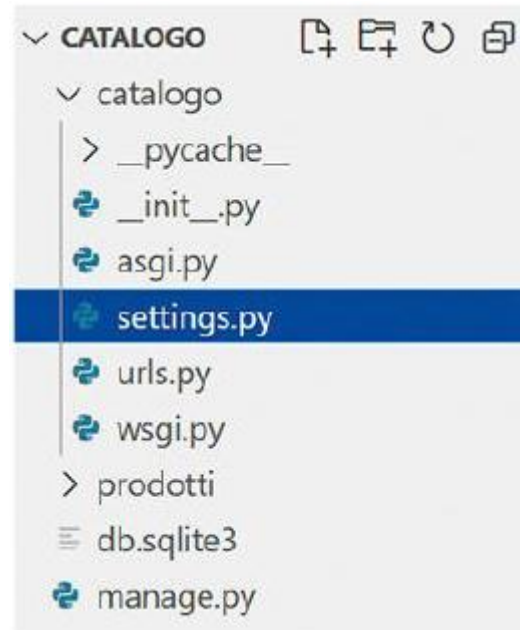
```
prodotti/  
  __init__.py  
  admin.py  
  apps.py  
  migrations/  
    __init__.py  
  models.py  
  tests.py  
  views.py
```

Creare Models - 1

Django usa ORM per la gestione del database, quindi per cominciare a progettare correttamente il sito dobbiamo creare lo schema E/R che rappresenta le entità e le associazioni tra loro



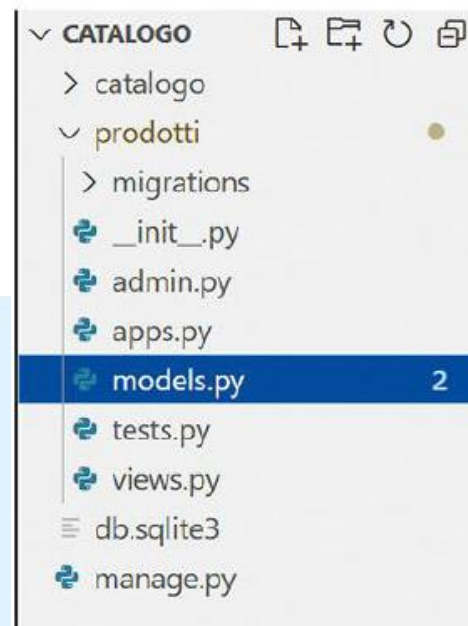
Se non usassimo l'ORM di Django, dovremmo derivare lo schema relazionale e poi predisporre le corrispondenti tabelle nel database; grazie a ORM, dallo schema E/R possiamo invece definire lo schema a oggetti, che fa uso delle classi. Normalmente Django utilizza il database SQLite che è incluso nella sua installazione, se si volesse usare un altro DBMS, ad esempio PostgreSQL dovremo andare a modificare il file di configurazione [settings.py](#)



Creare Models - 2

Lo schema logico a oggetti viene definito nel file *models.py*

```
1 from django.db import models
2 from django.contrib.auth.models import User
3
4 # Create your models here.
5 class Prodotto(models.Model):
6     nome = models.CharField(max_length = 50)
7     descrizione = models.CharField(max_length = 5000)
8     categoria = models.CharField(max_length = 80)
9     immagine = models.ImageField()
10
11 class Preferito(models.Model):
12     id_utente = models.ForeignKey(User, on_delete = models.CASCADE)
13     id_prodotto = models.ForeignKey(Prodotto, on_delete = models.CASCADE)
```



Le chiavi primarie vengono gestite automaticamente da Django.

Creare Models - 3

Il codice scritto nel `models.py` fornisce a Django le indicazioni per:

- creare lo schema del database (che corrisponde all'istruzione SQL CREATE TABLE) per le classi Prodotto e Preferito;
- creare una API Python per accedere ai dati delle due tabelle.

Le **API (Application Programming Interface)** sono librerie di funzioni che consentono la comunicazione (detta anche interfaccia) tra diverse applicazioni o sistemi software.

Per la classe Prodotto abbiamo previsto un attributo riferito alla sua immagine del prodotto: per poter usare il riferimento `ImageField()` è necessario installare la libreria `Pillow`, che permette di gestire le immagini.

Per farlo bisogna attivare l'ambiente virtuale e dal prompt dei comandi va inserito il comando che installa nell'ambiente virtuale la libreria Pillow.

```
python -m pip install Pillow
```

Configurare l'esistenza della app - 1

Possiamo ora procedere a modificare il file `settings.py` per indicare a Django che abbiamo creato l'app `prodotti`; la classe di configurazione dell'app si trova nel suo file di configurazione `prodotti/apps.py` e si chiama `ProdottiConfig`

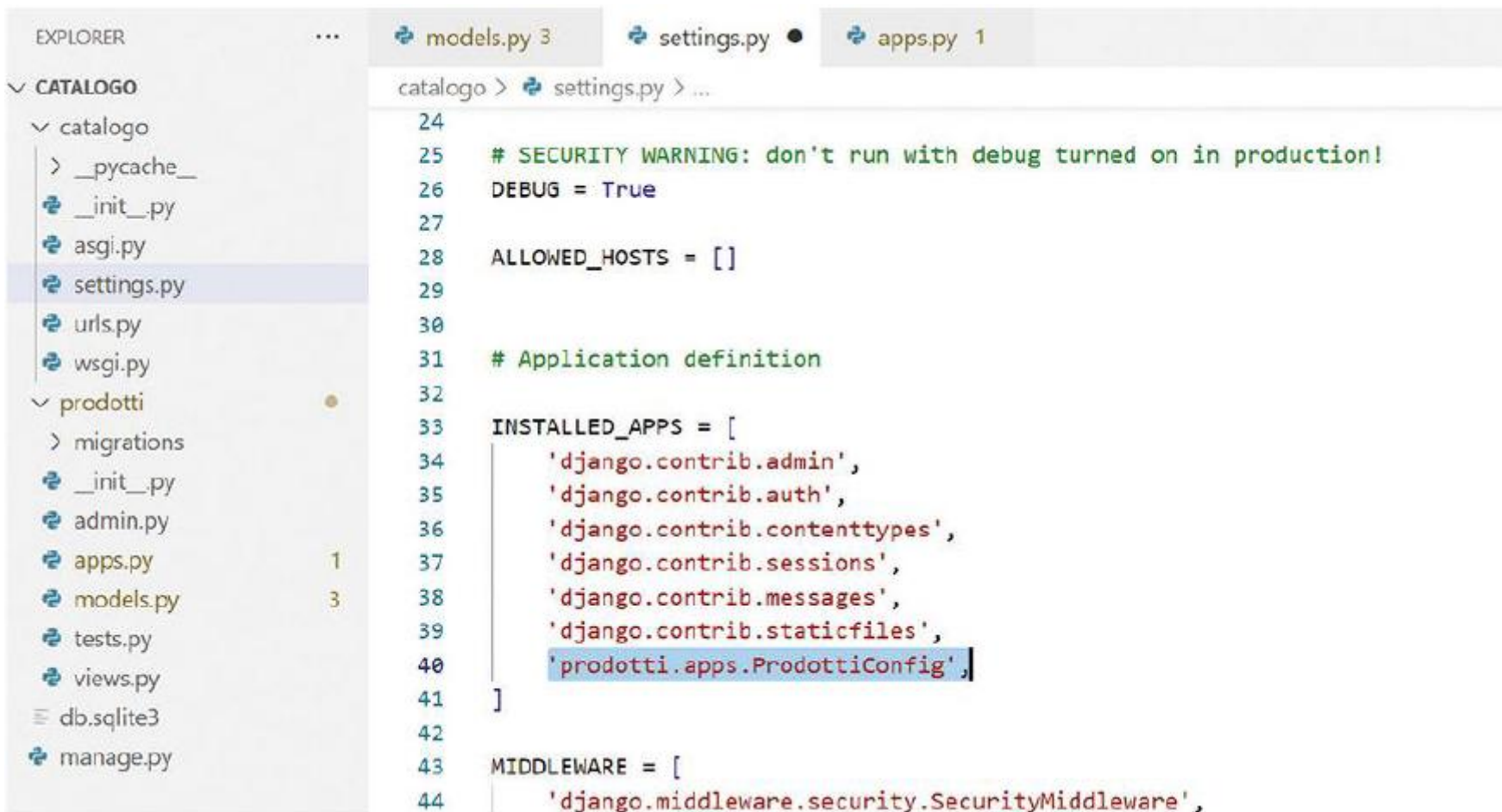


The screenshot shows a code editor with two panels. The left panel, titled 'EXPLORER', displays the project structure. The 'prodotti' app directory is expanded, showing files like `migrations`, `__init__.py`, `admin.py`, `apps.py` (highlighted), `models.py`, `tests.py`, `views.py`, `db.sqlite3`, and `manage.py`. The right panel shows the code in `prodotti > apps.py`. The code defines the `ProdottiConfig` class, which inherits from `AppConfig` and sets the `default_auto_field` to `django.db.models.BigAutoField` and the `name` to `'prodotti'`.

```
prodotti > apps.py > ...
1  from django.apps import AppConfig
2
3
4  class ProdottiConfig(AppConfig):
5      default_auto_field = 'django.db.models.BigAutoField'
6      name = 'prodotti'
7
```

Configurare l'esistenza della app - 2

Possiamo ora modificare il file *catalogo/settings.py* nella sezione *INSTALLED_APPS* aggiungendo una riga con il codice *prodotti.apps.ProdottiConfig*



```
EXPLORER
...
models.py 3
settings.py
apps.py 1

CATALOGO
  catalogo
    > __pycache__
    _init_.py
    asgi.py
    settings.py
    urls.py
    wsgi.py
  prodotti
    > migrations
    _init_.py
    admin.py
    apps.py 1
    models.py 3
    tests.py
    views.py
    db.sqlite3
    manage.py

catalogo > settings.py > ...
24
25 # SECURITY WARNING: don't run with debug turned on in production!
26 DEBUG = True
27
28 ALLOWED_HOSTS = []
29
30
31 # Application definition
32
33 INSTALLED_APPS = [
34     'django.contrib.admin',
35     'django.contrib.auth',
36     'django.contrib.contenttypes',
37     'django.contrib.sessions',
38     'django.contrib.messages',
39     'django.contrib.staticfiles',
40     'prodotti.apps.ProdottiConfig',
41 ]
42
43 MIDDLEWARE = [
44     'django.middleware.security.SecurityMiddleware',
```

Migrazione al database

Procediamo con indicare a Django di creare le tabelle fondamentali nel database

```
python manage.py migrate
```

e con indicare a Django che abbiamo effettuato delle modifiche nel *models.py* che devono essere memorizzate nel database

```
python manage.py makemigrations prodotti
```

Nella cartella *prodotti/migrations/* troviamo le migrazioni che sono state effettuate, ciascuna numerata a partire da 0001. Per vedere i comandi SQL che sono stati eseguiti da Django nel corso di queste migrazioni, bisogna usare il comando *sqlmigrate* e indicare il nome e il numero della migrazione che ci interessa visualizzare, che nel nostro caso è la prima migrazione fatta per l'app *prodotti*.

```
python manage.py sqlmigrate prodotti 0001
```

Infine, per applicare anche le ultime migrazioni relative alle nuove tabelle create usiamo ancora il comando *migrate*

```
python manage.py migrate
```

Modifiche al Models - 1

Ricordiamo la procedura in tre passaggi per fare le modifiche al models:

1. modificare le classi nel file *models.py*;
2. eseguire il comando `python manage.py makemigrations prodotti` per creare le migrazioni per queste modifiche;
3. eseguire il comando `python manage.py migrate` per applicare le modifiche al database

```
python manage.py makemigrations prodotti
```

```
python manage.py migrate
```

API per interagire col database

Django mette a disposizione una serie di API che consentono di effettuare le operazioni di creazione, lettura, aggiornamento e cancellazione dei dati sul database (CRUD), che corrispondono alle istruzioni SQL dei linguaggi DML e QL INSERT, SELECT, UPDATE e DELETE.

Usando queste API è possibile effettuare le operazioni CRUD sui dati del database senza utilizzare il linguaggio SQL, proprio grazie al meccanismo ORM usato da Django.

Stiamo usando uno schema logico a oggetti, quindi ogni istanza della nostra classe (un oggetto) diventa una riga della tabella corrispondente nel database. Per esempio, l'inserimento di un record, che corrisponde a una istruzione SQL INSERT, prevede la creazione di una nuova istanza della classe e un'operazione di salvataggio nel database con il metodo save().

Per poter lavorare con le API (senza scrivere script), possiamo usare la shell di Django, che è attivabile dal prompt dei comandi

```
python manage.py shell
```

Il cursore cambia di forma (>>>) per mostrare che siamo nella shell di Django.

Esempio di API -1

Creiamo ora una nuova istanza della classe Prodotto e inseriamo i valori relativi ai suoi attributi (istruzione INSERT):

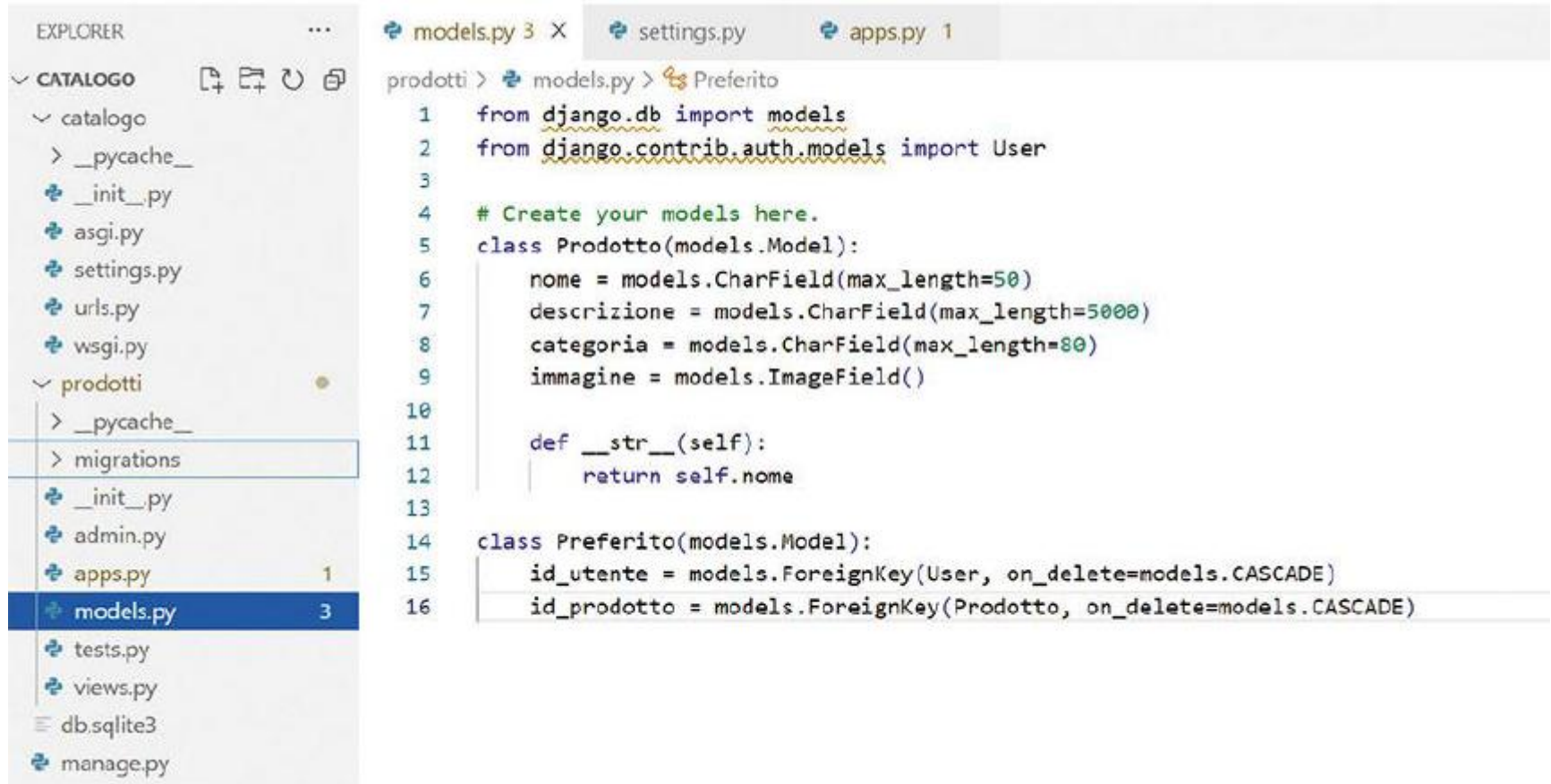
```
>>> from prodotti.models import Prodotto, Preferito
>>> prod = Prodotto(nome = "Bicchiere Baby", descrizione = "bicchiere antiscivolo, con
... beccuccio, indicato per bambini dai 24 mesi", categoria = "infanzia")
>>> prod.save()
>>> prod.id
1
>>>
```

Visualizziamo l'istanza appena inserita (istruzione SELECT), partendo dai singoli attributi e poi chiedendo la visualizzazione di tutte le istanze presenti:

```
>>> prod.nome
'Bicchiere Baby'
>>> prod.descrizione
'bicchiere antiscivolo, con beccuccio, indicato per bambini dai 24 mesi'
>>> prod.categoria
'infanzia'
>>> Prodotto.objects.all()
<QuerySet [<Prodotto: Prodotto object (1)>]>
>>>
```

Modifiche ai Models - 2

Aggiungiamo nel *models.py* il metodo `__str__()` alla classe *Prodotto*, che serve per visualizzare il nome del prodotto (e non il suo id):



```
EXPLORER
CATALOGO
  catalogo
    __pycache__
    __init__.py
    asgi.py
    settings.py
    urls.py
    wsgi.py
  prodotti
    __pycache__
    migrations
    __init__.py
    admin.py
    apps.py 1
    models.py 3
    tests.py
    views.py
    db.sqlite3
    manage.py

models.py 3 X settings.py apps.py 1
prodotti > models.py > Preferito
1  from django.db import models
2  from django.contrib.auth.models import User
3
4  # Create your models here.
5  class Prodotto(models.Model):
6      nome = models.CharField(max_length=50)
7      descrizione = models.CharField(max_length=5000)
8      categoria = models.CharField(max_length=80)
9      immagine = models.ImageField()
10
11     def __str__(self):
12         return self.nome
13
14 class Preferito(models.Model):
15     id_utente = models.ForeignKey(User, on_delete=models.CASCADE)
16     id_prodotto = models.ForeignKey(Prodotto, on_delete=models.CASCADE)
```


Esempio di API - 2

Quando vogliamo eseguire una visualizzazione di istanza usando un filtro (con la forma SELECT... WHERE...) possiamo usare il metodo `filter()`, che descrive le condizioni che l'attributo dell'istanza deve rispettare.

```
>>> Prodotto.objects.filter(id = 1)
<QuerySet [<Prodotto: Bicchiere Baby>]>
>>> Prodotto.objects.filter(nome__startswith = "Bab")
<QuerySet []>
>>> Prodotto.objects.filter(nome__startswith = "Bi")
<QuerySet [<Prodotto: Bicchiere Baby>]>
>>> Prodotto.objects.filter(nome__icontains = "B")
<QuerySet [<Prodotto: Bicchiere Baby>, <Prodotto: Set Bimbi>]>
>>>
```

Quando siamo sicuri che il risultato sia costituito da una sola istanza, per esempio se si effettua una ricerca per ID, al posto di `filter()` si può usare il metodo `get()` che restituisce un singolo valore:

```
>>> Prodotto.objects.filter(id = 2)
<QuerySet [<Prodotto: Set Bimbi>]>
>>> Prodotto.objects.get(id = 2)
<Prodotto: Set Bimbi>
>>>
```

Sito di amministrazione

Il framework Django contiene al suo interno un sito di amministrazione che permette a coloro che si occupano del sito web l'inserimento dei contenuti e la gestione degli utenti.

Il sito di amministrazione è accessibile solo [a un utente amministratore](#)

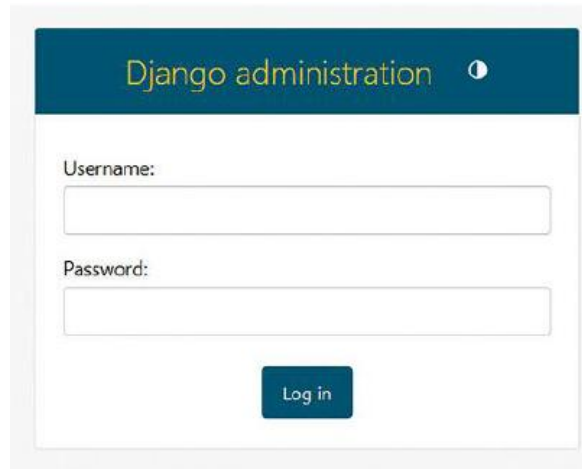
```
python manage.py createsuperuser
```

Il sito di amministrazione di Django è già attivato.

Per potervi accedere dobbiamo avviare il server Django

```
python manage.py runserver
```

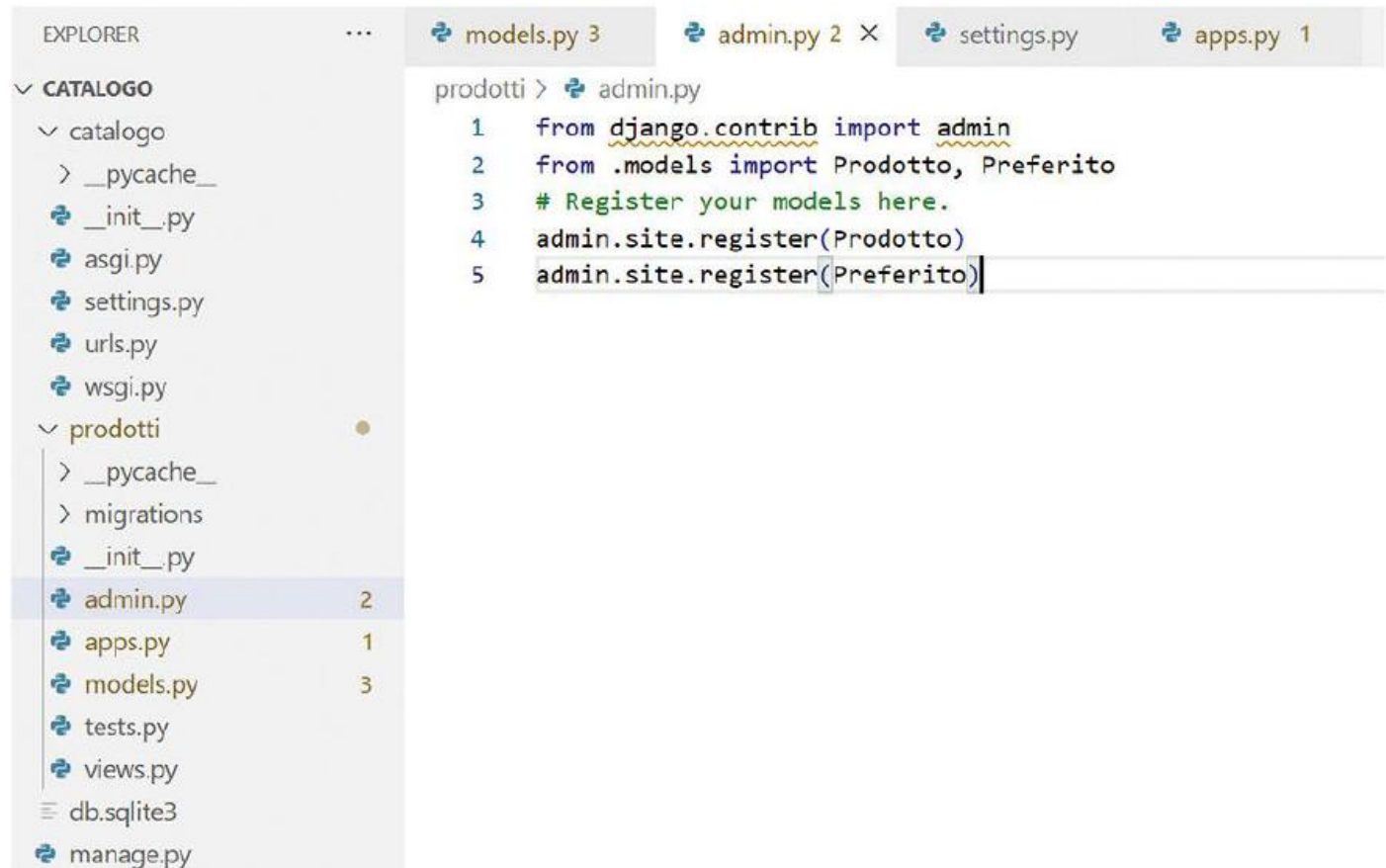
Possiamo quindi accedere dal browser all'indirizzo del server locale (di solito <http://127.0.0.1:8000/admin/>)

The image shows a screenshot of the Django administration login interface. At the top, there is a dark blue header with the text "Django administration" in a light green font, followed by a small circular icon. Below the header, the page has a white background. It features two input fields: the first is labeled "Username:" and the second is labeled "Password:". Both labels are in a small, dark font. Below the password field, there is a dark blue button with the text "Log in" in white. The entire login form is enclosed in a light gray border.

Configurare il sito di amministrazione

Al primo accesso il sito di amministrazione mostra solo la parte relativa alla gestione degli utenti, poiché non è ancora stato configurato per gestire anche la nostra app *prodotti*.

Per fare ciò bisogna aprire lo script *prodotti/admin.py* e inserire la dichiarazione della app



The screenshot shows a code editor with the Explorer panel on the left and the editor window on the right. The Explorer panel shows the project structure with the 'prodotti' app selected. The editor window shows the 'admin.py' file for the 'prodotti' app, which contains the following code:

```
prodotti > admin.py
1 from django.contrib import admin
2 from .models import Prodotto, Preferito
3 # Register your models here.
4 admin.site.register(Prodotto)
5 admin.site.register(Preferito)
```

Gestione file statici

La gestione dei file statici, come ad esempio le immagini, prevede che venga indicato a Django dove vogliamo che le immagini dei prodotti vengano salvate. Apriamo quindi il file *catalogo/settings.py*, che contiene le configurazioni del sito, scorriamo fino alla sezione relativa ai file statici e indichiamo che le immagini dei prodotti saranno posizionate in una cartella di nome *media* posta allo stesso livello di *catalogo*.



```
EXPLORER
  CATALOGO
    catalogo
      __pycache__
      __init__.py
      asgi.py
      settings.py
      urls.py
      wsgi.py
    media
    prodotti
    db.sqlite3
    manage.py

catalogo > settings.py > ...
107
108 LANGUAGE_CODE = 'en-us'
109
110 TIME_ZONE = 'UTC'
111
112 USE_I18N = True
113
114 USE_TZ = True
115
116
117 # Static files (CSS, JavaScript, Images)
118 # https://docs.djangoproject.com/en/5.0/howto/static-files/
119
120 STATIC_URL = 'static/'
121
122 MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
123 MEDIA_URL = '/media/'
124
125 # Default primary key field type
126 # https://docs.djangoproject.com/en/5.0/ref/settings/#default-auto-field
127
128 DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
129
```

Sistema di autenticazione

Il sistema di autenticazione di Django è un'altra delle caratteristiche del framework finalizzata a semplificare la vita del programmatore, in quanto offre:

1. un modello `user` preesistente, dotato dei seguenti attributi di base:
 - a) username;
 - b) password;
 - c) email;
 - d) nome;
 - e) cognome;
2. la funzione di creazione di un utente;
3. la funzione di autenticazione;
4. la funzione di cambio password;
5. le funzioni di assegnazione di permessi e di appartenenza a gruppi specifici;
6. le funzioni di login e di logout.

API per lo user

La creazione di un utente, se non usiamo il sito di amministrazione di Django, può anche essere effettuata attraverso le API, che poi ci serviranno per consentire a un utente che visiterà il sito di registrarsi al suo interno.

```
create_user(username, email = None, password = None, ** altri_campi)
```

per salvare sul database è necessario inserire anche il comando `save()`.

```
>>> from django.contrib.auth.models import User
>>> user = User.objects.create_user("mario", "mario@prova.com", "mario123")
>>> user.first_name = "Mario"
>>> user.last_name = "Rossi"
>>> user.save()
>>>
```

API per le Foreign Key

Per memorizzare nella tabella del database l'utente che, visualizzando un prodotto specifico, lo dichiara come preferito, è necessario:

- identificare l'utente;
- identificare il prodotto;
- inserire i due identificatori nella tabella dei Preferito.

Nel sito web l'utente viene identificato poiché si è registrato (altrimenti non avrebbe a disposizione questa funzionalità); anche il prodotto è identificato, perché il prodotto deve essere selezionato per poter vedere il dettaglio e sceglierlo come preferito.

Supponiamo ora che l'utente «mario» voglia contrassegnare come preferito il prodotto “Pennello in silicone”. Otteniamo gli id di entrambe le istanze e creiamo una nuova istanza della classe Preferito che salveremo nel database,

```
>>> utente = User.objects.get(username = "mario")
>>> prefer = Prodotto.objects.get(nome = "Pennello in silicone")
>>> Preferito.objects.create(id_utente = utente, id_prodotto = prefer)
<Preferito: Preferito object (1)>
>>> Preferito.objects.all()
<QuerySet [<Preferito: Preferito object (1)>]>
```

Creare View e Url

Una **view** è un tipo di pagina web che appartiene al sito, per esempio la home page, la pagina di dettaglio di un prodotto o la pagina di login.

In Django le pagine web e gli altri contenuti sono gestiti attraverso le **views**. Ogni view è rappresentata da una funzione Python (o, nel caso di views basate su classi, da un metodo): Django sceglie quale view utilizzare esaminando la URL che viene richiesta (per la precisione, la parte di URL che segue il nome di dominio).

Le views sono sviluppate e contenute nel file **views.py**.

Una **URL**, per come viene gestita da Django, ha una struttura del tipo `/prodotti/<nome>/`.

Per sapere quale view va utilizzata nella pagina che corrisponde a una URL, Django usa una mappa, che è contenuta nel file **urls.py**.

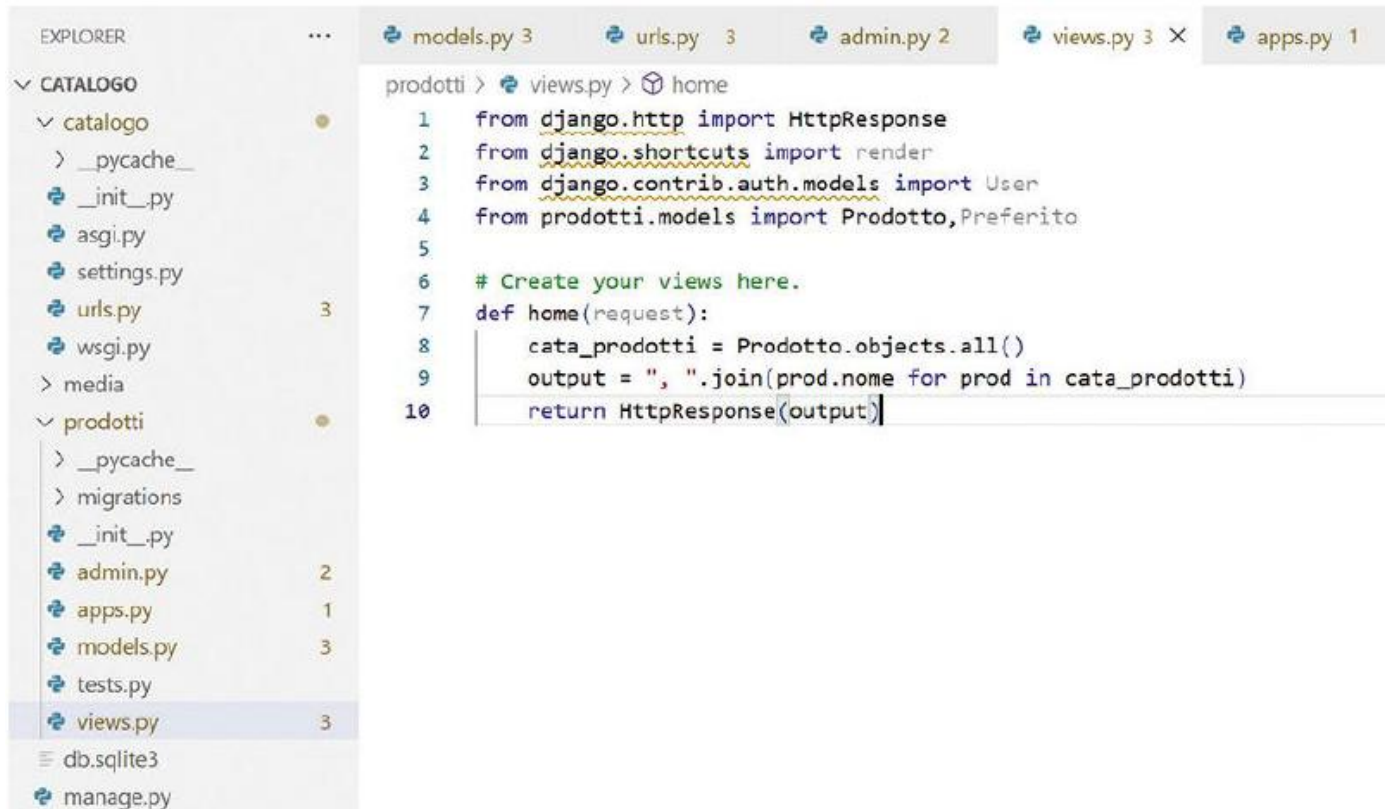
Di solito i due file relativi a views e URL vengono compilati in parallelo.

Creare View

Ogni view può fare una di queste due cose:

- restituire un oggetto `HttpResponse` con il contenuto della pagina richiesta;
- oppure sollevare un'eccezione, per esempio `Http404`.

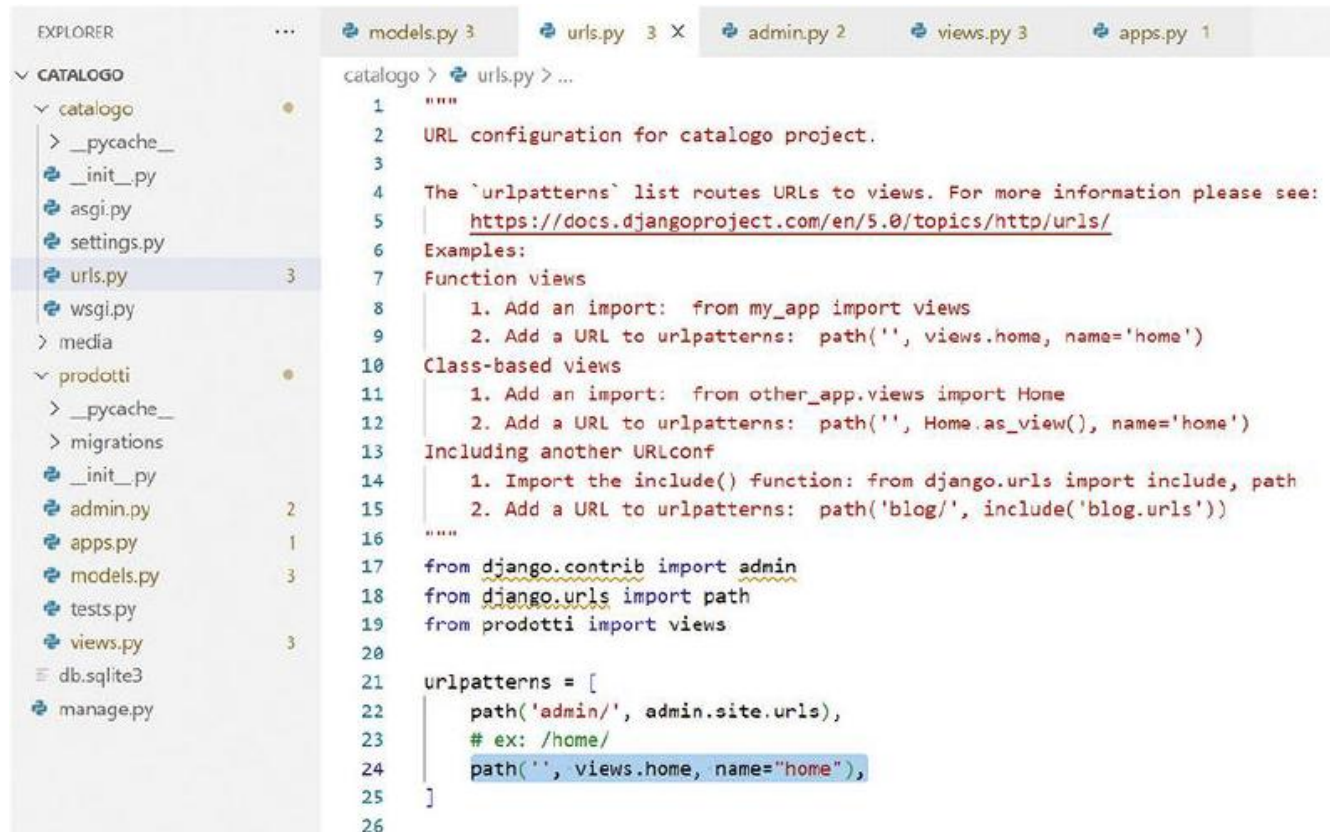
Cominciamo a scrivere la prima view, quella relativa alla home page, in cui vengono visualizzati i prodotti contenuti nel database.



```
prodotti > views.py > home
1  from django.http import HttpResponse
2  from django.shortcuts import render
3  from django.contrib.auth.models import User
4  from prodotti.models import Prodotto, Preferito
5
6  # Create your views here.
7  def home(request):
8      cata_prodotti = Prodotto.objects.all()
9      output = ", ".join(prod.nome for prod in cata_prodotti)
10     return HttpResponse(output)
```

Aggiornare Url

Bisogna quindi aggiornare il file *urls.py* per predisporlo a gestire la view appena preparata:



The screenshot shows a Django IDE interface. On the left, the 'EXPLORER' pane displays the project structure for 'catalogo'. The 'urls.py' file is highlighted under the 'catalogo' directory. On the right, the 'urls.py' file is open in the editor. The code includes a docstring with a link to the Django documentation, examples for function and class-based views, and the 'urlpatterns' list. The 'urlpatterns' list contains a path for the admin site and a path for the home view, which is highlighted in blue.

```
catalogo > urls.py > ...
1  """
2  URL configuration for catalogo project.
3
4  The `urlpatterns` list routes URLs to views. For more information please see:
5  | https://docs.djangoproject.com/en/5.0/topics/http/urls/
6  | Examples:
7  | Function views
8  |     1. Add an import: from my_app import views
9  |     2. Add a URL to urlpatterns: path('', views.home, name='home')
10 | Class-based views
11 |     1. Add an import: from other_app.views import Home
12 |     2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
13 | Including another URLconf
14 |     1. Import the include() function: from django.urls import include, path
15 |     2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16 | """
17 from django.contrib import admin
18 from django.urls import path
19 from prodotti import views
20
21 urlpatterns = [
22     path('admin/', admin.site.urls),
23     # ex: /home/
24     path('', views.home, name="home"),
25 ]
26
```

Per poter vedere qualcosa, dobbiamo avviare il server Django e collegarci all'indirizzo del server locale per vedere nel browser la nostra home page, che al momento è una pagina bianca con il nome dei tre prodotti separato da una virgola.

Il sistema di Template

Il sistema di template di Django permette di creare e usare semplici modelli di pagina HTML (con la possibilità di aggiungere fogli di stile CSS), ma anche di sfruttare template di terze parti, come ad esempio Bootstrap.

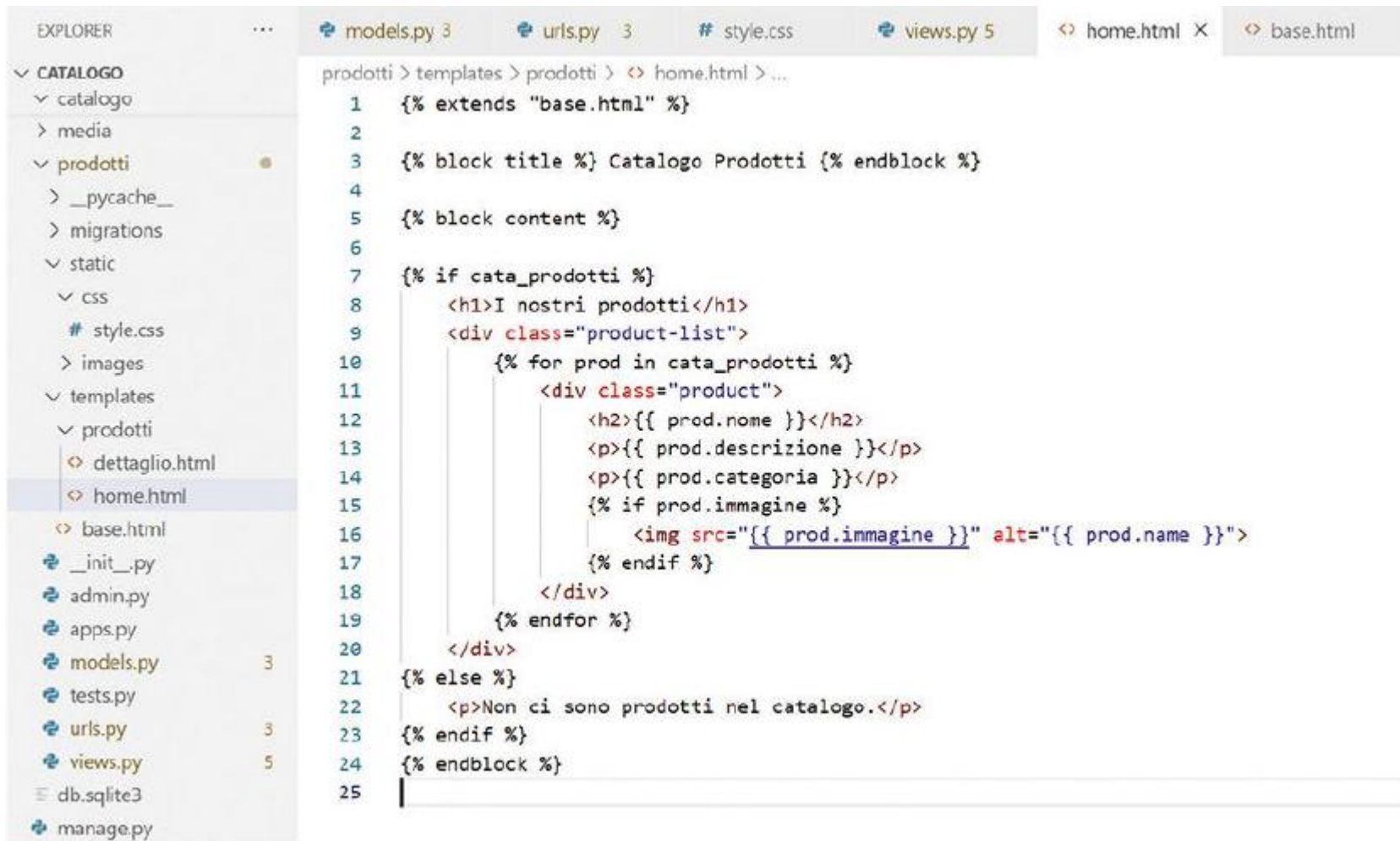
Per configurare correttamente il sistema di template in Django dobbiamo eseguire i passi seguenti.

1. Creiamo una directory di nome *templates* nella cartella dell'app *prodotti*: il nome della cartella deve essere rispettato perché Django andrà a cercare i template in questa cartella.
2. Al suo interno creiamo un'altra cartella che si chiama come l'app, cioè *prodotti*.
3. Nella cartella *templates* creiamo un file HTML di nome *base.html*, che useremo per definire i componenti base di ogni pagina web.
4. Nel file *base.html* includiamo un'immagine che useremo come logo del sito web. L'immagine deve essere contenuta in una cartella di nome *static/images* che dobbiamo creare all'interno della cartella dell'app *prodotti*.
5. Nel file *base.html* inseriamo anche il riferimento a un CSS di nome *style.css*, che salviamo in una cartella di nome *static/css/style.css*.
6. Nella cartella *templates/prodotti* creiamo il file HTML di nome *home.html* come estensione del file *base.html*, che è la home page del sito.

Il sistema di Template: base.html

```
prodotti > templates > <> base.html > html > body > nav.navbar
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      {% load static %}
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <link rel="stylesheet" href="{% static 'css/styles.css' %}">
8      <title>{% block title %}{% endblock %}</title>
9  </head>
10 <body>
11     <nav class="navbar">
12         <div class="logo">
13             
14             <h1>Catalogo Prodotti</h1>
15         </div>
16         <ul>
17             <li><a href="#">Home</a></li>
18             <li><a href="#">Prodotti</a></li>
19         </ul>
20     </nav>
21     {% block content %}{% endblock %}
22 </body>
23 </html>
24
25
26
```

Il sistema di Template: home.html

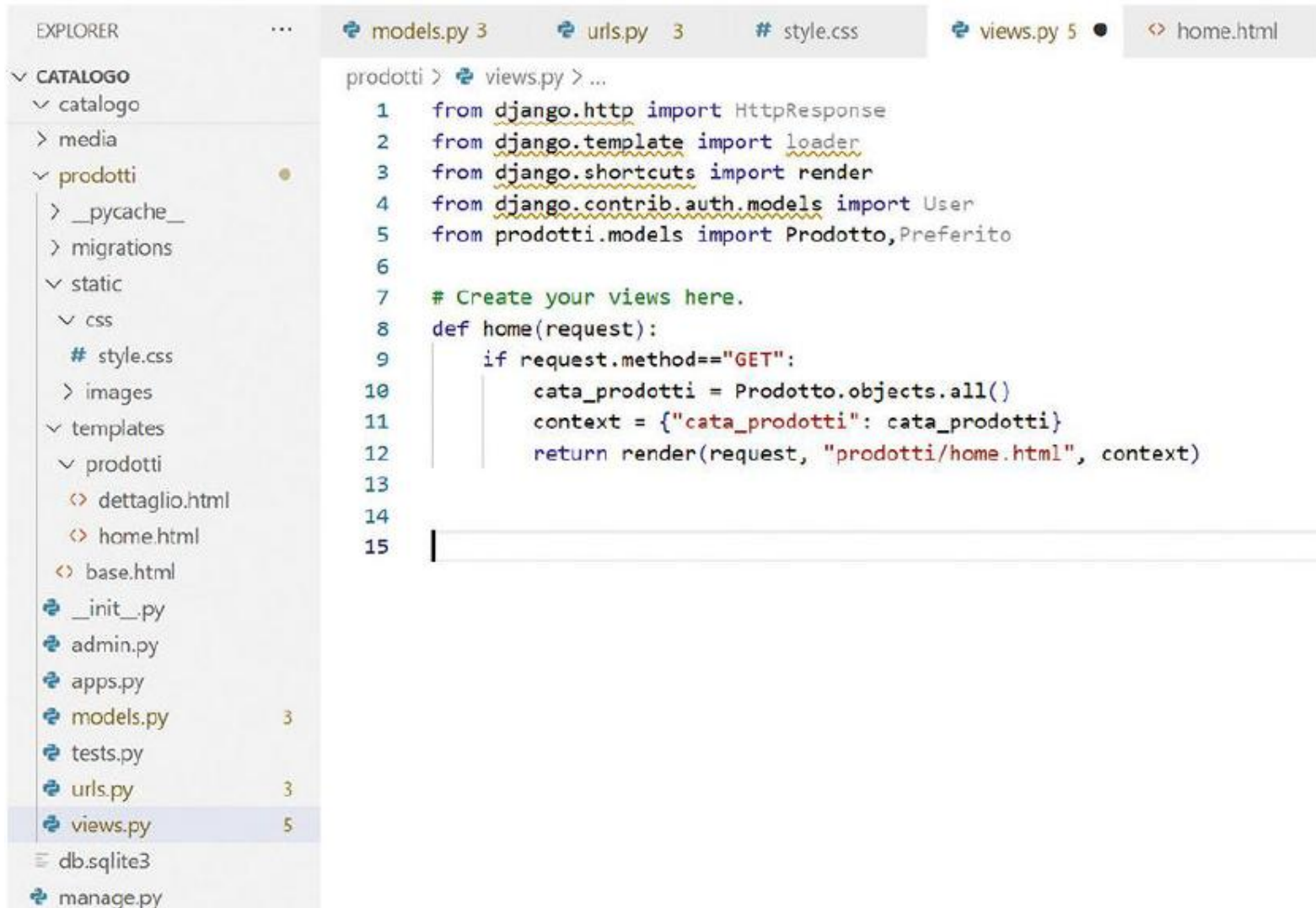


The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with a 'prodotti' directory containing 'dettaglio.html' and 'home.html'. The code editor shows the content of 'home.html', which is a Django template. The template extends 'base.html' and contains a block for the product list. It uses Django's template language to iterate over 'cata_prodotti' and display product details, including a name, description, category, and an image. The image is displayed using an tag with the product name as the alt text. The template also includes a fallback message 'Non ci sono prodotti nel catalogo.'

```
prodotti > templates > prodotti > <> home.html > ...
1  {% extends "base.html" %}
2
3  {% block title %} Catalogo Prodotti {% endblock %}
4
5  {% block content %}
6
7  {% if cata_prodotti %}
8      <h1>I nostri prodotti</h1>
9      <div class="product-list">
10         {% for prod in cata_prodotti %}
11             <div class="product">
12                 <h2>{{ prod.nome }}</h2>
13                 <p>{{ prod.descrizione }}</p>
14                 <p>{{ prod.categoria }}</p>
15                 {% if prod.immagine %}
16                     
17                 {% endif %}
18             </div>
19         {% endfor %}
20     </div>
21 {% else %}
22     <p>Non ci sono prodotti nel catalogo.</p>
23 {% endif %}
24 {% endblock %}
25
```


Modifica di views

Modifichiamo ora la view corrispondente per usare il template *home.html*



The screenshot shows a Django project editor with the following components:

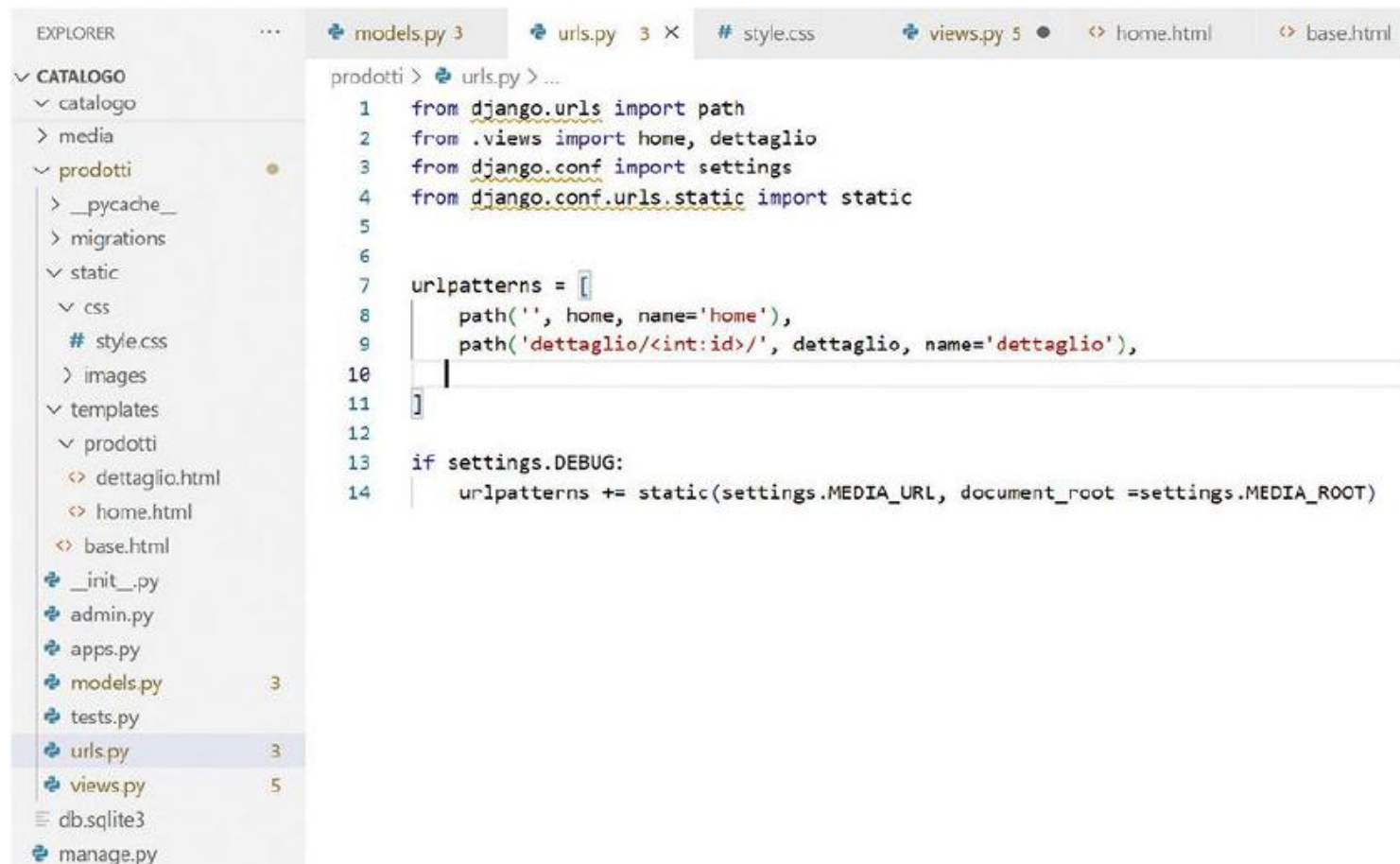
- EXPLORER (Left Panel):** Displays the project structure. The `prodotti` directory is expanded, showing `__pycache__`, `migrations`, `static` (containing `css` with `style.css` and `images`), `templates` (containing `prodotti` with `dettaglio.html`, `home.html`, and `base.html`), `__init__.py`, `admin.py`, `apps.py`, `models.py`, `tests.py`, `urls.py`, `views.py` (selected), `db.sqlite3`, and `manage.py`.
- Top Tabs:** `models.py 3`, `urls.py 3`, `# style.css`, `views.py 5` (active), and `<> home.html`.
- Code Editor (Right Panel):** Shows the `views.py` file with the following code:

```
prodotti > views.py > ...
1  from django.http import HttpResponseRedirect
2  from django.template import loader
3  from django.shortcuts import render
4  from django.contrib.auth.models import User
5  from prodotti.models import Prodotto, Preferito
6
7  # Create your views here.
8  def home(request):
9      if request.method=="GET":
10         cata_prodotti = Prodotto.objects.all()
11         context = {"cata_prodotti": cata_prodotti}
12         return render(request, "prodotti/home.html", context)
13
14
15
```

Creare Url per prodotti

Creiamo un file *urls.py* per la app *prodotti*, che contiene le URL specifiche per questa app, e successivamente aggiorniamo il file *urls.py* generale, cioè quello relativo al sito *catalogo*.

1. Creiamo un nuovo file */prodotti/urls.py*

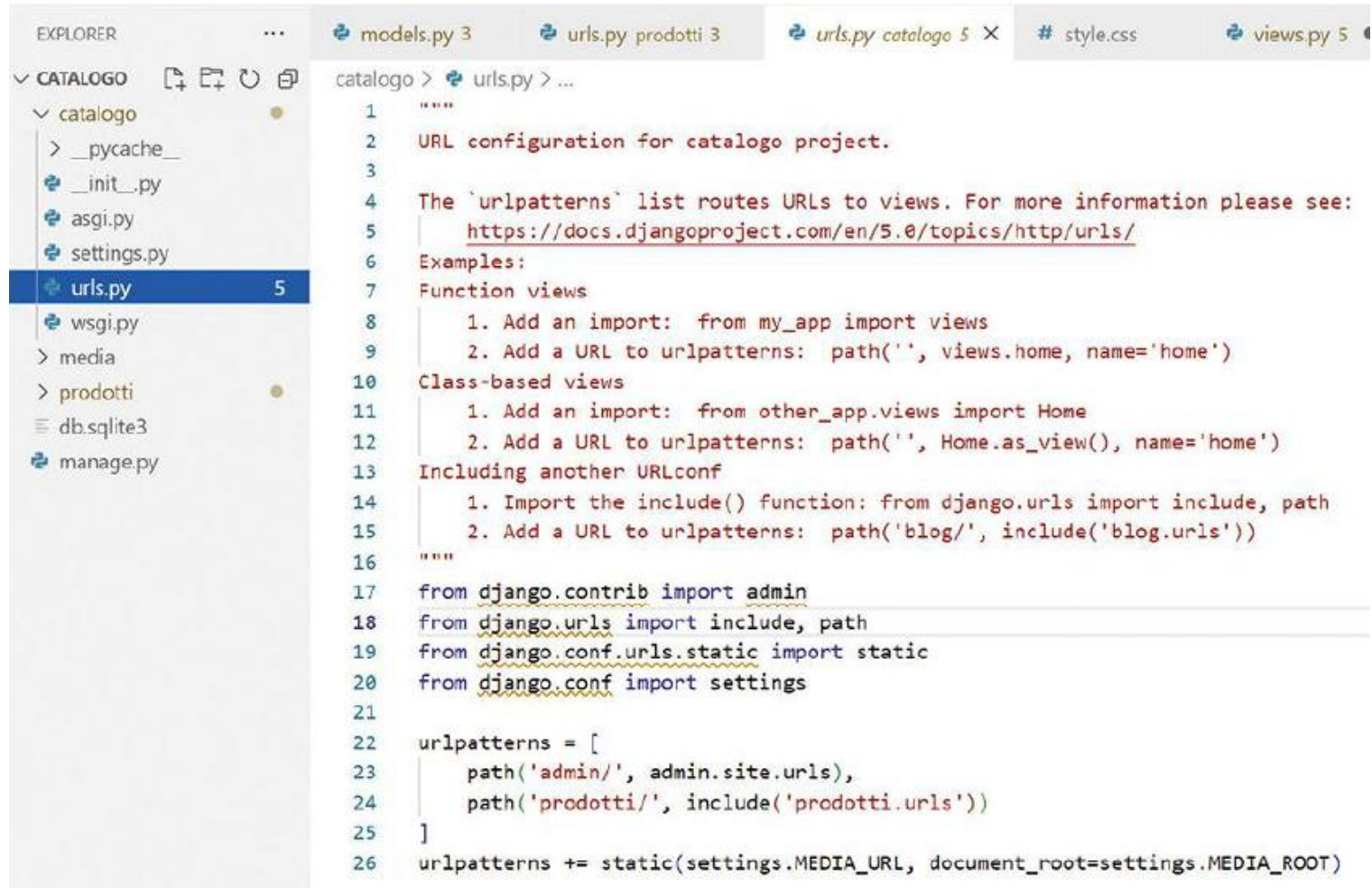


The screenshot shows a code editor with the Explorer panel on the left and the code editor on the right. The Explorer panel shows the project structure with the 'prodotti' app folder selected. The code editor shows the content of the new file 'urls.py' for the 'prodotti' app. The code includes imports for Django's URL handling and defines a list of URL patterns for the app. The file is named 'urls.py' and is located in the 'prodotti' app directory.

```
prodotti > urls.py > ...
1  from django.urls import path
2  from .views import home, dettaglio
3  from django.conf import settings
4  from django.conf.urls.static import static
5
6
7  urlpatterns = [
8      path('', home, name='home'),
9      path('dettaglio/<int:id>/', dettaglio, name='dettaglio'),
10
11 ]
12
13 if settings.DEBUG:
14     urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Aggiornare URL catalogo

2. Aggiorniamo il file *catalogo/urls.py* per indicare che le URL dell'app *prodotti* si trovano nella cartella dell'app *prodotti*. In questo modo, per ogni nuova URL che aggiungiamo aggiorneremo solo queste.



The screenshot shows a code editor with the Explorer panel on the left and the code editor on the right. The Explorer panel shows the project structure with the following folders and files:

- CATALOGO
 - catalogo
 - __pycache__
 - __init__.py
 - asgi.py
 - settings.py
 - urls.py (selected)
 - wsgi.py
 - media
 - prodotti
 - db.sqlite3
 - manage.py

The code editor shows the contents of the *catalogo/urls.py* file:

```
1  """
2  URL configuration for catalogo project.
3
4  The 'urlpatterns' list routes URLs to views. For more information please see:
5  | https://docs.djangoproject.com/en/5.0/topics/http/urls/
6  | Examples:
7  | Function views
8  |     1. Add an import: from my_app import views
9  |     2. Add a URL to urlpatterns: path('', views.home, name='home')
10 | Class-based views
11 |     1. Add an import: from other_app.views import Home
12 |     2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
13 | Including another URLconf
14 |     1. Import the include() function: from django.urls import include, path
15 |     2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16 | """
17 from django.contrib import admin
18 from django.urls import include, path
19 from django.conf.urls.static import static
20 from django.conf import settings
21
22 urlpatterns = [
23     path('admin/', admin.site.urls),
24     path('prodotti/', include('prodotti.urls'))
25 ]
26
27 urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```


Vedere la home page preparata

Avviare il server Django e per vedere se tutto il sistema che abbiamo preparato funziona, collegandoci alla URL del server locale e inserendo nella barra degli indirizzi */prodotti/*



Creare la pagina di dettaglio per prodotti - 1

1. Definiamo la view per visualizzare il dettaglio di un prodotto selezionato. Apriamo il file *prodotti/views.py* e aggiungiamo la view relativa a dettaglio.

```
1 from django.http import HttpResponse
2 from django.template import loader
3 from django.shortcuts import render
4 from django.contrib.auth.models import User
5 from prodotti.models import Prodotto, Preferito
6
7 # Create your views here.
8 def home(request):
9     if request.method=="GET":
10         cata_prodotti = Prodotto.objects.all()
11         context = {"cata_prodotti": cata_prodotti}
12         return render(request, "prodotti/home.html", context)
13
14 def dettaglio(request, id):
15     if request.method=="GET":
16         prod = Prodotto.objects.get(id=id)
17         context = {
18             "prod": prod,
19             "prod.nome": prod.nome,
20             "prod.descrizione": prod.descrizione,
21             "prod.categoria": prod.categoria,
22             "prod.immagine": prod.immagine
23         }
24         return render(request, "prodotti/dettaglio.html", context)
```

Creare la pagina di dettaglio per prodotti - 2

2. Prepariamo il template *dettaglio.html* che viene utilizzato dalla view.

```
1 {% extends "base.html" %}
2 {% block title %} Dettaglio prodotto {% endblock %}
3
4 {% block content %}
5
6     <div class="product-detail">
7         <h1>{{ prod.name }}</h1>
8         <p>{{ prod.descrizione }}</p>
9         <p>{{ prod.categoria }}</p>
10        {% if prod.immagine %}
11            
12        {% endif %}
13        <a href="{% url 'home' %}" class="back-link"> Ritorno a Prodotti</a>
14    </div>
15 {% endblock %}
```

Creare la pagina di dettaglio per prodotti - 3

3. Aggiorniamo il file *prodotti/urls.py* in modo da gestire correttamente la richiesta URL quando si seleziona un prodotto nella home page.

```
1 from django.urls import path
2 from .views import home, dettaglio
3 from django.conf import settings
4 from django.conf.urls.static import static
5 from . import views
6
7 urlpatterns = [
8     path('', home, name='home'),
9     path('<int:id>/', dettaglio, name='dettaglio'),
10
11 ]
12
13 if settings.DEBUG:
14     urlpatterns += static(settings.MEDIA_URL, document_root =settings.MEDIA_ROOT)
```



Creare la pagina di dettaglio per prodotti - 4

4. Aggiorniamo infine *home.html* affinché sia possibile selezionare ogni prodotto tramite link ipertestuale.

```
1 {% extends "base.html" %}
2
3 {% block title %} Catalogo Prodotti {% endblock %}
4
5 {% block content %}
6
7
8
9 {% if cata_prodotti %}
10     <h1>I nostri prodotti</h1>
11     <div class="product-list">
12         {% for prod in cata_prodotti %}
13             <a href="{% url 'dettaglio' prod.id %}">
14                 <div class="product">
15                     <h2>{{ prod.name }}</h2>
16                     <p>{{ prod.descrizione }}</p>
17                     <p>{{ prod.categoria }}</p>
18                     {% if prod.immagine %}
19                         
20                     {% endif %}
21                 </div>
22             </a>
23         {% endfor %}
24     </div>
25 {% else %}
26     <p>Non ci sono prodotti nel catalogo.</p>
27 {% endif %}
```

Cookie

I **cookie** sono piccoli file di testo che vengono memorizzati sul dispositivo dell'utente dal browser.

Ogni volta che l'utente visita il sito, il browser invia automaticamente i relativi cookie al server.

Questi file vengono usati per memorizzare dati come l'ID di sessione, le preferenze dell'utente o le informazioni di autenticazione, mantenendo lo stato dell'utente tra sessioni diverse (per esempio, mantenendolo loggato).

Possono essere configurati per avere una scadenza specifica (**cookie persistenti**) o per essere eliminati alla chiusura del browser (**cookie di sessione**).

Django usa i cookie per gestire alcune funzionalità fondamentali di un'applicazione web, come l'autenticazione e il tracciamento delle sessioni, e fornisce un modo semplice per gestirli tramite l'oggetto `HttpResponse` e i suoi metodi.

Per esempio, si può impostare un cookie in una risposta HTTP utilizzando il metodo `set_cookie()`, leggerlo tramite il metodo `COOKIES.get()` ed eliminarlo con il metodo `delete_cookie()`.

Cookie

Vediamo come farlo nel file *views.py*.

```
1  from django.http import HttpResponse
2
3  def set_cookie(request):
4      #Imposta il cookie
5      response = HttpResponse('Impostare un cookie')
6      response.set_cookie('user_preference', 'dark_mode', max_age = 3600)
7      return response
8
9  def get_cookie(request):
10     #Legge il cookie
11     preference = request.COOKIES.get('user_preference', 'default_mode')
12     return HttpResponse(f'La preferenza è: {preference}')
13
14  def delete_cookie(request):
15     #Elimina il cookie
16     response = HttpResponse('Cookie eliminato!')
17     response.delete_cookie('user_preference')
18     return response
```

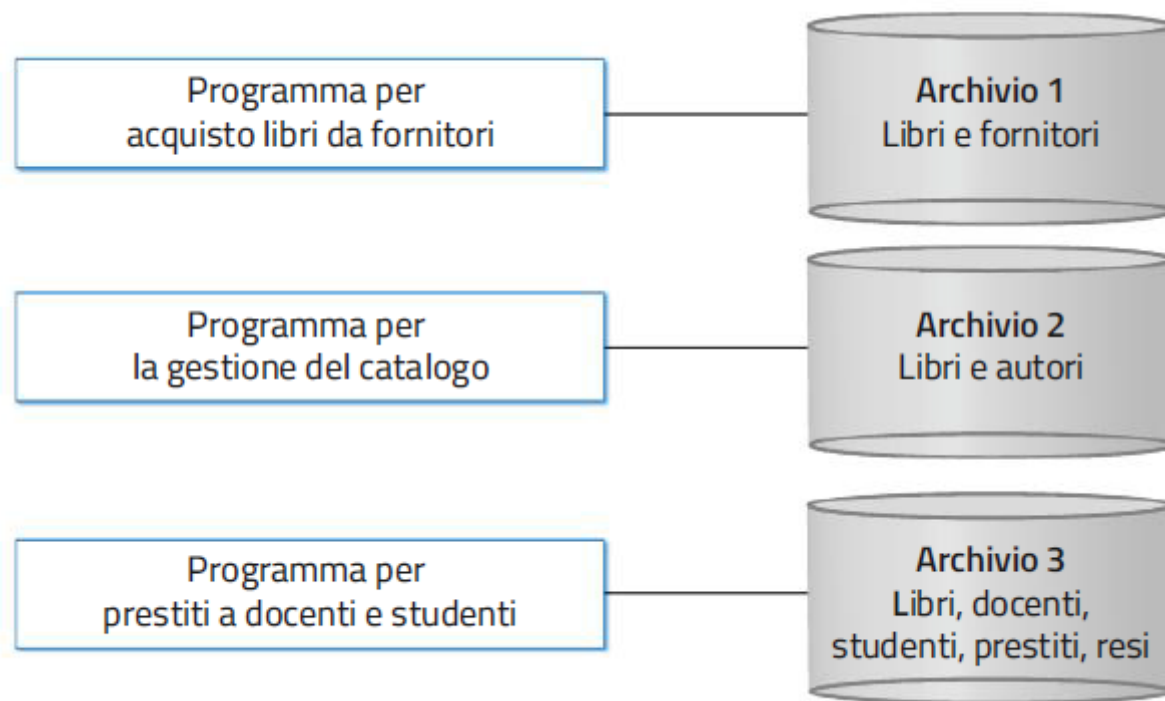
Capitolo 1

Introduzione alle basi di dati

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume
Informatica per progetti – Lavorare con le basi di dati, di C. Batini, M. Fiore, ISBN 9788808769978

Dagli archivi alle basi di dati

In una biblioteca di istituto, ad esempio, per gestire gli acquisti dei libri dai fornitori, il loro inserimento in un catalogo, e per tenere traccia dei prestiti agli insegnanti e agli studenti, possono essere scritti tre diversi programmi software, ciascuno che gestisce un insieme di dati, su cui opera; gli insiemi di dati sono chiamati **archivio**



Dagli archivi alle basi di dati

Il fatto che i dati siano locali ai programmi genera problemi poiché i *dati sui libri* sono utilizzati in tutti i programmi, generando:

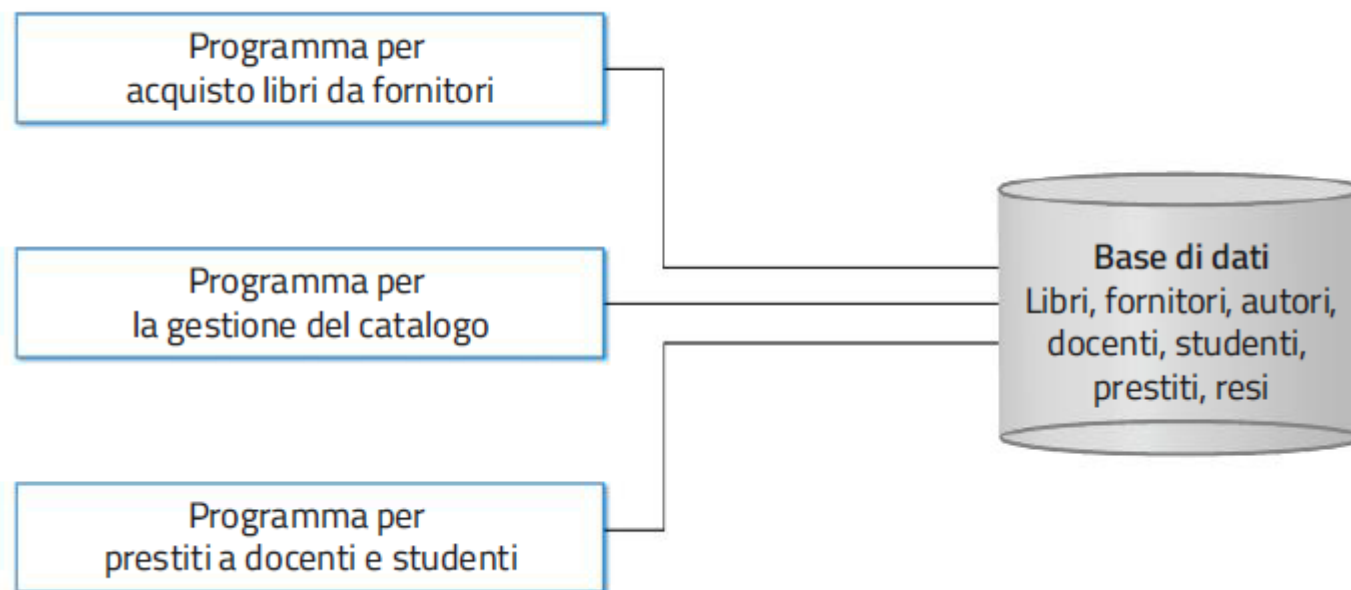
- una potenziale duplicazione di dati
- rischi di inconsistenza.

Le **basi di dati** nascono per risolvere questo tipo di problemi.

In una base di dati ogni dato viene rappresentato una sola volta e tutti i programmi accedono alla stessa copia del dato, di conseguenza non ci sono duplicazioni nei dati

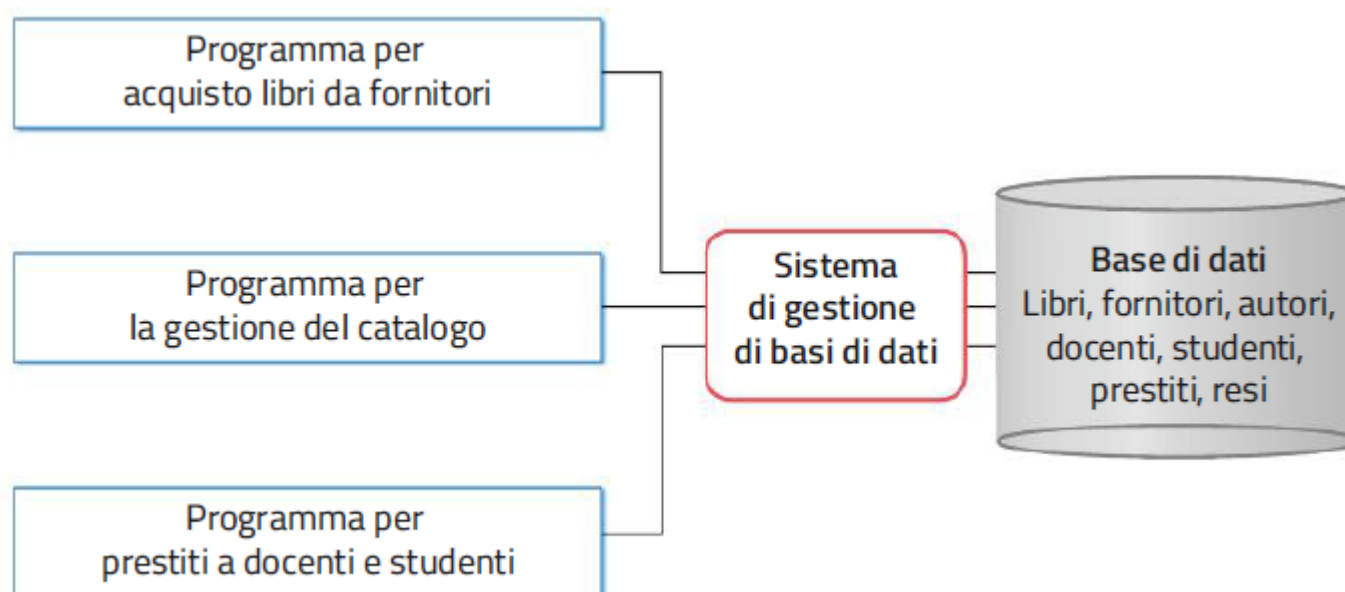
Dagli archivi alle basi di dati

Nel nostro esempio, quando un nuovo libro viene comprato e inserito nella base di dati dal *programma per acquisto di libri*, tutti i programmi possono immediatamente utilizzare i dati che lo riguardano, e non c'è bisogno di inserirli tre volte.



Data Base Management System (DBMS)

Se programmi diversi utilizzano la stessa base di dati, occorre garantire un corretto accesso concorrente: in questo caso è necessario utilizzare un componente software chiamato sistema di gestione di basi di dati (*Database Management System*, o *DBMS*)



Data Base Management System (DBMS)

Un **sistema di gestione di basi di dati** (*Data Base Management System, DBMS*) è un programma software che si interpone tra i programmi degli utenti e la base di dati rappresentata in memoria e che gestisce tutte le elaborazioni degli utenti.

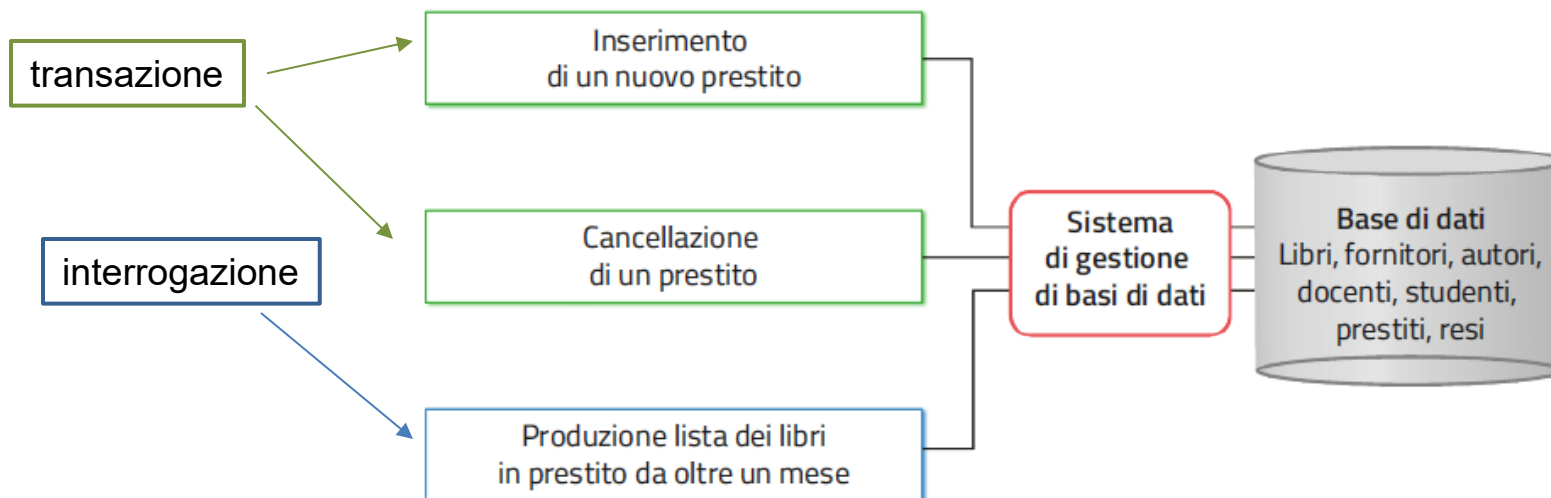
Il sistema di gestione di basi di dati deve garantire a tutti gli utenti il controllo di una base di dati che sia:

1. **grande**, nel senso che i dati non possono essere contenuti in memoria centrale, ma debbano essere permanentemente memorizzati in memoria secondaria;
2. **persistente**, perché anche in presenza di guasti devono esistere meccanismi che ripristinino lo stato della base di dati precedente al guasto;
3. **condivisa** tra una pluralità di utenti, per cui il DBMS deve garantire un corretto accesso concorrente ai programmi da parte degli utenti. La condivisione può essere limitata da opportuni criteri di accesso, che stabiliscono che solo alcuni utenti possano accedere a dati a loro riservati.

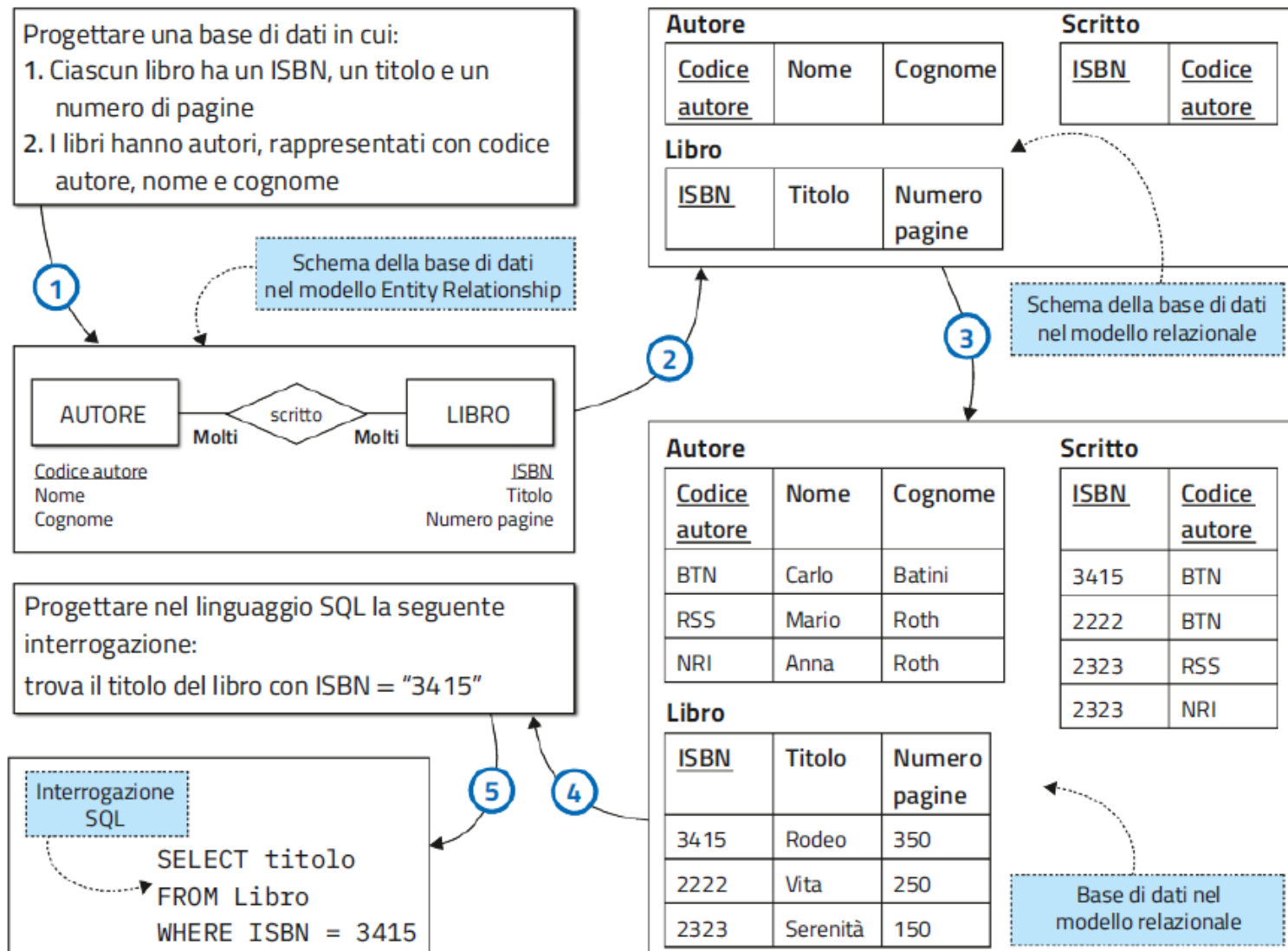
Operazioni su Data Base

Le operazioni che gli utenti eseguono tramite i programmi applicativi sono di due tipologie molto diverse tra loro:

- Una **transazione** è un'operazione realizzata da un programma software che esegue inserimenti, cancellazioni e modifiche ai dati all'interno della base di dati, *con ciò cambiando la base di dati*.
- Un'**interrogazione** è un'operazione realizzata da un programma software che estrae dalla base di dati i dati che rispettano determinate condizioni; una interrogazione, dunque, *non cambia la base di dati*.



Progettazione della Base Dati Biblioteca



Fasi della progettazione

1. Definizione dei dati necessari all'applicazione software e loro descrizione con un modello concettuale.
2. Traduzione dello schema dei dati dal modello concettuale al modello relazionale, che è il modello utilizzato da molti DBMS,
3. Inserimento e aggiornamento dei dati all'interno della base di dati.
4. Formulazione delle operazioni di interesse; interrogazioni e transazioni
5. Espressione delle interrogazioni di interesse nel linguaggio SQL, che è il linguaggio comunemente adottato nella gestione delle basi di dati.

Concetti principali di una Base Dati

Il modello più diffuso nei sistemi di gestione di basi di dati è il modello **relazionale**.

Una **relazione** o tabella è una struttura tabellare costituita dal nome della relazione, da una prima riga costituita dai nomi delle colonne e da un insieme di righe le cui celle contengono i dati di interesse, uno per cella.

Uno **schema di relazione** è costituito dal nome della relazione e dai nomi delle colonne, che vengono chiamate attributi della relazione.

Lo schema di relazione non cambia mai nel tempo. Può cambiare invece il contenuto della relazione, cioè la sua istanza.

I dati nel loro insieme costituiscono l'**istanza** della relazione.

Lo schema della relazione può essere rappresentato anche attraverso la notazione compatta: *Playlist (Titolo, Nome del cantante)*

Schema logico	{	Playlist		}	Relazione o tabella
		Titolo	Nome del cantante		
Istanza	{	Break my heart	Bruno Jupiter	}	
		Dance forever	Marc Leroy		
		Dance dance dance	Paulo Rosa		
		Non ti fermare	Giò Trap		

Concetti principali di una Base Dati

Una **struttura di rappresentazione** è una particolare modalità con cui possiamo organizzare e descrivere un insieme di dati, come, per esempio, una relazione, una riga di una relazione, una colonna di una relazione, un grafo.

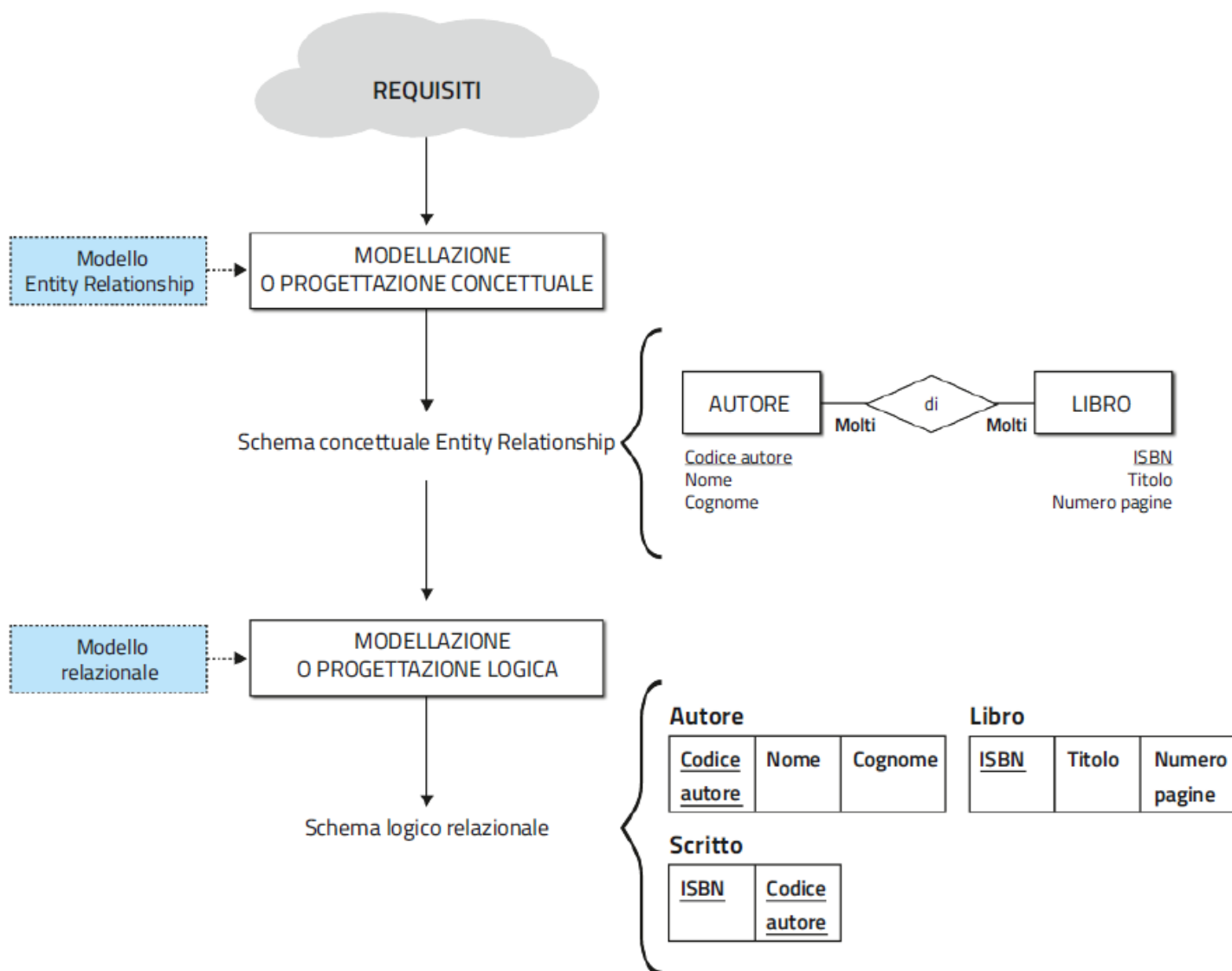
Un **modello dei dati** è un insieme di strutture di rappresentazione con cui si descrive una base di dati. Nel modello relazionale le strutture di rappresentazione sono la relazione e l'attributo.

Una **base di dati** è una collezione di dati di interesse di uno o più utenti; essa può essere definita a due livelli.

1. A **livello logico**, una base di dati è l'insieme delle strutture di rappresentazione dei dati usate per descrivere un frammento di mondo. Le strutture di rappresentazione elaborabili da un DBMS sono le relazioni.

2. A **livello fisico**, una base di dati è l'insieme strutturato delle celle di memoria digitale utilizzate per rappresentare la base di dati espressa a livello logico. La memoria digitale utilizzata può essere realizzata con varie tecnologie, come un disco o una memoria a stato solido.

Progettazione di una Base Dati



Requisiti

I testi che espongono i concetti e gli oggetti che intendiamo rappresentare all'interno di una base di dati sono chiamati **requisiti**, che possiamo definire come le descrizioni complete, accurate e non ambigue dei dati che sono coinvolti nella base di dati.

I requisiti esprimono le esigenze degli utenti in termini dei dati che devono essere memorizzati, ritrovati e utilizzati per rispondere a tali esigenze.

I requisiti vanno organizzati per punti.

Modelli dei dati

Il modello **Entity Relationship** è utilizzato in una fase intermedia della progettazione, perché è più ricco di strutture di rappresentazione rispetto al modello relazionale, ed è dotato di una rappresentazione grafica comprensibile anche per un utente non informatico.

Il modello **relazionale** è il modello i cui dati vengono visti dalle interrogazioni e dai programmi software.

Lo schema Entity Relationship, al contrario dello schema relazionale, non ha una corrispondente istanza associata, perché questa è creata e aggiornata solo nel momento di utilizzo della base dati, in cui è usato il modello relazionale.

Un modello diffusamente usato soprattutto per applicazioni di intelligenza artificiale e rappresentazione della conoscenza è il **modello a grafo**, in cui sono definite due strutture di rappresentazione, i nodi, che rappresentano sia concetti (per esempio Libro), sia oggetti (per esempio “La Divina Commedia”) del mondo reale, e gli archi, che mettono in relazione concetti e oggetti tra loro.

Modello Concettuale Entity Relationship E/R

I requisiti descrivono la realtà in termini di *concetti*, *caratteristiche* e *azioni/collegamenti logici* tra concetti.

I concetti espressi nei requisiti corrispondono a classi (o *insiemi*) di oggetti.

I concetti sono descritti da alcune loro caratteristiche e i requisiti indicano anche alcune azioni o collegamenti logici che coinvolgono i *concetti*.

L'essenza del modello Entity Relationship (modello E/R): nella realtà ci sono concetti (Entity), che hanno delle caratteristiche (o proprietà), e azioni o collegamenti logici (Relationship) tra concetti.

Le entità (*Entity*) sono i singoli *concetti* espressi nei requisiti.

Un'entità rappresenta una *classe di oggetti osservabili* nella realtà. Le entità hanno attributi, le *caratteristiche* delle entità a cui possiamo associare dei valori.

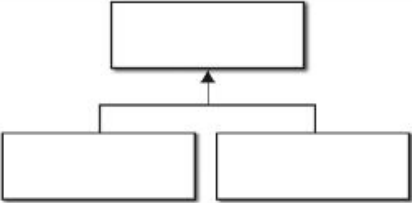
Le *associazioni* (*Relationship*) sono le *azioni* che collegano le entità o, più in generale, i legami logici tra le entità nel mondo reale.

Un'associazione si definisce su due o più entità, e rappresenta un legame logico tra tali entità.

Modello Concettuale Entity Relationship E/R

Le entità, gli attributi e le associazioni sono le strutture del modello concettuale Entity Relationship.

Il modello E/R è stato introdotto nel 1976 da Peter P. Chen; si usa nella prima fase concettuale della progettazione di una base dati e ha il vantaggio di utilizzare una notazione grafica facilmente comprensibile.

Struttura di rappresentazione	Simbolo grafico
Entità	
Attributo di entità	
Associazione	
Attributo di associazione	
È-un	
Generalizzazione	

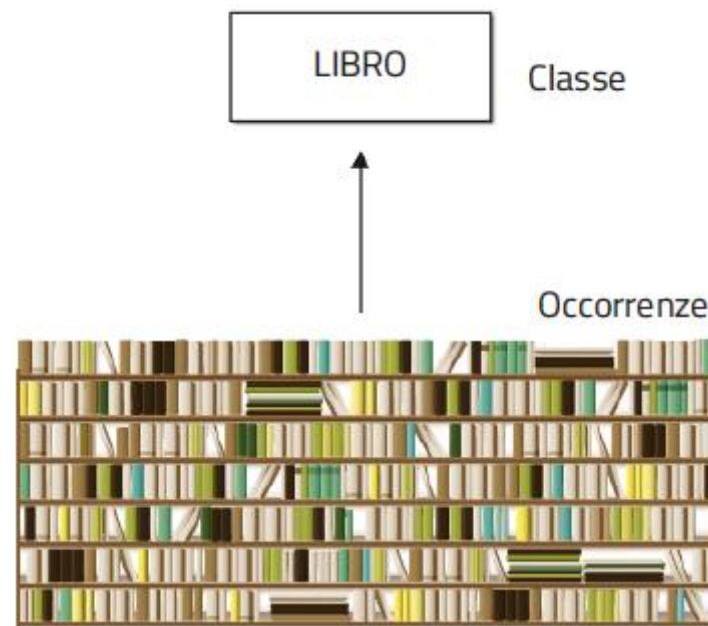
Entità e attributi

Un'entità (*Entity*) rappresenta una *classe di occorrenze (oggetti) osservabili* nella realtà.

1. Le occorrenze di un'entità hanno proprietà comuni; per esempio, nella biblioteca di Istituto di tutti i libri ci interessano i titoli.
2. Le singole occorrenze hanno una esistenza autonoma; per esempio, un libro ha una esistenza autonoma nella biblioteca rispetto a tutti gli altri.

Le entità rappresentano dunque classi di occorrenze osservabili del mondo reale, e sono quindi una *astrazione* rispetto alle caratteristiche proprie delle singole occorrenze.

Le entità si indicano con il nome al singolare
E la loro rappresentazione grafica è
un rettangolo, con il nome dell'entità
all'interno.



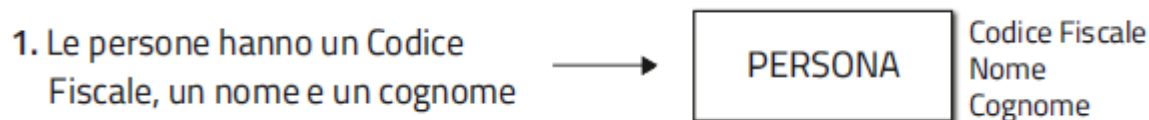
Entità e attributi

Un **attributo** di entità è una proprietà dell'entità, il cui valore per ogni specifica occorrenza dell'entità dipende solamente dall'occorrenza, e non da altri elementi dello schema.

Partendo dai requisiti, intendiamo rappresentare gli attributi delle entità, cioè le loro caratteristiche o proprietà:



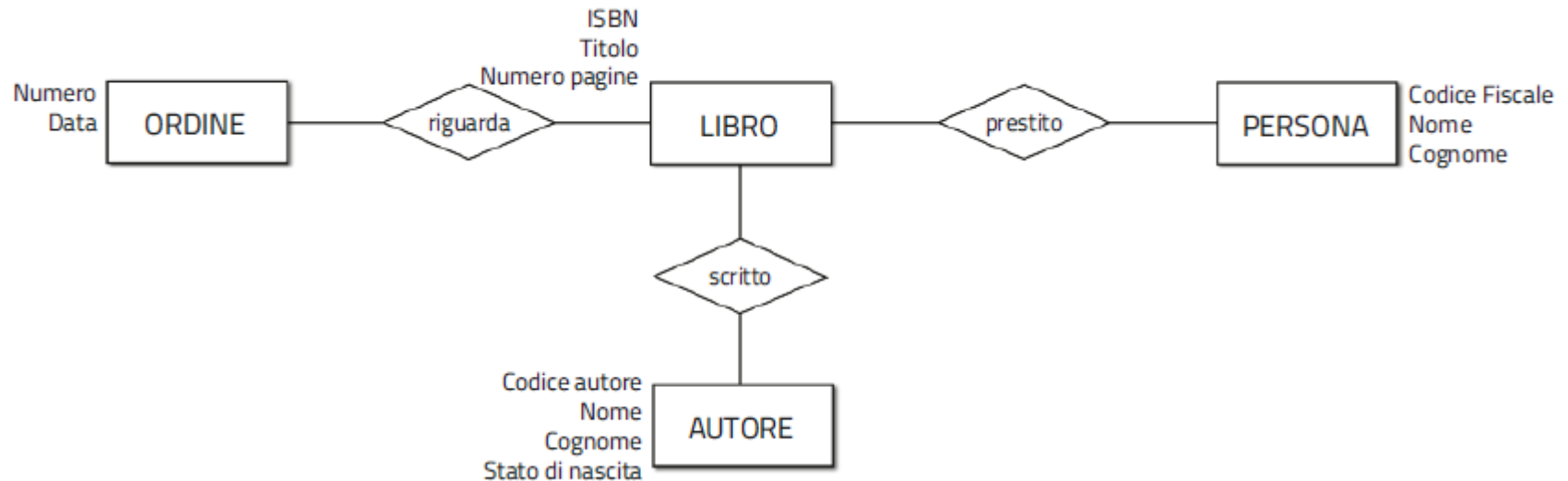
Gli attributi possono essere rappresentati semplicemente con i loro nomi accanto al rettangolo dell'entità



Associazioni e attributi di associazione

Un'**associazione** (*Relationship*) è un'azione o legame logico tra due o più entità nel mondo reale.

Si rappresenta con un rombo che collega le entità coinvolte nell'associazione:



Associazioni e attributi di associazione

Anche le associazioni possono avere degli attributi.



Certe volte non è facile distinguere se un attributo sia una caratteristica di un'entità o di un'associazione.

Per decidere è sufficiente porsi la seguente domanda: il valore del voto dipende solo dalla persona, come nel caso del Codice Fiscale? Evidentemente no, dal momento che una persona può dare voti diversi a libri diversi.

Possiamo allora chiederci se **Voto** dipende solo dal libro. Anche in questo caso la risposta è negativa, dal momento che un libro può avere diversi voti, assegnati da diverse persone; dunque **Voto** è attributo dell'associazione tra le due entità.

Attributi

Un attributo di entità o di associazione ha alcune caratteristiche principali, che sono:

- il **formato**, cioè il tipo di valori che assume, per esempio numero intero, oppure stringa di caratteri alfanumerici, oppure data ecc.;
- la **dimensione**, cioè la quantità massima di caratteri o di cifre che si possono utilizzare quando si assegna un valore all'attributo;
- la **opzionalità**, cioè la possibilità di definire se l'attributo è obbligatorio, e quindi deve avere un valore assegnato, oppure è facoltativo, e quindi può non avere un valore assegnato; in questo caso può assumere il valore *null*, corrispondente all'italiano *non lo conosco*.

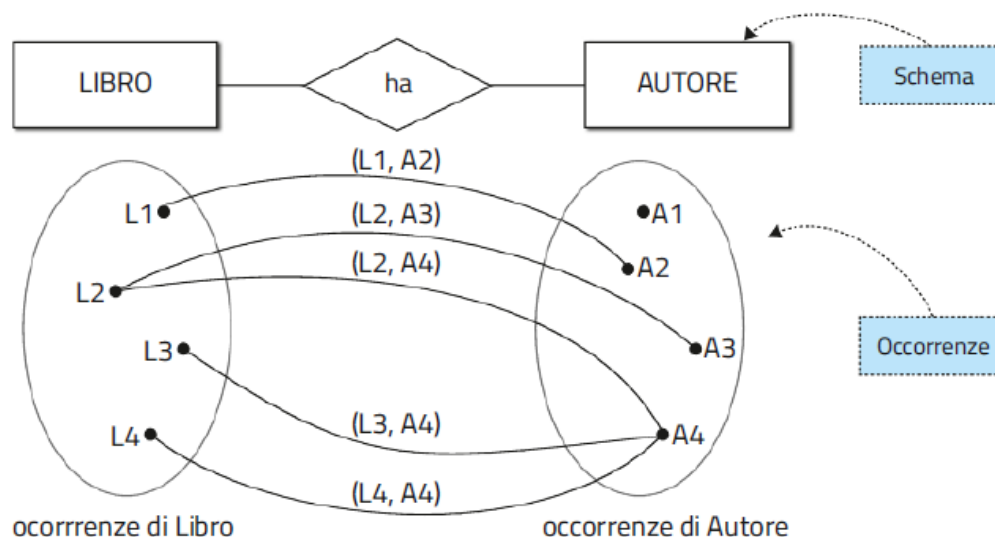
Il valore *null* rappresenta un valore a tutti gli effetti; corrisponde a una informazione mancante e non deve essere confuso con il valore zero o con la stringa vuota. Per esempio, se non conosciamo la data di nascita di uno studente, possiamo indicare il valore corrispondente con *null*.

Cardinalità

La cardinalità *di un'entità in un'associazione* serve a caratterizzare meglio la natura del legame tra l'entità e l'associazione.

Per esempio, un libro può avere molti autori e un autore può scrivere molti libri, un docente può insegnare in più classi e in una classe insegnano più docenti. Uno studente, invece, appartiene a un'unica classe; questo tipo di legame è chiamato **cardinalità**.

L'associazione che lega due entità si può rappresentare anche attraverso una rappresentazione grafica delle occorrenze delle entità collegate; in questo modo è più semplice individuare le cardinalità.



Cardinalità

La **cardinalità di un'entità E1** nell'associazione A che la collega con una seconda entità E2 è:

- **Uno**, quando a una qualunque occorrenza di E1 corrisponde nell'associazione A al massimo una occorrenza di E2;
- **Molti**, quando a una occorrenza di E1 corrispondono nell'associazione A una o più occorrenze di E2.



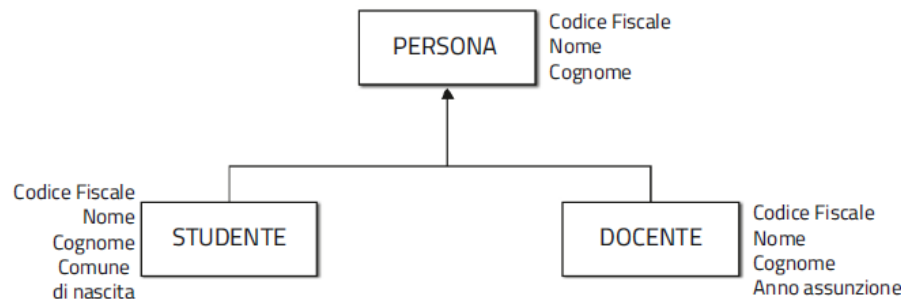
Cardinalità

Si possono suddividere le associazioni tra due entità come segue.

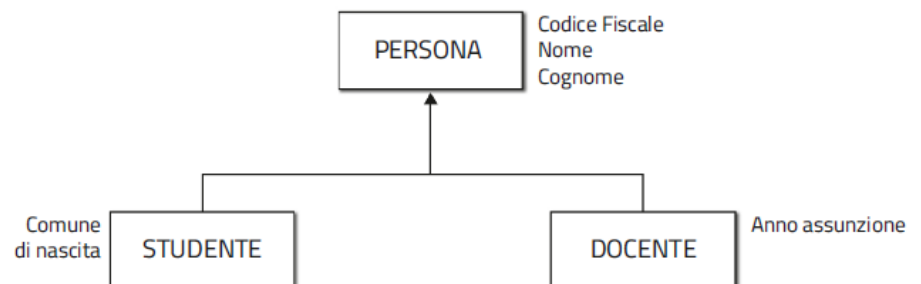
- Associazione **Molti a Molti**, indicata con Molti/Molti o M:N. Si ha quando a ogni occorrenza della prima entità E1 si possono associare in generale una o più occorrenze della seconda entità E2 e a ogni occorrenza della seconda entità E2 si possono associare una o più occorrenze della prima entità E1. Per esempio, uno studente svolge molti compiti, e un compito è svolto da molti studenti.
- Associazione **Uno a Molti**, indicata con Uno/Molti o 1:N. Si ha quando a ogni occorrenza della prima entità E1 è associata a una sola occorrenza della seconda entità E2, e a ogni occorrenza della seconda entità E2 sono associate in generale una o più occorrenze della prima entità E1. Per esempio, uno studente appartiene a una classe, mentre a una classe appartengono molti studenti.
- Associazione **Uno a Uno**, indicata con Uno/Uno o 1:1. Si ha quando a ogni occorrenza della prima entità E1 è associata al massimo una sola occorrenza della seconda entità E2 e viceversa. Per esempio, un sindaco è sindaco di un solo Comune, e un Comune ha un solo sindaco.

La generalizzazione

Tra un'entità A detta genitore e due entità B e C, dette figlie, è definita una **generalizzazione** se ogni occorrenza di B e C è anche occorrenza di A. L'entità genitore può avere più entità figlie.

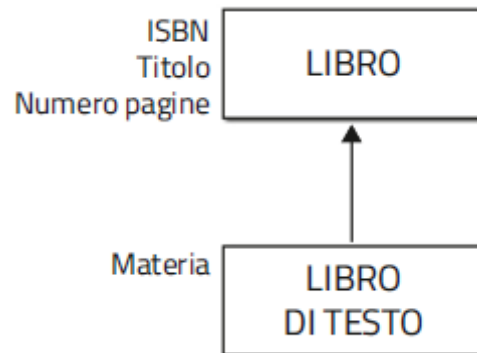


La generalizzazione gode della proprietà di **ereditarietà**.



È-un

Quando una generalizzazione è definita tra un'entità genitore e una sola entità figlia, viene chiamata con il termine di struttura *è-un*.



Identificatori

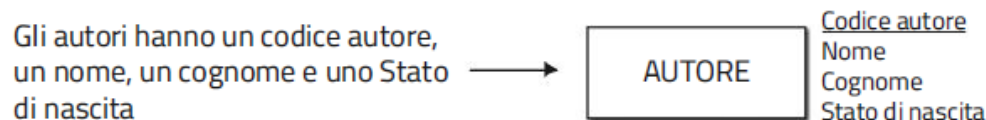
Un **identificatore** di un'entità è un insieme di uno o più attributi i cui valori permettono di identificare univocamente le occorrenze dell'entità. Nello schema del modello E/R, per rappresentare l'attributo *identificatore* questo viene sottolineato.



Si possono avere identificatori rappresentati da due attributi, oppure più identificatori:



Un'entità con un identificatore formato da due attributi



La stessa entità con un identificatore formato da un solo attributo



Un'entità con due identificatori

Definizione di modello Entity Relationship

Il **modello Entity Relationship** è un modello concettuale di dati le cui strutture di rappresentazione sono:

- l'entità,
- l'associazione,
- l'attributo,
- la generalizzazione,
- la struttura *è-un*,
- l'identificatore di entità,
- la cardinalità di entità in associazione.

Il modello logico relazionale

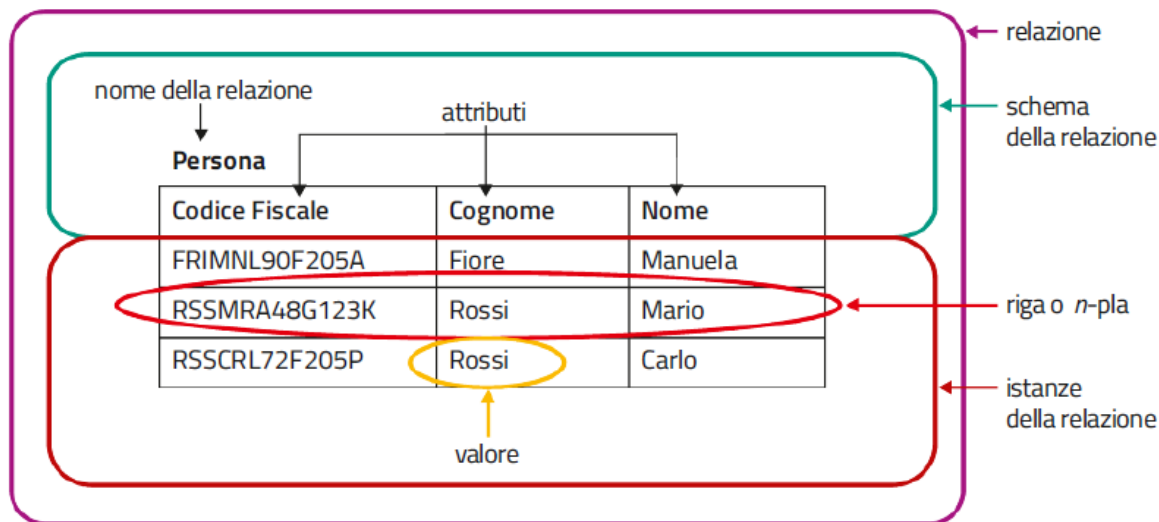
Il modello relazionale dei dati è stato proposto nel 1970 da Edgar “Ted” Codd; si basa sul concetto matematico di relazione.

Codd ideò un modello molto semplice, con due sole strutture di rappresentazione, la **relazione**, formata da righe tutte uguali, e l'**attributo**, che è una colonna della relazione, e che nelle sue celle può assumere solo valori elementari.

Nel modello relazionale l'entità è rappresentata con una *relazione o tabella*.

I concetti base associati a una relazione sono:

- schema della relazione;
- nome della relazione;
- attributo o colonna;
- istanza;
- n -upla o riga;
- valore.



Il modello logico relazionale

Spesso rappresenteremo uno schema relazionale, senza dunque l'istanza, con la seguente notazione testuale:

Persona (codice_fiscale, cognome, nome)

La rappresentazione di una base di dati mediante relazioni presenta grandi vantaggi quando stiamo cercando dei dati che ci interessano. Si usa dire che il modello relazionale è **un modello basato sui valori**, nel senso che sono i valori degli attributi l'informazione che ci permette di navigare tra le relazioni e formulare interrogazioni che coinvolgono più relazioni.



Persona

Codice Fiscale	Cognome	Nome
FRIMNL90F205A	Fiore	Manuela
RSSMRA48G123K	Rossi	Mario
RSSCRL72F205P	Rossi	Carlo

Chiavi e vincolo di integrità referenziale

Un **vincolo di integrità** è una proprietà che deve essere rispettata in tutte le istanze di una base di dati.

È chiamato **chiave** di una relazione un attributo o un insieme di attributi della relazione stessa i cui valori, in qualunque sua istanza, identificano i valori di tutti gli altri attributi nell'istanza. La chiave non cambia a seconda delle istanze della relazione, ed è una proprietà dello schema di relazione.

Le **chiavi** sono un primo esempio di vincolo di integrità inteso come proprietà che deve essere rispettata da tutte le istanze delle relazioni di una base di dati. Per tale ragione, le chiavi sono anche chiamate **vincoli di chiave**.

La chiave di una relazione si indica sottolineando l'attributo o gli attributi che la compongono.

Persona (codice_fiscale, cognome, nome, comune_nascita, regione)

Quando si hanno più chiavi, occorre sceglierne una, che viene chiamata **chiave primaria**, e che ha la proprietà di avere valori sempre diversi dal valore *null*.

Vincolo di integrità referenziale

Si dice che tra un attributo A1 di una relazione secondaria R1 e l'attributo chiave A2 di una relazione primaria R2 è definito un **vincolo di integrità referenziale** quando ogni valore dell'attributo A1 nella relazione R1 compare necessariamente come valore di A2 nella relazione R2.

Un vincolo di integrità referenziale è una proprietà di una coppia di attributi A1 e A2, o di due insiemi di attributi, appartenenti il primo a una relazione secondaria, e il secondo a una relazione primaria. Il vincolo stabilisce che se un valore dell'attributo A1 (o dell'insieme di attributi) compare nella relazione secondaria, deve anche comparire come valore di A2 nella relazione primaria.

Il **modello relazionale dei dati** è un modello che ha come strutture di rappresentazione la relazione o tabella e l'attributo, e come vincoli di integrità la chiave, la chiave primaria e il vincolo di integrità referenziale.

Definizione di concetto di relazione

Una **relazione** è caratterizzata dalle seguenti proprietà.

1. Tutte le righe di una relazione hanno lo stesso **numero di colonne**, cioè di attributi.
2. I valori di un attributo **appartengono a un dominio** di definizione. Per esempio, Regione ha come valori ammessi solo i nomi delle Regioni italiane. Quindi i valori di una colonna di una relazione sono tutti dello stesso tipo.
3. Dato che esiste la **chiave primaria** della relazione, tutte le righe sono diverse tra loro.
4. Gli attributi rappresentano **proprietà semplici** delle istanze, quindi le informazioni che rappresentano sono elementari, non sono cioè composte da insiemi di valori (come per esempio potrebbe essere un indirizzo composto da Via, Numero civico, Comune, CAP ecc.).
5. Le righe **non hanno un ordine specifico**, non sono cioè, per esempio, in ordine alfabetico sul valore di una colonna oppure in ordine di inserimento.

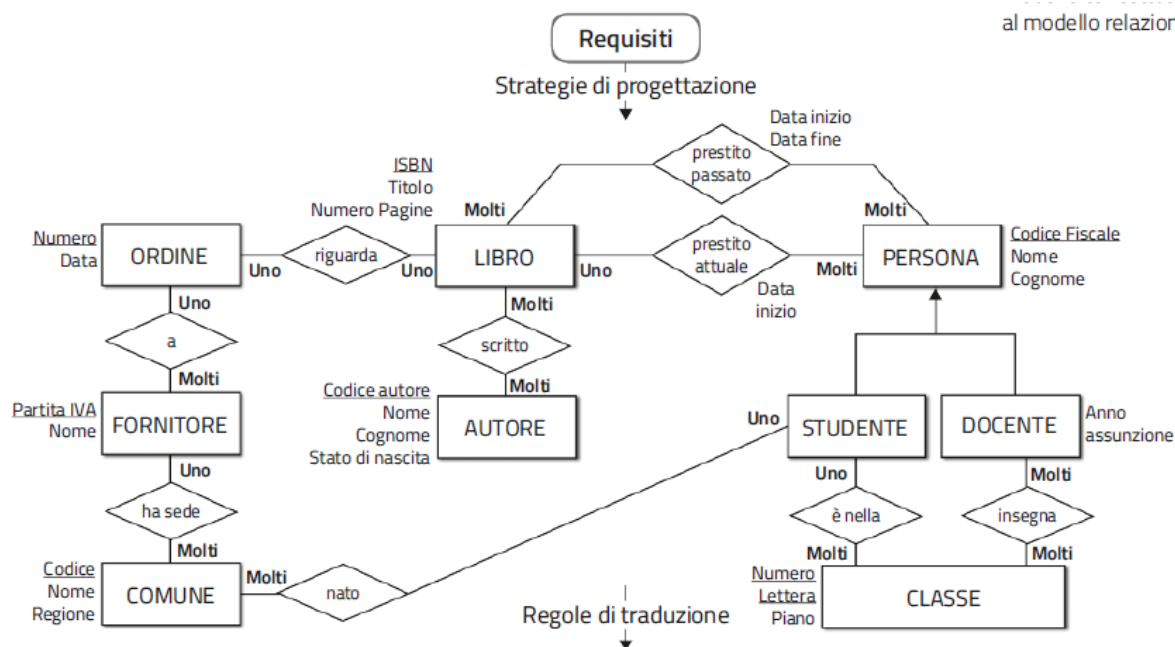
La **chiave** è la più importante proprietà nel modello relazionale.

Capitolo 2

La progettazione di basi di dati

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume
Informatica per progetti – Lavorare con le basi di dati, di C. Batini, M. Fiore, ISBN 9788808769978

Progettazione di una base dati



```

Persona (codice_fiscale, nome, cognome, ruolo, anno_assunzione, numero, lettera, codice_comune)
Insegna (codice_fiscale, numero, lettera)
Classe (numero, lettera, piano)
PrestitoPassato (codice_fiscale, isbn, data_inizio, data_fine)
Libro (isbn, titolo, codice_fiscale, data_inizio, numero, numero_pagine)
Scritto (isbn, codice_autore)
Autore (codice_autore, nome, cognome, stato_nascita)
Ordine (numero, data, partita_iva)
Fornitore (partita_iva, nome, codice_comune)
Comune (codice, nome, nome_regione)
    
```

Progettazione di una base dati

La progettazione di una base di dati va svolta in due fasi:

- una **prima fase di progettazione concettuale**
- una **seconda fase in cui lo schema concettuale** viene tradotto nello **schema relazionale corrispondente**.

Nella progettazione di basi di dati si possono fare delle scelte di progettazione che portano a produrre uno schema concettuale o uno schema logico in cui sono stati commessi degli errori.

È perciò importante che nel corso delle due fasi di progettazione concettuale e di progettazione logica siano previste verifiche di qualità degli schemi prodotti.

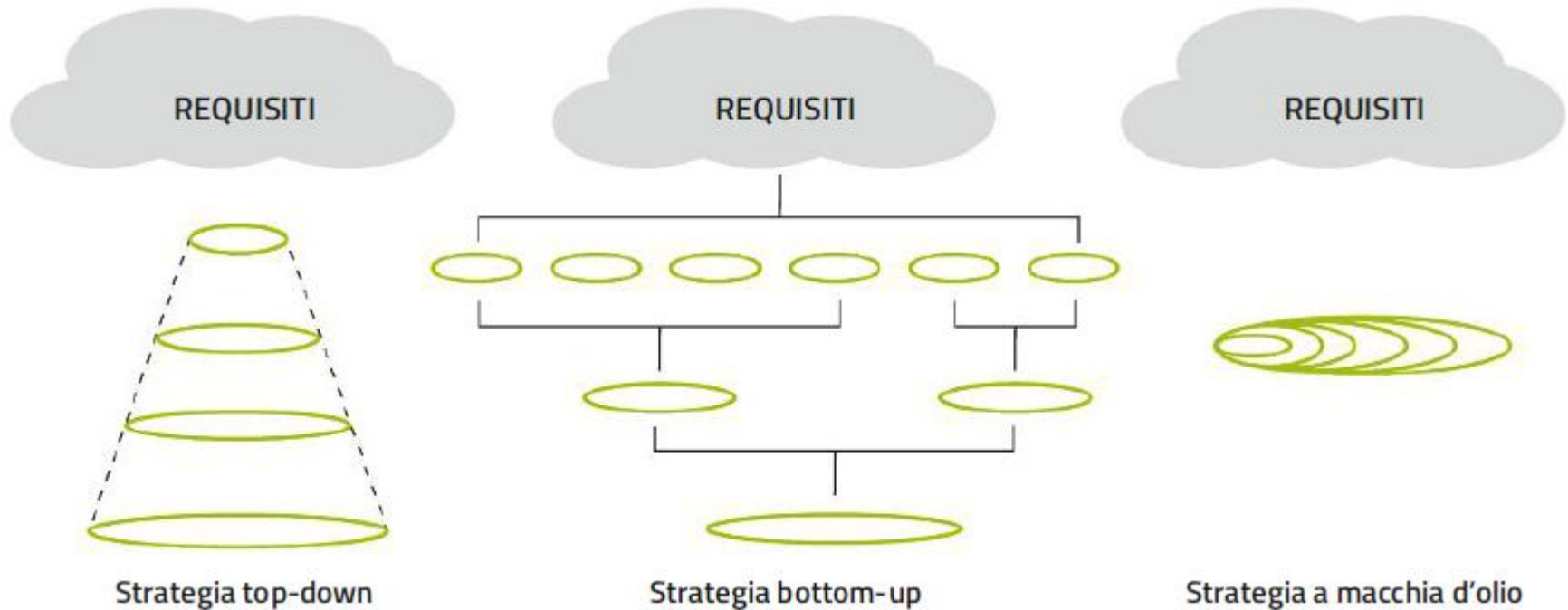
La progettazione concettuale

Quando si produce uno schema concettuale a partire da requisiti scritti in linguaggio naturale, è possibile seguire delle regole che sono chiamate strategie.

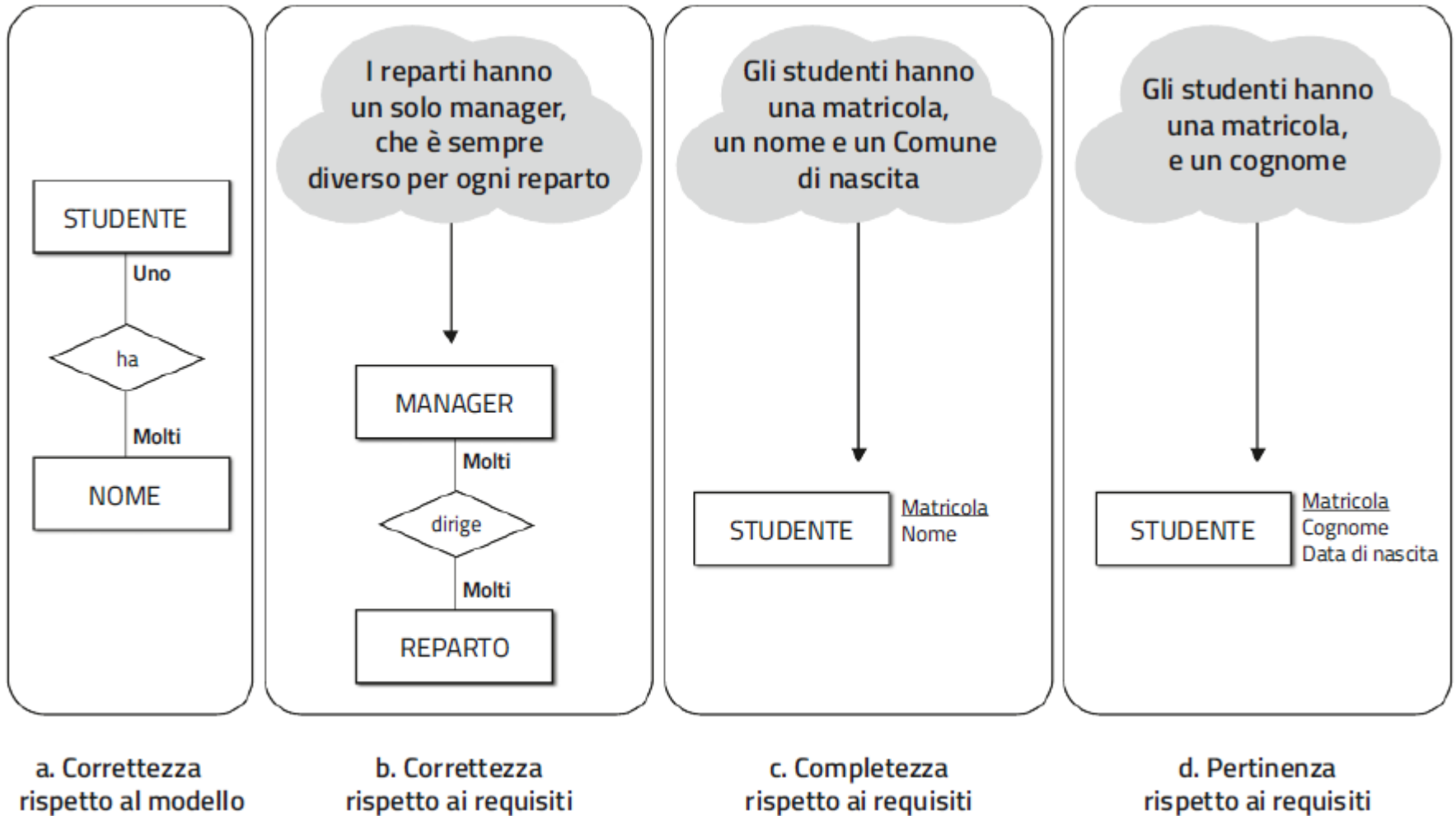
Principalmente esistono tre strategie:

- la strategia top-down, che opera dal generale al particolare;
- la strategia bottom-up, che, al contrario, opera dal particolare al generale;
- la strategia a macchia d'olio, in cui il risultato viene prodotto attraverso espansioni successive a partire da un concetto iniziale.

La progettazione concettuale



Correttezza dello schema concettuale



Correttezza dello schema concettuale

La **correttezza** (*dello schema concettuale*) **rispetto al modello** esprime il fatto che le strutture di rappresentazione utilizzate nello schema (entità, associazioni ecc.) siano coerenti con le loro definizioni.

La **correttezza rispetto ai requisiti** è la proprietà per cui tutti i requisiti sono rappresentati nello schema con strutture di rappresentazione coerenti con il loro significato nei requisiti.

La **completezza rispetto ai requisiti** misura quanti requisiti rispetto al totale sono rappresentati nello schema per mezzo di strutture del modello E/R.

La **pertinenza dello schema rispetto ai requisiti** è l'inverso della completezza e misura il numero di concetti nello schema (rispetto al totale) che non hanno una controparte nei requisiti.

Uno **schema concettuale** è **minimale** se al suo interno non esistono strutture di rappresentazione esprimibili mediante altre strutture di rappresentazione, ovvero se non è possibile eliminare strutture di rappresentazione senza alterare il contenuto informativo dello schema.

La progettazione logica

Per trasformare lo schema concettuale E/R nello schema logico relazionale occorre definire delle regole di traduzione per ciascuna delle strutture di rappresentazione del modello E/R.

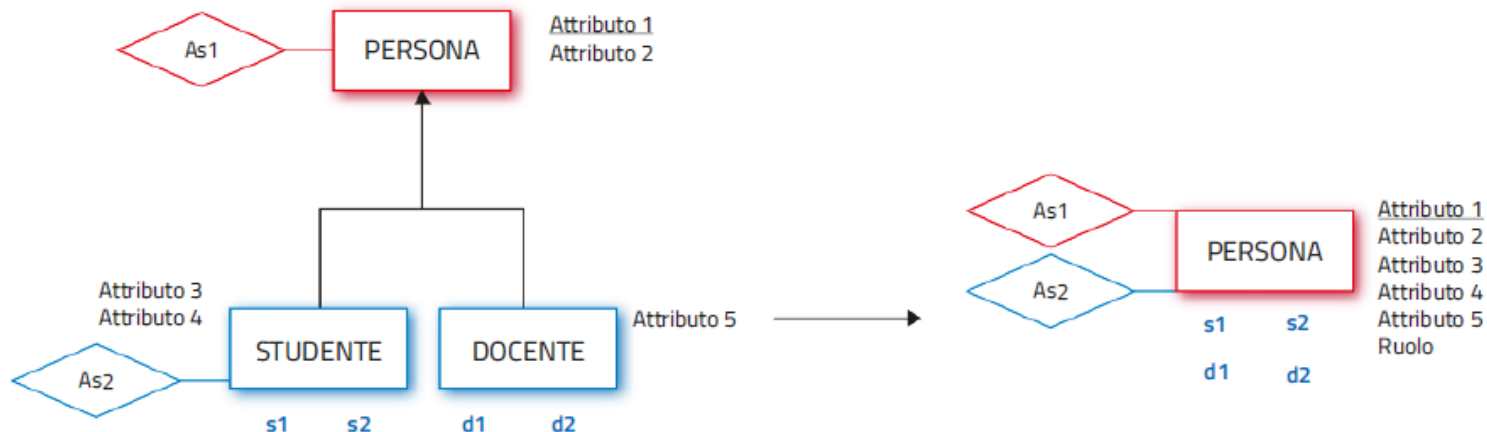
La traduzione riguarda sia le strutture di rappresentazione (ovvero lo schema), sia, di conseguenza, le occorrenze (ovvero i dati associati allo schema).

Le generalizzazioni non hanno una struttura corrispondente nello schema logico, quindi è necessario preventivamente tradurle in entità.

Regole di traduzione: generalizzazioni e è-un

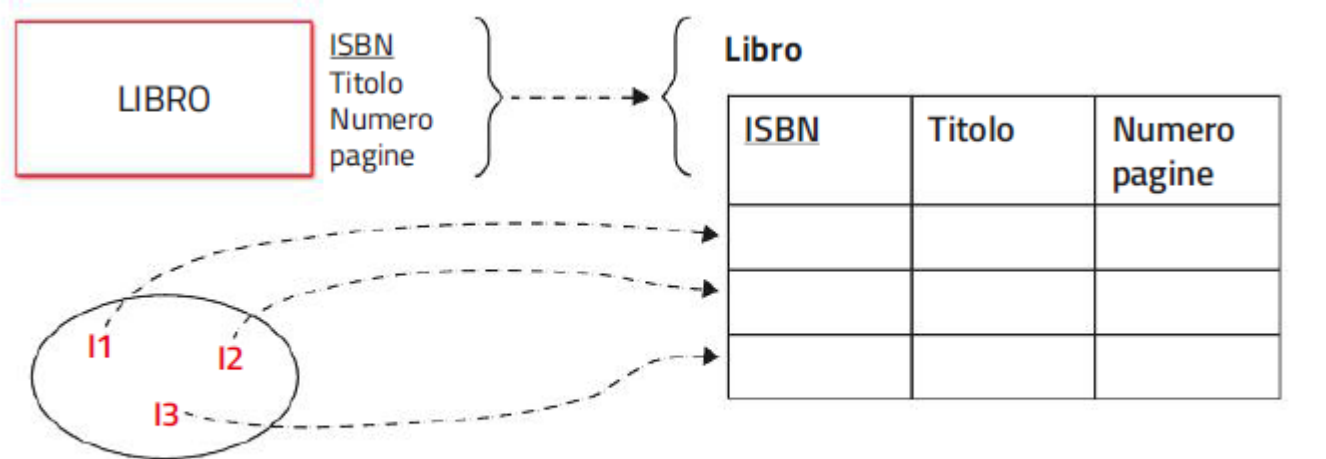
Le generalizzazioni e le strutture *è-un* non hanno una controparte nel modello relazionale, quindi vanno tradotte per prime, e, in particolare, vanno trasformate in entità del modello E/R.

Le generalizzazioni vengono tradotte rappresentando l'entità genitore *G* e le entità figlie in un'unica entità genitore *G*, a cui vengono assegnati tutti gli attributi e le associazioni delle entità figlie. Le occorrenze delle entità figlie nel loro insieme diventano poi le occorrenze dell'entità genitore. L'entità genitore ha un ulteriore attributo *Ruolo* che serve per discriminare quali delle occorrenze siano di una entità figlia o dell'altra.



Regole di traduzione: entità

Per tradurre un'entità, il nome dell'entità e i nomi dei suoi attributi vanno a costituire lo schema di relazione, mentre ogni occorrenza dell'entità viene rappresentata mediante una riga della relazione.



Libro (isbn, titolo, numero_pagine)

Notazione compatta

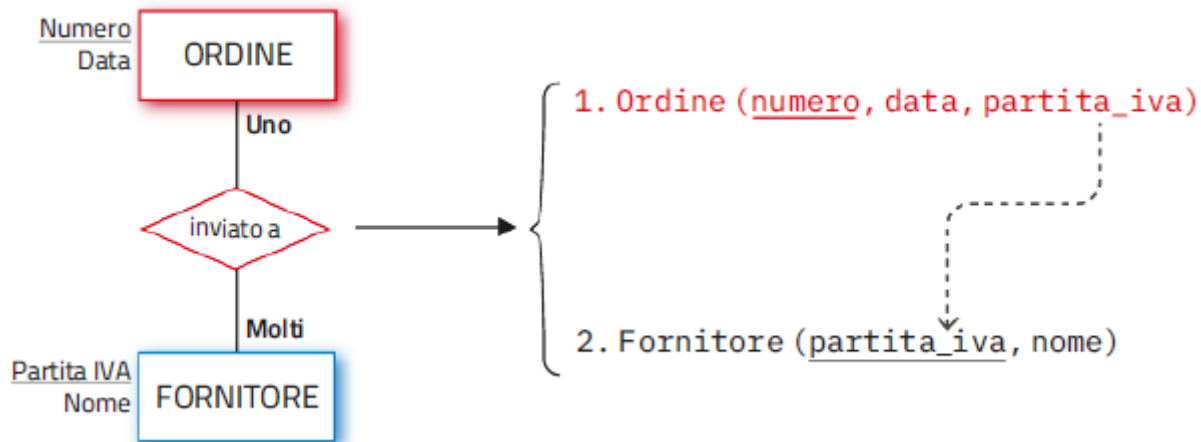
Regole di traduzione: associazione Molti a Molti

Nell'associazione Molti a Molti tra le entità Libro e Autore, a ogni occorrenza di Libro corrispondono una o più occorrenze di Autore, e viceversa. Per cui, accanto alla traduzione di ciascuna delle due entità, è necessario creare una nuova relazione, che rappresenta il legame tra autori e libri; tale legame è rappresentato includendo nella relazione le due chiavi di Autore e di Libro.



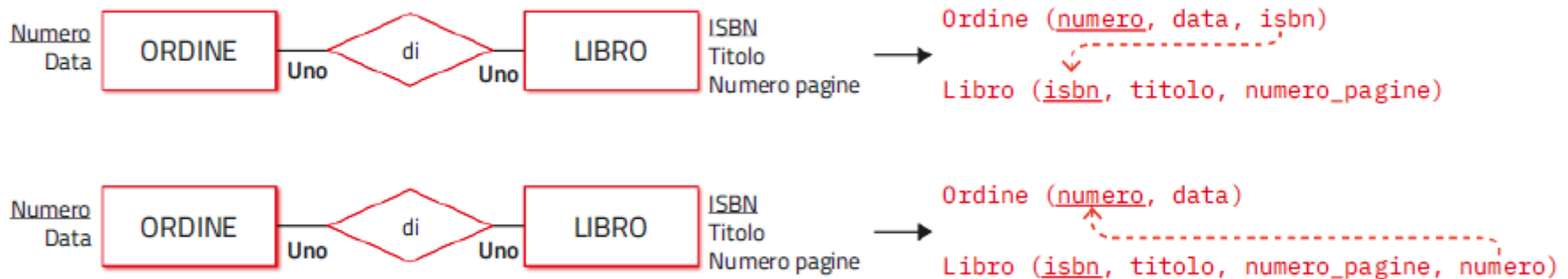
Regole di traduzione: associazione Uno a Molti

Nell'associazione Uno a Molti a ogni ordine corrisponde un fornitore. Perciò, invece di creare una terza relazione per rappresentare l'associazione Inviato a, possiamo aggiungere alla relazione Ordine la chiave della relazione Fornitore. Anche qui, occorre rappresentare il vincolo di integrità referenziale costituito dalla chiave esterna di Fornitore.



Regole di traduzione: associazione Uno a Uno

L'associazione Uno a Uno è un caso particolare dell'associazione Uno a Molti. Per esempio a ogni ordine corrisponde un libro e viceversa. Possiamo quindi rappresentare la chiave ISBN di Libro nella relazione Ordine, (in alto nella figura), oppure rappresentare la chiave di Ordine in Libro (in basso).



Regole di traduzione: riepilogo

Generalizzazione	
Struttura è-un	
Entità	
Associazione Molti a Molti	1. A (<u>KA</u>, A1) 2. B (<u>KA</u>, <u>KC</u>, B1) 3. C (<u>KC</u>, C1)
Associazione Uno a Molti	1. A (<u>KA</u>, A1, <u>KC</u>) 2. C (<u>KC</u>, C1)
Associazione Uno a Uno	1. A (<u>KA</u>, A1, <u>KC</u>) 2. C (<u>KC</u>, C1) oppure 1. A (<u>KA</u>, A1) 2. C (<u>KC</u>, C1, <u>KA</u>)

Qualità dello schema logico

Una **dipendenza funzionale** lega due attributi A e B (o due gruppi di attributi) di una relazione R ed esprime la proprietà per cui a ogni valore di A corrisponde nelle righe della relazione un solo valore di B. Se la proprietà stabilita dalla dipendenza funzionale **vale in tutte le istanze della relazione R**, allora una proprietà dello schema della relazione R è viene scritta come $A \Rightarrow B$.

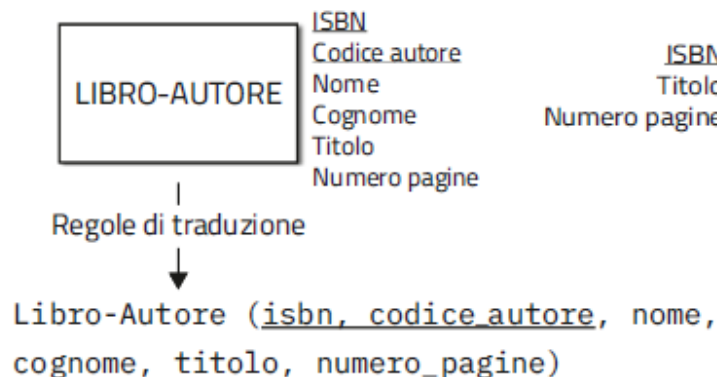
Una **dipendenza di chiave** è una dipendenza funzionale $A \Rightarrow B$ in cui A è una chiave. Una dipendenza di chiave è una proprietà di uno schema di relazione.

Forma normale di Boyce e Codd

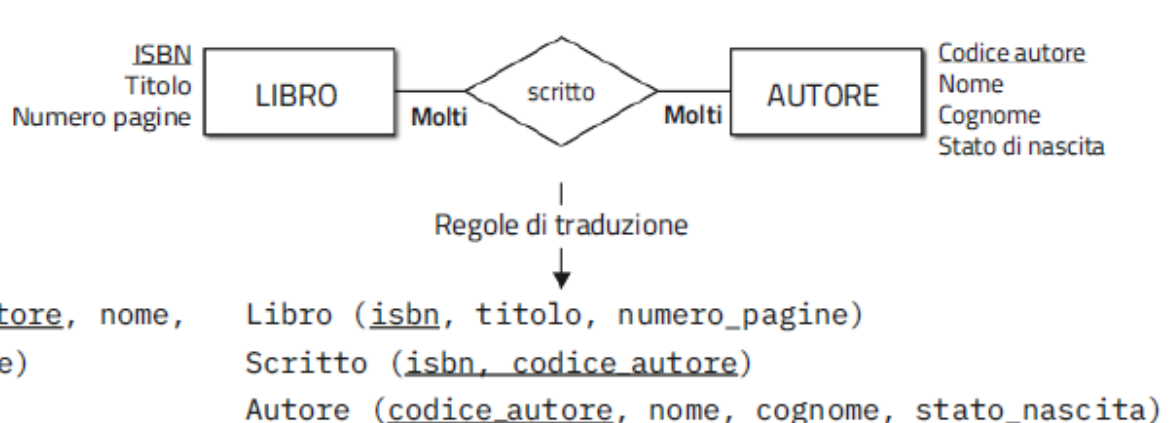
La **forma normale di Boyce e Codd** (BCNF, *Boyce Codd Normal Form*) è la proprietà di una relazione che esprime il fatto che in essa le uniche dipendenze funzionali sono dipendenze di chiave.

Si dice che una base di dati è «in forma normale di Boyce e Codd» quando tutte le sue relazioni sono in forma normale di Boyce e Codd. La proprietà di essere in BCNF garantisce che nella base di dati non vi siano ripetizioni di dati e rischi di anomalie, intese come la necessità di aggiornare un dato tutte le volte che si ripete all'interno della base di dati.

Schema relazionale NON in BNFC



Schema relazionale in BNFC



Normalizzazione

La **normalizzazione** è un procedimento che si applica a una base di dati che non rispetta la BCNF, per trasformarla, attraverso successive decomposizioni, in una base di dati in cui le dipendenze funzionali sono di chiave in tutte le relazioni; come conseguenza, la nuova base di dati che risulta dalla decomposizione rispetta la BCNF.

Data una relazione A e due relazioni B e C che sono il risultato di una decomposizione di A, diciamo che la **decomposizione è senza perdita di informazione** se è sempre possibile collegare le righe di B con le righe di C ottenendo la relazione di partenza A. La condizione affinché una decomposizione sia senza perdita di informazione è che l'insieme degli attributi comuni alle due relazioni B e C sia chiave di almeno una relazione tra B e C.

Diremo che una relazione R è in **terza forma normale (3NF)** se, per ogni dipendenza funzionale $X \Rightarrow Y$ definita su di essa, è verificata almeno una delle seguenti condizioni:

- X contiene una chiave K di R;
- ogni attributo in Y è contenuto in almeno una chiave di R.

Capitolo 3

Interrogare una base di dati

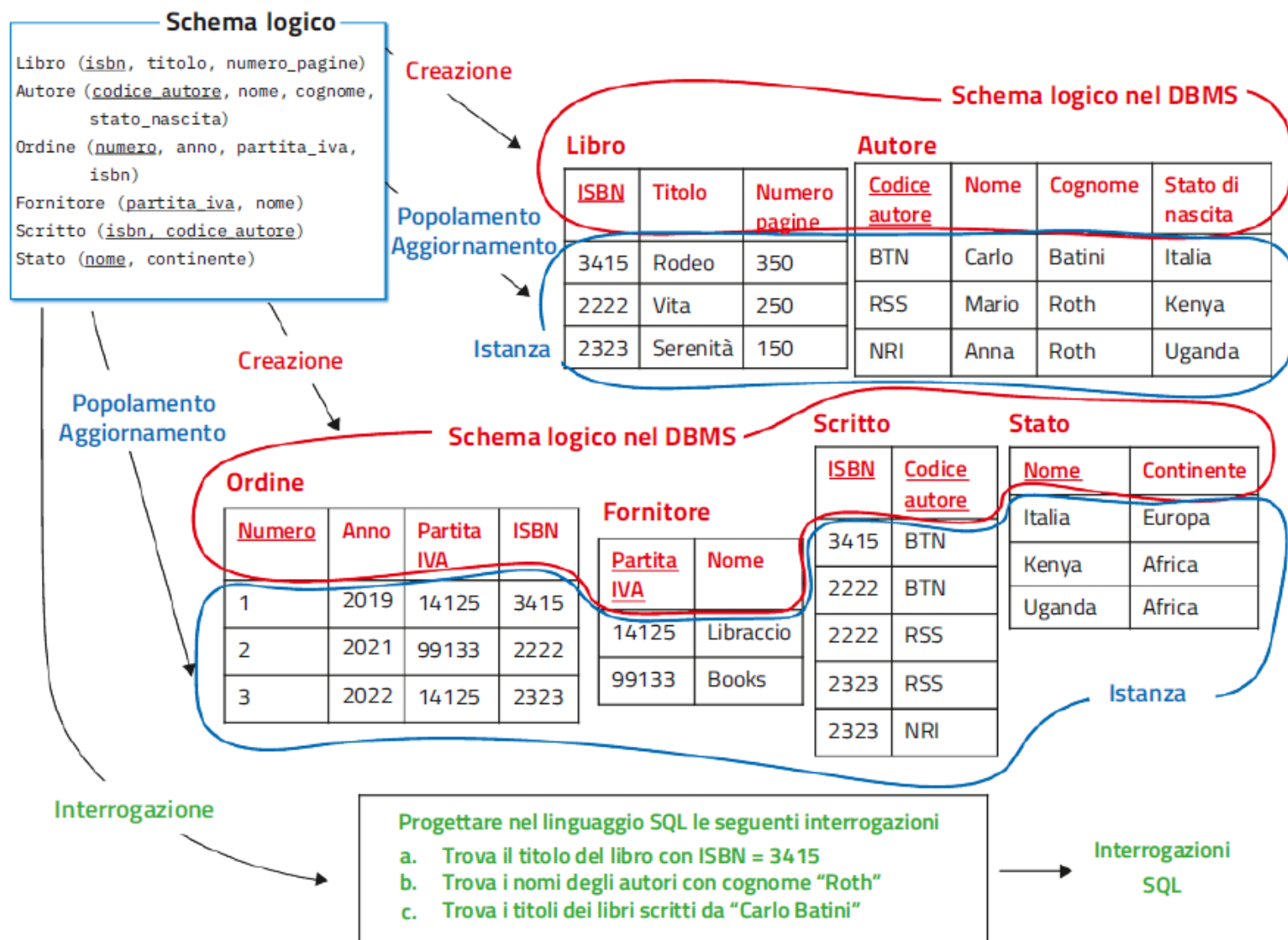
I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume
Informatica per progetti – Lavorare con le basi di dati, di C. Batini, M. Fiore, ISBN 9788808769978

II DBMS

Il sistema di gestione di basi di dati (*Database Management System*, DBMS) ci permette di eseguire sulla base di dati due tipi di operazioni, le transazioni e le interrogazioni:

- Una **transazione** è un'operazione, realizzata da un programma software ed eseguita dal DBMS, che effettua inserimenti, cancellazioni e modifiche ai dati nella base di dati, con ciò cambiando la base di dati.
- Un'**interrogazione** è un'operazione, realizzata da un programma software ed eseguita dal DBMS, che estrae dalla base di dati i dati che rispettano determinate condizioni; un'interrogazione, dunque, non cambia la base di dati.

Ciclo di vita di una base dati



Ciclo di vita di una base dati

Una volta terminata la progettazione logica, occorre procedere alle seguenti attività:

1. creazione dello schema logico della base di dati nel DBMS (in rosso);
2. popolamento iniziale dell'istanza della base di dati, che nel nostro esempio della biblioteca di Istituto è relativa ai libri, agli autori, agli ordini e ai fornitori dei libri attualmente presenti in biblioteca;
3. aggiornamento periodico della base di dati con i nuovi libri e relativi autori, i nuovi ordini e i fornitori.

Il linguaggio SQL

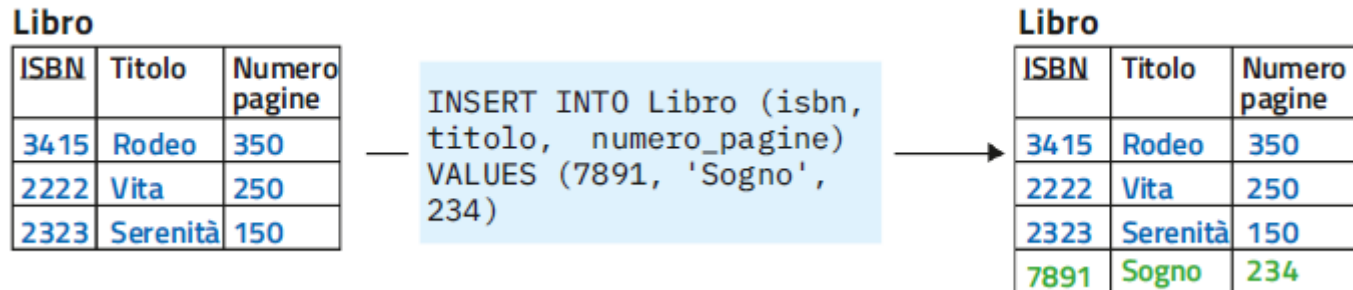
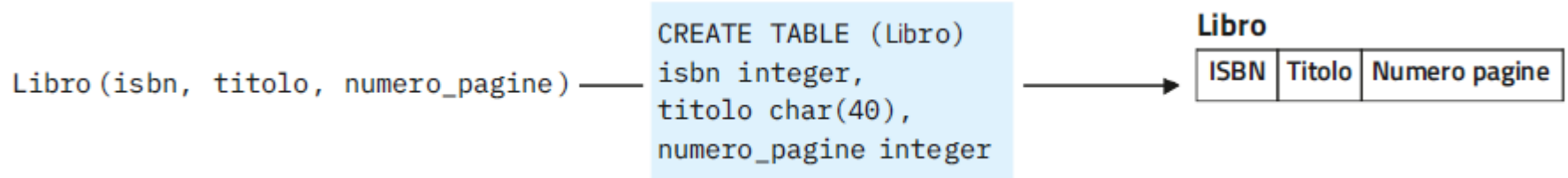
La creazione dello schema della base di dati è effettuata utilizzando il cosiddetto *Data Definition Language DDL*, iniziando con un'istruzione CREATE SCHEMA, e, a seguire, con le istruzioni CREATE TABLE per la definizione delle tabelle, dei loro attributi, delle chiavi, della chiave primaria e di tutti gli altri aspetti che abbiamo visto precedentemente.

Il popolamento iniziale e l'aggiornamento dell'istanza della base di dati sono effettuati utilizzando il *Data Manipulation Language DML* e le istruzioni INSERT, UPDATE e DELETE.

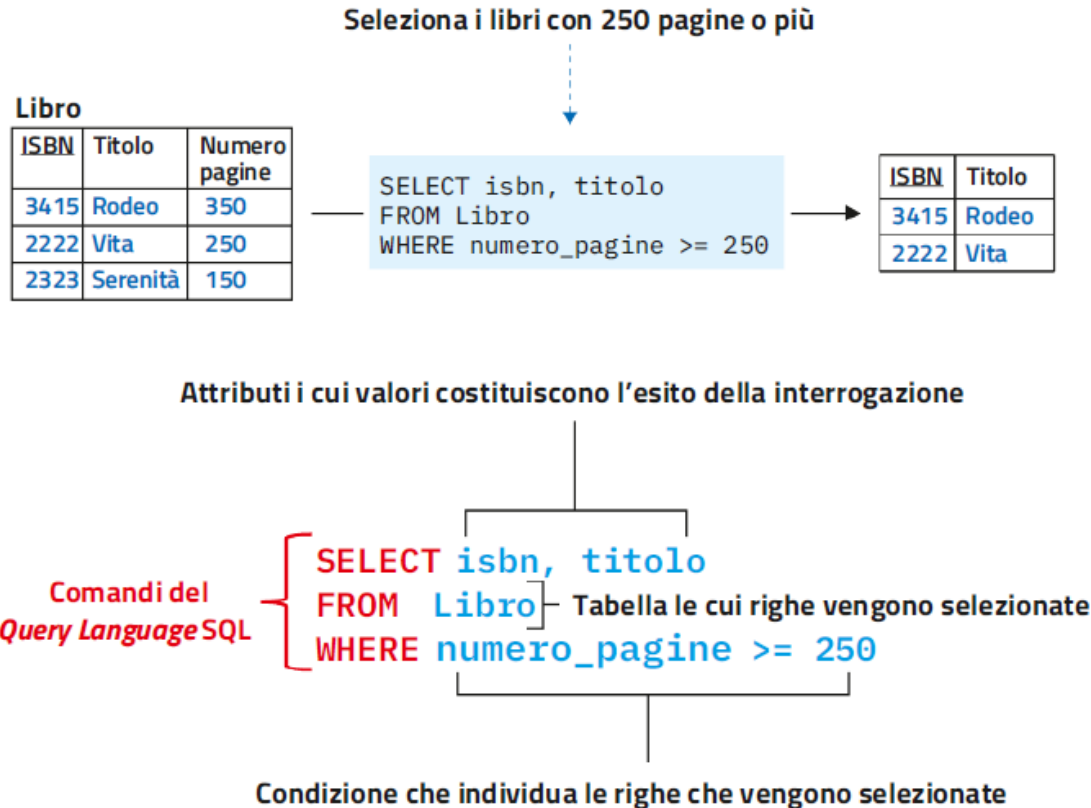
Infine, l'interrogazione della base di dati viene effettuata con un terzo linguaggio, il *Query Language QL* e le istruzioni SELECT e CREATE VIEW.

Data Definition Language, Data Manipulation Language e Query Language costituiscono nel loro insieme il linguaggio SQL, acronimo di *Structured Query Language*.

DDL e DML: esempi



QL: esempio SELECT



In particolare, nell'istruzione SELECT:

1. il comando SELECT definisce gli attributi i cui valori costituiscono il risultato dell'esecuzione dell'interrogazione (si può dire che si tratta dell'output dell'interrogazione);
2. il comando FROM individua la tabella o le tabelle su cui operare;
3. il comando WHERE esprime la condizione o le condizioni che le righe della tabella prodotte nell'output devono rispettare.

L'istruzione SELECT base

Per vedere come si interroga una base di dati, partiamo da alcune **query**. Utilizzeremo questo termine quando facciamo riferimento alla specifica dell'interrogazione tramite requisiti in linguaggio naturale. Utilizzeremo invece il termine **interrogazione** quando intendiamo una query espressa mediante un'istruzione SELECT.

La SELECT base opera su una sola tabella.

Per costruire l'interrogazione relativa a una query bisogna procedere in due passaggi:

1. dapprima individuiamo la tabella su cui opera la query;
2. successivamente esprimiamo l'interrogazione nel linguaggio SQL mediante l'istruzione SELECT, in cui riportiamo anche la tabella risultato.

Le stringhe di caratteri (per esempio "Carlo") vanno messe tra apici singoli ('Carlo').

L'istruzione SELECT base

Query – Trova lo Stato di nascita di Carlo Batini

1. Tabella su cui opera l'interrogazione

①

Libro

ISBN	Titolo	Numero di pagine
3415	Rodeo	350
2222	Vita	250
2323	Serenità	150

Autore

Codice autore	Nome	Cognome	Stato di nascita
BTN	Carlo	Batini	Italia
RSS	Mario	Roth	Kenya
NRI	Anna	Roth	Uganda

Ordine

Numero	Anno	Partita IVA	ISBN
1	2019	14125	3415
2	2021	99133	2222
3	2022	14125	2323

Fornitore

Partita IVA	Nome
14125	Libraccio
99133	Books

Scritto

ISBN	Codice autore
3415	BTN
2222	BTN
2222	RSS
2323	RSS
2323	NRI

Stato

Nome	Continente
Italia	Europa
Kenya	Africa
Uganda	Africa

2. Interrogazione in SQL

②

```
SELECT stato_nascita
FROM Autore
WHERE nome = 'Carlo' AND cognome = 'Batini'
```

3. Risultato

③

Stato di nascita
Italia

L'istruzione SELECT base

Query – Trova gli ISBN e i titoli dei libri il cui numero di pagine è compreso tra 200 e 300

①

Libro

<u>ISBN</u>	Titolo	Numero di pagine
34 15	Rodeo	350
2222	Vita	250
2323	Serenità	150

Autore

<u>Codice autore</u>	Nome	Cognome	Stato di nascita
BTN	Carlo	Batini	Italia
RSS	Mario	Roth	Kenya
NRI	Anna	Roth	Uganda

Ordine

<u>Numero</u>	Anno	Partita IVA	ISBN
1	2019	14125	34 15
2	2021	99133	2222
3	2022	14125	2323

Fornitore

<u>Partita IVA</u>	Nome
14125	Libraccio
99133	Books

Scritto

<u>ISBN</u>	<u>Codice autore</u>
34 15	BTN
2222	BTN
2222	RSS
2323	RSS
2323	NRI

Stato

<u>Nome</u>	<u>Continente</u>
Italia	Europa
Kenya	Africa
Uganda	Africa

②

```
SELECT isbn, titolo
FROM Libro
WHERE numero_pagine >= 200 AND numero_pagine <= 300
```

③

<u>ISBN</u>	Titolo
2222	Vita

L'istruzione SELECT base

Per migliorare la leggibilità del risultato dell'interrogazione possiamo modificare i nomi degli attributi dopo il comando SELECT, utilizzando il comando AS.

Query – Trova gli ISBN e i titoli dei libri il cui numero di pagine è compreso tra 200 e 300

①

Libro

ISBN	Titolo	Numero pagine
3415	Rodeo	350
2222	Vita	250
2323	Serenità	150

Autore

Codice autore	Nome	Cognome	Stato di nascita
BTN	Carlo	Batini	Italia
RSS	Mario	Roth	Kenya
NRI	Anna	Roth	Uganda

Ordine

Numero	Anno	Partita IVA	ISBN
1	2019	14125	3415
2	2021	99133	2222
3	2022	14125	2323

Fornitore

Partita IVA	Nome
14125	Libraccio
99133	Books

Scritto

ISBN	Codice autore
3415	BTN
2222	BTN
2222	RSS
2323	RSS
2323	NRI

Stato

Nome	Continente
Italia	Europa
Kenya	Africa
Uganda	Africa

②

```
SELECT isbn, titolo AS TitoloDelLibro
FROM Libro
WHERE numero_pagine >= 200 AND numero_pagine <= 300
```

③

ISBN	TitoloDelLibro
2222	Vita

Sintassi istruzione SELECT base

La **sintassi** dell'istruzione SELECT del linguaggio di interrogazione SQL è l'insieme di regole che disciplinano la corretta struttura dell'istruzione, i suoi comandi e i suoi elementi costitutivi.

```
SELECT lista di attributi della tabella separati da virgole  
FROM Tabella  
WHERE condizione
```

Al posto della lista di attributi può comparire il simbolo asterisco *, che indica l'intero insieme di attributi della tabella.

Condizioni comando WHERE

Una **condizione** è un'espressione booleana, che quindi ha valore vero o falso, ottenuta combinando con gli operatori logici OR, AND e NOT condizioni atomiche del tipo A CONFR B oppure A CONFR c, dove:

1. CONFR è un operatore di confronto: =, >, <, <=, >=, <> (in cui <> significa diverso);
2. A e B sono nomi di attributi sui cui valori abbia senso applicare l'operatore CONFR (per esempio intero = intero);
3. c è una costante a cui abbia senso applicare l'operatore CONFR (per esempio COSTO >= 5).

Esecuzione SELECT base

```
SELECT lista di attributi separati da virgola  
FROM Tabella  
WHERE condizione
```

L'esecuzione di questa istruzione comporta:

1. il calcolo del valore booleano (vero o falso) della condizione citata nella WHERE per tutte le righe della tabella citata nella FROM;
2. la selezione delle sole righe con valore vero;
3. la produzione di una tabella risultato che contiene i soli valori degli attributi nell'elenco che compare nella SELECT.

Mentre l'operatore NOT viene eseguito prima degli operatori AND e OR, la sintassi non definisce nessun ordine tra gli operatori AND e OR. Di conseguenza, se in una condizione compaiono entrambi gli operatori, se si vuole stabilire un ordine di esecuzione, esso va espresso con l'uso delle parentesi ().

Istruzione SELECT base su più tabelle con JOIN esplicito

Il **comando** JOIN collega le righe di due tabelle tramite i valori comuni di uno stesso attributo, creando in tal modo una nuova tabella.

```
SELECT lista di attributi separati da virgola
FROM Tabella1
JOIN Tabella2 ON Tabella1.attributo di Tabella1 = Tabella2.attributo di Tabella2
JOIN Tabella3 ON Tabella2.attributo di Tabella2 = Tabella3.attributo di Tabella3
JOIN ...
WHERE condizione
```

Nelle SELECT base l'interrogazione SQL può essere espressa come segue:

- **Se** le tabelle degli attributi contenenti i dati di input e gli attributi in output sono direttamente collegate attraverso vincoli di integrità referenziale,
- **allora** usiamo un unico JOIN esplicito nel comando FROM;
- **altrimenti**, usando i vincoli di integrità referenziale, cerchiamo un cammino che collega le tabelle e usiamo i JOIN espliciti che servono per esprimere nella FROM questo cammino.

Istruzione SELECT base su più tabelle con JOIN esplicito

Query – Trova il nome del fornitore dell'ordine numero 3

```
SELECT nome  
FROM Ordine JOIN Fornitore ON Ordine.partita_iva = Fornitore.partita_iva  
WHERE numero = 3
```

Ordine

Numero	Anno	Partita IVA	ISBN
1	2019	14125	3415
2	2021	99133	2222
3	2022	14125	2223

Fornitore

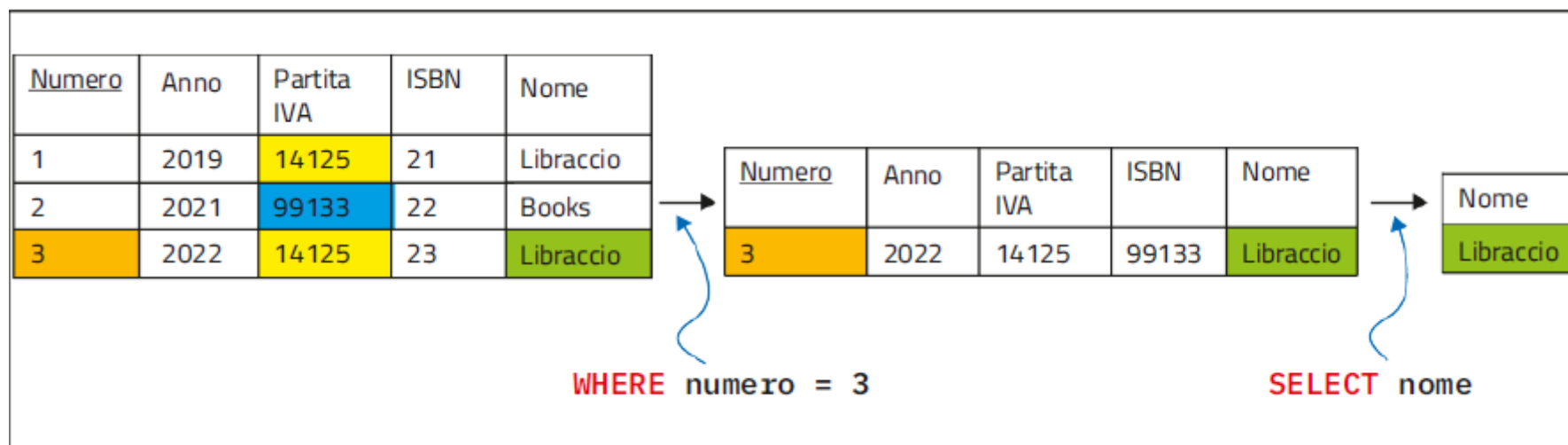
Partita IVA	Nome
14125	Libraccio
99133	Books

Numero	Anno	Partita IVA	Nome	ISBN
1	2019	14125	Libraccio	3415
2	2021	99133	Books	2222
3	2022	14125	Libraccio	2223

per eseguire l'interrogazione dobbiamo utilizzare:

1. la tabella Ordine, in cui si trova il numero d'ordine in input all'interrogazione;
 2. la tabella Fornitore, in cui troviamo l'attributo nome, il cui valore va prodotto in output.
- Inoltre, per trovare il nome del fornitore dobbiamo prima collegare le due tabelle tramite l'attributo comune partita_iva, evidenziato in blu

Istruzione SELECT base su più tabelle con JOIN esplicito



Una volta collegate le righe delle due tabelle, nella tabella risultante troviamo sia il numero dell'ordine sia il nome del fornitore.

Per mezzo del comando WHERE, possiamo quindi esprimere la condizione di selezione richiesta dalla **Query**.

Istruzione SELECT base su più tabelle con JOIN implicito

Accanto al JOIN esplicito, vediamo anche la struttura sintattica del JOIN implicito.

Nel JOIN implicito, la condizione di JOIN non viene introdotta nel comando FROM, ma, piuttosto, viene espressa nel comando WHERE, come condizione di uguaglianza tra valori di due attributi delle tabelle coinvolte.

Join esplicito:

```
SELECT lista di attributi separati da virgola
FROM Tabella1 JOIN Tabella2 ON Tabella1.attributo di Tabella1 = Tabella2.attributo di Tabella2
WHERE condizione
```

La precedente istruzione è equivalente alla seguente, in cui il JOIN è implicito perché espresso all'interno del comando WHERE:

```
SELECT lista di attributi separati da virgola
FROM Tabella1, Tabella2
WHERE condizione AND Tabella1.attributo di Tabella1 = Tabella2.attributo di Tabella2
```

Istruzione SELECT base su più tabelle

Query – Trova tutti i continenti di nascita degli autori dei libri

Libro

ISBN	Titolo	Numero pagine
3415	Rodeo	350
2222	Vita	250
2323	Serenità	150

Autore

Codice autore	Nome	Cognome	Stato di nascita
BTN	Carlo	Batini	Italia
RSS	Mario	Roth	Kenya
NRI	Anna	Roth	Uganda

Ordine

Numero	Anno	Partita IVA	ISBN
1	2019	14125	3415
2	2021	99133	2222
3	2022	14125	2323

Fornitore

Partita IVA	Nome
14125	Libraccio
99133	Books

Scritto

ISBN	Codice autore
3415	BTN
2222	BTN
2222	RSS
2323	RSS
2323	NRI

Stato

Nome	Continente
Italia	Europa
Kenya	Africa
Uganda	Africa

JOIN esplicito →

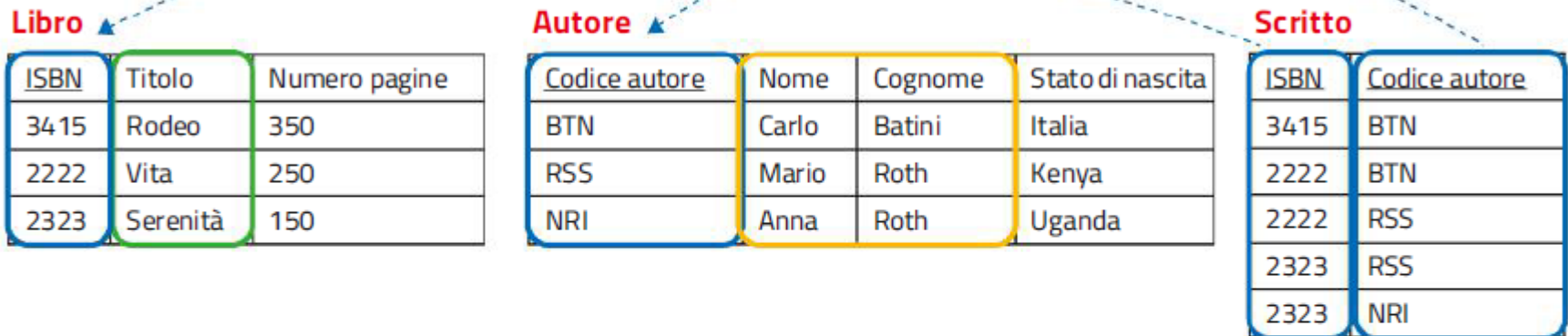
```
SELECT continente
FROM Autore JOIN Stato ON Autore.stato_nascita = Stato.nome
```

JOIN implicito →

```
SELECT continente
FROM Autore, Stato
WHERE Autore.stato_nascita = Stato.nome
```

Istruzione SELECT base su più tabelle

Query – Trova i titoli dei libri di cui è autore Carlo Batini – Passo 3



Fornitore

Partita IVA	Nome
14125	Libraccio
99133	Books

Ordine

Numero	Anno	Partita IVA	ISBN
1	2019	14125	3415
2	2021	99133	2222
3	2022	14125	2323

Stato

Nome	Continente
Italia	Europa
Kenya	Africa
Uganda	Africa

```
SELECT Libro.titolo
FROM Autore JOIN Scritto ON Autore.codice_autore = Scritto.codice_autore JOIN Libro
ON Libro.isbn = Scritto.isbn
WHERE Autore.nome = 'Carlo' AND Autore.cognome = 'Batini'
```

SELECT con aggregazioni: DISTINCT

L'istruzione SELECT produce in output tabelle in cui può accadere che alcuni valori compaiano più volte; per evitarlo, bisogna usare il comando DISTINCT.

Stato

Nome	Continente
Italia	Europa
Kenya	Africa
Uganda	Africa

SELECT continente
FROM Stato

Continente
Europa
Africa
Africa

Stato

Nome	Continente
Italia	Europa
Kenya	Africa
Uganda	Africa

SELECT DISTINCT continente
FROM Stato

Continente
Europa
Africa

SELECT con aggregazioni: count

L'operatore count, può essere visto come una funzione che, applicata ai valori di un attributo, genera un singolo valore; perciò viene chiamato operatore di aggregazione.

L'operatore count, conta il numero dei continenti prodotti in output dalla SELECT base. Se vogliamo contare non il numero di valori, ma il numero di valori distinti, allora dobbiamo usare anche il comando DISTINCT.

Stato

Nome	Continente
Italia	Europa
Kenya	Africa
Uganda	Africa

```
SELECT count(continente)  
AS NumeroContinenti  
FROM Stato
```

NumeroContinenti
3

Stato

Nome	Continente
Italia	Europa
Kenya	Africa
Uganda	Africa

```
SELECT count(DISTINCT continente)  
AS NumeroContinenti  
FROM Stato
```

NumeroContinenti
2

Operatori di aggregazione

Gli operatori di aggregazione entrano in gioco solo dopo che è stata eseguita la SELECT base; i valori NULL sono ignorati dagli operatori di aggregazione.

```
SELECT lista di attributi separati da virgola con operatori di aggregazione
FROM ...
WHERE ...
```

L'esecuzione di questa istruzione comporta:

1. prima di tutto, l'esecuzione dell'istruzione senza considerare gli operatori di aggregazione, ottenendo una tabella in output;
2. successivamente, il calcolo dei valori degli operatori di aggregazione applicati ai corrispondenti attributi della tabella in output.

Operatore	Calcola
count	il numero di valori dell'attributo nella tabella risultato
sum	la somma dei valori dell'attributo nella tabella risultato
max	il valore massimo dei valori dell'attributo nella tabella risultato
min	il valore minimo dei valori dell'attributo nella tabella risultato
avg	il valore medio dei valori dell'attributo nella tabella risultato

SELECT con raggruppamenti: GROUP BY

Query – Per ciascun libro in biblioteca, trova il numero di autori

```
SELECT isbn, count(codice_autore) AS NumeroAutori  
FROM Scritto  
GROUP BY isbn
```

Passo 1

```
SELECT isbn, codice_autore  
FROM Scritto
```

Scritto

ISBN	Codice autore
3415	BTN
2222	BTN
2222	RSS
2323	RSS
2323	NRI

Passo 2

```
SELECT isbn, codice_autore  
FROM Scritto  
GROUP BY isbn
```

ISBN	Codice autore
3415	BTN
2222	BTN
2222	RSS
2323	RSS
2323	NRI

Passo 3

```
SELECT isbn, count(codice_autore)  
AS NumeroAutori  
FROM Scritto  
GROUP BY isbn
```

ISBN	Codice autore
3415	BTN
2222	BTN
2222	RSS
2323	RSS
2323	NRI

ISBN	NumeroAutori
3415	1
2222	2
2323	2

SELECT con raggruppamenti: GROUP BY

La sintassi della SELECT con il comando GROUP BY e i tre passi della sua esecuzione prevede che l'operatore di aggregazione possa essere definito su più attributi e, in generale, al posto di attributo1, può esserci invece una lista di attributi, sia nella SELECT sia nella GROUP BY. In questo caso, le due liste devono coincidere.

```
SELECT attributo1, operatore di aggregazione  
FROM Tabelle collegate da JOIN  
WHERE condizione  
GROUP BY attributo1
```

L'esecuzione dell'istruzione comporta:

1. l'esecuzione dell'istruzione SELECT senza considerare gli operatori di aggregazione e di GROUP BY, ottenendo una tabella T1 in output (passo 1);
2. la creazione di una tabella T2 in cui a ogni valore "v" di attributo1 o, in generale, della lista di attributi sono associati i gruppi di valori dei restanti attributi (passo 2);
3. l'esecuzione del comando SELECT e il calcolo dell'operatore di aggregazione (passo 3).

SELECT con raggruppamenti: HAVING

Per mezzo del comando WHERE abbiamo potuto esprimere condizioni su singole righe delle tabelle prodotte dalla FROM; il comando GROUP BY, però, produce gruppi di valori per ogni valore “v” dell’Attributo1. Per esprimere condizioni su tali gruppi di valori, è possibile usare il comando HAVING.

Query – Per ciascun autore di almeno due libri,
trova il numero medio di pagine dei libri in biblioteca di cui è autore

Autore

Codice autore	Nome	Cognome	Stato di nascita
---------------	------	---------	------------------

Scritto

ISBN	Codice autore
------	---------------

Libro

ISBN	Titolo	Numero pagine
------	--------	---------------

```
SELECT Autore.codice_autore, avg(numero_pagine)
FROM Autore JOIN Scritto ON Autore.codice_autore =
Scritto.codice_autore JOIN Libro ON Scritto.isbn =
Libro.isbn
GROUP BY codice_autore
```

```
SELECT Autore.codice_autore, avg(numero_pagine)
FROM Autore JOIN Scritto ON Autore.codice_autore =
Scritto.codice_autore JOIN Libro ON Scritto.isbn =
Libro.isbn
GROUP BY codice_autore
HAVING count(isbn) >= 2
```

SELECT con raggruppamenti: HAVING

La SELECT senza il comando HAVING crea una tabella le cui righe sono raggruppate per codice autore. Il comando HAVING seleziona le sole righe della tabella in cui il numero di ISBN è maggiore o uguale a 2, e solo per queste viene calcolato il numero medio di pagine.

```
SELECT attributo1, operatore di aggregazione  
FROM Tabelle collegate da JOIN  
WHERE condizione  
GROUP BY attributo1  
HAVING condizione
```

L'esecuzione dell'istruzione comporta:

1. l'esecuzione dell'istruzione SELECT senza considerare gli operatori di aggregazione e di GROUP BY, ottenendo una tabella T1 in output (passo 1);
2. la creazione di una tabella T2, in cui a ogni valore "v" di Attributo1 sono associati i gruppi di valori dei restanti attributi (passo 2);
3. la selezione nella tabella T2 dei soli gruppi di valori creati nel passo 2 che rispettano la condizione nel comando HAVING (passo 3);
4. l'esecuzione del comando SELECT e il calcolo dell'operatore di aggregazione (passo 4).

SELECT con ordinamenti: ORDER BY

Nell'istruzione SELECT è possibile eseguire un ordinamento, grazie al comando ORDER BY.

Il comando ORDER BY viene eseguito alla fine dell'esecuzione della SELECT base e dei comandi GROUP BY e HAVING. Nella ORDER BY è possibile citare uno o più attributi secondo cui esprimere l'ordinamento, ed è possibile indicare se l'ordinamento debba essere ascendente o discendente, con i comandi ASC e DESC.

Libro

ISBN	Titolo	Numero pagine
3415	Rodeo	250
2222	Vita	350
2323	Serenità	150

```
SELECT *  
FROM Libro  
ORDER BY numero_pagine ASC
```

ISBN	Titolo	Numero pagine
2323	Serenità	150
3415	Rodeo	250
2222	Vita	350

Libro

ISBN	Titolo	Numero pagine
3415	Rodeo	250
2222	Vita	350
2323	Serenità	150

```
SELECT *  
FROM Libro  
ORDER BY numero_pagine DESC
```

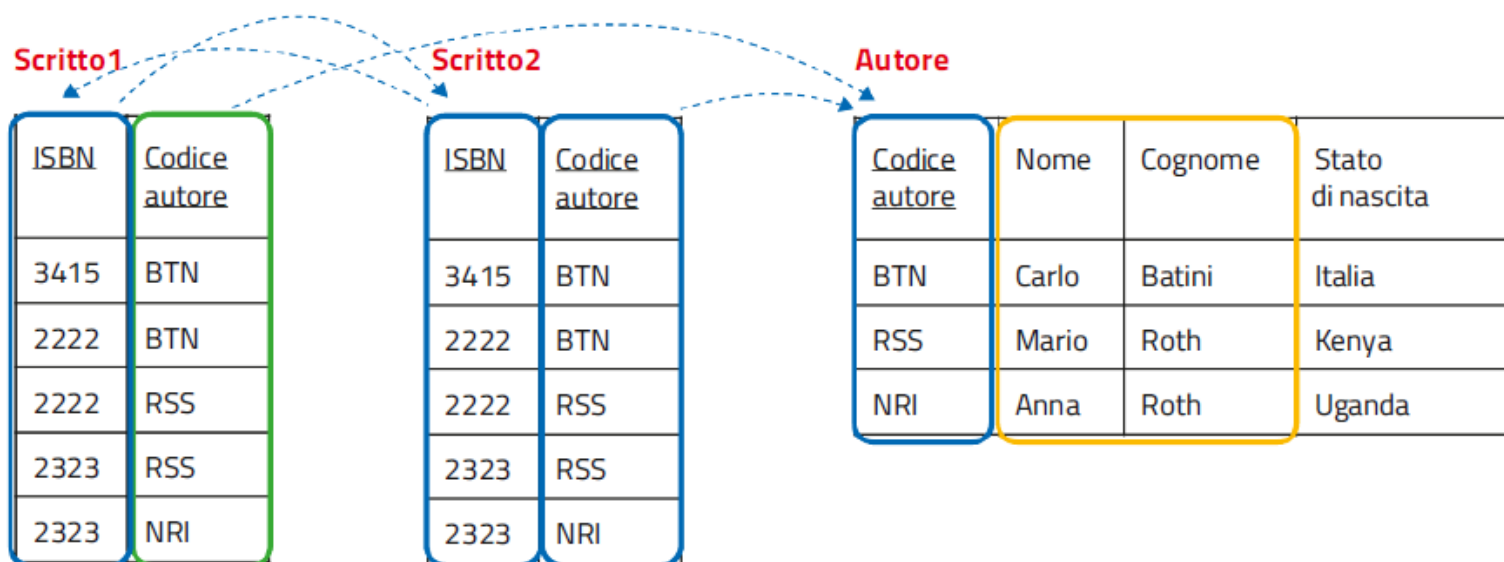
ISBN	Titolo	Numero pagine
2222	Vita	350
3415	Rodeo	250
2323	Serenità	150

Ordine di esecuzione SELECT

1. comando FROM della SELECT base
2. comando WHERE della SELECT base
3. comando GROUP BY
4. comando HAVING
5. comando SELECT e eventuali operatori citati nel comando
6. comando ORDER BY

Istruzione CREATE VIEW

Query – Trova gli Stati di nascita degli autori che hanno un libro in comune con Carlo Batini
(Carlo Batini compreso)



```
SELECT DISTINCT stato_nascita
FROM Autore JOIN Scritto AS Scritto1 ON Scritto1.codice_autore = Autore.codice_autore
      JOIN Scritto AS Scritto2 ON Scritto1.isbn = Scritto2.isbn
WHERE Autore.nome = 'Carlo' AND Autore.cognome = 'Batini'
```

CREATE VIEW

L'istruzione CREATE VIEW ci permette di definire un'interrogazione che crea una nuova tabella, a cui è associato un nome che può essere citato in una successiva SELECT.

Con l'istruzione CREATE VIEW possiamo, con una strategia di tipo divide et impera, creare tabelle che corrispondono a passi intermedi di una SELECT, semplificando i passi successivi.

```
CREATE VIEW NomeDiTabella AS istruzione SELECT
```

Query – Trova gli Stati di nascita degli autori che hanno un libro in comune con Carlo Batini
(Carlo Batini compreso)

Autore

Codice autore	Nome	Cognome	Stato di nascita
---------------	------	---------	------------------

Scritto

ISBN	Codice autore
------	---------------

1. Trova gli ISBN dei libri con autore Carlo Batini e chiama questa tabella ISBNLibriBatini

```
CREATE VIEW ISBNLibriBatini AS
SELECT isbn
FROM Autore JOIN Scritto ON Autore.codice_autore = Scritto.codice_autore
WHERE Autore.nome = 'Carlo' AND Autore.cognome = 'Batini'
```

2. Trova gli Stati di nascita degli autori di libri che hanno ISBN in ISBNLibriBatini

```
SELECT DISTINCT stato_nascita
FROM ISBNLibriBatini JOIN Scritto ON ISBNLibriBatini.isbn = Scritto.isbn
```


Comandi SQL

Comando	Che cosa accade quando viene eseguito
SELECT	Indica gli attributi da produrre in output; gli attributi possono essere specificati da nomi, espressioni aritmetiche e operatori di aggregazione.
AS	Assegna nuovi nomi più espressivi ad attributi e tabelle.
DISTINCT	Applicato a un attributo nella SELECT, specifica che gli eventuali duplicati vengono cancellati dall'output.
FROM	Indica la tabella o le tabelle che sono utilizzate nel calcolo della tabella in output. Nella FROM possono anche comparire comandi di tipo JOIN ON (o JOIN esplicito).
JOIN ON	Crea una tabella in cui le righe di due tabelle sono collegate sulla base di uguali valori di loro attributi.
condizione	Espressione booleana formata da condizioni atomiche collegate da operatori logici AND, OR o NOT, eventualmente raggruppate con parentesi per indicare l'ordine di valutazione.
WHERE	Specifica la condizione che devono rispettare le righe della tabella in output.
count, min, max, avg, sum	Operatori di aggregazione che, applicati a un insieme di valori, producono un unico valore.
GROUP BY	Produce gruppi di valori che, nella tabella calcolata dalla SELECT base, sono associati a uno stesso valore degli attributi specificati nella GROUP BY.
HAVING	Esprime una condizione sui gruppi di valori prodotti dalla GROUP BY.
ORDER BY	Ordina i valori della tabella prodotta in output secondo i valori degli attributi specificati nella ORDER BY.

Qualità interrogazioni SQL

- Un'interrogazione SQL che esprime una query Q, eseguita su una base di dati B, è **sintatticamente corretta** se i diversi comandi di cui è composta rispettano la sintassi del *Query Language* adottato dal DBMS.
- Un'interrogazione SQL che esprime una query Q, eseguita su una base di dati B, è **corretta (in esecuzione)** se per tutte le possibili istanze della base dati B il risultato in output è coerente con quanto espresso dalla query Q.
- Un'interrogazione SQL che esprime una query Q, eseguita su una base di dati B, è **leggibile** se le elaborazioni che effettua sono immediatamente comprensibili dal testo dell'interrogazione, senza ulteriore necessità di spiegazioni.
- Un'interrogazione SQL che esprime una query Q, eseguita su una base di dati B, è **efficiente** se la sua esecuzione con un DBMS richiede un limitato impegno di tempo di calcolo (efficienza temporale) e di memoria occupata (efficienza nell'occupazione di spazio).

Capitolo 4

II DBMS

PostgreSQL

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume *Informatica per progetti – Lavorare con le basi di dati*, di C. Batini, M. Fiore, ISBN 9788808769978

DBMS PostgreSQL

Il Database Management System (DBMS) **PostgreSQL** è un sistema di gestione di basi di dati relazionale molto utilizzato in tutto il mondo. È conosciuto e apprezzato, soprattutto in ambiente professionale, per la sua affidabilità, scalabilità, robustezza e conformità agli standard ANSI SQL, cioè gli standard ufficiali del linguaggio SQL.

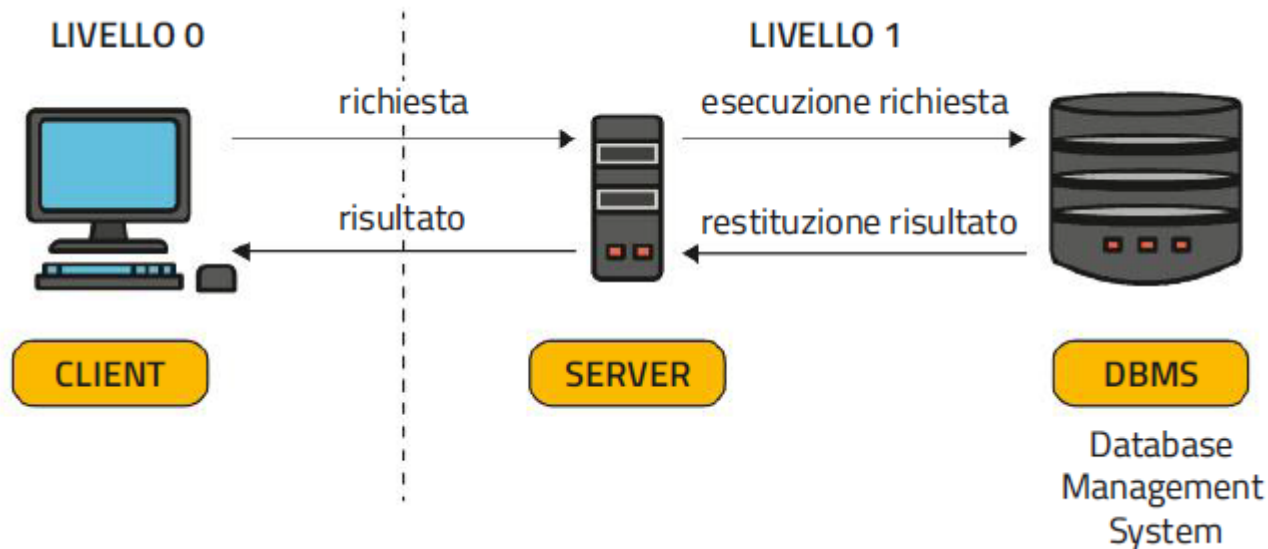
PostgreSQL è un progetto **open source** cioè non è un prodotto commerciale.

Il software è diffuso sotto licenza BSD (*Berkeley Software Distribution*).

Supporta le transazioni **ACID** (sigla che indica alcune caratteristiche fondamentali del DBMS: *Atomicità, Coerenza, Isolamento e Durabilità*), garantendo l'integrità dei dati e la concorrenza delle transazioni, oltre alla possibilità di definire vincoli complessi e procedure memorizzate tramite il linguaggio di programmazione PL/pgSQL, molto simile al linguaggio SQL ma con maggiori funzioni di controllo.

DBMS PostgreSQL

Dal punto di vista architetturale, PostgreSQL è un servizio e viene generalmente eseguito dal sistema operativo sul server che eroga tale servizio in ottica client/server: il servizio di PostgreSQL può gestire più basi di dati e ogni singola base di dati è rappresentata da uno spazio fisico in cui vengono memorizzati i dati.



pgAdmin

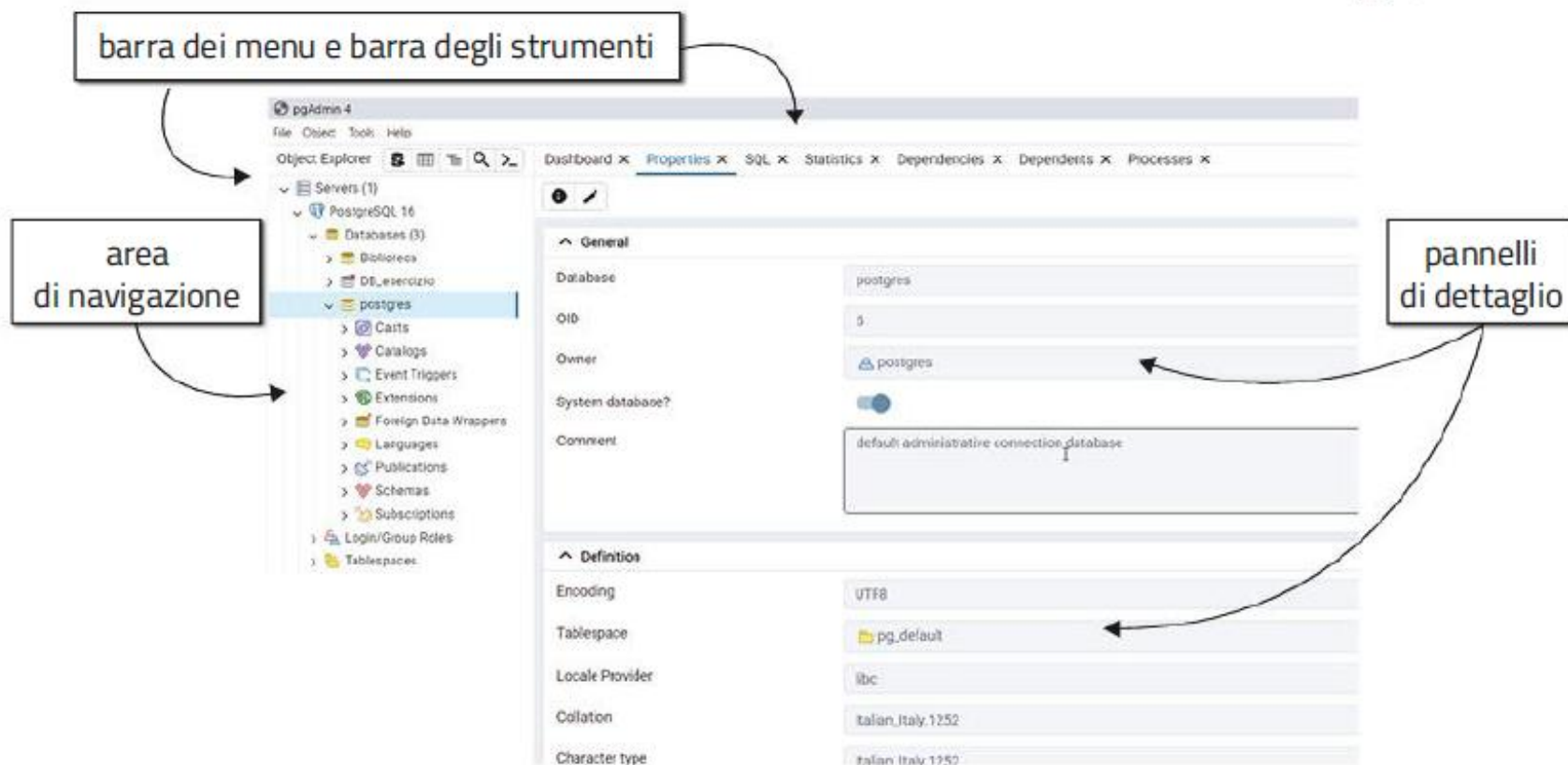
pgAdmin è un client open source con interfaccia grafica progettato per semplificare la gestione delle basi di dati PostgreSQL.

Il client pgAdmin è indirizzato sia agli amministratori dei database sia agli utenti, poiché, oltre alle funzionalità di navigazione dei database e di esecuzione delle interrogazioni SQL, mette a disposizione funzionalità avanzate come il monitoraggio delle prestazioni e la gestione degli utenti.

Attraverso pgAdmin, quindi senza scrivere istruzioni SQL, è possibile:

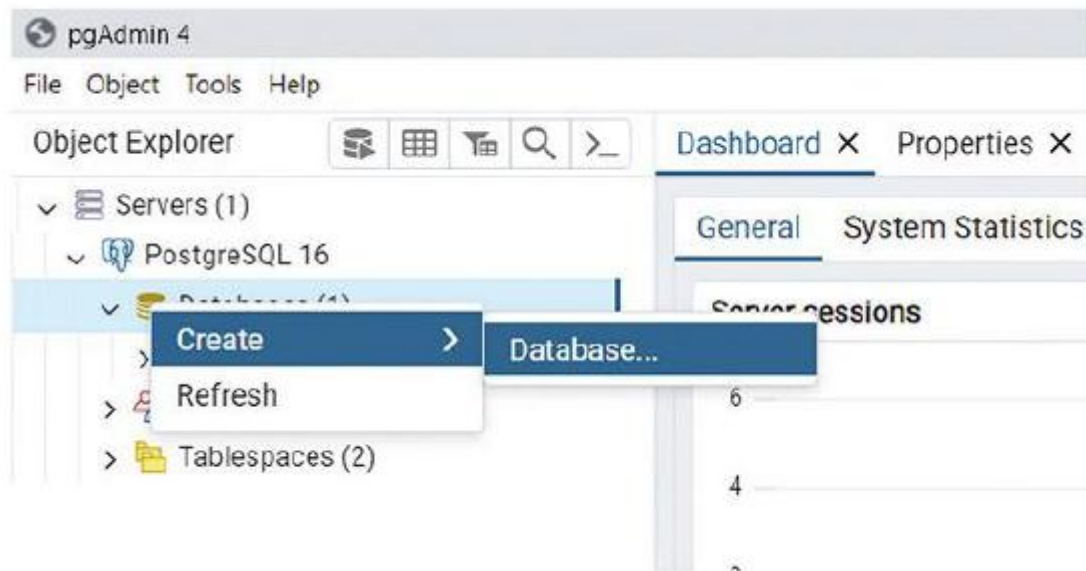
1. creare schemi;
2. modificare schemi;
3. inserire dati nelle tabelle;
4. visualizzare dati;
5. verificare la sintassi delle interrogazioni SQL;
6. monitorare le prestazioni del database.

Interfaccia pgAdmin



Creazione database

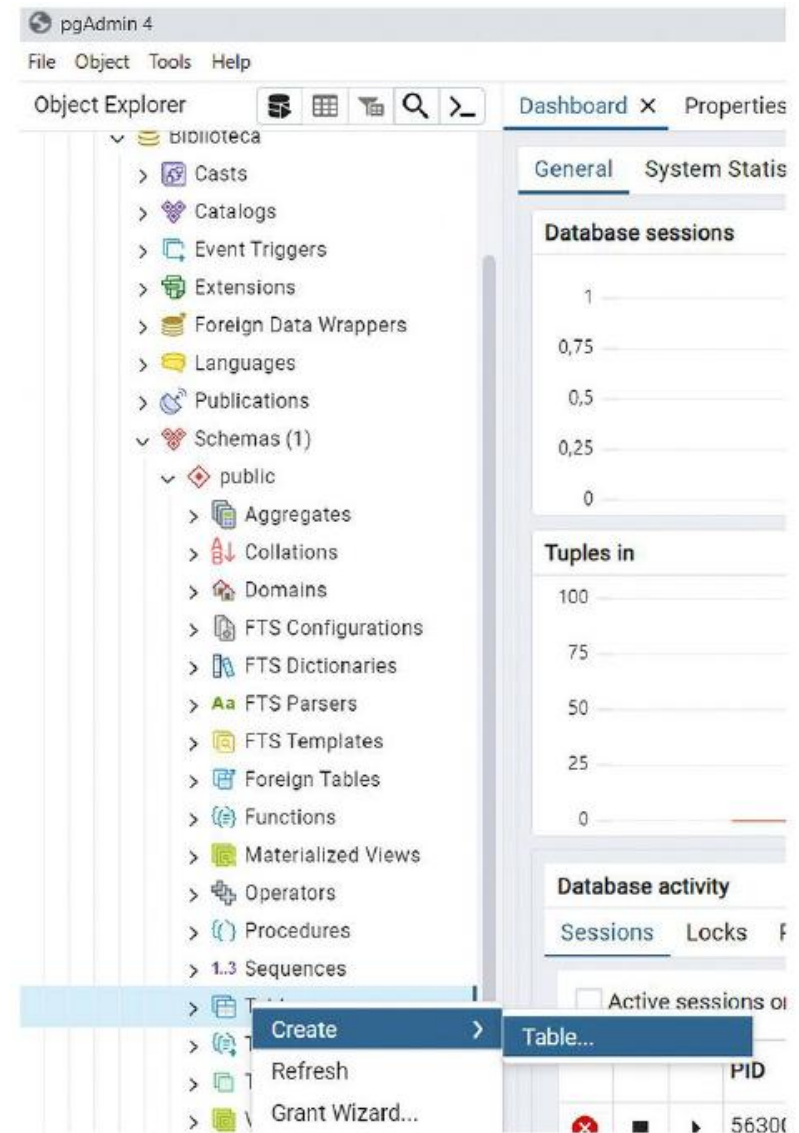
Selezioniamo nell'area di navigazione di sinistra la voce *Databases* e facendo clic con il tasto destro selezioniamo la voce *Create*



Creazione tabelle

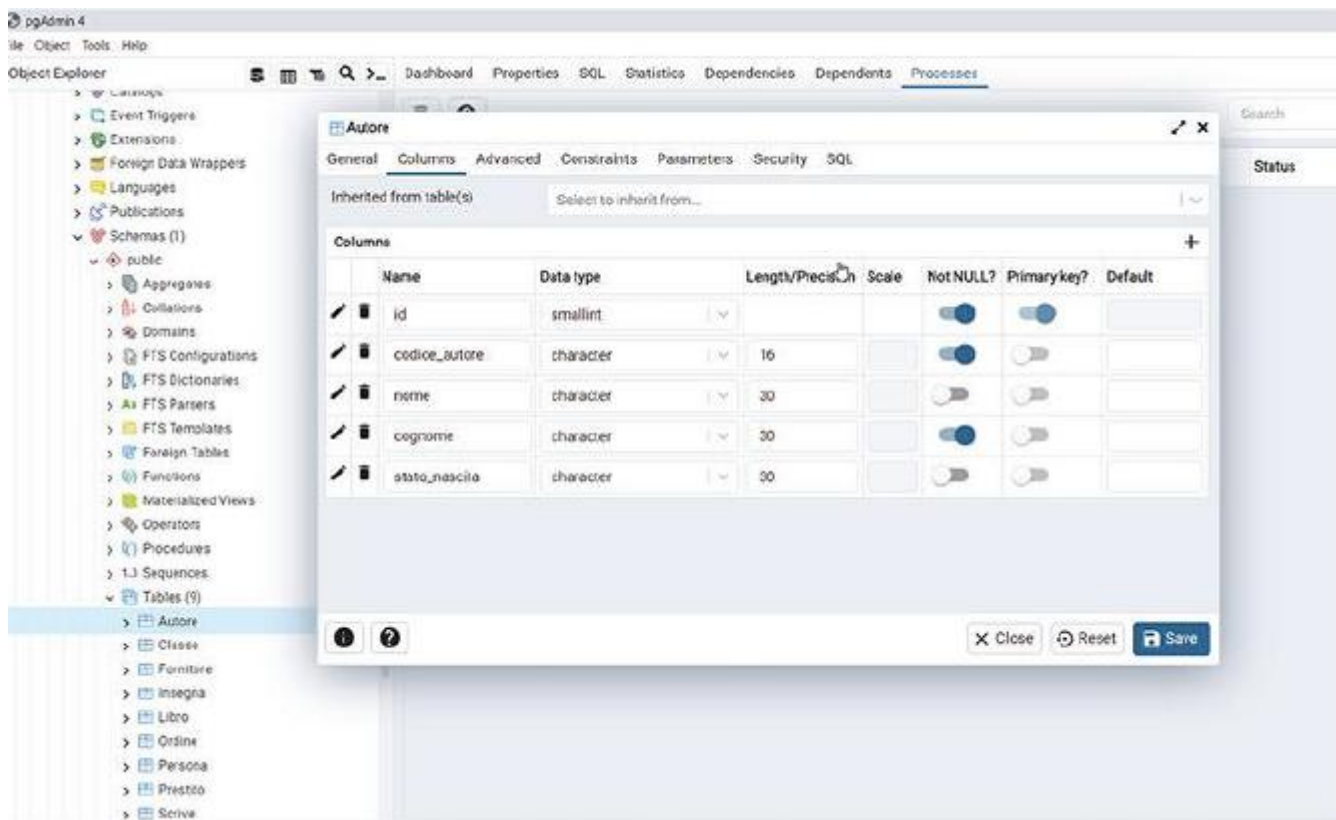
Per creare le tabelle ci posizioniamo sulla voce *Schemas* di Biblioteca, facciamo clic per aprire le voci sottostanti e selezioniamo la voce *Tables*.

Facciamo quindi clic con il tasto destro, selezioniamo la voce *Create* e poi *Table*



Creazione tabelle – inserimento campi

Nell'inserimento del nome e dei campi della tabella si faccia attenzione a indicare anche il tipo di dato relativo agli attributi e la dimensione. Per l'attributo chiave della tabella dovremo indicare nell'apposita colonna che l'attributo è *Primary key*, ovvero chiave primaria.



Primary key come IDENTITY

La definizione della *Primary Key* come *IDENTITY* prevede di utilizzare un meccanismo del DBMS molto utile e che garantisce il vincolo di integrità di chiave primaria:

la chiave primaria è rappresentata da un numero intero che parte da 1 e si incrementa in modo automatico di 1 ogni volta che viene inserita una nuova istanza di dati nella tabella.

In questo modo non ci dobbiamo preoccupare di inserire un valore per la chiave primaria, poiché il DBMS lo farà in modo automatico e garantendo l'univocità della chiave.

Il concetto di chiave primaria identifica univocamente un soggetto, come il Codice Fiscale, e deve essere sempre pensato come identificatore del soggetto: le interrogazioni SQL relative a una persona le faremo usando il suo Codice Fiscale e non l'id che gli è stato assegnato da PostgreSQL.

Il meccanismo di *IDENTITY* rimarrà solo un automatismo atto a evitare errori e problemi di integrità, rendendo la gestione del database più efficiente da parte del DBMS.

Primary key come IDENTITY

L'attributo id che abbiamo definito come *Primary key* e che corrisponde alla chiave della tabella definita nello schema relazionale: in PostgreSQL è necessario specificare che questo attributo è di tipo *Identity*.

The screenshot shows the 'Autore' table editor with the following configuration:

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	smallint			☒	☒	

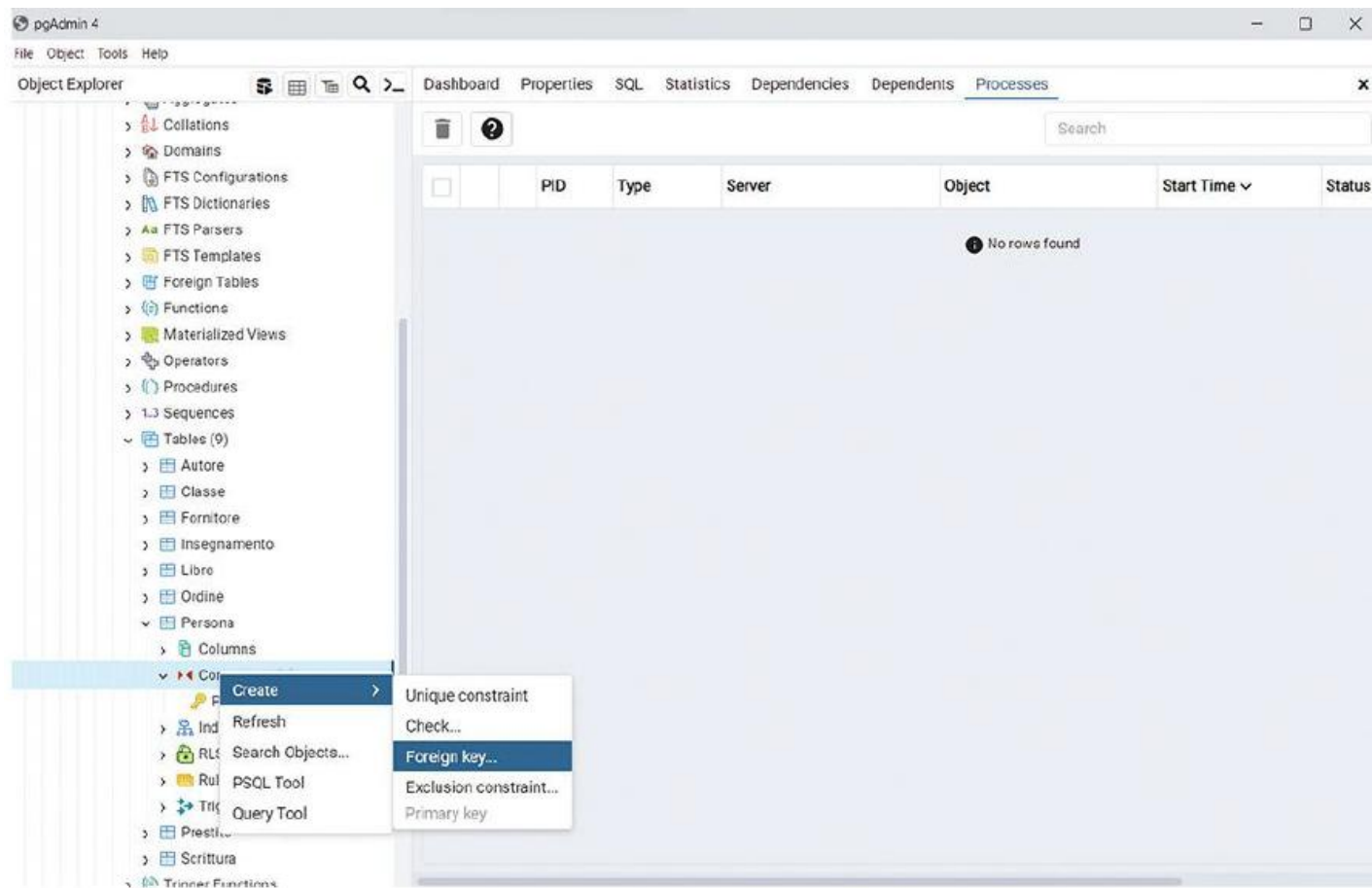
In the 'Constraints' tab, the configuration for the 'id' column is as follows:

Property	Value
Default	
Not NULL?	☒
Type	☒ IDENTITY
Identity	ALWAYS
Increment	1
Start	1
Minimum	1

È possibile anche decidere di non utilizzare il meccanismo di *IDENTITY* offerto dal DBMS: in questo caso useremo l'identificatore originario Codice autore e specificheremo semplicemente che è *Primary Key* senza definirlo come *IDENTITY*.

Foreign key – vincolo referenziale

Per definire correttamente il vincolo di integrità referenziale di chiave esterna, nell'area di navigazione di sinistra, sotto la tabella Persona troviamo la voce *Constraints*; selezioniamo quindi *Create* e poi *Foreign Key*



DDL – comando CREATE TABLE

```
CREATE TABLE la_mia_tabella (  
  prima_colonna text,  
  seconda_colonna integer);
```

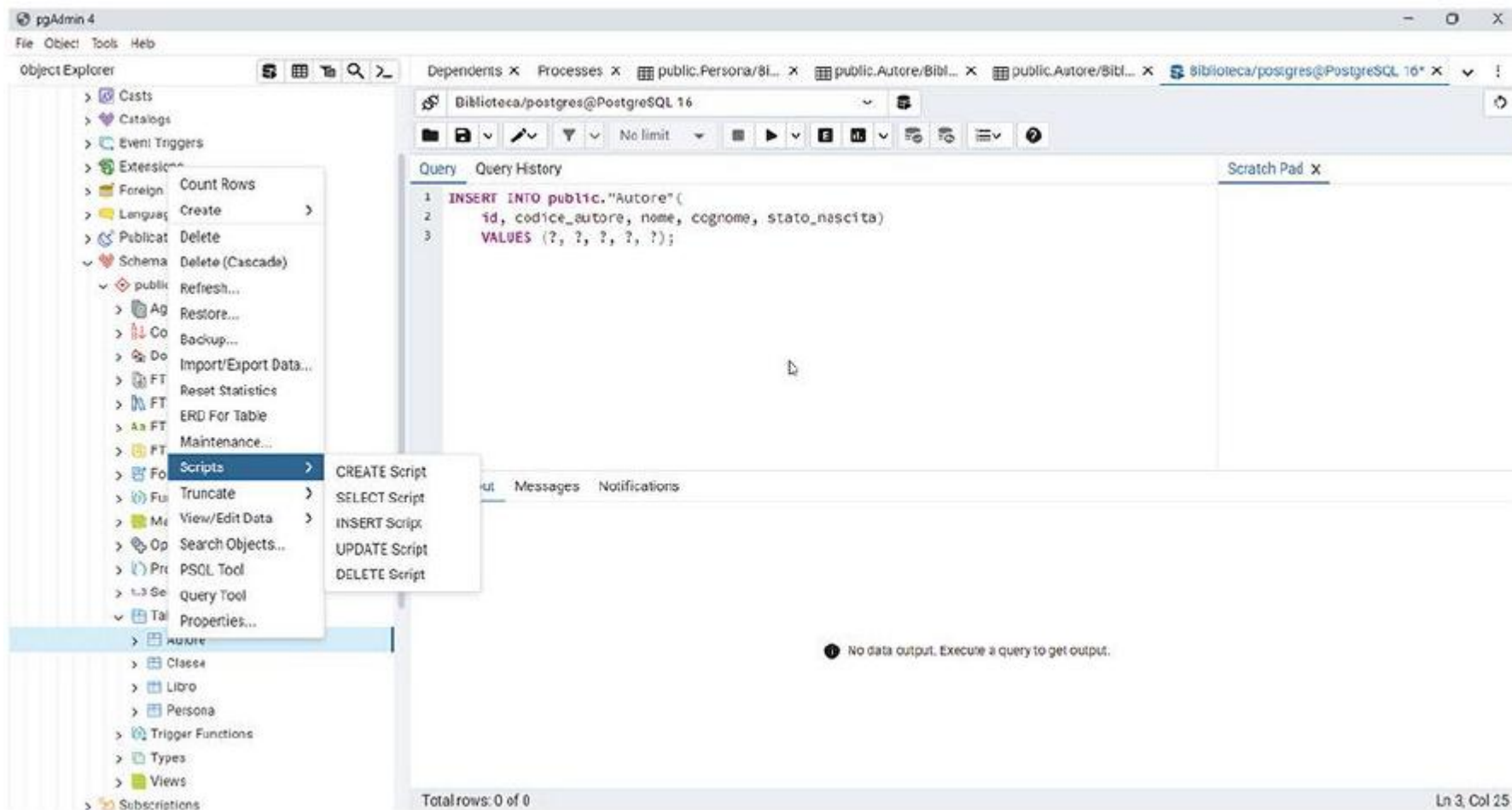
Create - Table ×

General Columns Advanced Constraints Partitions Parameters Security SQL

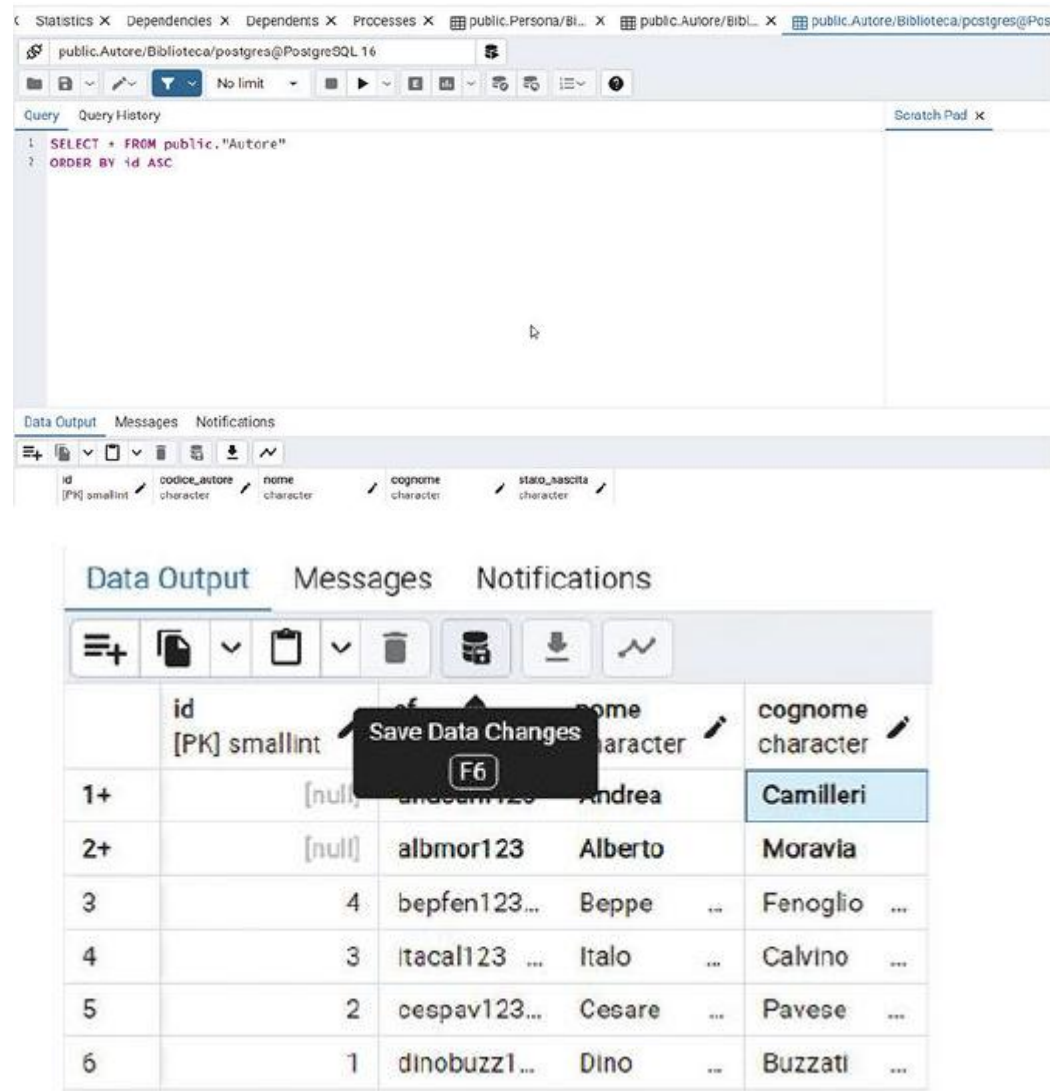
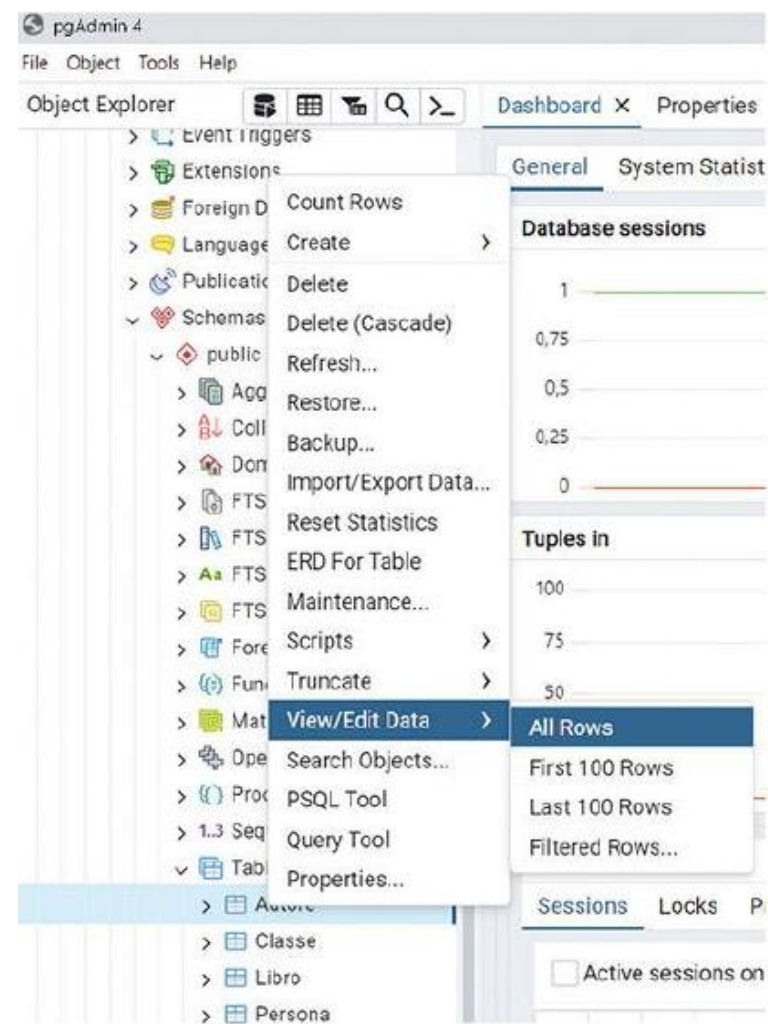
```
1 CREATE TABLE public."Libro"  
2 (  
3   id smallint NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 ),  
4   isbn character(20) NOT NULL,  
5   titolo character(40) NOT NULL,  
6   PRIMARY KEY (id)  
7 );  
8  
9 ALTER TABLE IF EXISTS public."Libro"  
10  OWNER to postgres;
```

i ? × Close ↺ Reset 💾 Save

DML – INSERT, UPDATE, DELETE



Popolare il database con pgAdmin

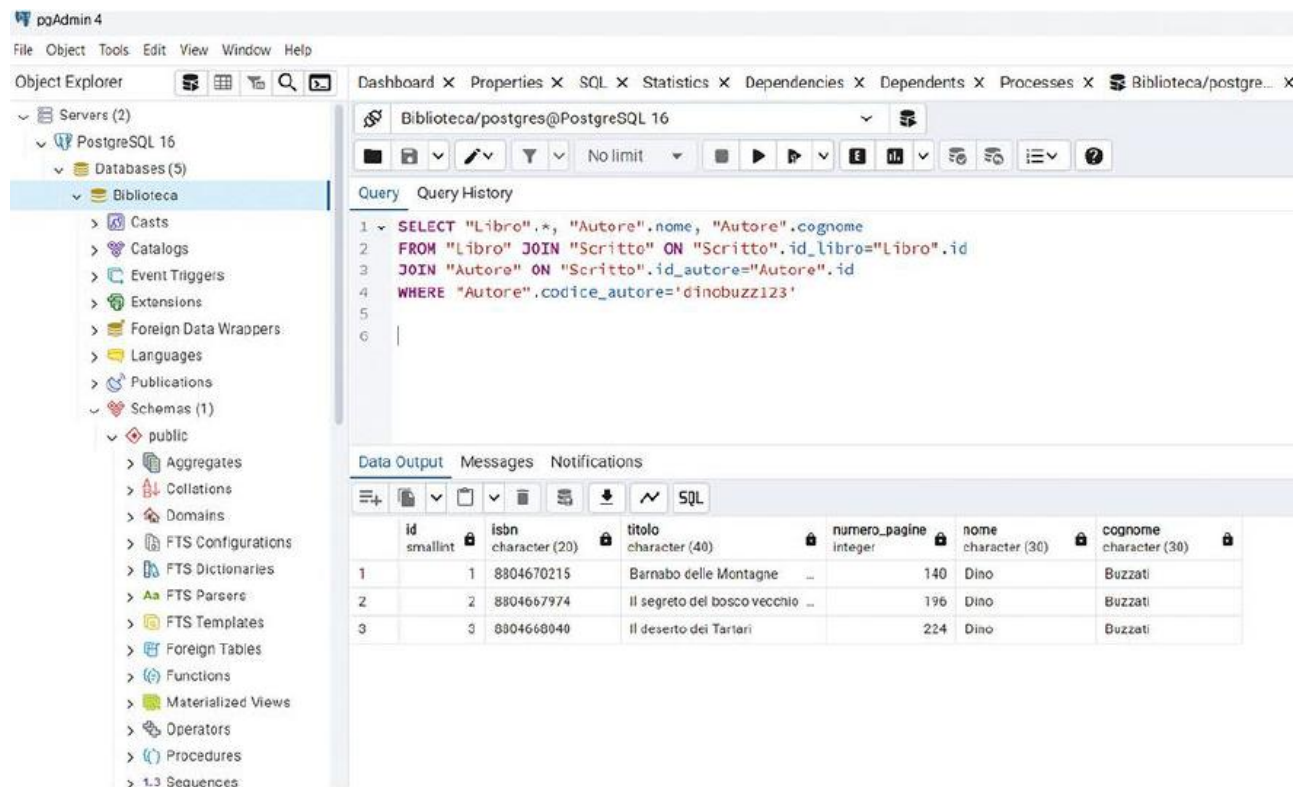


Interrogazioni SQL – Query tool

PgAdmin al suo interno ha integrato un editor di interrogazioni SQL, [Query Tool](#), per scrivere, verificare la sintassi ed eseguire le interrogazioni direttamente dall'interfaccia del client.

È possibile scrivere le proprie interrogazioni SQL, salvarle per utilizzi successivi o usare quelle salvate in precedenza.

Una volta eseguita l'interrogazione col tasto *Execute*, è possibile visualizzare i risultati direttamente nell'interfaccia di pgAdmin. Questo strumento fornisce anche informazioni utili per il monitoraggio delle interrogazioni in esecuzione, come il tempo di esecuzione e il numero di righe restituite.



The screenshot displays the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure, including Servers, PostgreSQL 16, Databases (5), and the public schema. The main window is the Query Tool, which shows the following SQL query:

```
1 SELECT "Libro".*, "Autore".nome, "Autore".cognome
2 FROM "Libro" JOIN "Scritto" ON "Scritto".id_libro="Libro".id
3 JOIN "Autore" ON "Scritto".id_autore="Autore".id
4 WHERE "Autore".codice_autore='dinobuzz123'
5
6
```

Below the query editor, the Data Output tab is active, showing the results of the query in a table format:

id	isbn	titolo	numero_pagine	nome	cognome
1	8804670215	Barnabo delle Montagne	140	Dino	Buzzati
2	8804667974	Il segreto del bosco vecchio	196	Dino	Buzzati
3	8804668040	Il deserto dei Tartari	224	Dino	Buzzati

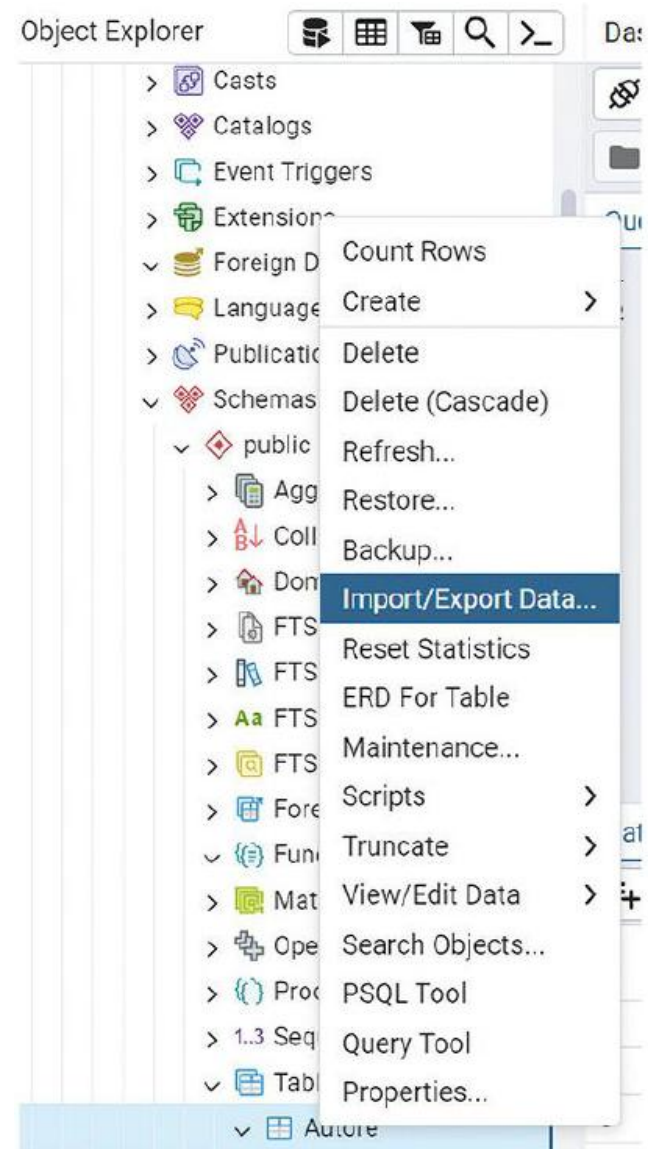
Importazione ed esportazione dati

Con pgAdmin è possibile sia importare sia esportare dati in un database utilizzando un file per trasferire dati tra diversi sistemi.

Se per esempio i dati per popolare le tabelle sono contenuti in un file CSV, possiamo fare in modo che pgAdmin importi i dati direttamente da questo file, senza doverli inserire manualmente.

Se invece desideriamo esportare i dati del database, per esempio per fare un *backup* o per consentirne l'uso in altri sistemi, è possibile farlo in diversi formati.

I principali formati per l'importazione e l'esportazione di dati sono file CSV, TXT e SQL.

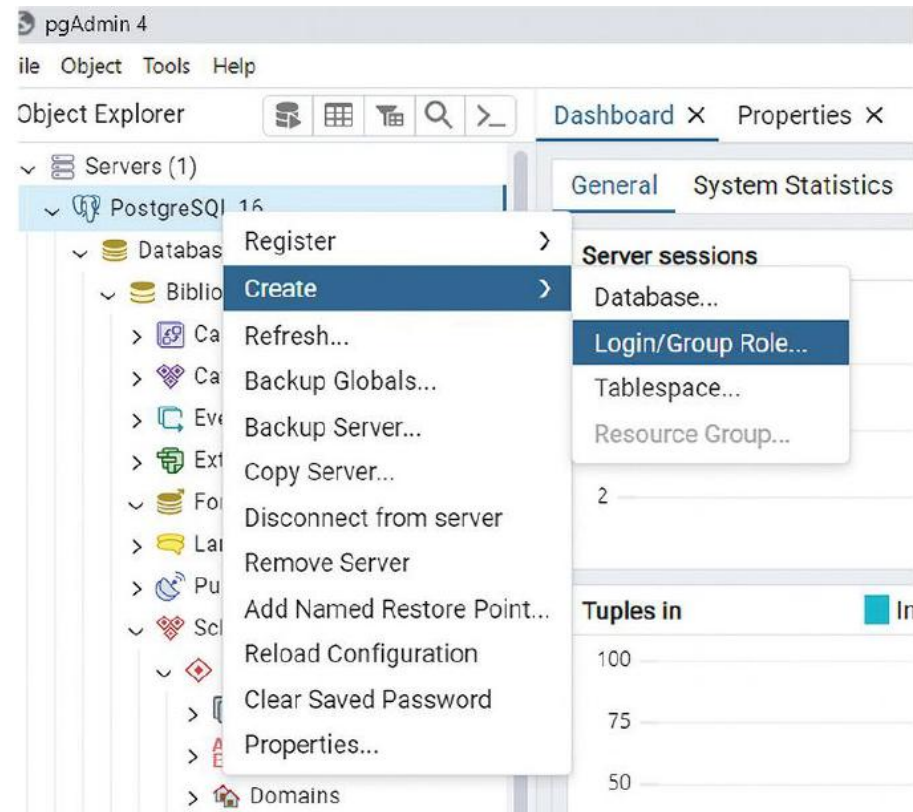


Gestione degli utenti

Per interagire con un database in PostgreSQL (ma anche negli altri DBMS), è necessario avere un utente che possiede i diritti di accesso sui dati e ha il compito di amministrare tutto il database.

Contestualmente alla creazione del database viene creato di default un utente *admin* (chiamato *postgres*), che viene usato per effettuare le operazioni di base.

Quando vogliamo permettere ad altre persone di accedere al database e ai dati, ovvero eseguire interrogazioni ma non modifiche a dati o a tabelle, per sicurezza è meglio utilizzare utenti diversi da quello di amministrazione e dare a questi utenti privilegi differenti in base alle operazioni che sono autorizzati a svolgere.



Capitolo 5

Un progetto Client/Server

I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume
Informatica per progetti – Lavorare con le basi di dati, di C. Batini, M. Fiore, ISBN 9788808769978

Applicazione gestionale

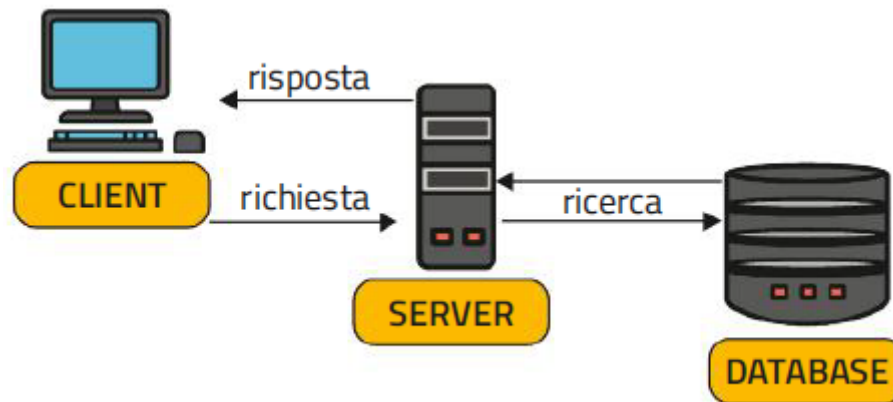
Un'**applicazione gestionale** è un'applicazione in cui il software applicativo consente la gestione completa dei dati contenuti in un database, senza aver bisogno di utilizzare a questo scopo il client del DBMS.

La sigla **CRUD** (creazione, lettura, aggiornamento, cancellazione) indica le principali operazioni che si effettuano sui dati di un database.

- *Create* corrisponde a INSERT,
- *Read* a SELECT,
- *Update* a UPDATE
- *Delete* a DELETE.

Applicazione client/server

Architettura client/server: la tecnologia utilizzata dal **server** è Python, che si occupa anche di gestire gli accessi al DBMS PostgreSQL, eseguendo le operazioni di lettura e scrittura e le interrogazioni al database. Il **client** è rappresentato da un'interfaccia testuale anch'essa realizzata in Python, che prevede un menu di scelte disponibile sul computer.



La progettazione

Le principali fasi di progettazione sono:

1. raccolta e analisi dei requisiti;
2. definizione dell'architettura;
3. progettazione del database (livello dati), comprensivo dello schema E/R, dello schema relazionale e della definizione delle tabelle;
4. progettazione dell'applicazione lato server (server side o back-end), delle operazioni CRUD e degli obiettivi che l'applicazione intende raggiungere;
5. progettazione dell'applicazione lato client (client side o front-end), delle interfacce utente e delle logiche di presentazione;
6. test di qualità finale;
7. manutenzione.

La documentazione

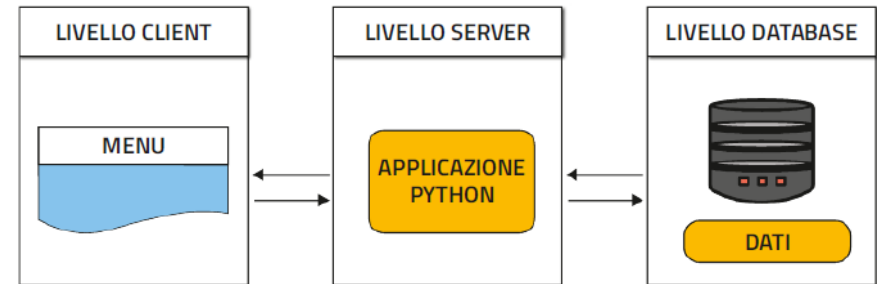
Ogni fase indicata deve essere documentata correttamente, per facilitare lo sviluppo e la manutenzione del software che viene prodotto, oltre a permettere un riutilizzo delle funzioni e delle procedure prodotte.

La documentazione di solito riguarda:

1. i requisiti;
2. l'architettura;
3. la documentazione tecnica, comprensiva anche di schemi E/R, schema relazionale e interrogazioni SQL;
4. il monitoraggio temporale del progetto;
5. il manuale utente.

Deve essere il più possibile aggiornata, precisa e ricca di dettagli, al fine di facilitare sviluppo e manutenzione, ma anche di sostenere la collaborazione tra gli operatori e la qualità del software prodotto.

Architettura applicativa



Livello client (menu di interfaccia)

1. realizzare i menu con le voci e le sottovoci di gestione
2. rendere semplice la navigazione tra le voci di menù

Livello server (applicazione)

1. realizzare le funzioni di accesso al database
2. realizzare le operazioni CRUD che integrano le interrogazioni SQL
3. realizzare il codice che esegue le interrogazioni SQL per mostrare i risultati

Livello database

1. realizzare lo schema concettuale E/R
2. derivare il corrispondente schema logico relazionale
3. implementare le tabelle del database rispettando i vincoli di integrità referenziale definiti nello schema logico
4. popolare il database con i dati iniziali
5. scrivere le interrogazioni SQL corrispondenti alla gestione dei dati

Accesso al database

Per consentire a un'applicazione Python di accedere al database PostgreSQL è necessario utilizzare una libreria specifica chiamata *psycopg2* che è un adattatore PostgreSQL per il linguaggio di programmazione Python.

Ogni DBMS ha un adattatore specifico, quindi a seconda del tipo di DBMS è necessario utilizzare l'adattatore corretto.

La libreria *psycopg2* si installa come tutte le librerie Python, utilizzando il comando *pip*,

```
pip install psycopg2
```

Dopo aver installato la libreria con il comando *pip*, l'applicazione dovrà prevedere l'importazione della libreria così che questa possa venire utilizzata all'interno dello script,

```
1 | import psycopg2
```

Operazioni per accesso al database

Le operazioni che devono essere eseguite dall'applicazione Python per utilizzare correttamente il database PostgreSQL sono:

1. Installare la libreria *psycopg2*.
2. Importare la libreria e connettersi al database.
3. Gestire gli errori.
4. Creare il cursore.
5. Scrivere l'interrogazione.
6. Ottenere i risultati.
7. Chiudere il cursore e la connessione.

Connessione al database

```
1  try:
2      #Stabilire la connessione al database
3      conn = psycopg2.connect(
4          dbname = "nome_database",
5          user = "nome_utente",
6          password = "password_utente",
7          host = "indirizzo_host",
8          port = "porta"
9      )
10 except Exception as e:
11     print(f"Errore: {e}")
```

In seguito alla corretta connessione al database è necessario creare il cursore.

```
12  cur = conn.cursor()
```

Creare ed eseguire query

```
13      #Esempio di query
14      query = '''
15      SELECT "Persona".id, "Persona".nome, "Persona".cognome,
16              "Persona".anno_assunzione, "Classe".id, "Classe".numero,
17              "Classe".lettera, "Libro".id, "Libro".isbn, "Libro".titolo
18      FROM "Persona"
19      JOIN "Insegna" ON "Insegna".id_docente = "Persona".id
20      JOIN "Classe" ON "Insegna".id_classe = "Classe".id
21      JOIN "Prestito" ON "Prestito".id_persona = "Persona".id
22      JOIN "Libro" ON "Prestito".id_libro = "Libro".id
23      WHERE "Persona".ruolo = 'docente'
24      AND "Prestito".data_fine IS NULL;
25      '''
26
27      #Eseguire la query
28      cur.execute(query)
29
29      #Recuperare i risultati
30      results = cur.fetchall()
31      for row in results:
32          print(row)
```

Ottenere i risultati

Il metodo `fetchall()` recupera tutto il risultato dell'interrogazione eseguita, costituito da più righe.

È anche possibile utilizzare `fetchone()` per recuperare una riga alla volta (includendolo dunque in un ciclo di `for`) o `fetchmany(size)` per recuperare un numero specifico di righe, a partire dalla prima.

Infine dovremo chiudere tutti i collegamenti effettuati per liberare le risorse (cursore e database) che non utilizzeremo più.

```
33 # Chiudere il cursore e la connessione
34     cur.close()
35     conn.close()
```

DDL: comando INSERT da Python

```
1 import psycopg2
2
3 try:
4     conn = psycopg2.connect(
5         dbname = "nome_database",
6         user = "nome_utente",
7         password = "password",
8         host = "indirizzo_host",
9         port = "porta"
10    )
11    cur = conn.cursor()
12
13    insert_query = '''
14    INSERT INTO "Libro" (id, isbn, titolo, numero_pagine)
15    VALUES (%s, %s, %s, %s)
16    '''
17
18    #Lista di tuple con i dati da inserire
19    data_list = [
20        (2, '1234567', 'Lonesome Dove', 976),
21        (3, '3456789', 'Le strade di Laredo', 516)
22    ]
23
24    for data in data_list:
25        cur.execute(insert_query, data)
26
27        conn.commit()
28
29        print("Dati inseriti con successo.")
30
31        cur.close()
32        conn.close()
33
34    except Exception as e:
35        print(f"Errore: {e}")
```

DDL: comando UPDATE da Python

```
1 import psycopg2
2
3 try:
4     conn = psycopg2.connect(
5         dbname = "nome_database",
6         user = "nome_utente",
7         password = "password",
8         host = "indirizzo_host",
9         port = "porta"
10    )
11    cur = conn.cursor()
12
13    update_query = '''
14    UPDATE "Libro"
15    SET isbn = %s
16    WHERE titolo = %s
17    '''
18
19    #Lista di tuple con i nuovi dati da aggiornare
20    data_to_update = [
21        ('9876543', 'Lonesome Dove'),
22        ('9875555', 'Le strade di Laredo')
23    ]
24
25    for data in data_to_update:
26        cur.execute(update_query, data)
27
28    conn.commit()
```

```
29
30     print("Dati aggiornati con successo.")
31
32     cur.close()
33     conn.close()
34
35 except Exception as e:
36     print(f"Errore: {e}")
```


DDL: comando DELETE da Python

```
1 import psycopg2
2
3 try:
4     conn = psycopg2.connect(
5         dbname = "nome_database",
6         user = "nome_utente",
7         password = "password",
8         host = "indirizzo_host",
9         port = "porta"
10    )
11    cur = conn.cursor()
12
13    delete_query = '''
14    DELETE FROM "Libro"
15    WHERE id = %s
16    '''
17
18    #Lista di ID dei record da cancellare
19    ids_to_delete = [(1,), (2,)]
20
21    for record_id in ids_to_delete:
22        cur.execute(delete_query, record_id)
23
24    conn.commit()
25
26    print("Record cancellati con successo.")
27
28    cur.close()
29    conn.close()
30
31 except Exception as e:
32    print(f"Errore: {e}")
```

Transazioni: Commit, Rollback

Una **transazione** è una sequenza di una o più istruzioni SQL del DML (INSERT, UPDATE, DELETE) eseguite come un'unica unità di lavoro. Il DBMS garantisce che la transazione, costituita da un gruppo di istruzioni, venga completata interamente oppure cancellata completamente, per esempio in seguito a un errore.

Questa caratteristica viene detta **atomicità**.

Una transazione termina sempre con una operazione di **commit**.

L'operazione commit rende permanenti tutte le modifiche eseguite in una transazione, cioè conferma che tutte le operazioni SQL (come INSERT, UPDATE o DELETE) eseguite dall'inizio della transazione sono corrette e che i risultati delle operazioni possono essere scritti in modo permanente nel database.

Una transazione inizia con la prima istruzione SQL eseguita e termina con una operazione di commit oppure con una operazione di rollback:

- commit per rendere le modifiche permanenti;
- rollback per annullare tutte le modifiche effettuate nella transazione corrente, tipicamente in caso di errore.

Transazioni

```
1 import psycopg2
2
3 try:
4     #Stabilire la connessione al database
5     conn = psycopg2.connect(
6         dbname = "nome_database",
7         user = "nome_utente",
8         password = "password",
9         host = "indirizzo_host",
10        port = "porta"
11    )
12    #Creare un cursore
13    cur = conn.cursor()
14
15    #Eseguire alcune operazioni SQL
16    insert_query = '''
17    INSERT INTO "Persona" (id, nome, cognome, anno_assunzione, ruolo)
18    VALUES (%s, %s, %s, %s, %s)
19    '''
20    update_query = '''
21    UPDATE "Persona"
22    SET nome = %s, cognome = %s
23    WHERE id = %s
24    '''
25    delete_query = '''
26    DELETE FROM "Persona"
27    WHERE id = %s
28    '''
29
30    #Dati da inserire
31    data_to_insert = (4, 'Luca', 'Gialli', 2023, 'studente')
32    #Dati da aggiornare
33    new_data = ('Luigi', 'Verdi', 2)
34    #ID del record da cancellare
35    id_to_delete = (3,)
36
37    #Eseguire le query
38    cur.execute(insert_query, data_to_insert)
39    cur.execute(update_query, new_data)
40    cur.execute(delete_query, id_to_delete)
41
42    #Commit delle modifiche
43    conn.commit()
44
45    print("Modifiche salvate con successo.")
46
47    #Chiudere il cursore e la connessione
48    cur.close()
49    conn.close()
50
51 except Exception as e:
52     if conn:
53         conn.rollback() #Annullare le modifiche in caso di errore
54     print(f"Errore: {e}")
55
```

Manutenzione

La **manutenzione correttiva** è l'insieme delle attività di correzione di difetti o errori nel software che vengono scoperti dopo che l'applicazione è stata rilasciata all'utente (in gergo, «messa in produzione»). Questi difetti possono essere bug, problemi di sicurezza o malfunzionamenti.

La **manutenzione evolutiva** riguarda le modifiche da apportare all'applicazione per migliorarne le funzionalità, adattarla a nuovi requisiti richiesti dall'utente e per aggiornarla in seguito alla disponibilità di nuove tecnologie, come nuove versioni del DBMS o degli ambienti di sviluppo.

Capitolo 6

Big Data e analisi dei dati

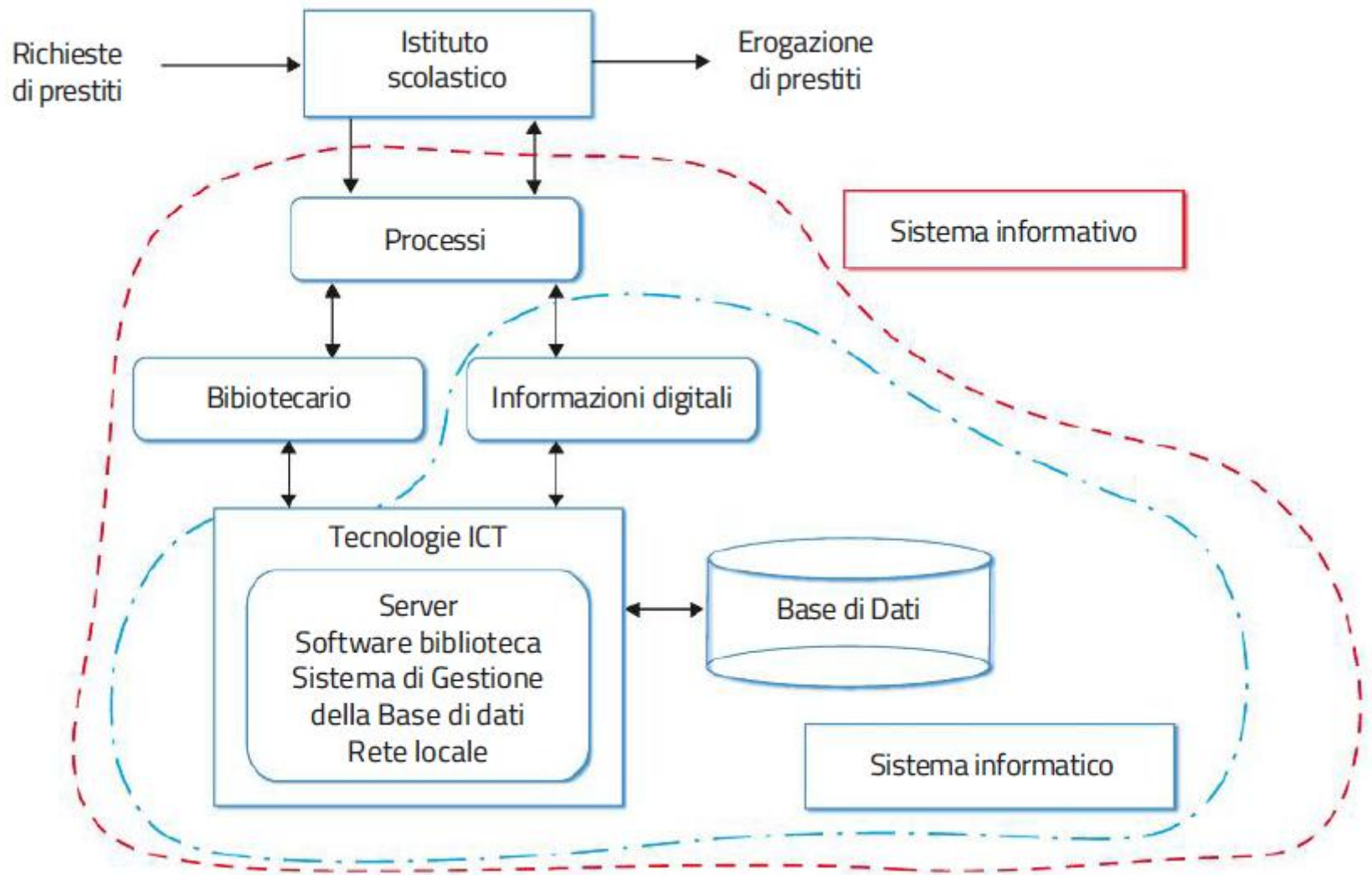
I crediti fotografici delle immagini presenti in questo PowerPoint sono riportati all'interno del volume
Informatica per progetti – Lavorare con le basi di dati, di C. Batini, M. Fiore, ISBN 9788808769978

Sistema Informativo vs Sistema Informatico

Le basi di dati sono una tecnologia diffusamente usata nei sistemi informativi di molte organizzazioni.

Per esempio, in un **sistema informativo** di un Istituto scolastico che, al fine di erogare prestiti di libri a seguito delle richieste degli utenti, svolge delle attività, o meglio, dei **processi**, queste attività sono svolte dal bibliotecario con il supporto di un'applicazione software che opera su una base di dati e fa uso di un DBMS e di tecnologie hardware e di rete.

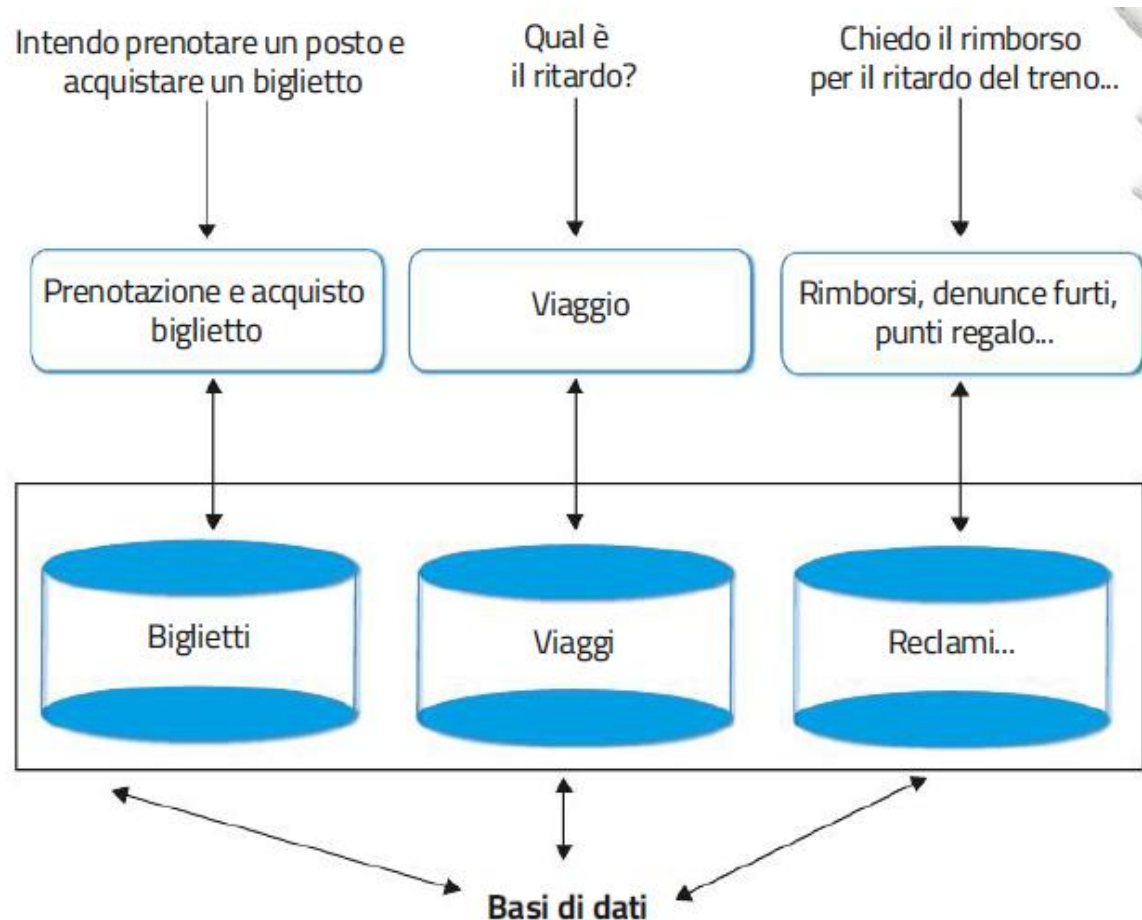
Queste tecnologie nel loro insieme costituiscono il **sistema informatico**, ovvero la parte *automatizzata* di un sistema informativo.



OLTP (On Line Transaction Processing)

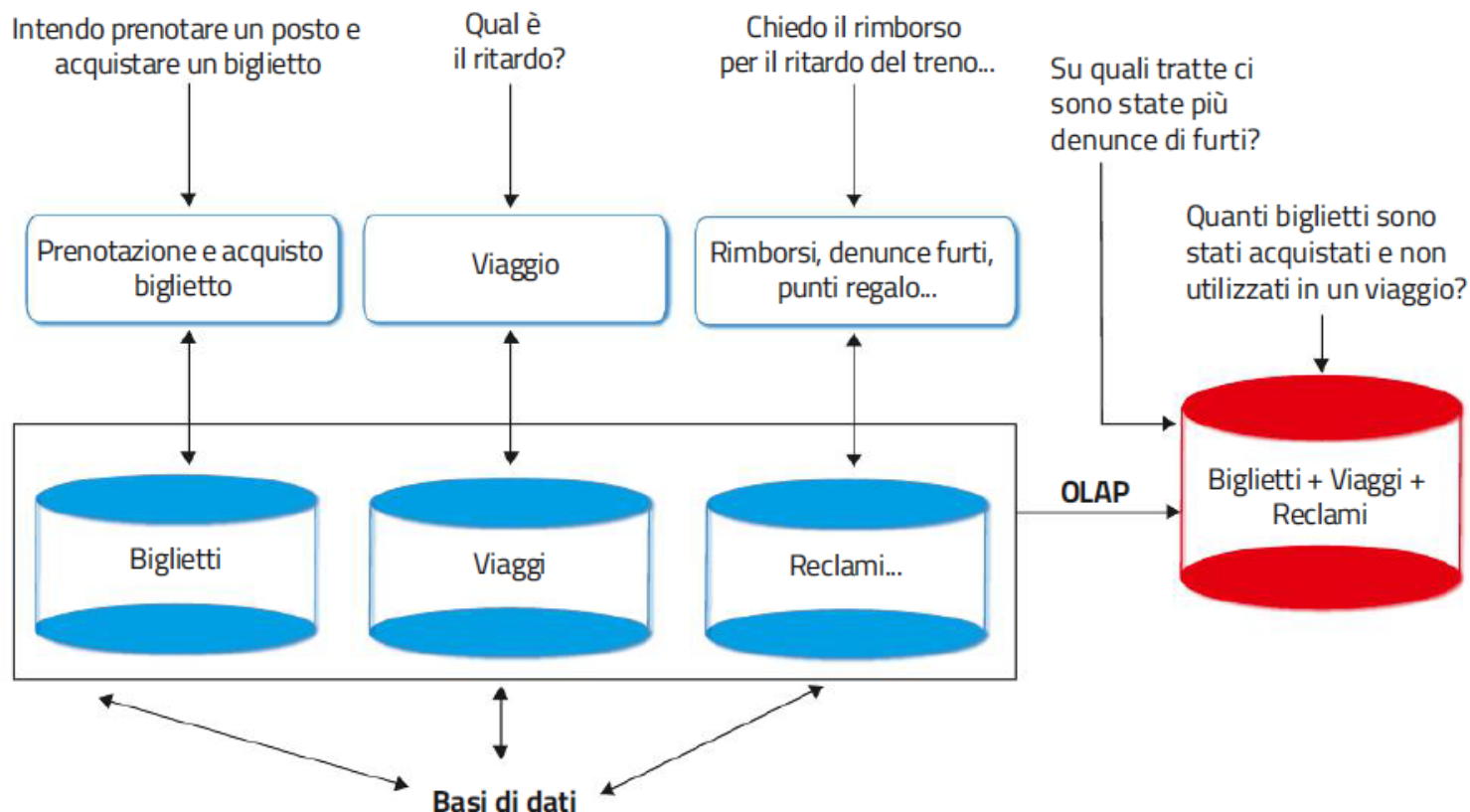
Un'azienda o una Pubblica amministrazione possono avere sistemi informativi dotati di molteplici basi di dati. Per esempio, un'azienda ferroviaria può avere una base di dati per la prenotazione e l'acquisto di biglietti, una seconda base di dati per fornire informazioni sui ritardi dei treni, una terza per le richieste di rimborso e così via.

Il tipo di elaborazione realizzata nel sistema informativo viene chiamata **On Line Transaction Processing (OLTP)**, cioè elaborazione basata su transazioni.



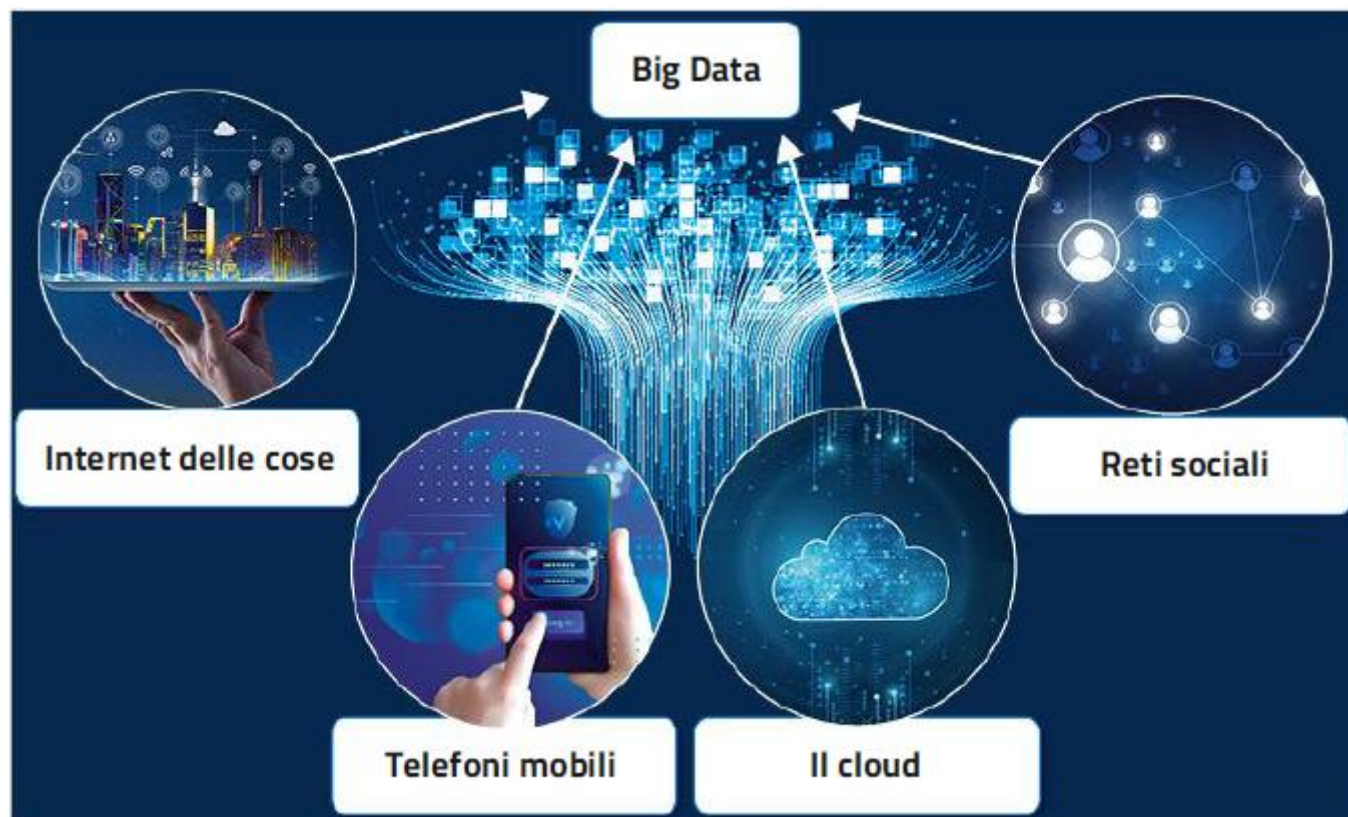
OLAP (On Line Analytical Processing)

Oltre a queste operazioni gestite grazie alle basi di dati, per organizzare le proprie attività l'azienda di trasporti può essere interessata a produrre periodicamente informazioni di sintesi, richieste attraverso interrogazioni. Questo tipo di elaborazioni viene chiamato **On Line Analytical Processing (OLAP)**.



Tecnologie che producono i Big Data

A partire dall'inizio del nuovo secolo, sono state prodotte e messe a nostra disposizione nuove tecnologie che, acquisendo, operando e trasmettendo dati digitali, hanno accresciuto la disponibilità dei dati nella rappresentazione del mondo fisico, dando luogo al fenomeno chiamato **Big Data**.



Big Data

I **Big Data** si riferiscono a *dataset* (insiemi di dati) il cui volume è talmente grande che eccede la capacità dei sistemi di basi di dati tradizionali di acquisire, immagazzinare, gestire e analizzare i dati digitali.

Secondo alcune stime di crescita dei Big Data i dati diffusi e scambiati nel web *raddoppiano in volume* ogni anno e mezzo.

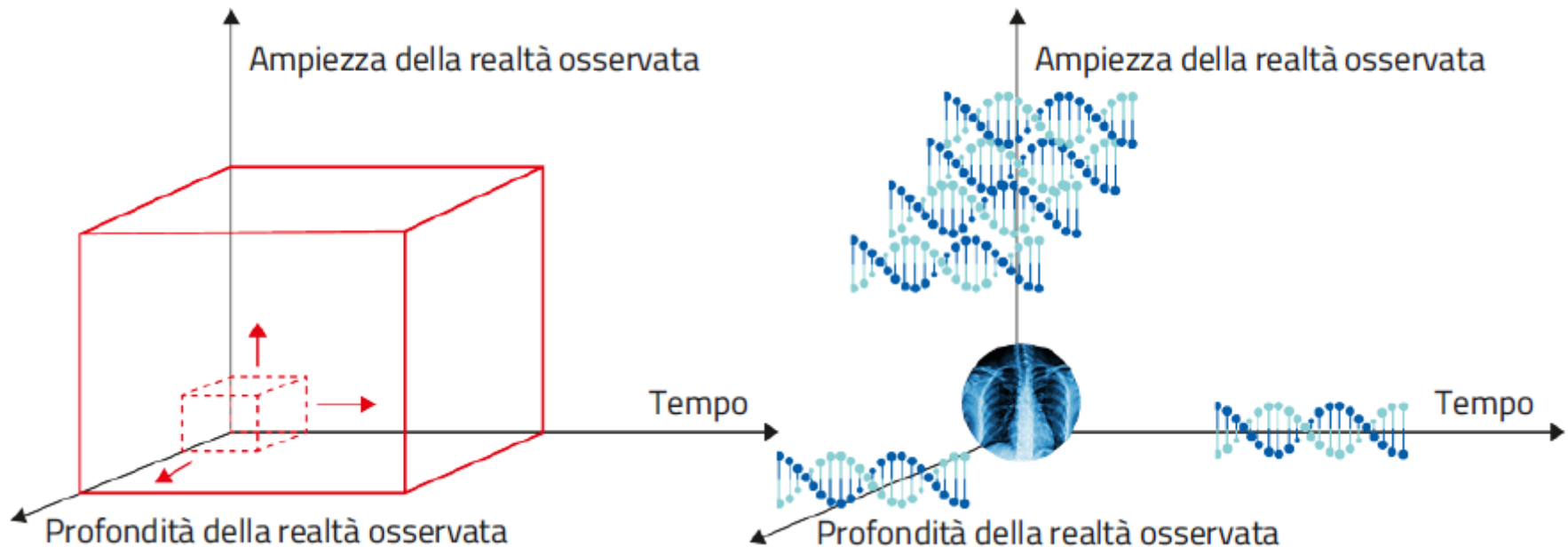
Come conseguenza della grande e crescente diffusione dei Big Data, i dati digitali permeano la nostra vita, modificano la comunicazione pubblica e privata, influenzano in modo radicale l'economia e forniscono materiale prezioso alla ricerca scientifica, presentando grandi opportunità ma anche grandi rischi per l'umanità.

L'elaborazione e l'analisi di dati digitali comportano l'adozione di tecniche nuove, che si distaccano dai modelli e linguaggi delle basi di dati; tali tecniche si collocano nelle discipline della [statistica inferenziale](#), [dell'apprendimento automatico](#) (*Machine learning*), [dell'elaborazione del linguaggio naturale](#) (*natural language processing*) e [dell'Intelligenza Artificiale](#).

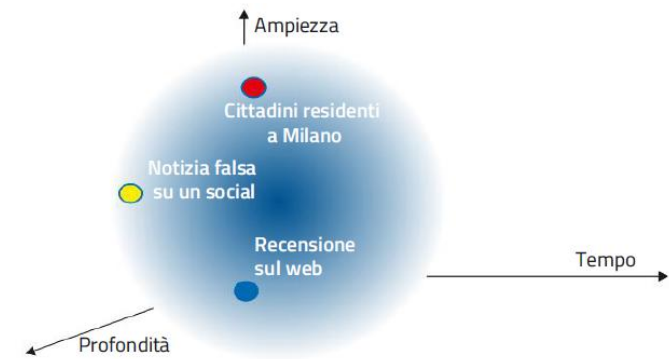
Dimensioni di espansione dei Big Data

Con i Big Data i dati disponibili per affrontare e risolvere problemi crescono continuamente e accade secondo tre coordinate:

1. l'ampiezza della conoscenza sulla realtà osservata;
2. la profondità della conoscenza sulla realtà osservata;
3. il tempo.



Caratteristiche dei Big Data



Le principali caratteristiche dei Big Data sono:

- il **volume**, con riferimento alla dimensione dei dati nell'ambito delle tre coordinate
- la **velocità**, intesa come tasso di generazione e trasmissione dei dati nell'unità di tempo;
- la **varietà**, in termini di eterogeneità dei tipi di dati.
- la **veridicità**, intesa come accuratezza del dato nel rappresentare un fatto o un fenomeno del mondo reale.

Nelle basi di dati, i dati rappresentati sono dotati di una struttura tabellare; questi tipi di dati sono detti **dati strutturati**.

I dati digitali sono evoluti verso formati **semistruutturati** o **non strutturati** come i documenti, che utilizzano, come forma espressiva prevalente, il linguaggio naturale, oppure verso formati visuali di rappresentazione, come le mappe geografiche, le immagini, i video.

Ciclo di vita dei dati digitali

Nel nuovo mondo dei Big Data, che cosa si intende per «ciclo di vita del dato»?

1. Formulazione del problema
2. Scelta delle fonti e acquisizione dei dati grezzi
3. Preparazione dei dati: profilazione.
4. Preparazione dei dati: normalizzazione
5. Preparazione dei dati: valutazione e miglioramento della qualità dei dati
6. Significato dei dati: Arricchimento semantico
7. Significato dei dati: Trasformazione di modello
8. Significato dei dati: Deduplicazione
9. Significato dei dati: Integrazione
10. Significato dei dati: Fusione dei dati eterogenei
11. Analisi dei dati e produzione di nuova conoscenza
12. Visualizzazione

Ciclo di vita dei dati digitali

Formulazione del problema: si cerca di comprendere bene l'obiettivo finale, ovvero il fine per cui i dati vengono raccolti ed elaborati. Questa fase sembra ovvia, ma accade più spesso di quanto non si creda che si inizi ad accumulare tanti dati, e solo successivamente ci si ponga il problema del perché vengono raccolti e di come utilizzarli.

Scelta delle fonti e acquisizione dei dati grezzi: questo passo è fondamentale; accade, infatti, spesso di non riuscire a trovare le fonti necessarie per raggiungere i nostri obiettivi, soprattutto nel mondo dei Big Data in cui ci servono tanti dati. In questi casi dobbiamo avere l'umiltà di rinunciare, perché, se sono pochi, i dati sono inutilizzabili e porterebbero quindi a risultati probabilmente errati.

Ciclo di vita dei dati digitali

Preparazione dei dati: profilazione.

La preparazione prevede un insieme di fasi che sono propedeutiche alle attività di effettiva elaborazione e analisi dei dati per raggiungere l'obiettivo finale.

La profilazione prevede di assegnare un nome espressivo agli attributi per cui il loro significato non è per nulla chiaro.

Preparazione dei dati: normalizzazione

La normalizzazione prevede di rendere omogenei i valori degli attributi di nostro interesse che, ad esempio, in alcuni casi sono numerici, mentre in altri sono costituiti da stringhe alfabetiche, oppure in alcuni casi sono espressi in lingue differenti.

Ciclo di vita dei dati digitali

Preparazione dei dati: valutazione e miglioramento della qualità dei dati.

Se i dati in input di una programma sono «sporchi», anche l'output del programma ne risentirà.

Consideriamo tre caratteristiche di qualità:

- l'**accuratezza** dei dati, cioè la coincidenza o vicinanza del valore numerico o alfabetico con il valore vero;
- la **completezza**, ovvero il fatto che non vi siano valori nulli;
- la **consistenza**, cioè la proprietà per cui i dati che hanno tra di loro un legame funzionale rispettino questo legame.

Per esempio, se vi sono tre attributi che esprimono il prezzo netto, l'Iva e il prezzo lordo, deve essere valida l'uguaglianza

Prezzo lordo = Prezzo netto + Iva.

Ciclo di vita dei dati digitali

Significato dei dati: arricchimento semantico.

È il passo in cui i dati eterogenei rappresentati nelle fonti, vengono espressi in modo semanticamente più ricco. Si usa dire che il dato è trasformato in conoscenza.

Si deve estrarre dal testo i dati e associare a essi un significato, per esempio associando alla stringa di caratteri “Giacomo Corsi” il significato di “Indagato”.

L’associazione di significato può essere fatta con algoritmi di elaborazione del linguaggio naturale (*Natural Language Processing*, NLP) confrontando, per esempio, i nomi e cognomi insieme con liste di nomi e cognomi di indagati e di persone, ovvero con altri documenti in cui “Giacomo Corsi” sia stato già annotato come “Indagato” e “Ciro Bini” come “Persona”.

Ciclo di vita dei dati digitali

Significato dei dati: trasformazione di modello.

Si adotta il modello dei grafi semantici per trasformare gli schemi tabellari in un unico modello di conoscenza.

Significato dei dati: deduplicazione.

Questa fase ha come obiettivo individuare, ed eliminare, eventuali duplicazioni degli stessi dati inseriti per errore più volte attraverso una tecnica che misura la “distanza” tra due righe calcolando il numero di caratteri da aggiungere, cancellare o modificare per trasformare la prima stringa di caratteri nella seconda (il valore “null” ha distanza zero).

Significato dei dati: integrazione.

L'integrazione è un'attività progettuale importante in molti ambiti in cui dati che provengono da diverse fonti vengono rappresentati insieme.

Significato dei dati: fusione di dati eterogenei.

Quando nelle attività di deduplicazione e integrazione accade di integrare dati diversi che fanno riferimento alla stessa realtà, bisogna scegliere, nella rappresentazione integrata, uno tra i dati, e non considerare più gli altri.

Ciclo di vita dei dati digitali

Strategie di fusione

Strategia	Classificazione	Descrizione
PASS IT ON	ignora il conflitto	trasferisce la soluzione del conflitto all'utente o all'applicazione
CONSIDER ALL POSSIBILITIES	ignora il conflitto	considera tutte le combinazioni di valori
TAKE THE INFORMATION	evita il conflitto, basata sull'istanza	preferisce valori reali rispetto a valori nulli
NO GOSSIPING	evita il conflitto, basata sull'istanza	seleziona solo tuple consistenti
TRUST YOUR FRIENDS	evita il conflitto, basata sui metadati	seleziona il valore di una fonte fidata
CRY WITH THE WOLVES	risolve il conflitto, basata sull'istanza	seleziona il valore più frequente
ROLL THE DICE	risolve il conflitto, basata sull'istanza	seleziona un valore a caso
MEET IN THE MIDDLE	risolve il conflitto, basata sull'istanza	seleziona un valore medio
KEEP UP TO DATE	risolve il conflitto, basata sui metadati	seleziona il valore più recente

Ciclo di vita dei dati digitali

Analisi dei dati e produzione di nuova conoscenza.

Tutto ciò fatto finora trova il suo valore nelle analisi che possono essere fatte sui dati estratti dalle fonti, dati che sono stati omogeneizzati, migliorati nella qualità, arricchiti, rappresentati con lo stesso modello, integrati, fusi insieme. Queste analisi possono avere come fine la produzione di nuova conoscenza, oppure la produzione di modelli descrittivi, classificatori, predittivi.

Visualizzazione dei dati

La visualizzazione mostra i dati all'utente finale in modo che siano immediatamente percepibili e comprensibili.

Modelli e tecniche per usare i Big Data

La grande disponibilità di dati digitali rende possibile realizzare differenti descrizioni ed elaborazioni della realtà rappresentata dai dati digitali, che chiamiamo con il nome unificante di modelli.

Un **modello** è una rappresentazione concettuale di una parte del mondo reale, capace di spiegare e descrivere in maniera generale un determinato fenomeno.

Un **modello matematico** è una rappresentazione quantitativa di un fenomeno naturale.

Nel caso dei dati, il termine **modello** viene usato per esprimere un insieme di strutture di rappresentazione; per esempio, il modello relazionale dei dati ha come strutture di rappresentazione la relazione e l'attributo. Viene invece usato il termine **schema** quando vogliamo indicare la rappresentazione di un frammento di realtà per mezzo delle strutture del modello.

Modelli e tecniche per usare i Big Data

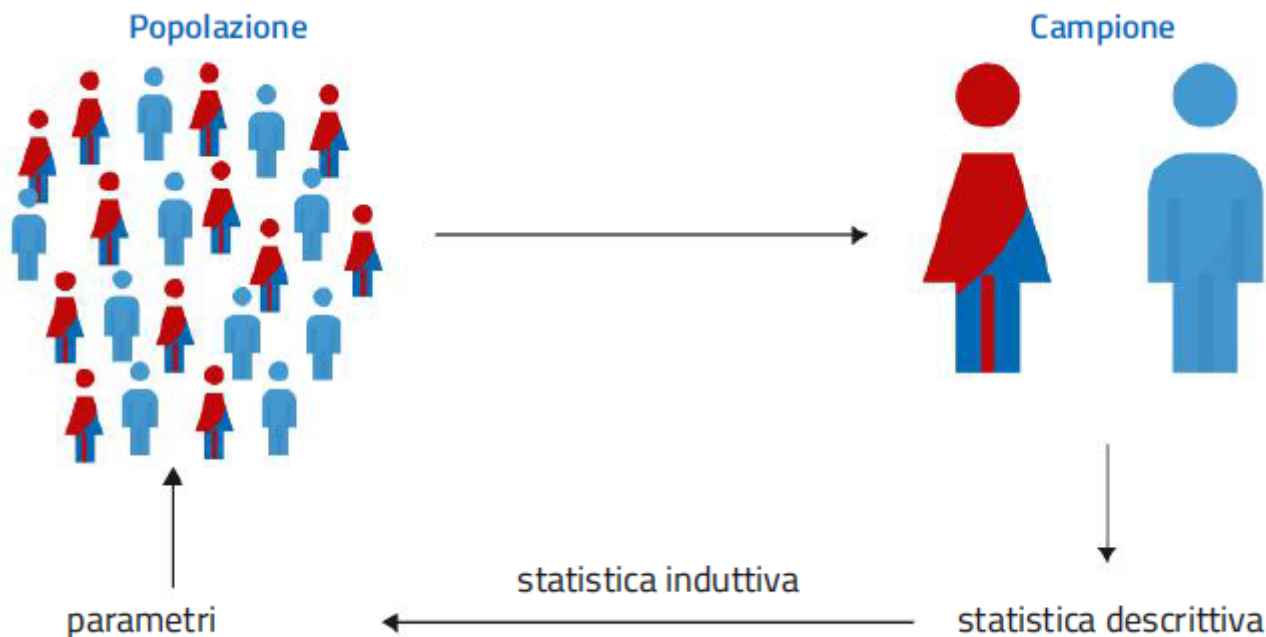
I **modelli statistici descrittivi** (le statistiche descrittive) ambiscono a esprimere le caratteristiche rilevanti di un fenomeno della realtà visto nella sua interezza. Questi modelli utilizzano tecniche di analisi statistica per estrarre informazioni significative dai dati, come la media, la deviazione standard, le distribuzioni e le correlazioni. Le statistiche descrittive sono rivolte al passato e al presente.

I **modelli classificatori** o classificazioni hanno lo scopo di stimare il valore di una caratteristica y (per esempio, per gli alberi, il tipo di albero, se faggio, frassino o altro) a partire dai valori che assumono altre caratteristiche x_1, x_2, x_n (per esempio l'altezza dell'albero, il colore del tronco, la forma delle foglie, l'ampiezza della chioma ecc.). Anche i modelli classificatori sono rivolti al passato e al presente.

I **modelli predittivi** sono modelli classificatori in cui la caratteristica y , risultato della classificazione, riguarda *un fenomeno che accadrà nel futuro*. Essi riguardano dunque il futuro.

Statistiche descrittive

Le statistiche descrittive sono strumenti fondamentali nell'analisi dei dati, poiché la statistica insegna a individuare i modi in cui un fenomeno si manifesta, a descriverlo sinteticamente e a trarre da esso conclusioni più generali di fenomeni più ampi.



Statistiche descrittive

Le statistiche descrittive includono una serie di funzioni per riassumere e descrivere le proprietà di un *dataset*, come *la media, la mediana, la moda, la varianza e la deviazione standard*.

Questi strumenti permettono di avere una **visione di insieme dei dati**, facilitando l'identificazione di schemi, tendenze ed eventuali errori e anomalie e, attraverso strumenti di rappresentazione come grafici e tabelle, permettono di comprendere, sintetizzare e comunicare le caratteristiche principali dei dati.

La statistica descrittiva ottiene dei risultati che si basano su valori effettivi, mentre **nel caso della statistica induttiva (inferenziale e bayesiana) i risultati si basano su stime**.

La statistica descrittiva serve a capire le caratteristiche proprie della variabili e delle unità statistiche che sono oggetto di analisi.

Machine Learning

Il **Machine learning** è la disciplina dell'Intelligenza Artificiale che permette di costruire modelli classificatori e predittivi utilizzando tecniche automatiche di apprendimento.

Il Machine learning è utilizzato in un vasto insieme di ambiti dell'attività umana, tra cui:
il riconoscimento e la classificazione di immagini e, in particolare, il riconoscimento facciale; la traduzione di testi tra lingue; la produzione di testi nei dialoghi con umani (la famiglia di modelli ChatGPT); la scoperta di nuovi farmaci e nuove cure; le auto a guida autonoma; i sistemi di raccomandazione che vengono usati quando ci viene suggerito un acquisto su siti di e-commerce; i giochi; gli investimenti finanziari; i sistemi militari e di puntamento; i sistemi di sorveglianza.

Le tipologie di Machine learning più studiate sono l'*apprendimento supervisionato*, l'*apprendimento non supervisionato* e l'*apprendimento per rinforzo*.

L'apprendimento supervisionato

Nell'apprendimento supervisionato sono disponibili, come esempio, *un campione di dati di input e i dati di output a essi collegati tramite una funzione*; per i dati di esempio, dunque, la funzione tra dati di input e dati di output è nota.

La tecnica di apprendimento produce il modello classificatorio o predittivo per tutti i dati di input.

L'apprendimento supervisionato è applicato in moltissime aree. Oltre a quelle menzionate in precedenza, è utilizzato per esempio per la concessione di prestiti ai clienti di una banca: per decidere, le banche devono determinare qual è il livello di rischio che il cliente non restituisca il prestito (il rischio di credito).

L'apprendimento non supervisionato

Nell'apprendimento non supervisionato sono disponibili, come esempio, *dati di input non associati a valori di output*; l'obiettivo, in questo caso, è riconoscere schemi, strutture o proprietà nell'insieme dei dati fornito.

Per esempio consideriamo un insieme di libri di una biblioteca: l'obiettivo di un possibile modello classificatorio è di raggruppare (con una operazione che è detta «*clustering*») i libri in due categorie: libri di saggistica e romanzi. Poiché non disponiamo di esempi di libri classificati nelle due categorie, dovremo utilizzare altre proprietà e caratteristiche dei libri per procedere nella classificazione.

L'apprendimento per rinforzo

Nell'apprendimento per rinforzo il modello predittivo viene generato mediante apprendimento sulla base delle risposte dell'ambiente alle scelte e decisioni che il modello prevede.

Per esempio, se lo scopo del modello è giocare a scacchi, vengono fatte giocare al modello tante partite in cui le singole mosse e le sequenze di mosse sono premiate o penalizzate sulla base dell'esito finale della partita (vittoria o sconfitta).

Possono essere generati diversi modelli che giocano tra di loro, così che alla fine prevalga quello che presenta le migliori prestazioni.

Alberi di decisione

Allo scopo di costruire un modello classificatorio o predittivo con l'apprendimento supervisionato, possono essere usate svariate tipologie di tecniche.

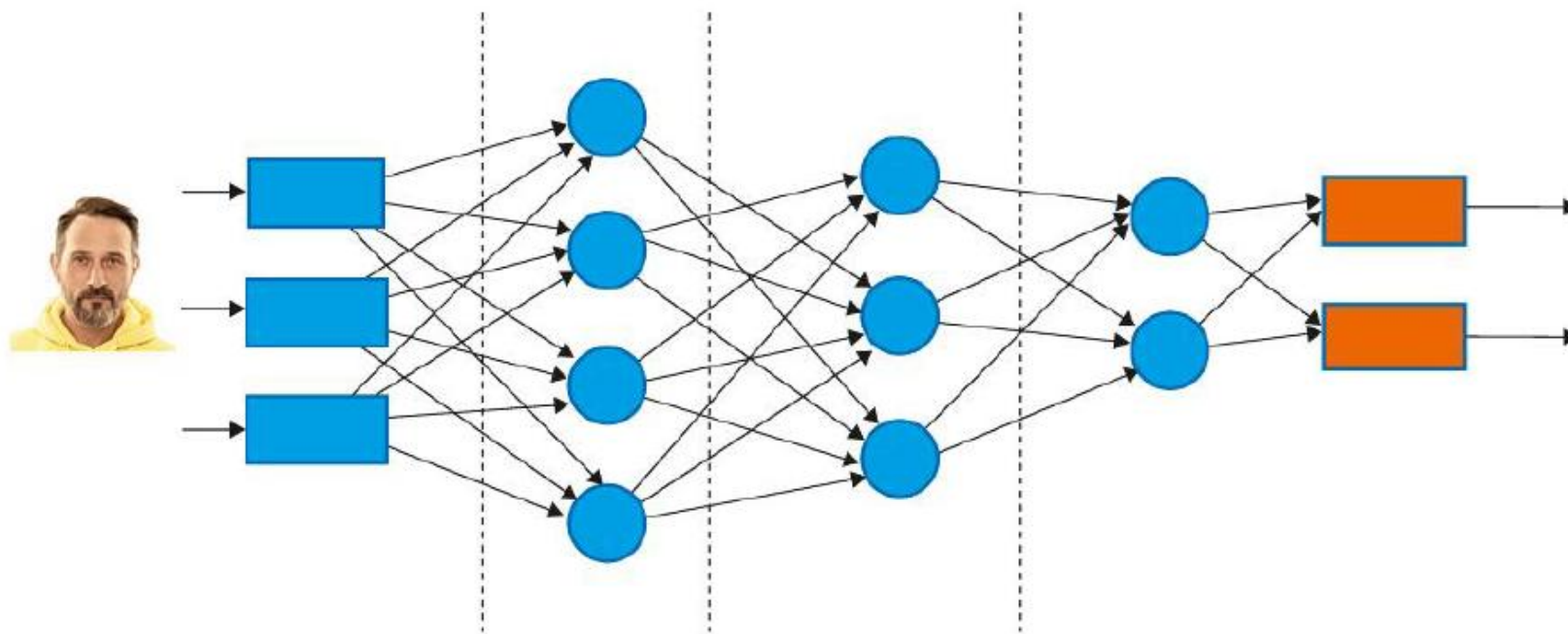
Alcune sono gli alberi di decisione e le reti neurali.

Un albero di decisione ha lo scopo di classificare un insieme, per esempio, in gruppi con caratteristiche omogenee. Ogni ramo si divide in parti: una relativa a gruppi **che hanno** quella caratteristica e una relativa a gruppi **che non hanno** quella caratteristica.

Le foglie rappresentano il valore predittivo che l'albero di decisione permette di ottenere in base ai casi noti del passato per tutte le combinazioni possibili.

Reti neurali

Una **rete neurale** è costituita da un grafo di interconnessioni di informazioni costituite da neuroni artificiali. Essa riceve segnali esterni su uno strato di nodi di ingresso, ciascuno dei quali è collegato con nodi interni, organizzati in più livelli; ogni nodo elabora i segnali ricevuti e trasmette il risultato ai nodi successivi con cui è collegato.



Bias

Con il termine *bias* intendiamo una distorsione di un processo cognitivo che ci porta a percepire la realtà in modo soggettivo, esprimere una valutazione non obiettiva o prendere una decisione non ottimale.

I **bias cognitivi** sono distorsioni del ragionamento; per esempio il *bias* di generalizzazione è quel *bias* che, sulla base di pochi elementi (per esempio «sono stato in Inghilterra tre volte e ha sempre piovuto») induce una legge generale (per es. «in Inghilterra piove sempre»).

I **bias sociali** sono distorsioni nella nostra visione del mondo condizionate da esperienze sociali. Un esempio è un atteggiamento di maggiore o minore accettazione delle diverse comunità etniche nella società in cui viviamo, chiamato, nei casi più estremi, razzismo.

I **bias algoritmici** sono distorsioni delle tecniche di apprendimento e/o dei dati su cui operano. Portano a produrre modelli classificatori o predittivi in cui l'accuratezza e/o l'equità del modello abbiano valori asimmetrici e diversi per particolari categorie sociali a cui il modello viene applicato. I *bias* algoritmici possono essere dovuti all'esistenza di *bias* cognitivi o sociali incorsi nella scelta e costruzione della tecnica di apprendimento e/o dei dati su cui essa opera.

Open Data

Gli **Open Data** sono dati (informazioni, dati numerici, dati alfanumerici ecc.) ad accesso libero, che possono essere utilizzati, riutilizzati e distribuiti.

In Italia dal 2020 è disponibile la Italian Open Data Licence 2.0 (<https://www.dati.gov.it/iodl/2.0/>), che prevede che l'utente possa liberamente:

- consultare, estrarre, scaricare, copiare, pubblicare, distribuire e trasmettere i dati;
- creare un lavoro derivato, per esempio attraverso la combinazione con altre informazioni, includendole in un prodotto o sviluppando un'applicazione informatica che le utilizzi come base di dati.

In cambio, all'utente viene chiesto solo di indicare la fonte dei dati e il nome del soggetto che fornisce il dato, includendo, se possibile, un link alla licenza.

Gli Open Data sono quindi disponibili in un formato aperto, standardizzato e leggibile da applicazioni software, in modo da semplificare la loro consultazione e il loro utilizzo in altre applicazioni informatiche:

Vantaggi degli Open Data

- 1) Trasparenza e responsabilità: gli Open Data permettono ai cittadini di monitorare l'operato delle istituzioni pubbliche, promuovendo un'amministrazione più trasparente e responsabile.
- 2) Innovazione e sviluppo economico: gli Open Data sostengono l'innovazione per la creazione di nuovi prodotti e servizi da parte delle aziende.
- 3) Miglioramento dei servizi pubblici: la pianificazione e l'erogazione dei servizi possono essere migliorati con l'uso degli Open Data.
- 4) Partecipazione e coinvolgimento civico: una maggiore partecipazione dei cittadini alla vita pubblica è favorita dagli Open Data, poiché, attraverso i portali di accesso ai dati, i cittadini possono accedere facilmente alle informazioni e contribuire con idee e soluzioni.

Fonti degli Open Data

- 1) Portali governativi: i Governi di molti Paesi hanno creato portali dedicati in cui sono raccolte e messe a disposizione del pubblico diverse tipologie di dati, come bilanci comunali, statistiche sanitarie ecc.
- 2) Università: molte università e centri di ricerca, nel processo di realizzazione di progetti scientifici e di collaborazioni internazionali, hanno messo a disposizione sempre più spesso i loro dati agli altri ricercatori e al pubblico, favorendo la collaborazione e il progresso scientifico.
- 3) Iniziative private: anche le aziende private hanno iniziato a condividere i loro dati. Google, per esempio, ha reso disponibili i dati delle sue ricerche sulla mobilità urbana, che possono essere utilizzati per migliorare la pianificazione del territorio e delle infrastrutture.